

- [Basic calculations](#)
- [Tutorial](#)
- [Reference](#)

Basic calculations

Symja can be used to calculate basic stuff:

```
>> 1 + 2  
3
```

To submit a command to Symja, press Shift+Return in the Web interface or Return in the console interface. The result will be printed in a new line below your query.

Symja understands all basic arithmetic operators and applies the usual operator precedence. Use parentheses when needed:

```
>> 1 - 2 * (3 + 5) / 4  
-3
```

The multiplication can be omitted:

```
>> 1 - 2 (3 + 5) / 4  
-3
```

But function `f(x)` notation isn't interpreted as `f*(x)`

```
>> f(x)
```

Powers expressions like 3^4 can be entered using `^`:

```
>> 3 ^ 4  
81
```

Integer divisions yield rational numbers like $\frac{6}{4}$:

```
>> 6 / 4  
3/2
```

To convert the result to a floating point number, apply the function `N`:

```
>> N(6 / 4)
1.5
```

For integer number bases other than `10` the `^^` operator can be used. Here's an example for a hexadecimal number:

```
>> 16^^abcdefff
2882400255
```

The number can be converted back to hexadecimal form with the `BaseForm` function:

```
>> BaseForm(2882400255, 16)
Subscript[abcdefff,16]
```

Symbols with upper and lower case characters

Functions are applied using an identifier and parentheses `(` and `)`. In general an identifier is a user-defined or built-in name for a variable, function or constant.

Only the identifiers which consists of only one character are case sensitive. For all other identifiers the input parser doesn't distinguish between lower and upper case characters.

For example: the upper-case identifiers `D`, `E`, `I`, `N`, are different from the identifiers `d`, `e`, `i`, `n`, whereas the functions like `Factorial`, `Integrate` can be entered as `factorial(100)` or `integrate(sin(x),x)`. If you type `SIN(x)` or `sin(x)`, Symja assumes you always mean the same built-in `Sin` function.

Symja provides many common mathematical functions and constants.

For example `Log` calculates the natural logarithm for base `E`. `Log2` and `Log10` are variants for logarithm to the bases `2` and `10`.

```
>> Log(E)
1

>> Sin(Pi)
0

>> Cos(0.5)
0.877583
```

When entering floating point numbers in your query, Symja will perform a numerical evaluation and present a numerical result, pretty much like if you had applied `N`.

Complex numbers

Of course, Symja has complex numbers and uses the equation:

$$\sqrt{-1}=I$$

In Symja the imaginary unit is represented by the uppercase letter **I**:

```
>> Sqrt(-4)
2*I

>> I ^ 2
-1

>> (3 + 2*I) ^ 4
-119+I*120

>> (3 + 2*I) ^ (2.5 - I)
43.663+I*8.28556

>> Tan(I + 0.5)
0.195577+I*0.842966
```

Abs calculates absolute values for complex numbers:

```
>> Abs(-3)
3

>> Abs(3 + 4*I)
5
```

Symja can operate with pretty huge numbers:

```
>> 100!
9332621544394415268169923885626670049071596826438162146859296389521759999322991\
56089414639761565182862536979208272237582511852109168640000000000000000000000000
```

(! denotes the factorial function.) The precision of numerical evaluation can be set:

```
>> N(Pi, 100)
3.1415926535897932384626433832795028841971693993751058209749445923078164062862089986280
```

Division by zero is forbidden and prints the message `Power: Infinite expression 1/0 encountered.`

```
>> 1 / 0
ComplexInfinity
```

Other expressions involving `Infinity` are evaluated:

```
>> Infinity + 2*Infinity
Infinity
```

In contrast to combinatorial belief, 0^0 is undefined and prints the message `Power: Indeterminate expression 0^0 encountered.`:

```
>> 0 ^ 0
Indeterminate
```

The result of the previous query to Symja can be accessed by `%`:

```
>> 3 + 4
7

>> % ^ 2
49
```

In the console available functions can be determined with the `?` operator

```
>> ?ArcC*
ArcCos, ArcCosh, ArcCot, ArcCoth, ArcCsc, ArcCsch
```

Documentation can be displayed by asking for information for the function name.

```
>> ?Integrate
```

Tutorial

The following sections are introductions to the basic principles of the Symja language. A few examples and functions are presented. Only their most common usages are listed; for a full description of their possible arguments, options, etc., see their entry in the "function reference" of built-in symbols.

- [Symbols and assignments](#)
- [Comparisons and Boolean logic](#)
- [Strings](#)
- [Lists](#)

- [The structure of things](#)
- [Functions and patterns](#)
- [Control statements](#)
- [Scoping](#)
- [Plotting graphs and functions](#)
- [Curve sketching](#)
- [Linear algebra](#)
- [Semantic import and Datasets](#)

Reference

- [Expression types](#)
 - [Function by category](#)
 - [Reference of functions and built-in symbols](#)
-

Symbols and assignments

Symbols need not be declared in Symja, they can just be entered and remain variable (Only the symbols which consists of exactly one character are case sensitive. For all other identifiers the input parser doesn't distinguish between lower and upper case characters):

```
>> x
x
```

Basic simplifications are performed:

```
>> x + 2 * x
3*x
```

Symbols can have any name that consists of characters and digits:

```
>> iAm1Symbol ^ 2
iam1symbol^2
```

You can assign values to symbols:

```
>> a = 2
2

>> a ^ 3
8
```

```
>> a = 4
4

>> a ^ 3
64
```

Assigning a value returns that value. If you want to suppress the output of any result, add a `;` to the end of your query:

```
>> a = 4;
```

Values can be copied from one variable to another:

```
>> b = a;
```

Now changing a does not affect b:

```
>> a = 3;

>> b
4
```

Such a dependency can be achieved by using “delayed assignment” with the `:=` operator (which does not return anything, as the right side is not even evaluated):

```
>> b := a ^ 2

>> b
9

>> a = 5;

>> b
25
```

Comparisons and Boolean logic

Values can be compared for equality using the `==` operator:

```
>> 3 == 3
True

>> 3 == 4
False
```

The special symbols `True` and `False` are used to denote truth values. Naturally, there are inequality comparisons as well:

```
>> 3 > 4
False
```

Truth values can be negated using `!` (logical not) and combined using `&&` (logical and) and `||` (logical or):

```
>> !True
False

>> !False
True

>> 3 < 4 && 6 > 5
True
```

`&&` has higher precedence than `||`, i.e. it binds stronger:

```
>> True && True || False && False
True

>> True && (True || False) && False
False
```

Strings

Strings can be entered with `"` as delimiters:

```
>> "Hello world!"
Hello world!"
```

Strings can be joined using the `<>` operator:

```
>> "Hello" <> " " <> "world!"
Hello world!
```

Numbers cannot be joined to strings:

```
>> "Debian" <> 6
Debian<>6
```

They have to be converted to strings using `ToString` first:

```
>> "Debian" <> ToString(6)
Debian6
```

You can input a named Unicode character like this: `\[Name]`

```
>> "\\[Epsilon]"
```

You can input a 2 or 4 digits hexadecimal Unicode character like this: `\.xx` or `\:xxxx`

```
>> "\\:20AC"
€
```

Lists

Lists can be entered in Symja with curly braces `{` and `}`:

```
>> mylist = {a, b, c, d}
{a,b,c,d}
```

There are various functions for constructing lists:

```
>> Range(5)
{1,2,3,4,5}

>> Array(f, 4)
{f(1),f(2),f(3),f(4)}

>> ConstantArray(x, 4)
{x,x,x,x}

>> Table(n ^ 2, {n, 2, 5})
{4,9,16,25}
```

The number of elements of a list can be determined with `Length`:

```
>> Length(mylist)
4
```

Elements can be extracted using double square braces:

```
>> mylist[[3]]  
c
```

Negative indices count from the end:

```
>> mylist[[-3]]  
b
```

Lists can be nested:

```
>> mymatrix = {{1, 2}, {3, 4}, {5, 6}};
```

There are various ways of extracting elements from a list:

```
>> mymatrix[[2, 1]]  
3  
  
>> mymatrix[;;;, 2]  
{2,4,6}  
  
>> Take(mylist, 3)  
{a,b,c}  
  
>> Take(mylist, -2)  
{c,d}  
  
>> Drop(mylist, 2)  
{c,d}  
  
>> First(mymatrix)  
{1,2}  
  
>> Last(mylist)  
d  
  
>> Most(mylist)  
{a,b,c}  
  
>> Rest(mylist)  
{b,c,d}
```

Lists can be used to assign values to multiple variables at once:

```
>> {a, b} = {1, 2};  
  
>> a  
1
```

```
>> b  
2
```

Many operations, like addition and multiplication, “thread” over lists, i.e. lists are combined element-wise:

```
>> {1, 2, 3} + {4, 5, 6}  
{5, 7, 9}  
  
>> {1, 2, 3} * {4, 5, 6}  
{4, 10, 18}
```

It is an error to combine lists with unequal lengths:

```
>> {1, 2} + {4, 5, 6}  
Lists of unequal length cannot be combined: {1,2}+{4,5,6}  
{1,2}+{4,5,6}
```

The structure of things

Every expression in Symja is built upon the same principle: it consists of a head and an arbitrary number of children, unless it is an atom, i.e. it can not be subdivided any further. To put it another way: everything is a function call. This can be best seen when displaying expressions in their “full form”:

```
>> FullForm(a + b + c)  
Plus(a, b, c)
```

Nested calculations are nested function calls:

```
>> FullForm(a + b * (c + d))  
Plus(a, Times(b, Plus(c, d)))
```

Even lists are function calls of the function `List`:

```
>> FullForm({1, 2, 3})  
List(1, 2, 3)
```

The head of an expression can be determined with `Head`:

```
>> Head(a + b + c)  
Plus
```

The children of an expression can be accessed like list elements:

```
>> (a + b + c)[[2]]  
b
```

The head is the 0th element:

```
>> (a + b + c)[[0]]  
Plus
```

The head of an expression can be exchanged using the function `Apply`:

```
>> Apply(g, f(x, y))  
g(x,y)  
  
>> Apply(Plus, a * b * c)  
a+b+c
```

`Apply` can be written using the operator `@@`:

```
>> Times @@ {1, 2, 3, 4}  
24
```

This exchanges the head `List` of `{1, 2, 3, 4}` with `Times`, and then the expression `Times(1, 2, 3, 4)` is evaluated, yielding `24`.

`Apply` can also be applied on a certain level of an expression:

```
>> Apply(f, {{1, 2}, {3, 4}}, {1})  
{f(1,2),f(3,4)}
```

Or even on a range of levels:

```
>> Apply(f, {{1, 2}, {3, 4}}, {0, 2})  
f(f(1,2),f(3,4))
```

`Apply` is similar to `Map` (operator `/@`):

```
>> Map(f, {1, 2, 3, 4})  
{f(1),f(2),f(3),f(4)}
```

```
>> f /@ {{1, 2}, {3, 4}}
{f({1,2}),f({3,4})}
```

The atoms of Symja are numbers, symbols, and strings. `AtomQ` tests whether an expression is an atom:

```
>> AtomQ(5)
True

>> AtomQ(a + b)
False
```

The full form of rational and complex numbers looks like they were compound expressions:

```
>> FullForm(3 / 5)
Rational(3,5)

>> FullForm(3 + 4 * I)
Complex(3,4)
```

However, they are still atoms, thus unaffected by applying functions, for instance:

```
>> f @@ Complex(3, 4)
3+I*4
```

Nevertheless, every atom has a head:

```
>> Head /@ {1, 1/2, 2.0, I, "a string", x}
Integer,Rational,Real,Complex,String,Symbol}
```

The operator `===` tests whether two expressions are the same on a structural level:

```
>> 3 === 3
True

>> 3 == 3.0
True
```

But

```
>> 3 === 3.0
False
```

because `3` (an `Integer`) and `3.0` (a `Real`) are structurally different.

Functions and patterns

Symja implements a [Term rewriting system](#). Here we describe how the rewriting rules can be defined with function and pattern notation.

Functions can be defined in the following way:

```
>> f(x_) := x ^ 2
```

This tells Symja to replace every occurrence of `f` with one (arbitrary) parameter `x` with `x ^ 2`.

```
>> f(3)
9

>> f(a)
a^2
```

The definition of `f` does not specify anything for two parameters, so any such call will stay unevaluated:

```
>> f(1, 2)
f(1,2)
```

In fact, functions in Symja are just one aspect of patterns: `f(x_)` is a pattern that matches expressions like `f(3)` and `f(a)`. The following patterns are available:

```
—
```

matches one expression.

```
x_
```

matches one expression and stores it in `x`.

```
—
```

matches a sequence of one or more expressions.

```
—
```

matches a sequence of zero or more expressions.

```
_h
```

matches one expression with head `h`.

```
x_h
```

matches one expression with head `h` and stores it in `x`.

```
p | q
```

or

```
Alternatives(p, q)
```

matches either pattern `p` or `q`.

```
p ? t
```

or

```
PatternTest(p, t)
```

matches `p` if the test `t(p)` yields `True`.

```
p /; c
```

or

```
Condition(p, c)
```

matches `p` if condition `c` holds.

As before, patterns can be used to define functions:

```
>> g(s____) := Plus(s) ^ 2
```

```
>> g(1, 2, 3)
```

```
36
```

`MatchQ(e, p)` tests whether `e` matches `p`:

```
>> MatchQ(a + b, x_ + y_)
True

>> MatchQ(6, _Integer)
True
```

`ReplaceAll` (operator `/.`) replaces all occurrences of a pattern in an expression using a `Rule` given by `->`:

```
>> {2, "a", 3, 2.5, "b", c} /. x_Integer -> x ^ 2
{4, "a", 9, 2.5, "b", c}
```

You can also specify a list of rules:

```
>> {2, "a", 3, 2.5, "b", c} /. {x_Integer -> x ^ 2.0, y_String -> 10}
{4.0, 10, 9.0, 2.5, 10, c}
```

`ReplaceRepeated` (operator `//.`) applies a set of rules repeatedly, until the expression doesn't change anymore:

```
>> {2, "a", 3, 2.5, "b", c} //. {x_Integer -> x ^ 2.0, y_String -> 10}
{4.0, 100.0, 9.0, 2.5, 100.0, c}
```

There is a “delayed” version of `Rule` which can be specified by `:>` (similar to the relation of `:=` to `=`):

```
>> a :> 1 + 2
a:>1+2

>> a -> 1 + 2
a->3
```

This is useful when the right side of a rule should not be evaluated immediately (before matching):

```
>> {1, 2} /. x_Integer -> N(x)
{1, 2}
```

Here, `N` is applied to `x` before the actual matching, simply yielding `x`. With a delayed rule this can be avoided:

```
>> {1, 2} /. x_Integer :> N(x)
{1.0, 2.0}
```

In addition to defining functions as rules for certain patterns, there are pure functions that can be defined using the `&` postfix operator, where everything before it is treated as the function body and `#` can be used as argument placeholder:

```
>> h = # ^ 2 &;  
  
>> h(3)  
9
```

Multiple arguments can simply be indexed:

```
>> s = #1 + #2 &;  
  
>> s(4, 6)  
10
```

It is also possible to name arguments using `Function`:

```
>> p = Function({x, y}, x * y);  
  
>> p(4, 6)  
24
```

Pure functions are very handy when functions are used only locally, e.g., when combined with operators like `Map`:

```
>> # ^ 2 & /@ Range(5)  
{1, 4, 9, 16, 25}
```

Sort using the second element of a list as a key:

```
>> Sort({{x, 10}, {y, 2}, {z, 5}}, #1[[2]] < #2[[2]] &)  
{{y, 2}, {z, 5}, {x, 10}}
```

Functions can be applied using prefix or postfix notation, in addition to using `()`:

```
>> h @ 3  
9
```

```
>> 3 // h  
9
```

Control statements

Like most programming languages, Symja has common control statements for conditions, loops, etc.:

```
If(cond, pos, neg)
```

returns `pos` if `cond` evaluates to `True`, and `neg` if it evaluates to `False`.

```
Which(cond1, expr1, cond2, expr2, ...)
```

yields `expr1` if `cond1` evaluates to `True`, `expr2` if `cond2` evaluates to `True`, etc.

```
Do(expr, {i, max})
```

evaluates `expr` `max` times, substituting `i` in `expr` with values from `1` to `max`.

```
For(start, test, incr, body)
```

evaluates `start`, and then iteratively `body` and `incr` as long as `test` evaluates to `True`.

```
While(test, body)
```

evaluates `body` as long as `test` evaluates to `True`.

```
Nest(f, expr, n)
```

returns an expression with `f` applied `n` times to `expr`.

```
NestWhile(f, expr, test)
```

applies a function `f` repeatedly on an expression `expr`, until applying `test` on the result no longer yields `True`.

```
FixedPoint(f, expr)
```

starting with `expr`, repeatedly applies `f` until the result no longer changes.

```
>> If(2 < 3, a, b)
a

>> x = 3; Which(x < 2, a, x > 4, b, x < 5, c)
c
```

Compound statements can be entered with `;`. The result of a compound expression is its last part or `Null` if it ends with a `;`.

```
>> 1; 2; 3
3

>> 1; 2; 3;
```

Inside `For`, `while`, and `Do` loops, `Break()` exits the loop and `continue()` continues to the next iteration.

```
>> For(i = 1, i <= 5, i++, If(i == 4, Break())); Print(i)
1
2
3
```

Scoping

By default, all symbols are “global” in Symja, i.e. they can be read and written in any part of your program. However, sometimes “local” variables are needed in order not to disturb the global namespace. Symja provides two ways to support this:

- lexical scoping by `Module`, and
- dynamic scoping by `Block`.

```
Module({vars}, expr)
```

localizes variables by giving them a temporary name of the form `name$number`, where `number` is the current internal Symja value of “module number”. Each time a module is evaluated, “module number” is incremented.

```
Block({vars}, expr)
```

temporarily stores the definitions of certain variables, evaluates `expr` with reset values and restores the original definitions afterwards.

Both scoping constructs shield inner variables from affecting outer ones:

```
>> t = 3
3

>> Module({t}, t = 2)
2

>> Block({t}, t = 2)
2

>> t
3
```

`Module` creates new variables:

```
>> y = x ^ 3;
>> Module({x = 2}, x * y)
2*x^3
```

`Block` does not:

```
>> Block({x = 2}, x * y)
16
```

Thus, `Block` can be used to temporarily assign a value to a variable:

```
>> expr = x ^ 2 + x;
>> Block({x = 3}, expr)
12

>> x
x
```

It is common to use scoping constructs for function definitions with local variables:

```
>> fac(n_) := Module({k, p}, p = 1; For(k = 1, k <= n, ++k, p *= k); p)
>> fac(10)
3628800

>> 10!
3628800
```

Plotting graphs and functions

- [Examples](#)
- [Used JavaScript libraries](#)
- [Generating JavaScript output](#)

Examples

These are some functions integrated in Symja, which allow the output of a graphical "JavaScript control".

- [BarChart](#)
- [BoxWhiskerChart](#)
- [DensityHistogram](#)
- [Histogram](#)
- [PieChart](#)
- [ListLinePlot](#)
- [ListPlot](#)
- [ListPointPlot3D](#)
- [Manipulate](#)
- [ParametricPlot](#)
- [Plot](#)
- [Plot3D](#)

Here are some corresponding examples:

```
>> ListLinePlot(Table({n, n ^ 0.5}, {n, 10}))

>> Manipulate(ListPlot(Table({Sin(t), Cos(t*a)}, {t, 100})), {a,1,4,1})

>> Manipulate(ListPointPlot3D(Table({Sin(t), Cos(t*a), Cos(t^2) }, {t, 500})), {a,1,4,1})

>> Manipulate(Plot3D(Sin(a*x*y), {x, -1.5, 1.5}, {y, -1.5, 1.5}), {a,1,5})

>> ParametricPlot({Sin(t), Cos(t^2)}, {t, 0, 2*Pi})

>> Plot(Sin(x)*Cos(1 + x), {x, 0, 2*Pi})

>> Graphics3D({Darker(Yellow), Sphere({{-1, 0, 0}, {1, 0, 0}, {0, 0, Sqrt(3.0)}}), 1)})
```

The following example displays an undirected weighted [Graph](#) from graph theory functions:

```
>> Graph({1 <-> 2, 2 <-> 3, 3 <-> 4, 4 <-> 1},{EdgeWeight->{2.0,3.0,4.0, 5.0}})
```

[TreeForm](#) visualizes the structure of an expression:

```
>> TreeForm(a+(b*q*s)^(2*y)+Sin(c)^(3-z))
```

Used JavaScript libraries

For the generation of these controls, Symja uses the following JavaScript libraries

- [Math](#) for evaluating common mathematical operations in JavaScript
- [JSXGraph](#) for interactive function plotting and charting
- [MathCell](#) for displaying 3D function plots
- [vis-network](#) for (graph) network views

You can get the HTML source code of the generated controls in most browser by doing a right-mouse-click on the control and selecting menu: `This Frame -> View Frame Source` and save the HTML source code as standalone example on your file system.

Generating JavaScript output

If you would like to use the output from the plot or graph functions in your own web pages, you can generate the JavaScript source code with the [JSForm](#) function.

You can for example display the generated JavaScript form of the [Manipulate](#) function:

```
>> Manipulate(PLOT(Sin(x)*Cos(1 + a*x), {x, 0, 2*Pi}), {a,0,10}) // JSForm
```

and insert it in a HTML template from the [JSXGraph.org](#) project.

Curve sketching

Let's sketch the function

```
>> f(x_) := 4 x / (x ^ 2 + 3 x + 5)
```

The derivatives are

```
>> {f'(x), f''(x), f'''(x)} // Together
{(20-4*x^2)/(25+30*x+19*x^2+6*x^3+x^4), (-120-120*x+8*x^3)/(125+225*x+210*x^2+117*x^3+42*x^4+9*x^5+x^6), (480+1440*x+720*x^2-24*x^4)/(625+1500*x+1850*x^2+1440*x^3+771*x^4+288*x^5+74*x^6+12*x^7+x^8)}
```

To get the extreme values of f , compute the zeroes of the first derivatives:

```
>> extremes = Solve(f'(x) == 0, x)
{{x->-Sqrt(5)},{x->Sqrt(5)}}
```

And test the second derivative:

```
>> f''(x) /. extremes // N
{1.65086, -0.064079}
```

Thus, there is a local maximum at $x = \sqrt{5}$ and a local minimum at $x = -\sqrt{5}$.

Compute the inflection points numerically, chopping imaginary parts close to 0:

```
>> inflections = Solve(f''(x) == 0, x) // N // Chop
{{x->4.29983}, {x->-1.0852}, {x->-3.21463}}
```

Insert into the third derivative:

```
>> f'''(x) /. inflections
{0.00671894, -3.67683, 0.694905}
```

Being different from 0, all three points are actual inflection points. f is not defined where its denominator is 0:

```
>> Solve(Denominator(f(x)) == 0, x)
{{x->1/2*(-3-I*Sqrt(11))}, {x->1/2*(-3+I*Sqrt(11))}}
```

These are non-real numbers, consequently f is defined on all real numbers. The behaviour of f at the boundaries of its definition:

```
>> Limit(f(x), x -> Infinity)
0

>> Limit(f(x), x -> -Infinity)
0
```

Finally, let's plot f :

```
>> Plot(f(x), {x, -8, 6})
```

Linear Algebra

Let's consider the matrix

```
>> A = {{1, 1, 0}, {1, 0, 1}, {0, 1, 1}};
```

The derivatives are

```
>> MatrixForm(A)
```

We can compute its eigenvalues and eigenvectors:

```
>> Eigenvalues(A)
>> Eigenvectors(A)
```

This yields the diagonalization of A:

```
>> T = Transpose(Eigenvectors(A)); MatrixForm(T)
>> Inverse(T) . A . T // MatrixForm
>> % == DiagonalMatrix(Eigenvalues(A))
```

We can solve linear systems:

```
>> LinearSolve(A, {1, 2, 3})
>> A . %
```

In this case, the solution is unique:

```
>> NullSpace(A)
```

Let's consider a singular matrix:

```
>> B = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
>> MatrixRank(B)
>> s = LinearSolve(B, {1, 2, 3})
>> NullSpace(B)
>> B . (RandomInteger(100) * %[[1]] + s)
```

Semantic import and Datasets

The semantic import and datasets functionality is implemented with the help of the [Tablesaw](#) dataframe and visualization library. We'll use Tablesaw to look at data about Tornadoes.

Exploring Tornadoes

To give a better sense of how `SemanticImport` and `Dataset` works, we'll use a tornado data set from NOAA. Here's what we'll cover:

- Read a CSV file
- Viewing dataset metadata
- Previewing data
- Sorting
- Running descriptive stats (mean, min, max, etc.)
- Filtering rows
- Totals and sub-totals
- Saving your data

All the data is in the Symja *data* folder.

Read a CSV file

Here we read a `csv` file of tornado data. The `SemanticImport` function infers the column types by sampling the data and returns a `Dataset` variable named `ds`.

```
>> ds = SemanticImport("https://raw.githubusercontent.com/axkr/symja_android_library/ma
```

Note: that the file is addressed relative to the current working directory. You may have to change it for your code.

If you would like to create smaller datasets you can use the `SemanticImportString` function, which creates a `Dataset` from a String representation:

```
>> ds = SemanticImportString("Products,Sales,Market_Share\nna,5500,3\nb,12200,4\nc,60000
```

Viewing table metadata

Often, the best way to start is to print the column names for reference:

```
>> Keys(ds)
```

The `Dimensions` function displays the row and column counts:

```
>> Dimensions(ds)
```


`Structure` shows the index, name and type of each column. Like many other `Dataset` functions, `Structure` returns a `Dataset`.

```
>> ts=Structure(ds)
```

You can then produce a string representation for display. For convenience, calling `ToString` on a `Dataset` produces a string representation of the table.

```
>> ts // ToString // InputForm
```

You can also perform other table operations on it. For example, the code below removes all columns whose type isn't `DOUBLE`:

```
>> ts(Select("#Column Type" == "STRING" &))
```

Of course, that also returned a `Dataset`. We'll cover selecting rows in more detail later.

Previewing data

The `Span` operator `;;` returns a new table containing the first 3 rows.

```
>> ds(1;;3)
```

This will create a new table containing all rows but only the `Distillery` column

```
>> ds(All, "Distillery")
```

This will create a new table containing all rows but only the columns 3 to 5

```
>> ds(All,3;;5)
```

This will create a new table containing rows 1 to 10 but only the columns `Distillery`, `Latitude` and `Longitude`

```
>> ds(1;;10,{"Distillery", "Latitude", "Longitude"})
```

The `Normal` function converts a table into a list of Symja associations `<|"column-name1"->value1, ... |>`.

```
>> Normal(ds(1;;3))
```

The `InputForm` function shows that the column names are converted to keys of type `String` in the associations.

```
>> Normal(ds(1;;3)) // InputForm
```

Sorting

Now lets sort the table in reverse order by the id column. The negative sign before the name indicates a descending sort.

```
TODO insert Sort() example
```

Descriptive statistics

Descriptive statistics are calculated using the `Summary` function:

```
>> Summary(ds)
```

Filtering

You can write your own `Select` function to filter rows.

```
>> ds(Select(Slot("Latitude") > 30000 && Slot("Longitude") > 80000 &))
```

The next example returns a `Dataset` containing only the columns named in the parameter list, rather than all the columns in the original.

```
>> ds(All, {"Distillery", "Postcode"})
```

Totals and sub-totals

Column metrics can be calculated using methods like `Total`, `Mean`, `Max`, etc.

You can apply those methods to a table, calculating results on one column, grouped by the values in another.

```
>> ds(Total, "Sweetness")
```

```
>> ds(Mean, "Smoky")
```

Saving your data

To save a table, you can write it as a CSV file:

```
>> Export("whiskey_first_3_rows.csv", ds(1;;3))
```

If you would like to create a `string` representation you can use the function `ExportString`

```
>> ExportString(ds(1;;3), "csv")
```

Expression types

- [Arithmetic](#)
- [Combinatorial](#)
- [Comparison](#)
- [Control flow statements](#)
- [Diophantine Equations](#)
- [Formatting output](#)
- [Graph theory](#)
- [Linear algebra](#)
- [Logic](#)
- [Number theory](#)
- [Pattern matching](#)
- [Plot functions](#)
- [Solvers](#)

Arithmetic

- [CubeRoot](#)
- [Divide](#)
- [Minus](#)
- [Plus](#)
- [Power](#)
- [Sqrt](#)
- [Subtract](#)
- [Times](#)
- [Accumulate](#)
- [Total](#)

Combinatorial

- [BernoulliB](#)

- Binomial
- CartesianProduct
- CatalanNumber
- DiceDissimilarity
- Factorial
- Factorial2
- Fibonacci
- IntegerPartitions
- Intersection
- JaccardDissimilarity
- MatchingDissimilarity
- Multinomial
- Partition
- PartitionsP
- PartitionsQ
- Permutations
- RogersTanimotoDissimilarity
- StirlingS1
- StirlingS2
- Subsets
- RussellRaoDissimilarity
- SokalSneathDissimilarity
- Tuples
- Union
- YuleDissimilarity

Comparison

- Equal
- Greater
- GreaterEqual
- Less
- LessEqual
- Max
- Min
- Negative
- NonNegative
- NonPositive
- Positive
- SameQ
- True
- TrueQ

- Unequal
- UnsameQ
- ValueQ

Control flow statements

- Abort
- Block
- Break
- CompoundExpression
- Continue
- Do
- FixedPoint
- FixedPointList
- For
- If
- Nest
- NestList
- NestWhile
- Return
- Switch
- Which
- While

Diophantine Equations

- ChineseRemainder
- FrobeniusNumber
- FrobeniusSolve
- IntegerPartitions
- MultiplicativeOrder
- PartitionsP
- PartitionsQ
- PowerMod

Formatting output

- HornerForm
- InputForm
- MatrixForm
- MathMLForm
- TeXForm

- [TreeForm](#)

Graph theory

- [BetweennessCentrality](#)
- [ClosenessCentrality](#)
- [GraphCenter](#)
- [GraphDiameter](#)
- [GraphPeriphery](#)
- [GraphRadius](#)
- [AdjacencyMatrix](#)
- [EdgeList](#)
- [EdgeQ](#)
- [EulerianGraphQ](#)
- [FindEulerianCycle](#)
- [FindHamiltonianCycle](#)
- [FindVertexCover](#)
- [FindShortestPath](#)
- [FindSpanningTree](#)
- [GraphQ](#)
- [HamiltonianGraphQ](#),
- [VertexEccentricity](#)
- [VertexList](#)
- [VertexQ](#)

Linear algebra

- [ArrayDepth](#)
- [ArrayQ](#)
- [BrayCurtisDistance](#)
- [CanberraDistance](#)
- [CharacteristicPolynomial](#)
- [ChessboardDistance](#)
- [ConjugateTranspose](#)
- [CosineDistance](#)
- [Cross](#)
- [DesignMatrix](#)
- [Det](#)
- [DiagonalMatrix](#)
- [Dimensions](#)
- [Dot](#)
- [Eigenvalues](#)

- Eigenvectors
- EuclideanDistance
- HilbertMatrix
- IdentityMatrix
- Inner
- Inverse
- JacobiMatrix
- LinearProgramming
- LinearSolve
- LUDecomposition
- MatrixPower
- MatrixQ
- MatrixRank
- Norm
- Normalize
- NullSpace
- Orthogonalize
- Outer
- PseudoInverse
- QRDecomposition
- RowReduce
- SingularValueDecomposition
- SquaredEuclideanDistance
- Transpose
- VandermondeMatrix
- VectorAngle
- VectorQ

Logic

- AllTrue
- AnyTrue
- And
- Boole
- BooleanConvert
- BooleanMinimize
- BooleanQ
- Booleans
- Equivalent
- False
- Implies
- NoneTrue

- [Nand](#)
- [Nor](#)
- [Not](#)
- [Or](#)
- [SatisfiabilityCount](#)
- [SatisfiabilityInstances](#)
- [SatisfiableQ](#)
- [TautologyQ](#)
- [True](#)
- [TrueQ](#)
- [Xor](#)

Number theory

- [CoprimeQ](#)
- [ContinuedFraction](#)
- [Divisors](#)
- [EvenQ](#)
- [FactorInteger](#)
- [FromContinuedFraction](#)
- [GCD](#)
- [IntegerExponent](#)
- [JacobiSymbol](#)
- [LCM](#)
- [MersennePrimeExponent](#)
- [MersennePrimeExponentQ](#)
- [Mod](#)
- [NextPrime](#)
- [OddQ](#)
- [PerfectNumber](#)
- [PerfectNumberQ](#)
- [Prime](#)
- [PrimePi](#)
- [PrimePowerQ](#)
- [PrimeQ](#)
- [Quotient](#)

Pattern matching

- [Alternatives](#)
- [Condition](#)
- [Except](#)

- [MatchQ](#)
- [Optional](#)
- [PatternTest](#)
- [Rule](#)
- [RuleDelayed](#)
- [Set](#)
- [SetDelayed](#)

Plot functions

- [ListLinePlot](#)
- [ListPlot](#)
- [ListPlot3D](#)
- [ListPointPlot3D](#)
- [Manipulate](#)
- [ParametricPlot](#)
- [Plot](#)
- [Plot3D](#)

Solvers

- [DSolve](#)
- [Eliminate](#)
- [GroebnerBasis](#)
- [FindInstance](#)
- [FindRoot](#)
- [NRoots](#)
- [NSolve](#)

Reference of functions and built-in symbols

Supported features icon:

- - the symbol / function is supported. Note that this doesn't mean that every possible symbolic evaluation is supported
- - the symbol / function is partially implemented and might not support most basic features of the function
- - the symbol / function is currently not supported
- - the symbol / function is deprecated and will not be further improved
- - the symbol / function is an experimental implementation. It may not fully behave as expected
- - the symbol / function is supported on Java virtual machine

- - the symbol / function is supported on Windows
- - the symbol / function is an alias for another function

Functions in alphabetical order:

- [\\$Assumptions](#)
- [\\$HistoryLength](#)
- [\\$IterationLimit](#)
- [\\$Line](#)
- [\\$MaxMachineNumber](#)
- [\\$MinMachineNumber](#)
- [\\$OperatingSystem](#)
- [\\$RecursionLimit](#)
- [\\$ScriptCommandLine](#)
- [AASTriangle](#)
- [Abort](#)
- [Abs](#)
- [AbsArg](#)
- [AbsoluteTiming](#)
- [Accumulate](#)
- [AddSides](#)
- [AddTo](#)
- [AdjacencyGraph](#)
- [AdjacencyMatrix](#)
- [Adjugate](#)
- [AiryAi](#)
- [AiryAiPrime](#)
- [AiryBi](#)
- [AiryBiPrime](#)
- [All](#)
- [AllTrue](#)
- [Alphabet](#)
- [Alternatives](#)
- [And](#)
- [AnglePath](#)
- [AngleVector](#)
- [Annuity](#)
- [AnnuityDue](#)
- [AntihermitianMatrixQ](#)
- [AntisymmetricMatrixQ](#)
- [AnyTrue](#)
- [Apart](#)
- [Append](#)

- [AppendTo](#)
- [Apply](#)
- [ApplySides](#)
- [ArcCos](#)
- [ArcCosh](#)
- [ArcCot](#)
- [ArcCoth](#)
- [ArcCsc](#)
- [ArcCsch](#)
- [ArcLength](#)
- [ArcSec](#)
- [ArcSech](#)
- [ArcSin](#)
- [ArcSinh](#)
- [ArcTan](#)
- [ArcTanh](#)
- [Area](#)
- [Arg](#)
- [ArgMax](#)
- [ArgMin](#)
- [ArithmeticGeometricMean](#)
- [Array](#)
- [ArrayDepth](#)
- [ArrayPad](#)
- [ArrayPlot](#)
- [ArrayQ](#)
- [ArrayReduce](#)
- [ArrayReshape](#)
- [ArrayRules](#)
- [Arrow](#)
- [ASATriangle](#)
- [AssociateTo](#)
- [Association](#)
- [AssociationMap](#)
- [AssociationQ](#)
- [AssociationThread](#)
- [Assuming](#)
- [AsymptoticDSolveValue](#)
- [AsymptoticRSolveValue](#)
- [AsymptoticSolve](#)
- [AtomQ](#)
- [Attributes](#)

- [BarChart](#)
- [BaseDecode](#)
- [BaseEncode](#)
- [BaseForm](#)
- [Begin](#)
- [BeginPackage](#)
- [BellB](#)
- [BellY](#)
- [BernoulliB](#)
- [BernoulliDistribution](#)
- [BernsteinBasis](#)
- [BesselI](#)
- [BesselJ](#)
- [BesselJZero](#)
- [BesselK](#)
- [BesselY](#)
- [BesselYZero](#)
- [Beta](#)
- [Between](#)
- [BetweennessCentrality](#)
- [BezierFunction](#)
- [BinaryDeserialize](#)
- [BinaryDistance](#)
- [BinarySerialize](#)
- [BinCounts](#)
- [Binomial](#)
- [BinomialDistribution](#)
- [BipartiteGraphQ](#)
- [BitAnd](#)
- [BitClear](#)
- [BitFlip](#)
- [BitGet](#)
- [BitLength](#)
- [BitNot](#)
- [BitOr](#)
- [BitSet](#)
- [BitXor](#)
- [Black](#)
- [Block](#)
- [Blue](#)
- [Boole](#)
- [BooleanConvert](#)

- [BooleanCountingFunction](#)
- [BooleanFunction](#)
- [BooleanMaxterms](#)
- [BooleanMinimize](#)
- [BooleanMinterms](#)
- [BooleanQ](#)
- [Booleans](#)
- [BooleanTable](#)
- [BooleanVariables](#)
- [BoxMatrix](#)
- [BoxWhiskerChart](#)
- [BrayCurtisDistance](#)
- [Break](#)
- [Brown](#)
- [ByteArray](#)
- [ByteArrayQ](#)
- [ByteArrayToString](#)
- [C](#)
- [CanberraDistance](#)
- [Cancel](#)
- [CarlsonRC](#)
- [CarlsonRD](#)
- [CarlsonRF](#)
- [CarlsonRG](#)
- [CarlsonRJ](#)
- [CarmichaelLambda](#)
- [CartesianProduct](#)
- [Cases](#)
- [Catalan](#)
- [CatalanNumber](#)
- [Catch](#)
- [Catenate](#)
- [CauchyDistribution](#)
- [CDF](#)
- [Ceiling](#)
- [CellularAutomaton](#)
- [CentralFeature](#)
- [CentralMoment](#)
- [CharacteristicPolynomial](#)
- [CharacterRange](#)
- [ChebyshevT](#)
- [ChebyshevU](#)

- [Check](#)
- [CheckAbort](#)
- [ChessboardDistance](#)
- [ChineseRemainder](#)
- [CholeskyDecomposition](#)
- [Chop](#)
- [CirclePoints](#)
- [Clear](#)
- [ClearAll](#)
- [ClearAttributes](#)
- [ClebschGordan](#)
- [Clip](#)
- [Close](#)
- [ClosenessCentrality](#)
- [Coefficient](#)
- [CoefficientArrays](#)
- [CoefficientList](#)
- [CoefficientRules](#)
- [Cofactor](#)
- [Collect](#)
- [CollinearPoints](#)
- [Commonest](#)
- [Compile](#)
- [CompiledFunction](#)
- [CompilePrint](#)
- [Complement](#)
- [CompleteGraph](#)
- [CompleteKaryTree](#)
- [Complex](#)
- [Complexes](#)
- [ComplexExpand](#)
- [ComplexInfinity](#)
- [ComplexPlot3D](#)
- [ComposeList](#)
- [ComposeSeries](#)
- [CompositeQ](#)
- [Composition](#)
- [CompoundExpression](#)
- [Compress](#)
- [Condition](#)
- [ConditionalExpression](#)
- [Conjugate](#)

- [ConjugateTranspose](#)
- [ConnectedGraphQ](#)
- [Constant](#)
- [ConstantArray](#)
- [ContainsAll](#)
- [ContainsAny](#)
- [ContainsExactly](#)
- [ContainsNone](#)
- [ContainsOnly](#)
- [Context](#)
- [Contexts](#)
- [Continue](#)
- [ContinuedFraction](#)
- [Convergents](#)
- [CoordinateBoundingBox](#)
- [CoplanarPoints](#)
- [CoprimeQ](#)
- [Correlation](#)
- [CorrelationDistance](#)
- [Cos](#)
- [Cosh](#)
- [CoshIntegral](#)
- [CosineDistance](#)
- [CosIntegral](#)
- [Cot](#)
- [Coth](#)
- [Count](#)
- [CountDistinct](#)
- [Counts](#)
- [Covariance](#)
- [CreateUUID](#)
- [Cross](#)
- [Csc](#)
- [Csch](#)
- [CubeRoot](#)
- [Cuboid](#)
- [Curl](#)
- [Cyan](#)
- [CycleGraph](#)
- [Cycles](#)
- [Cyclotomic](#)
- [Cylinder](#)

- D
- Dataset
- DateObject
- DateString
- DateValue
- DeBruijnSequence
- Decrement
- DedekindNumber
- Default
- DefaultValues
- Defer
- Definition
- Degree
- Delete
- DeleteCases
- DeleteDuplicates
- DeleteDuplicatesBy
- Denominator
- DensityHistogram
- Depth
- Derivative
- DesignMatrix
- Det
- Diagonal
- DiagonalMatrix
- DiagonalMatrixQ
- DialogInput
- DiceDissimilarity
- Diff
- DifferenceDelta
- DifferenceQuotient
- DifferenceRoot
- DigitCharacter
- DigitCount
- DigitQ
- Dimensions
- DiracDelta
- DirectedEdge
- DirectedInfinity
- DirichletBeta
- DirichletEta
- DirichletLambda

- [DiscreteDelta](#)
- [DiscretePlot](#)
- [DiscreteRatio](#)
- [DiscreteShift](#)
- [DiscreteUniformDistribution](#)
- [Discriminant](#)
- [Dispatch](#)
- [Distribute](#)
- [Div](#)
- [Divergence](#)
- [Divide](#)
- [DivideBy](#)
- [DivideSides](#)
- [Divisible](#)
- [Divisors](#)
- [DivisorSigma](#)
- [DivisorSum](#)
- [Do](#)
- [Dot](#)
- [DownValues](#)
- [Drop](#)
- [DSolve](#)
- [DSolveValue](#)
- [Dt](#)
- [E](#)
- [Echo](#)
- [EchoFunction](#)
- [EdgeCount](#)
- [EdgeList](#)
- [EdgeQ](#)
- [EdgeRules](#)
- [EditDistance](#)
- [EffectiveInterest](#)
- [Eigensystem](#)
- [Eigenvalues](#)
- [Eigenvectors](#)
- [Element](#)
- [ElementData](#)
- [Eliminate](#)
- [EllipticE](#)
- [EllipticF](#)
- [EllipticK](#)

- [EllipticPi](#)
- [End](#)
- [EndPackage](#)
- [Entropy](#)
- [Equal](#)
- [Equivalent](#)
- [Erf](#)
- [Erfc](#)
- [Erfi](#)
- [ErlangDistribution](#)
- [EuclideanDistance](#)
- [EulerE](#)
- [EulerGamma](#)
- [EulerianGraphQ](#)
- [EulerPhi](#)
- [EvalF](#)
- [Evaluate](#)
- [EvenQ](#)
- [ExactNumberQ](#)
- [Except](#)
- [Exp](#)
- [Expand](#)
- [ExpandAll](#)
- [ExpandDenominator](#)
- [ExpandNumerator](#)
- [Expectation](#)
- [ExpIntegralE](#)
- [ExpIntegralEi](#)
- [Exponent](#)
- [ExponentialDistribution](#)
- [Export](#)
- [ExportString](#)
- [ExtendedGCD](#)
- [Extract](#)
- [Factor](#)
- [Factorial](#)
- [Factorial2](#)
- [FactorialPower](#)
- [FactorInteger](#)
- [FactorSquareFree](#)
- [FactorSquareFreeList](#)
- [FactorTerms](#)

- [FactorTermsList](#)
- [False](#)
- [Fibonacci](#)
- [FileHash](#)
- [FileNames](#)
- [FilePrint](#)
- [FilterRules](#)
- [FindClusters](#)
- [FindCycle](#)
- [FindEulerianCycle](#)
- [FindFit](#)
- [FindGeneratingFunction](#)
- [FindGraphIsomorphism](#)
- [FindHamiltonianCycle](#)
- [FindInstance](#)
- [FindLinearRecurrence](#)
- [FindMaximum](#)
- [FindMinimum](#)
- [FindPermutation](#)
- [FindRoot](#)
- [FindSequenceFunction](#)
- [FindShortestPath](#)
- [FindShortestTour](#)
- [FindSpanningTree](#)
- [FindVertexCover](#)
- [FiniteAbelianGroupCount](#)
- [FiniteGroupCount](#)
- [First](#)
- [FirstCase](#)
- [FirstPosition](#)
- [Fit](#)
- [FittedModel](#)
- [FiveNum](#)
- [FixedPoint](#)
- [FixedPointList](#)
- [Flat](#)
- [Flatten](#)
- [FlattenAt](#)
- [Floor](#)
- [Fold](#)
- [FoldList](#)
- [For](#)

- [Fourier](#)
- [FourierDCTMatrix](#)
- [FourierDSTMatrix](#)
- [FourierMatrix](#)
- [FractionalPart](#)
- [FrechetDistribution](#)
- [FreeQ](#)
- [FrobeniusNumber](#)
- [FrobeniusSolve](#)
- [FromCharacterCode](#)
- [FromContinuedFraction](#)
- [FromDigits](#)
- [FromLetterNumber](#)
- [FromPolarCoordinates](#)
- [FromRomanNumeral](#)
- [FromSphericalCoordinates](#)
- [FullDefinition](#)
- [FullForm](#)
- [FullSimplify](#)
- [Function](#)
- [FunctionExpand](#)
- [FunctionURL](#)
- [Gamma](#)
- [GammaDistribution](#)
- [Gather](#)
- [GatherBy](#)
- [GCD](#)
- [GegenbauerC](#)
- [GeoDistance](#)
- [GeometricDistribution](#)
- [GeometricMean](#)
- [Get](#)
- [Glaisher](#)
- [GoldbachList](#)
- [GoldenAngle](#)
- [GoldenRatio](#)
- [Grad](#)
- [Graph](#)
- [GraphCenter](#)
- [GraphComplement](#)
- [GraphDiameter](#)
- [GraphDifference](#)

- [GraphDisjointUnion](#)
- [Graphics](#)
- [Graphics3D](#)
- [GraphIntersection](#)
- [GraphPeriphery](#)
- [GraphPower](#)
- [GraphQ](#)
- [GraphRadius](#)
- [GraphUnion](#)
- [Gray](#)
- [Greater](#)
- [GreaterEqual](#)
- [GreaterEqualThan](#)
- [GreaterThan](#)
- [Green](#)
- [GridGraph](#)
- [GroebnerBasis](#)
- [GroupBy](#)
- [Gudermannian](#)
- [GumbelDistribution](#)
- [HamiltonianGraphQ](#)
- [HammingDistance](#)
- [HankelH1](#)
- [HankelH2](#)
- [HankelMatrix](#)
- [HarmonicMean](#)
- [HarmonicNumber](#)
- [Hash](#)
- [Haversine](#)
- [Head](#)
- [HeavisideTheta](#)
- [HermiteDecomposition](#)
- [HermiteH](#)
- [HermitianMatrixQ](#)
- [HessenbergDecomposition](#)
- [HexadecimalCharacter](#)
- [HilbertMatrix](#)
- [Histogram](#)
- [HodgeDual](#)
- [Hold](#)
- [HoldAll](#)
- [HoldAllComplete](#)

- [HoldComplete](#)
- [HoldFirst](#)
- [HoldForm](#)
- [HoldPattern](#)
- [HoldRest](#)
- [HornerForm](#)
- [HurwitzLerchPhi](#)
- [HurwitzZeta](#)
- [HypercubeGraph](#)
- [Hyperfactorial](#)
- [Hypergeometric0F1](#)
- [Hypergeometric1F1](#)
- [Hypergeometric2F1](#)
- [HypergeometricDistribution](#)
- [HypergeometricPFQ](#)
- [HypergeometricU](#)
- [I](#)
- [Identity](#)
- [IdentityMatrix](#)
- [If](#)
- [Im](#)
- [Implies](#)
- [Import](#)
- [ImportString](#)
- [In](#)
- [Increment](#)
- [Indeterminate](#)
- [InexactNumberQ](#)
- [Infinity](#)
- [Inner](#)
- [Input](#)
- [InputForm](#)
- [InputString](#)
- [Insert](#)
- [InstanceOf](#)
- [Int](#)
- [Integer](#)
- [IntegerDigits](#)
- [IntegerExponent](#)
- [IntegerLength](#)
- [IntegerName](#)
- [IntegerPart](#)

- IntegerPartitions
- IntegerQ
- Integers
- Integrate
- InterpolatingFunction
- InterpolatingPolynomial
- Interpolation
- InterquartileRange
- Interrupt
- Intersection
- Interval
- IntervalComplement
- IntervalData
- IntervalIntersection
- IntervalMemberQ
- IntervalUnion
- Inverse
- InverseCDF
- InverseErf
- InverseErfc
- InverseFourier
- InverseFunction
- InverseGammaDistribution
- InverseGudermannian
- InverseHaversine
- InverseLaplaceTransform
- InverseSeries
- InverseZTransform
- IsomorphicGraphQ
- JaccardDissimilarity
- JacobiAmplitude
- JacobiCD
- JacobiCN
- JacobiDN
- JacobiMatrix
- JacobiP
- JacobiSC
- JacobiSD
- JacobiSN
- JacobiSymbol
- JavaClass
- JavaForm

- [JavaNew](#)
- [JavaObject](#)
- [JavaObjectQ](#)
- [JavaShow](#)
- [Join](#)
- [JordanDecomposition](#)
- [JSForm](#)
- [KaryTree](#)
- [Key](#)
- [KeyDrop](#)
- [Keys](#)
- [KeySelect](#)
- [KeysExistsQ](#)
- [KeySort](#)
- [KeyTake](#)
- [KeyValueMap](#)
- [KeyValuePattern](#)
- [Khinchin](#)
- [KolmogorovSmirnovTest](#)
- [KroneckerDelta](#)
- [KroneckerProduct](#)
- [Kurtosis](#)
- [LaguerreL](#)
- [LaplaceTransform](#)
- [Laplacian](#)
- [Last](#)
- [LCM](#)
- [LeafCount](#)
- [LeastSquares](#)
- [LegendreP](#)
- [LegendreQ](#)
- [Length](#)
- [LengthWhile](#)
- [LerchPhi](#)
- [Less](#)
- [LessEqual](#)
- [LessEqualThan](#)
- [LessThan](#)
- [LetterCharacter](#)
- [LetterCounts](#)
- [LetterNumber](#)
- [LetterQ](#)

- [Level](#)
- [LevelQ](#)
- [LeviCivitaTensor](#)
- [LightBlue](#)
- [LightBrown](#)
- [LightCyan](#)
- [LightGray](#)
- [LightGreen](#)
- [LightMagenta](#)
- [LightOrange](#)
- [LightPink](#)
- [LightPurple](#)
- [LightRed](#)
- [LightYellow](#)
- [Limit](#)
- [LinearModelFit](#)
- [LinearProgramming](#)
- [LinearRecurrence](#)
- [LinearSolve](#)
- [List](#)
- [Listable](#)
- [ListConvolve](#)
- [ListCorrelate](#)
- [ListLinePlot](#)
- [ListLinePlot3D](#)
- [ListLogLogPlot](#)
- [ListLogPlot](#)
- [ListPlot](#)
- [ListPlot3D](#)
- [ListPointPlot3D](#)
- [ListQ](#)
- [Ln](#)
- [LoadJavaClass](#)
- [Log](#)
- [Log10](#)
- [Log2](#)
- [LogGamma](#)
- [LogIntegral](#)
- [LogisticSigmoid](#)
- [LogNormalDistribution](#)
- [Lookup](#)
- [LowerCaseQ](#)

- [LowerTriangularize](#)
- [LowerTriangularMatrixQ](#)
- [LucasL](#)
- [LUDecomposition](#)
- [MachineNumberQ](#)
- [Magenta](#)
- [MangoldtLambda](#)
- [ManhattanDistance](#)
- [Manipulate](#)
- [Map](#)
- [MapApply](#)
- [MapAt](#)
- [MapIndexed](#)
- [MapThread](#)
- [MatchingDissimilarity](#)
- [MatchQ](#)
- [MathMLForm](#)
- [MatrixD](#)
- [MatrixExp](#)
- [MatrixForm](#)
- [MatrixFunction](#)
- [MatrixLog](#)
- [MatrixMinimalPolynomial](#)
- [MatrixPlot](#)
- [MatrixPower](#)
- [MatrixQ](#)
- [MatrixRank](#)
- [Max](#)
- [MaxFilter](#)
- [MaximalBy](#)
- [Maximize](#)
- [Mean](#)
- [MeanFilter](#)
- [Median](#)
- [MedianFilter](#)
- [MeijerG](#)
- [MemberQ](#)
- [Merge](#)
- [MersennePrimeExponent](#)
- [MersennePrimeExponentQ](#)
- [Message](#)
- [MessageName](#)

- Messages
- Min
- MinFilter
- MinimalBy
- Minimize
- Minors
- Minus
- MissingQ
- Mod
- ModularInverse
- Module
- MoebiusMu
- MonomialList
- Most
- Multinomial
- MultiplicativeOrder
- MultiplySides
- N
- NakagamiDistribution
- Names
- Nand
- ND
- NDSolve
- Nearest
- NearestTo
- Negative
- Nest
- NestList
- NestWhile
- NestWhileList
- NExpectation
- NextPrime
- NHoldAll
- NHoldFirst
- NHoldRest
- NIntegrate
- NMaximize
- NMinimize
- None
- NoneTrue
- NonNegative
- NonPositive

- Nor
- Norm
- Normal
- NormalDistribution
- Normalize
- Not
- Nothing
- NRoots
- NSolve
- NSum
- Null
- NullSpace
- NumberLinePlot
- NumberQ
- NumberString
- Numerator
- NumericalOrder
- NumericalSort
- NumericFunction
- NumericQ
- OddQ
- Off
- On
- OneIdentity
- OpenAppend
- OpenWrite
- Operate
- OptimizeExpression
- Optional
- Options
- OptionsPattern
- OptionValue
- Or
- Orange
- Order
- OrderedQ
- Ordering
- Orderless
- Orthogonalize
- OrthogonalMatrixQ
- Out
- Outer

- [OutputStream](#)
- [Overflow](#)
- [OwnValues](#)
- [PadLeft](#)
- [PadRight](#)
- [ParametricPlot](#)
- [Parenthesis](#)
- [ParetoDistribution](#)
- [Part](#)
- [Partition](#)
- [PartitionsP](#)
- [PartitionsQ](#)
- [PathGraph](#)
- [PatternTest](#)
- [PauliMatrix](#)
- [Pause](#)
- [PDF](#)
- [PearsonCorrelationTest](#)
- [PerfectNumber](#)
- [PerfectNumberQ](#)
- [Perimeter](#)
- [PermutationCycles](#)
- [PermutationCyclesQ](#)
- [PermutationList](#)
- [PermutationListQ](#)
- [PermutationReplace](#)
- [Permutations](#)
- [Permute](#)
- [PetersenGraph](#)
- [Pi](#)
- [Pick](#)
- [Piecewise](#)
- [PiecewiseExpand](#)
- [PieChart](#)
- [Pink](#)
- [PlanarGraphQ](#)
- [Plot](#)
- [Plot3D](#)
- [Plus](#)
- [PlusMinus](#)
- [Pochhammer](#)
- [Point](#)

- [PoissonDistribution](#)
- [PolarPlot](#)
- [PolyGamma](#)
- [Polygon](#)
- [PolygonalNumber](#)
- [PolyLog](#)
- [PolynomialExtendedGCD](#)
- [PolynomialGCD](#)
- [PolynomialLCM](#)
- [PolynomialQ](#)
- [PolynomialQuotient](#)
- [PolynomialQuotientRemainder](#)
- [PolynomialRemainder](#)
- [Position](#)
- [Positive](#)
- [PossibleZeroQ](#)
- [Power](#)
- [PowerExpand](#)
- [PowerMod](#)
- [PowerRange](#)
- [PowersRepresentations](#)
- [PreDecrement](#)
- [PreIncrement](#)
- [Prepend](#)
- [PrependTo](#)
- [Prime](#)
- [PrimeOmega](#)
- [PrimePi](#)
- [PrimePowerQ](#)
- [PrimeQ](#)
- [PrimeZetaP](#)
- [PrimitiveRootList](#)
- [PrincipleComponents](#)
- [Print](#)
- [PrintableASCIIQ](#)
- [Probability](#)
- [Product](#)
- [ProductLog](#)
- [Projection](#)
- [PseudoInverse](#)
- [Purple](#)
- [QRDecomposition](#)

- QuadraticIrrationalQ
- Quantile
- Quantity
- QuantityMagnitude
- QuantityUnit
- Quartiles
- Quiet
- Quotient
- QuotientRemainder
- Ramp
- RamseyNumber
- Random
- RandomChoice
- RandomComplex
- RandomGraph
- RandomInteger
- RandomPermutation
- RandomPrime
- RandomReal
- RandomSample
- RandomVariate
- Range
- RankedMax
- RankedMin
- Rational
- Rationalize
- Ratios
- Re
- Read
- ReadList
- Real
- RealAbs
- Reals
- RealSign
- RealValuedNumberQ
- Reap
- Red
- Reduce
- Refine
- RegularExpression
- ReIm
- ReleaseHold

- [RemoveDiacritics](#)
- [RepeatedTiming](#)
- [Replace](#)
- [ReplaceAll](#)
- [ReplaceAt](#)
- [ReplaceList](#)
- [ReplacePart](#)
- [ReplaceRepeated](#)
- [Rescale](#)
- [Rest](#)
- [Resultant](#)
- [Return](#)
- [Reverse](#)
- [ReverseSort](#)
- [RiccatiSolve](#)
- [Riffle](#)
- [RightComposition](#)
- [RogersTanimotoDissimilarity](#)
- [RomanNumeral](#)
- [RootMeanSquare](#)
- [Roots](#)
- [RotateLeft](#)
- [RotateRight](#)
- [RotationMatrix](#)
- [RotationTransform](#)
- [Round](#)
- [RowReduce](#)
- [RSolve](#)
- [RSolveValue](#)
- [Rule](#)
- [RuleDelayed](#)
- [RussellRaoDissimilarity](#)
- [SameObjectQ](#)
- [SameQ](#)
- [SASTriangle](#)
- [SatisfiabilityCount](#)
- [SatisfiabilityInstances](#)
- [SatisfiableQ](#)
- [Save](#)
- [SawtoothWave](#)
- [ScalingTransform](#)
- [Scan](#)

- [SchurDecomposition](#)
- [Sec](#)
- [Sech](#)
- [Select](#)
- [SelectFirst](#)
- [SemanticImport](#)
- [SemanticImportString](#)
- [Sequence](#)
- [SequenceHold](#)
- [Series](#)
- [SeriesCoefficient](#)
- [SeriesData](#)
- [Set](#)
- [SetAttributes](#)
- [SetDelayed](#)
- [Share](#)
- [ShearingTransform](#)
- [Sign](#)
- [Signature](#)
- [Simplify](#)
- [Sin](#)
- [Sinc](#)
- [SingularValueDecomposition](#)
- [Sinh](#)
- [SinhIntegral](#)
- [SinIntegral](#)
- [SixJSymbol](#)
- [Skewness](#)
- [Slot](#)
- [SlotSequence](#)
- [SokalSneathDissimilarity](#)
- [Solve](#)
- [Sort](#)
- [SortBy](#)
- [Sow](#)
- [Span](#)
- [SparseArray](#)
- [SparseArrayQ](#)
- [SpecialsFreeQ](#)
- [Sphere](#)
- [SphericalBesselJ](#)
- [SphericalBesselY](#)

- [SphericalHarmonicY](#)
- [Splice](#)
- [Split](#)
- [SplitBy](#)
- [Sqrt](#)
- [SquaredEuclideanDistance](#)
- [SquareFreeQ](#)
- [SquareMatrixQ](#)
- [SquaresR](#)
- [SSSTriangle](#)
- [Stack](#)
- [StackBegin](#)
- [StandardDeviation](#)
- [Standardize](#)
- [StarGraph](#)
- [StartOfLine](#)
- [StartOfString](#)
- [StieltjesGamma](#)
- [StirlingS1](#)
- [StirlingS2](#)
- [String](#)
- [StringCases](#)
- [StringContainsQ](#)
- [StringCount](#)
- [StringExpression](#)
- [StringFreeQ](#)
- [StringInsert](#)
- [StringJoin](#)
- [StringLength](#)
- [StringMatchQ](#)
- [StringPart](#)
- [StringPosition](#)
- [StringQ](#)
- [StringReplace](#)
- [StringReverse](#)
- [StringRiffle](#)
- [StringSplit](#)
- [StringTake](#)
- [StringTemplate](#)
- [StringToByteArray](#)
- [StringToStream](#)
- [StringTrim](#)

- [StruveH](#)
- [StruveL](#)
- [StudentTDistribution](#)
- [Subdivide](#)
- [Subfactorial](#)
- [Subresultants](#)
- [SubsetCases](#)
- [SubsetQ](#)
- [SubsetReplace](#)
- [Subsets](#)
- [Subtract](#)
- [SubtractFrom](#)
- [SubtractSides](#)
- [SudokuSolve](#)
- [Sum](#)
- [Surd](#)
- [SurvivalFunction](#)
- [Switch](#)
- [Symbol](#)
- [SymbolName](#)
- [SymbolQ](#)
- [SymmetricMatrixQ](#)
- [SyntaxQ](#)
- [SystemDialogInput](#)
- [Table](#)
- [TagSet](#)
- [TagSetDelayed](#)
- [Take](#)
- [TakeLargest](#)
- [TakeLargestBy](#)
- [TakeSmallest](#)
- [TakeSmallestBy](#)
- [TakeWhile](#)
- [Tally](#)
- [Tan](#)
- [Tanh](#)
- [TautologyQ](#)
- [TemplateApply](#)
- [TemplateIf](#)
- [TemplateSlot](#)
- [TensorDimensions](#)
- [TensorProduct](#)

- [TensorRank](#)
- [TestReport](#)
- [TestResultObject](#)
- [TeXForm](#)
- [Thread](#)
- [ThreeJSymbol](#)
- [Through](#)
- [Throw](#)
- [TimeConstrained](#)
- [TimeObject](#)
- [Times](#)
- [TimesBy](#)
- [TimeValue](#)
- [Timing](#)
- [ToCharacterCode](#)
- [ToeplitzMatrix](#)
- [ToExpression](#)
- [Together](#)
- [ToLowerCase](#)
- [ToPolarCoordinates](#)
- [ToSphericalCoordinates](#)
- [ToString](#)
- [Total](#)
- [ToUnicode](#)
- [ToUpperCase](#)
- [Tr](#)
- [Trace](#)
- [TransformationFunction](#)
- [TranslationTransform](#)
- [Transliterate](#)
- [Transpose](#)
- [TreeForm](#)
- [TreePlot](#)
- [TrigExpand](#)
- [TrigReduce](#)
- [TrigToExp](#)
- [True](#)
- [TrueQ](#)
- [TTest](#)
- [Tuples](#)
- [Uncompress](#)
- [Undefined](#)

- Underflow
- UndirectedEdge
- Unequal
- Unevaluated
- UniformDistribution
- Union
- Unique
- UnitaryMatrixQ
- UnitConvert
- Unitize
- UnitStep
- UnitVector
- UnsameQ
- Unset
- UpperCaseQ
- UpperTriangularize
- UpperTriangularMatrixQ
- UpValues
- URLDecode
- URLEncode
- ValueQ
- Values
- VandermondeMatrix
- Variables
- Variance
- VectorAngle
- VectorGreater
- VectorGreaterEqual
- VectorLess
- VectorLessEqual
- VectorQ
- Verbatim
- VerificationTest
- VertexCount
- VertexEccentricity
- VertexList
- VertexQ
- WeibullDistribution
- WeierstrassP
- WeightedAdjacencyMatrix
- WeightedGraphQ
- WheelGraph

- [Which](#)
 - [While](#)
 - [White](#)
 - [Whitespace](#)
 - [WhitespaceCharacter](#)
 - [WhittakerM](#)
 - [WhittakerW](#)
 - [WignerD](#)
 - [With](#)
 - [WordBoundary](#)
 - [Xor](#)
 - [Yellow](#)
 - [YuleDissimilarity](#)
 - [ZernikeR](#)
 - [Zeta](#)
 - [ZTransform](#)
-

Installation

The different kinds of installations are described in the [Github Wiki Installation](#) page.

Tutorial

The following sections are introductions to the basic principles of the language of Symja. A few examples and functions are presented. Only their most common usages are listed; for a full description of their possible arguments, options, etc., see their entry in the "function reference" of built-in symbols.

- [Getting started](#)
- [Symbols and assignments](#)
- [Comparisons and Boolean logic](#)
- [Strings](#)
- [Lists](#)
- [The structure of things](#)
- [Functions and patterns](#)
- [Control statements](#)
- [Scoping](#)
- [Plotting graphs and functions](#)
- [Curve sketching](#)
- [Linear algebra](#)
- [Semantic import and Datasets](#)

Reference

- [Expression types](#)
 - [Function by category](#)
 - [Reference of functions and built-in symbols](#)
-

\$Assumptions

\$Assumptions

contains the default assumptions for `Integrate`, `Refine` and `Simplify`.

Related terms

[Assuming](#)

Implementation status

- - full supported

Github

- [Implementation of \\$Assumptions](#)
-

\$HistoryLength

\$HistoryLength

specifies the maximum number of `In` and `out` entries.

Implementation status

- - full supported

Github

- [Implementation of \\$HistoryLength](#)
-

\$IterationLimit

`$IterationLimit`

specifies the maximum number of times a reevaluation of an expression may happen.

Implementation status

- - full supported

Github

- [Implementation of \\$IterationLimit](#)
-

\$Line

```
$Line
```

holds the current input line number.

Implementation status

- - full supported

Github

- [Implementation of \\$Line](#)
-

\$MaxMachineNumber

\$MaxMachineNumber

return the largest positive finite Java `double` value (`Double.MAX_VALUE` approx.
`1.7976931348623157*^308`)

Implementation status

- - full supported

Github

- [Implementation of \\$MaxMachineNumber](#)
-

\$MinMachineNumber

\$MinMachineNumber

return the smallest positive normal Java `double` value (`Double.MIN_NORMAL` approx. $2.2250738585072014 \times 10^{-308}$)

Implementation status

- - full supported

Github

- [Implementation of \\$MinMachineNumber](#)
-

\$OperatingSystem

`$OperatingSystem`

gives the type of operating system ("Windows", "MacOSX", or "Unix") running Symja.

Implementation status

- - full supported

Github

- [Implementation of \\$OperatingSystem](#)
-

\$RecursionLimit

`$RecursionLimit`

holds the current input line number

Implementation status

- - full supported

Github

- [Implementation of \\$RecursionLimit](#)
-

\$ScriptCommandLine

```
$ScriptCommandLine
```

is a list of string arguments when running Symja in script mode. The list starts with the name of the script.

Implementation status

- - full supported

Github

- [Implementation of \\$ScriptCommandLine](#)
-

AASTriangle

```
AASTriangle(alpha, beta, a)
```

returns a triangle from 2 angles `alpha`, `beta` and side `a` (which is not between the angles).

See:

- [Wikipedia - Solution of triangles](#)

Examples

```
>> AASTriangle(Pi/2, Pi/3, b)
Triangle({{0,0},{b/2,0},{0,1/2*Sqrt(3)*b}})

>> ASATriangle(Pi/2, b, Pi/3)
Triangle({{0,0},{b,0},{0,Sqrt(3)*b}})
```

Related terms

[ASATriangle](#), [SASTriangle](#), [SSSTriangle](#)

Implementation status

- - partially implemented

Github

- [Implementation of AASTriangle](#)
-

Abort

```
Abort()
```

aborts an evaluation completely and returns `$Aborted`.

Examples

```
>> Print("a"); Abort(); Print("b")
$Aborted
```

Implementation status

- - full supported

Github

- [Implementation of Abort](#)
-

Abs

```
Abs(expr)
```

returns the absolute value of the real or complex number `expr`.

See:

- [Wikipedia - Absolute value](#)

Examples

```
>> Abs(-3)
3
```

The derivative of `Abs` will not be evaluated because the system assumes `x` could be non-real:

```
>> D(Abs(x), x)
Abs'(x)
```

Use `RealAbs` to calculate the derivative:

```
>> D(RealAbs(x), x)
x/RealAbs(x)
```

Implementation status

- - full supported

Github

- [Implementation of Abs](#)
 - [Rule definitions of Abs](#)
-

AbsArg

```
AbsArg(expr)
```

returns a list of 2 values of the complex number `Abs(expr)`, `Arg(expr)`.

Examples

```
>> AbsArg(I)
{1, Pi/2}
```

Implementation status

- - full supported

Github

- [Implementation of AbsArg](#)
-

AbsoluteTiming

```
AbsoluteTiming(x)
```

returns a list with the first entry containing the evaluation time of `x` and the second entry is the evaluation result of `x`.

Examples

```
>> AbsoluteTiming(x = 1; Pause(x); x + 3)[[1]] > 1  
True
```

Related terms

[Pause](#), [TimeConstrained](#), [Timing](#)

Implementation status

- - full supported

Github

- [Implementation of AbsoluteTiming](#)
-

Accumulate

```
Accumulate(list)
```

accumulate the values of `list` returning a new list.

Examples

```
>> Accumulate({1, 2, 3})  
{1, 3, 6}
```

Implementation status

- - full supported

Github

- [Implementation of Accumulate](#)
-

AddSides

```
AddSides(compare-expr, value)
```

add `value` to all elements of the `compare-expr`. `compare-expr` can be `True`, `False` or an comparison expression with head `Equal`, `Unequal`, `Less`, `LessEqual`, `Greater`, `GreaterEqual`.

Examples

```
>> AddSides(1 < 0, x)
False

>> AddSides(a==b, x)
a+x==b+x
```

Implementation status

- - full supported

Github

- [Implementation of AddSides](#)
-

AddTo

```
AddTo(x, dx)
```

```
x += dx
```

is equivalent to `x = x + dx`.

Examples

```
>> a = 10  
>> a += 2  
12  
  
>> a  
12
```

Implementation status

- - full supported

Github

- [Implementation of AddTo](#)
-

AdjacencyGraph

```
AdjacencyGraph(matrix)
```

convert the adjacency `matrix` into a graph expression.

See:

- [Wikipedia - Adjacency matrix](#)

Examples

The sparse array can be converted to an adjacency matrix in `List` format with the `Normal` function:

```
>> AdjacencyGraph({{0,1,1,0},{0,0,1,0},{0,0,0,0},{0,1,0,0}})
Graph({1,2,3,4},{1->2,1->3,2->3,4->2})
```

Related terms

[AdjacencyMatrix](#), [GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [Normal](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#), [WeightedAdjacencyMatrix](#)

Implementation status

- - full supported

Github

- [Implementation of AdjacencyGraph](#)
-

AdjacencyMatrix

```
AdjacencyMatrix(graph)
```

convert the `graph` into a adjacency matrix in sparse array format.

See:

- [Wikipedia - Adjacency matrix](#)

Examples

The sparse array can be converted to an adjacency matrix in `List` format with the `Normal` function:

```
>> AdjacencyMatrix(Graph({1 -> 2, 2 -> 3, 1 -> 3, 4 -> 2})) // Normal
{{0,1,1,0},
 {0,0,1,0},
 {0,0,0,0},
 {0,1,0,0}}
```

Related terms

[AdjacencyGraph](#), [GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [Normal](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#), [WeightedAdjacencyMatrix](#)

Implementation status

- - partially implemented

Github

- [Implementation of AdjacencyMatrix](#)
-

Adjugate

`Adjugate(matrix)`

calculate the adjugate matrix `Inverse(matrix)*Det(matrix)`

See:

- [Wikipedia - Adjugate matrix](#)

Examples

The 3×3 matrix example from Wikipedia:

```
>> Adjugate({{-3,2,-5}, {-1,0,-2}, {3,-4,1}})
{{-8,18,-4},
 {-5,12,-1},
 {4,-6,2}}
```

Implementation status

- - partially implemented

Github

- [Implementation of Adjugate](#)
-

AiryAi

`AiryAi(z)`

returns the Airy function of the first kind of `z`.

See

- [Wikipedia - Airy function](#)
- [Fungrim - Airy functions](#)

Implementation status

- - experimental

Github

- [Implementation of AiryAi](#)
-

AiryAiPrime

```
AiryAiPrime(z)
```

returns the derivative of the `AiryAi` function.

See

- [Wikipedia - Airy function](#)
- [Fungrim - Airy functions](#)

Implementation status

- - experimental

Github

- [Implementation of AiryAiPrime](#)
-

AiryBi

`AiryBi(z)`

returns the Airy function of the second kind of `z`.

See

- [Wikipedia - Airy function](#)
- [Fungrim - Airy functions](#)

Implementation status

- - experimental

Github

- [Implementation of AiryBi](#)
-

AiryBiPrime

```
AiryBiPrime(z)
```

returns the derivative of the `AiryBi` function.

See

- [Wikipedia - Airy function](#)
- [Fungrim - Airy functions](#)

Implementation status

- - experimental

Github

- [Implementation of AiryBiPrime](#)
-

All

All

is a value for a number of functions indicating to include everything. For example it is a possible value for `Span`, `Part` and `Quiet`.

Examples

In `Part All`, extracts into a first column vector the first element of each of the list elements:

```
>> {{1, 3}, {5, 7}}[[All, 1]]  
{1, 5}
```

While in `Take All`, extracts as a column matrix the first element as a list for each of the list elements:

```
>> Take({{1, 3}, {5, 7}}, All, {1})  
{{1}, {5}}
```

Related terms

[Part](#), [Quiet](#), [Span](#), [Take](#)

AllTrue

```
AllTrue({expr1, expr2, ...}, test)
```

returns `True` if all applications of `test` to `expr1`, `expr2`, ... evaluate to `True`.

```
AllTrue(list, test, level)
```

returns `True` if all applications of `test` to items of `list` at `level` evaluate to `True`.

```
AllTrue(test)
```

gives an operator that may be applied to expressions.

Examples

```
>> AllTrue({2, 4, 6}, EvenQ)
True
>> AllTrue({2, 4, 7}, EvenQ)
False
>> AllTrue({}, EvenQ)
True
```

Implementation status

- - full supported

Github

- [Implementation of AllTrue](#)
-

Alphabet

```
Alphabet()
```

gives the list of lowercase letters `a-z` in the English or Latin alphabet .

```
Alphabet(language-string)
```

returns the languages alphabet as a list of lowercase letters.

Examples

```
>> Alphabet()  
{a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,v,w,x,y,z}  
  
>> Alphabet("Dutch")  
{a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,v,w,x,y,ij,z}
```

Related terms

[FromLetterNumber](#), [LetterNumber](#)

Implementation status

- - full supported

Github

- [Implementation of Alphabet](#)
-

Alternatives

```
Alternatives(p1, p2, ..., p_i)
```

or

```
p1 | p2 | ... | p_i
```

is a pattern that matches any of the patterns `p1, p2, ..., p_i`.

Examples

```
>> a+b+c+d/(a|b)->t  
c + d + 2 t
```

And

```
And(expr1, expr2, ...)
```

`expr1 && expr2 && ...` evaluates each expression in turn, returning `False` as soon as an expression evaluates to `False`. If all expressions evaluate to `True`, `And` returns `True`.

Examples

```
>> True && True && False
False
```

If an expression does not evaluate to `True` or `False`, `And` returns a result in symbolic form:

```
>> a && b && True && c
a && b && c
```

Implementation status

- - full supported

Github

- [Implementation of And](#)
-

AnglePath

```
AnglePath({phi1, phi2, ...})
```

returns the points formed by a turtle starting at $\{0, 0\}$ and angled at 0 degrees going through the turns given by angles phi1 , phi2 , ... and using distance 1 for each step.

```
AnglePath({{r1, phi1}, {r2, phi2}, ...})
```

instead of using 1 as distance, use r_1 , r_2 , ... as distances for the respective steps.

```
AnglePath({{x, y}, phi0}, {phi1, phi2, ...})
```

specifies initial position $\{x, y\}$ and initial direction phi0 .

Examples

```
>> AnglePath({90*Degree, 90*Degree, 90*Degree, 90*Degree})
{{0, 0}, {0, 1}, {-1, 1}, {-1, 0}, {0, 0}}

>> AnglePath({{1, 1}, 90*Degree}, {{1, 90*Degree}, {2, 90*Degree}, {1, 90*Degree}, {2,
{{1, 1}, {0, 1}, {0, -1}, {1, -1}, {1, 1}}

>> AnglePath({a, b})
{{0, 0}, {Cos(a), Sin(a)}, {Cos(a) + Cos(a+b), Sin(a) + Sin(a+b)}}

>> Graphics(Line(AnglePath(Table(1.7, {50}))))
-Graphics-

>> Graphics(Line(AnglePath(RandomReal({-1, 1}, {100}))))
-Graphics-
```

Implementation status

- - full supported

Github

- [Implementation of AnglePath](#)
-

AngleVector

```
AngleVector(phi)
```

returns the point at angle `phi` on the unit circle.

```
AngleVector({r, phi})
```

returns the point at angle `phi` on a circle of radius `r`.

```
AngleVector({x, y}, phi)
```

returns the point at angle `phi` on a circle of radius 1 centered at `{x, y}`.

```
AngleVector({x, y}, {r, phi})
```

returns point at angle `phi` on a circle of radius `r` centered at `{x, y}`.

Examples

```
>> AngleVector(90*Degree)
{0,1}

>> AngleVector({1, 10}, a)
{1+Cos(a), 10+Sin(a)}
```

Implementation status

- - full supported

Github

- [Implementation of AngleVector](#)
-

Annuity

```
Annuity(p, t)
```

or

```
Annuity(p, t, q)
```

returns an annuity object.

See:

- [Wikipedia - Annuity](#)

Examples

```
>> TimeValue(Annuity(100, 12), .01, 0)
1125.508

>> TimeValue(Annuity(100, 12), EffectiveInterest(.01, 0.25), 12)
1268.515
```

Related terms

[AnnuityDue](#), [EffectiveInterest](#), [TimeValue](#)

Implementation status

- - full supported

Github

- [Implementation of Annuity](#)
-

AnnuityDue

```
AnnuityDue(p, t)
```

or

```
AnnuityDue(p, t, q)
```

returns an annuity due object.

See:

- [Wikipedia - Annuity-due](#)

Examples

```
>> TimeValue(AnnuityDue(100, 12), 0.1, 0)
749.5061
```

Related terms

[Annuity](#), [EffectiveInterest](#), [TimeValue](#)

Implementation status

- - full supported

Github

- [Implementation of AnnuityDue](#)
-

AntihermitianMatrixQ

```
AntihermitianMatrixQ(m)
```

returns `True` if `m` is a anti hermitian matrix.

See:

- [Wikipedia - Skew-Hermitian matrix](#)

Examples

```
>> AntihermitianMatrixQ({{42, 7 + 11*I}, {-7 + 11*I, 21}})
True
```

Implementation status

- - full supported

Github

- [Implementation of AntihermitianMatrixQ](#)
-

AntisymmetricMatrixQ

```
AntisymmetricMatrixQ(m)
```

returns `True` if `m` is a anti symmetric matrix.

See:

- [Wikipedia - Skew-symmetric matrix](#)

Examples

```
>> AntisymmetricMatrixQ({{42, 7 + 11*I}, {-7 + 11*I, 21}})
False
```

Implementation status

- - full supported

Github

- [Implementation of AntisymmetricMatrixQ](#)
-

AnyTrue

```
AnyTrue({expr1, expr2, ...}, test)
```

returns `True` if any application of `test` to `expr1`, `expr2`, ... evaluates to `True`.

```
AnyTrue(list, test, level)
```

returns `True` if any application of `test` to items of `list` at `level` evaluates to `True`.

```
AnyTrue(test)
```

gives an operator that may be applied to expressions.

Examples

```
>> AnyTrue({1, 3, 5}, EvenQ)
False
>> AnyTrue({1, 4, 5}, EvenQ)
True
>> AnyTrue({}, EvenQ)
False
```

Implementation status

- - full supported

Github

- [Implementation of AnyTrue](#)
-

Apart

```
Apart(expr)
```

rewrites `expr` as a sum of individual fractions.

```
Apart(expr, var)
```

treats `var` as main variable.

Examples

```
>> Apart((x-1)/(x^2+x))
2/(x+1)-1/x

>> Apart(1 / (x^2 + 5x + 6))
1/(2+x)+1/(-3-x)
```

When several variables are involved, the results can be different depending on the main variable:

```
>> Apart(1 / (x^2 - y^2), x)
-1 / (2 y (x + y)) + 1 / (2 y (x - y))

>> Apart(1 / (x^2 - y^2), y)
1 / (2 x (x + y)) + 1 / (2 x (x - y))
```

`Apart` is Listable:

```
>> Apart({1 / (x^2 + 5x + 6)})
{1/(2+x)+1/(-3-x)}
```

But it does not touch other expressions:

```
>> Sin(1 / (x ^ 2 - y ^ 2)) // Apart
Sin(1/(x^2-y^2))
```

Implementation status

- - full supported

Github

- [Implementation of Apart](#)
-

Append

```
Append(expr, item)
```

returns `expr` with `item` appended to its leaves.

Examples

```
>> Append({1, 2, 3}, 4)
{1, 2, 3, 4}
```

`Append` works on expressions with heads other than `List`:

```
>> Append(f(a, b), c)
f(a, b, c)
```

Unlike `Join`, `Append` does not flatten lists in `item`:

```
>> Append({a, b}, {c, d})
{a, b, {c, d}}
```

Nonatomic expression expected.

```
>> Append(a, b)
Append(a, b)
```

Implementation status

- - full supported

Github

- [Implementation of Append](#)
-

AppendTo

```
AppendTo(s, item)
```

append `item` to value of `s` and sets `s` to the result.

Examples

```
>> s = {}  
>> AppendTo(s, 1)  
{1}  
  
>> s  
{1}
```

`Append` works on expressions with heads other than `List`:

```
>> y = f()  
>> AppendTo(y, x)  
f(x)  
  
>> y  
f(x)
```

`{}` is not a variable with a value, so its value cannot be changed.

```
>> AppendTo({}, 1)  
AppendTo({}, 1)
```

`a` is not a variable with a value, so its value cannot be changed.

```
>> AppendTo(a, b)  
AppendTo(a, b)
```

Implementation status

- - full supported

Github

- [Implementation of AppendTo](#)
-

Apply

```
f @ expr
```

returns `f(expr)`

```
Apply(f, expr)
```

```
f @@ expr
```

replaces the head of `expr` with `f`.

```
Apply(f, expr, levelspec)
```

applies `f` on the parts specified by `levelspec`.

Examples

```
>> f @@ {1, 2, 3}
f(1, 2, 3)
>> Plus @@ {1, 2, 3}
6
```

The head of `expr` need not be `List`:

```
>> f @@ (a + b + c)
f(a, b, c)
```

Apply on level 1:

```
>> Apply(f, {a + b, g(c, d, e * f), 3}, {1})
{f(a, b), f(c, d, e*f), 3}
```

The default level is 0:

```
>> Apply(f, {a, b, c}, {0})
f(a, b, c)
```

Range of levels, including negative level (counting from bottom):


```
>> Apply(f, {{{{{a}}}}}, {2, -3})
{{f(f({a}))}}
```

Convert all operations to lists:

```
>> Apply(List, a + b * c ^ e * f(g), {0, Infinity})
{a, {b, {c, e}, {g}}}
```

Level specification $x + y$ is not of the form n , $\{n\}$, or $\{m, n\}$.

```
>> Apply(f, {a, b, c}, x+y)
Apply(f, {a, b, c}, x + y)
```

The [A001597](#) Perfect powers: m^k where $m > 0$ and $k \geq 2$

```
>> $min = 0; $max = 10^4; Union@ Flatten@ Table( n^expo, {expo, Prime@ Range@ PrimePi@
{1,4,8,9,16,25,27,32,36,49,64,81,100,121,125,128,144,169,196,216,225,243,256,289,"324,3
```

Implementation status

- - full supported

Github

- [Implementation of Apply](#)
-

ApplySides

```
ApplySides(compare-expr, value)
```

divides all elements of the `compare-expr` by `value`. `compare-expr` can be `True`, `False` or a comparison expression with head `Equal`, `Unequal`, `Less`, `LessEqual`, `Greater`, `GreaterEqual`.

Examples

```
>> ApplySides(f, a==a)
True

>> ApplySides(Log, E^2==b)
2==Log(b)
```

Implementation status

- - full supported

Github

- [Implementation of ApplySides](#)
-

ArcCos

`ArcCos(expr)`

returns the arc cosine (inverse cosine) of `expr` (measured in radians).

`ArcCos(expr)` will evaluate automatically in the cases `Infinity`, `-Infinity`, `0`, `1`, `-1`.

See:

- [Wikipedia - Inverse trigonometric functions](#)

Examples

```
>> ArcCos(0)
Pi/2

>> ArcCos(1)
0

>> Integrate(ArcCos(x), {x, -1, 1})
Pi
```

Implementation status

- - full supported

Github

- [Implementation of ArcCos](#)
 - [Rule definitions of ArcCos](#)
-

ArcCosh

ArcCosh(z)

returns the inverse hyperbolic cosine of z .

See:

- [Wikipedia: Inverse hyperbolic function](#)

Examples

```
>> ArcCosh(0)
I*1/2*Pi

>> ArcCosh(0.0)
I*1.5707963267948966

>> ArcCosh(1.4)
0.867014726490565
```

Implementation status

- - full supported

Github

- [Implementation of ArcCosh](#)
 - [Rule definitions of ArcCosh](#)
-

ArcCot

`ArcCot(z)`

returns the inverse cotangent of `z`.

See:

- [Wikipedia - Inverse trigonometric functions](#)

Examples

```
>> ArcCot(0)
Pi/2

>> ArcCot(1)
Pi/4
```

Implementation status

- - full supported

Github

- [Implementation of ArcCot](#)
 - [Rule definitions of ArcCot](#)
-

ArcCoth

ArcCoth(z)

returns the inverse hyperbolic cotangent of z .

See:

- [Wikipedia: Inverse hyperbolic function](#)

Examples

```
>> ArcCoth(0)
I*1/2*Pi

>> ArcCoth(0.0)
I*1.5707963267948966

>> ArcCoth(0.5)
0.5493061443340549+I*(-1.5707963267948966)
```

Implementation status

- - full supported

Github

- [Implementation of ArcCoth](#)
 - [Rule definitions of ArcCoth](#)
-

ArcCsc

`ArcCsc(z)`

returns the inverse cosecant of `z`.

See:

- [Wikipedia - Inverse trigonometric functions](#)

Examples

```
>> ArcCsc(1)
Pi/2

>> ArcCsc(-1)
-Pi/2
```

Implementation status

- - full supported

Github

- [Implementation of ArcCsc](#)
 - [Rule definitions of ArcCsc](#)
-

ArcCsch

ArcCsch(z)

returns the inverse hyperbolic cosecant of z .

See:

- [Wikipedia: Inverse hyperbolic function](#)

Examples

```
>> ArcCsch(0)
ComplexInfinity

>> ArcCsch(1.0)
0.8813735870195429
```

Implementation status

- - full supported

Github

- [Implementation of ArcCsch](#)
 - [Rule definitions of ArcCsch](#)
-

ArcLength

```
ArcLength(geometric-form)
```

returns the length of the `geometric-form`.

See:

- [Wikipedia - Arc length](#)

Examples

```
>> ArcLength(Line({{a,b},{c,d},{e,f}}))  
Sqrt((a-c)^2+(b-d)^2)+Sqrt((c-e)^2+(d-f)^2)
```

Implementation status

- - partially implemented

Github

- [Implementation of ArcLength](#)
-

ArcSec

ArcSec(z)

returns the inverse secant of z .

See:

- [Wikipedia - Inverse trigonometric functions](#)

Examples

```
>> ArcSec(1)
0

>> ArcSec(-1)
Pi
```

Implementation status

- - full supported

Github

- [Implementation of ArcSec](#)
 - [Rule definitions of ArcSec](#)
-

ArcSech

ArcSech(z)

returns the inverse hyperbolic secant of z .

See:

- [Wikipedia: Inverse hyperbolic function](#)

Examples

```
>> ArcSech(0)
Infinity

>> ArcSech(1)
0

>> ArcSech(0.5)
1.3169578969248166
```

Implementation status

- - full supported

Github

- [Implementation of ArcSech](#)
 - [Rule definitions of ArcSech](#)
-

ArcSin

```
ArcSin(expr)
```

returns the arc sine (inverse sine) of `expr` (measured in radians).

`ArcSin(expr)` will evaluate automatically in the cases `Infinity`, `-Infinity`, `0`, `1`, `-1`.

See:

- [Wikipedia - Inverse trigonometric functions](#)

Examples

```
>> ArcSin(0)
0

>> ArcSin(1)
Pi/2
```

Implementation status

- - full supported

Github

- [Implementation of ArcSin](#)
 - [Rule definitions of ArcSin](#)
-

ArcSinh

ArcSinh(z)

returns the inverse hyperbolic sine of z .

See:

- [Wikipedia: Inverse hyperbolic function](#)

Examples

```
>> ArcSinh(0)
0

>> ArcSinh(0.0)
0.0

>> ArcSinh(1.0)
0.8813735870195429
```

Implementation status

- - full supported

Github

- [Implementation of ArcSinh](#)
 - [Rule definitions of ArcSinh](#)
-

ArcTan

```
ArcTan(expr)
```

returns the arc tangent (inverse tangent) of `expr` (measured in radians).

`ArcTan(expr)` will evaluate automatically in the cases `Infinity`, `-Infinity`, `0`, `1`, `-1`.

See:

- [Wikipedia - Inverse trigonometric functions](#)
- [Wikipedia - Atan2](#)
- [Fungrim - Inverse tangent](#)

Examples

```
>> ArcTan(1)
Pi/4

>> ArcTan(1.0)
0.7853981633974483

>> ArcTan(-1.0)
-0.7853981633974483

>> ArcTan(1, 1)
Pi/4

>> ArcTan(-1, 1)
3/4*Pi

>> ArcTan(1, -1)
-Pi/4

>> ArcTan(-1, -1)
-3/4*Pi

>> ArcTan(1, 0)
0

>> ArcTan(-1, 0)
Pi

>> ArcTan(0, 1)
Pi/2

>> ArcTan(0, -1)
-Pi/2
```

Implementation status

- - full supported

Github

- [Implementation of ArcTan](#)
 - [Rule definitions of ArcTan](#)
-

ArcTanh

ArcTanh(z)

returns the inverse hyperbolic tangent of z .

See:

- [Wikipedia: Inverse hyperbolic function](#)

Examples

```
>> ArcTanh(0)
0

>> ArcTanh(1)
Infinity

>> ArcTanh(.5 + 2*I)
0.09641562020299621+I*1.1265564408348223

>> ArcTanh(2+I)
ArcTanh(2+I)
```

Implementation status

- - full supported

Github

- [Implementation of ArcTanh](#)
 - [Rule definitions of ArcTanh](#)
-

Area

```
Area(geometric-form)
```

returns the area of the `geometric-form`.

See:

- [Wikipedia - Area](#)

Examples

```
>> Area(Disk({1,2}))  
Pi
```

Implementation status

- - partially implemented

Github

- [Implementation of Area](#)
-

Arg

```
Arg(expr)
```

returns the argument of the complex number `expr`.

See:

- [Wikipedia - Argument \(complex analysis\)](#)

Examples

```
>> Arg(1+I)
Pi/4
```

`Arg` evaluates the direction of `DirectedInfinity` quantities by the `Arg` of its arguments:

```
>> Arg(DirectedInfinity(1+I))
Pi/4
```

Implementation status

- - full supported

Github

- [Implementation of Arg](#)
-

ArgMax

```
ArgMax(function, variable)
```

returns a maximizer point for a univariate `function`.

See:

- [Wikipedia - Arg max](#)

Examples

```
>> ArgMax(x*10-x^2, x)
5
```

Implementation status

- - full supported

Github

- [Implementation of ArgMax](#)
-

ArgMin

```
ArgMin(function, variable)
```

returns a minimizer point for a univariate `function`.

See:

- [Wikipedia - Arg max](#)

Examples

```
>> ArgMin(x*10+x^2, x)
-5
```

Implementation status

- - full supported

Github

- [Implementation of ArgMin](#)
-

ArithmeticGeometricMean

```
ArithmeticGeometricMean({a, b, c,...})
```

returns the arithmetic geometric mean of `{a, b, c,...}`.

See:

- [Wikipedia - Arithmetic-geometric mean](#)
- [Fungrim - Arithmetic-geometric mean](#)

Examples

```
>> ArithmeticGeometricMean({1.0, 2.0, 3.0, 4.0}, 42.0)
{12.874, 14.88314, 16.37375, 17.62155}
```

Related terms

[GeometricMean](#), [HarmonicMean](#), [Mean](#), [Median](#)

Implementation status

- - full supported

Github

- [Implementation of ArithmeticGeometricMean](#)
-

Array

```
Array(f, n)
```

returns the n -element list $\{f(1), \dots, f(n)\}$.

```
Array(f, n, a)
```

returns the n -element list $\{f(a), \dots, f(a + n)\}$.

```
Array(f, {n, m}, {a, b})
```

returns an n -by- m matrix created by applying f to indices ranging from (a, b) to $(a + n, b + m)$.

```
Array(f, dims, origins, h)
```

returns an expression with the specified dimensions and index origins, with head h (instead of `List`).

Examples

```
>> Array(f, 4)
{f(1), f(2), f(3), f(4)}

>> Array(f, {2, 3})
{{f(1, 1), f(1, 2), f(1, 3)}, {f(2, 1), f(2, 2), f(2, 3)}}

>> Array(f, {2, 3}, {4, 6})
{{f(4, 6), f(4, 7), f(4, 8)}, {f(5, 6), f(5, 7), f(5, 8)}}

>> Array(f, 4)
{f(1), f(2), f(3), f(4)}

>> Array(f, {2, 3})
{{f(1, 1), f(1, 2), f(1, 3)}, {f(2, 1), f(2, 2), f(2, 3)}}

>> Array(f, {2, 3}, 3)
{{f(3, 3), f(3, 4), f(3, 5)}, {f(4, 3), f(4, 4), f(4, 5)}}

>> Array(f, {2, 3}, {4, 6})
{{f(4, 6), f(4, 7), f(4, 8)}, {f(5, 6), f(5, 7), f(5, 8)}}
```

The next line gives 12 samples of $\cos(x/4)$ starting at $(x=4)$.

```
>> Array(Cos(#/4)&, 12, 4)
{Cos(1), Cos(5/4), Cos(3/2), Cos(7/4), Cos(2), Cos(9/4), Cos(5/2), Cos(11/4), Cos(3), Cos(13/4),
```

Array can take a fourth argument which should be applied to the result instead of `List`. The next line finds the minimum of the samples. Notice the function to be applied must be the fourth argument, so you must provide an starting value as a third argument.

```
>> Array(Cos(#/4)&, 12, 4, Min)
Cos(13/4)

>> Array(f, {2, 3}, 1, Plus)
f(1,1)+f(1,2)+f(1,3)+f(2,1)+f(2,2)+f(2,3)
```

`{2, 3}` and `{1, 2, 3}` should have the same length.

```
>> Array(f, {2, 3}, {1, 2, 3})
Array(f, {2, 3}, {1, 2, 3})
```

Single or list of non-negative integers expected at position 2.

```
>> Array(f, a)
Array(f, a)
```

Single or list of non-negative integers expected at position 3.

```
>> Array(f, 2, b)
Array(f, 2, b)
```

Implementation status

- - full supported

Github

- [Implementation of Array](#)
-

ArrayDepth

`ArrayDepth(a)`

returns the depth of the non-ragged array `a`, defined as `Length(Dimensions(a))`.

Examples

```
>> ArrayDepth({{a,b},{c,d}})
2

>> ArrayDepth(x)
0

>> ArrayDepth(SparseArray({{1, 2, 3} -> a}, {2, 3, 4}))
3
```

Implementation status

- - full supported

Github

- [Implementation of ArrayDepth](#)
-

ArrayPad

```
ArrayPad(list, n)
```

adds n times 0 on the left and right of the `list`.

```
ArrayPad(list, {m,n})
```

adds m times 0 on the left and n times 0 on the right.

```
ArrayPad(list, {m, n}, x)
```

adds m times x on the left and n times x on the right.

Examples

```
>> ArrayPad({a, b, c}, 1, x)
{x, a, b, c, x}
```

Implementation status

- - full supported

Github

- [Implementation of ArrayPad](#)
-

ArrayPlot

```
ArrayPlot( matrix-of-values )
```

generate a rectangle image for the `matrix-of-values`.

Examples

```
>> ArrayPlot(RandomReal(1, {10, 20}))
```

Related terms

[ListPlot](#), [ListLogPlot](#), [ListLogLogPlot](#), [Manipulate](#), [ParametricPlot](#), [Plot](#), [Plot3D](#)

Implementation status

- - experimental

Github

- [Implementation of ArrayPlot](#)
-

ArrayQ

```
ArrayQ(expr)
```

tests whether `expr` is a full array.

```
ArrayQ(expr, pattern)
```

also tests whether the array depth of `expr` matches `pattern`.

```
ArrayQ(expr, pattern, test)
```

furthermore tests whether `test` yields `True` for all elements of `expr`.

Examples

```
>> ArrayQ(a)
False

>> ArrayQ({a})
True

>> ArrayQ({{a}}, {{b,c}})
False

>> ArrayQ({{a, b}, {c, d}}, 2, SymbolQ)
True
```

Implementation status

- - full supported

Github

- [Implementation of ArrayQ](#)
-

ArrayReduce

```
ArrayReduce(function, list-of-values, n)
```

returns the `list-of-values` structure reduced for dimension `n`.

Examples

```
>> arr = ArrayReshape(Range(24), {2, 3, 4})
{{{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}}, {{13, 14, 15, 16}, {17, 18, 19, 20}, {21, 22, 23, 24}}}
```

```
>> ArrayReduce(f, arr, 1)
{{f({1, 13}), f({2, 14}), f({3, 15}), f({4, 16})}, {f({5, 17}), f({6, 18}), f({7, 19}), f({8, 20})}, {f({9, 21}), f({10, 22}), f({11, 23}), f({12, 24})}}
```

```
>> ArrayReduce(f, arr, 2)
{{f({1, 5, 9}), f({2, 6, 10}), f({3, 7, 11}), f({4, 8, 12})}, {f({13, 17, 21}), f({14, 18, 22}), f({15, 19, 23}), f({16, 20, 24})}}
```

```
>> ArrayReduce(f, arr, 3)
{{f({1, 2, 3, 4}), f({5, 6, 7, 8}), f({9, 10, 11, 12})}, {f({13, 14, 15, 16}), f({17, 18, 19, 20}), f({21, 22, 23, 24})}}
```

Implementation status

- - experimental

Github

- [Implementation of ArrayReduce](#)
-

ArrayReshape

```
ArrayReshape(list-of-values, list-of-dimension)
```

returns the `list-of-values` elements reshaped as nested list with dimensions according to the `list-of-dimension`.

```
ArrayReshape(list-of-values, list-of-dimension, expr)
```

Use `expr` to fill up elements, if there are too little elements in the `list-of-values`.

Examples

A list of non-negative integers is expected at position 2. The optional third argument `x` is used to fill up the structure:

```
>> ArrayReshape({a, b, c, d, e, f}, {2, 3, 3, 2}, x)
{{{a, b}, {c, d}, {e, f}}, {{x, x}, {x, x}, {x, x}}, {{x, x}, {x, x}, {x, x}}, {{x, x}, {x, x}, {x, x}}}
```

Ignore unnecessary elements

```
>> ArrayReshape(Range(1000), {3, 2, 2})
{{{1, 2}, {3, 4}}, {{5, 6}, {7, 8}}, {{9, 10}, {11, 12}}}
```

Implementation status

- - partially implemented

Github

- [Implementation of ArrayReshape](#)
-

ArrayRules

```
ArrayRules(sparse-array)
```

return the array of rules which define the sparse array.

```
ArrayRules(sparse-array, defaultValue)
```

return the array of rules which define the sparse array and set the new `defaultValue`

.

```
ArrayRules(nestedLists)
```

return the array of rules which define the nested lists.

Examples

```
>> a = {{{0,0},{1,1}},{{0,1},{0,1}}}
      {{{0,0},{1,1}},{{0,1},{0,1}}}

>> ArrayRules(a)
      {{1,2,1}->1,{1,2,2}->1,{2,1,2}->1,{2,2,2}->1,{_,_,_}->0}
```

Related terms

[SparseArray](#)

Implementation status

- - partially implemented

Github

- [Implementation of ArrayRules](#)
-

Arrow

```
Arrow({p1, p2})
```

represents a line from `p1` to `p2` that ends with an arrow at `p2`.

```
Arrow({p1, p2}, s)
```

represents a line with arrow that keeps a distance of `s` from `p1` and `p2`.

```
Arrow({point_1, point_2}, {s1, s2})
```

represents a line with arrow that keeps a distance of `s1` from `p1` and a distance of `s2` from `p2`.

```
Arrow({point_1, point_2}, {s1, s2})
```

represents a line with arrow that keeps a distance of `s1` from `p1` and a distance of `s2` from `p2`.

Examples

Arrows can also be drawn in 3D by giving points in three dimensions:

```
>> Graphics3D(Arrow({{1, 1, -1}, {2, 2, 0}, {3, 3, -1}, {4, 4, 0}}))  
-Graphics3D-
```

Implementation status

- - full supported

Github

- [Implementation of Arrow](#)
-

ASATriangle

```
ASATriangle(alpha, c, beta)
```

returns a triangle from 2 angles `alpha`, `beta` and side `c` (which is between the angles).

See:

- [Wikipedia - Solution of triangles](#)

Examples

```
>> ASATriangle(Pi/2, b, Pi/3)
Triangle({{0,0},{b,0},{0,Sqrt(3)*b}})
```

Related terms

[AASTriangle](#), [SASTriangle](#), [SSSTriangle](#)

Implementation status

- - partially implemented

Github

- [Implementation of ASATriangle](#)
-

AssociateTo

```
AssociateTo(assoc, rule)
```

append `rule` to the association `assoc` and assign the result to `assoc`.

```
AssociateTo(assoc, list-of-rules)
```

append the `list-of-rules` to the association `assoc` and assign the result to `assoc`.

Examples

```
>> assoc = <|"A" -> <|"a" -> 1, "b" -> 2, "c" -> 3|>|>
<|A-><|a->1,b->2,c->3|>|>

>> AssociateTo(assoc, "A" -> 11)
<|A->11|>
```

Related terms

[Association](#), [AssociationQ](#), [AssociationMap](#), [AssociationThread](#), [Counts](#), [Lookup](#), [KeyExistsQ](#), [Keys](#), [KeySort](#), [Values](#)

Implementation status

- - full supported

Github

- [Implementation of AssociateTo](#)
-

Association

```
Association[key1 -> val1, key2 -> val2, ...>
```

or

```
<|key1 -> val1, key2 -> val2, ...|>
```

represents an association between `key`s and `value`s.

Examples

```
>> Association({akey->avalue, bkey->bvalue, ckey->cvalue})  
<|akey->avalue, bkey->bvalue, ckey->cvalue|>
```

`Association` is the head of associations:

```
>> Head(<|a -> x, b -> y, c -> z|>)  
Association  
  
>> <|a -> x, b -> y|>  
<|a -> x, b -> y|>  
  
>> Association({a -> x, b -> y})  
<|a -> x, b -> y|>
```

Associations can be nested:

```
>> <|a -> x, b -> y, <|a -> z, d -> t|>|>  
<|a -> z, b -> y, d -> t|>
```

Related terms

[AssociationQ](#), [Counts](#), [Keys](#), [KeySort](#), [Lookup](#), [Values](#)

Implementation status

- - partially implemented

Github

- [Implementation of Association](#)
-

AssociationMap

```
AssociationMap(header, <|k1->v1, k2->v2, ...|>)
```

create an association `<|header(k1->v1), header(k2->v2), ...|>` with the rules mapped by the `header`.

```
AssociationMap(header, {k1, k2, ...})
```

create an association `<|k1->header(k1), k2->header(k2), ...|>` with the rules mapped by the `header`.

Examples

```
>> AssociationMap(Reverse, <|U->1, V->2|>)
<|1->U, 2->V|>

>> AssociationMap(f, {U, V})
<|U->f(U), V->f(V)|>
```

Related terms

[Association](#), [AssociationQ](#), [AssociationThread](#), [Counts](#), [Lookup](#), [KeyExistsQ](#), [Keys](#), [KeySort](#), [Values](#)

Implementation status

- - full supported

Github

- [Implementation of AssociationMap](#)
-

AssociationQ

```
AssociationQ(expr)
```

returns `True` if `expr` is an association, and `False` otherwise.

Examples

```
>> AssociationQ(<|ahey->avalue, bkey->bvalue, ckey->cvalue|>)
True

>> AssociationQ(<|a -> 1, b :> 2|>)
True

>> AssociationQ(<|a, b|>)
False
```

Related terms

[Association](#), [Counts](#), [Keys](#), [KeySort](#), [Lookup](#), [Values](#)

Github

- [Implementation of AssociationQ](#)
-

AssociationThread

```
AssociationThread({k1, k2, ...}, {v1, v2, ...})
```

create an association with rules from the keys `{k1, k2, ...}` and values `{v1, v2, ...}`.

```
AssociationThread({k1, k2, ...} -> {v1, v2, ...})
```

create an association with rules from the keys `{k1, k2, ...}` and values `{v1, v2, ...}`.

Examples

```
>> AssociationThread({"U", "V"}, {1, 2})  
<|U->1, V->2|>  
  
>> AssociationThread({"U", "V"} :> {1, 2})  
<|U:>1, V:>2|>
```

Related terms

[AssociateTo](#), [Association](#), [AssociationQ](#), [AssociationMap](#), [Counts](#), [Lookup](#), [KeyExistsQ](#), [Keys](#), [KeySort](#), [Values](#)

Implementation status

- - full supported

Github

- [Implementation of AssociationThread](#)
-

Assuming

```
Assuming(assumption, expression)
```

evaluate the `expression` with the assumptions appended to the default `$Assumptions` assumptions.

Examples

```
>> Assuming(x>0, Simplify(Sqrt(x^2)))  
x
```

Related terms

[\\$Assumptions](#)

Implementation status

- - full supported

Github

- [Implementation of Assuming](#)
-

AsymptoticDSolveValue

```
AsymptoticDSolveValue(equation, f(x), {x,a,order})
```

returns an approximation of order `order` for the differential `equation` for the function `f(x)` and variable `x`.

See:

- [Wikipedia - Asymptotic analysis](#)

Examples

```
>> AsymptoticDSolveValue({y'(x) == y(x), y(0) == 1}, y(x), {x, 0, 3})  
1+x+x^2/2+x^3/6
```

AsymptoticRSolveValue

```
AsymptoticRSolveValue(equation, f(x), {x,a,order})
```

returns an approximation for the difference `equation` for the function `f(x)` and variable `x` to order `order`.

Examples

```
>> AsymptoticRSolveValue({a(n) == 3*a(n - 1), a(0) == 5}, a(n), {n, Infinity, 3})  
5*3^n
```

AsymptoticSolve

```
AsymptoticSolve(equation, v->b, {x,a,order})
```

returns an asymptotic approximation for the `equation` to order `order`.

Examples

```
>> AsymptoticSolve(y^2 + y - 2 - x == 0, y -> 1, {x, 0, 2})  
{{y->1+x/3-x^2/27}}
```

AtomQ

`AtomQ(x)`

is true if `x` is an atom (an object such as a number or string, which cannot be divided into subexpressions using 'Part').

Examples

```
>> AtomQ(x)
True

>> AtomQ(1.2)
True

>> AtomQ(2 + I)
True

>> AtomQ(2 / 3)
True

>> AtomQ(x + y)
False
```

Github

- [Implementation of AtomQ](#)
-

Attributes

```
Attributes(symbol)
```

returns the list of attributes which are assigned to `symbol`

Examples

```
>> Attributes(Plus)
{Flat, Listable, OneIdentity, Orderless, NumericFunction}
```

Related terms

[ClearAttributes](#), [Constant](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#), [SetAttributes](#)

Implementation status

- - full supported

Github

- [Implementation of Attributes](#)
-

BarChart

```
BarChart(list-of-values, options)
```

plot a bar chart for a `list-of-values` with option `BarOrigin->Bottom` Or `BarOrigin->Bottom`

Examples

```
>> BarChart({1, -2, 3}, BarOrigin->Bottom)
```

Implementation status

- - experimental

Github

- [Implementation of BarChart](#)
-

BaseDecode

```
BaseDecode(string)
```

decodes a Base64 encoded `string` into a `ByteArray` using the Base64 encoding scheme.

See:

- [Wikipedia - Base64](#)

Examples

The example from Wikipedia:

```
>> ba1 = BaseEncode(StringToByteArray("Man is distinguished, not only by his reason, but
TWFuIGlzIGRpc3Rpbmd1aXNoZWQsIG5vdCBvbmx5IGJ5IGhpcyByZWZb24sIGJ1dCBieSB0aGlzIHNpbmd1bGF
>> ba2 = BaseDecode(ba1)
ByteArray[269 Bytes]

>> ByteArrayToString(ba2)
Man is distinguished, not only by his reason, but by this singular passion from other a
```

Related terms

[BaseEncode](#), [ByteArrayToString](#), [StringToByteArray](#)

Implementation status

- - full supported

Github

- [Implementation of BaseDecode](#)
-

BaseEncode

```
BaseEncode(byte-array)
```

encodes the specified `byte-array` into a string using the Base64 encoding scheme.

See:

- [Wikipedia - Base64](#)

Examples

The example from Wikipedia:

```
>> ba1 = BaseEncode(StringToByteArray("Man is distinguished, not only by his reason, but
TWFuIGlzIGRpc3Rpbmd1aXNoZWQsIG5vdCBvbmx5IGJ5IGhpcyByZWZb24sIGJ1dCBieSB0aGlzIHNpbmd1bGF

>> ba2 = BaseDecode(ba1)
ByteArray[269 Bytes]

>> ByteArrayToString(ba2)
Man is distinguished, not only by his reason, but by this singular passion from other a
```

Related terms

[BaseDecode](#), [ByteArrayToString](#), [StringToByteArray](#)

Implementation status

- - full supported

Github

- [Implementation of BaseEncode](#)
-

BaseForm

```
BaseForm(integer, radix)
```

prints the `integer` number in base `radix` form.

See:

- [Wikipedia - Positional notation - Base conversion](#)
- [Wikipedia - Hexadecimal](#)

Examples

In the Android interface the following output looks like

`$$\textnormal{abcdefff}_{16}$$`

```
>> BaseForm(2882400255, 16)
Subscript("abcdefff",16)

>> 16^^abcdefff
2882400255
```

Implementation status

- - partially implemented

Github

- [Implementation of BaseForm](#)
-

Begin

```
Begin("<context-name>")
```

start a new context definition

Examples

```
>> Begin("mytest`")

>> Context()
mytest`

>> $ContextPath
{System`,Global`}

>> End()
mytest`

>> Context()
Global`

>> $ContextPath
{System`,Global`}
```

Implementation status

- - full supported

Github

- [Implementation of Begin](#)
-

BeginPackage

```
BeginPackage("<context-name>")
```

start a new package definition

Examples

```
>> BeginPackage("Test`")
```

```
>> $ContextPath  
{Test`,System`}
```

```
>> EndPackage( )
```

```
>> $ContextPath  
{Test`,System`,Global`}
```

Implementation status

- - full supported

Github

- [Implementation of BeginPackage](#)
-

BellB

`BellB(n)`

the Bell number function counts the number of different ways to partition a set that has exactly `n` elements

See:

- [Wikipedia - Bell number](#)
- [Fungrim - Bell numbers](#)

Examples

```
>> BellB(15)
1382958545

>> BellB(5, x)
x+15*x^2+25*x^3+10*x^4+x^5
```

Implementation status

- - partially implemented

Github

- [Implementation of BellB](#)
-

BellY

```
BellY(n, k, {x1, x2, ... , xN})
```

the second kind of Bell polynomials (incomplete Bell polynomials).

See:

- [Wikipedia - Bell polynomials](#)

Examples

```
>> BellY(6, 2, {x1, x2, x3, x4, x5})  
10*x3^2+15*x2*x4+6*x1*x5
```

Implementation status

- - partially implemented

Github

- [Implementation of BellY](#)
-

BernoulliB

```
BernoulliB(expr)
```

computes the Bernoulli number of the first kind.

```
BernoulliB(n, expr)
```

computes the Bernoulli polynomial.

See:

- [Wikipedia - Bernoulli number](#)
- [Fungrim - Bernoulli numbers and polynomials](#)

Examples

```
>> BernoulliB(42)
1520097643918070802691/1806
```

Implementation status

- - partially implemented

Github

- [Implementation of BernoulliB](#)
-

BernoulliDistribution

```
BernoulliDistribution(p)
```

returns the Bernoulli distribution.

See:

- [Wikipedia - Bernoulli distribution](#)

Examples

The probability density function of the Bernoulli distribution is

```
>> PDF(BernoulliDistribution(p), x)
Piecewise({{1-p, x==0}, {p, x==1}}, 0)
```

The cumulative distribution function of the Bernoulli distribution is

```
>> CDF(BernoulliDistribution(p), x)
Piecewise({{0, x<0}, {1-p, 0<=x&& x<1}}, 1)
```

The mean of the Bernoulli distribution is

```
>> Mean(BernoulliDistribution(p))
p
```

The standard deviation of the Bernoulli distribution is

```
>> StandardDeviation(BernoulliDistribution(p))
Sqrt((1-p)*p)
```

The variance of the Bernoulli distribution is

```
>> Variance(BernoulliDistribution(p))
(1-p)*p
```

The random variates of a Bernoulli distribution can be generated with function `RandomVariate`

```
>> RandomVariate(BernoulliDistribution(0.25), 10^1)
{1, 0, 0, 0, 1, 1, 0, 0, 0, 0}
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of BernoulliDistribution](#)
-

BernsteinBasis

```
BernsteinBasis(n, v, expr)
```

computes the Bernstein basis for the expression `expr`.

See:

- [Wikipedia - Bernstein polynomial](#)

Examples

```
>> BernsteinBasis(3,2,0.3)
0.189
```

Implementation status

- - full supported

Github

- [Implementation of BernsteinBasis](#)
-

Bessell

```
BesselI(n, z)
```

modified Bessel function of the first kind.

See

- [Wikipedia - Bessel function](#)
- [Fungrim - Bessel functions](#)

Examples

```
>> BesselI(1, 3.6)  
6.79271
```

Implementation status

- - full supported

Github

- [Implementation of Bessell](#)
 - [Rule definitions of Bessell](#)
-

BesselJ

```
BesselJ(n, z)
```

Bessel function of the first kind.

See

- [Wikipedia - Bessel function](#)
- [Fungrim - Bessel functions](#)

Examples

```
>> BesselJ(1, 3.6)  
0.0954655
```

Implementation status

- - full supported

Github

- [Implementation of BesselJ](#)
 - [Rule definitions of BesselJ](#)
-

BesselJZero

```
BesselJZero(n, z)
```

is the k th zero of the `BesselJ(n, z)` function.

See

- [Wikipedia - Bessel function](#)
- [Fungrim - Bessel functions](#)

Examples

```
>> BesselJZero(1.3, 3)
10.61381
```

Implementation status

- - experimental

Github

- [Implementation of BesselJZero](#)
-

BesselK

```
BesselK(n, z)
```

modified Bessel function of the second kind.

See

- [Wikipedia - Bessel function](#)
- [Fungrim - Bessel functions](#)

Examples

```
>> BesselK(1, 3.6)  
0.019795
```

Implementation status

- - full supported

Github

- [Implementation of BesselK](#)
 - [Rule definitions of BesselK](#)
-

BesselY

```
BesselY(n, z)
```

Bessel function of the second kind.

See

- [Wikipedia - Bessel function](#)
- [Fungrim - Bessel functions](#)

Examples

```
>> BesselY(1, 3.6)  
0.415392
```

Implementation status

- - full supported

Github

- [Implementation of BesselY](#)
 - [Rule definitions of BesselY](#)
-

BesselyZero

```
BesselyZero(n, z)
```

is the k th zero of the `Bessely(n, z)` function.

See

- [Wikipedia - Bessel function](#)
- [Fungrim - Bessel functions](#)

Examples

```
>> BesselyZero(1.3, 3)
9.031260842335175
```

Implementation status

- - full supported

Github

- [Implementation of BesselyZero](#)
-

Beta

```
Beta(a, b)
```

is the beta function of the numbers a, b .

```
Beta(x, a, b)
```

The incomplete beta function is a generalization of the beta function. For $x = 1$, the incomplete beta function coincides with the complete beta function.

See

- [Wikipedia - Beta function](#)
- [Fungrim - Beta function](#)

Examples

```
>> Beta(2.3, 3.2)
0.0540298

>> Beta(2, 3)
1/12

>> 12*Beta(1.0, 2, 3)
1.0
```

Implementation status

- - partially implemented

Github

- [Implementation of Beta](#)
-

Between

```
Between(expr, {min, max})
```

```
Between(expr, Interval({min, max}))
```

equivalent to `(min <= expr) && (expr <= max)`.

```
Between(expr, {{min1, max1}, {min2, max2}, ...})
```

```
Between(expr, Interval({min1, max1}, {min2, max2}, ...))
```

equivalent to `(min1 <= expr) && (expr <= max1) || (min2 <= expr) && (expr <= max2) || ...`.

```
Between({min, max})
```

```
Between({{min1, max1}, {min2, max2}, ...})
```

operator form that yields `Between(expr, range)` when applied to expression `expr` for `{min, max}, ...` ranges.

See

- [Wikipedia - Interval arithmetic](#)
- [Wikipedia - Interval \(mathematics\)](#)

Examples

Check that `6` is in range `4..10`:

```
>> Between(6, {4, 10})
True
```

Same as above in operator form:

```
>> Between({4, 10})[6]
True

>> Between({4, 10})[206]
False
```

Implementation status

- - full supported

Github

- [Implementation of Between](#)
-

BetweennessCentrality

```
BetweennessCentrality(graph)
```

Computes the betweenness centrality of each vertex of a `graph`.

See

- [Wikipedia - Betweenness centrality](#)
- [Youtube - Betweenness centrality](#)

Examples

```
>> BetweennessCentrality( Graph({agent1,agent2,agent3,agent4,agent5}, {agent1<->agent2,  
{0.0,0.0,5.0,0.0,0.0}  
  
>> BetweennessCentrality( Graph({1, 3, 2, 6, 4, 5}, { 2->5, 3->6, 4->6, 1->5, 5->4, 6->  
{5.0,0.0,0.0,7.0,5.0,7.0}
```

Implementation status

- - partially implemented

Github

- [Implementation of BetweennessCentrality](#)
-

BezierFunction

```
BezierFunction(list-of-control-points)
```

Bezier curve constructed by `list-of-control-points`

See

- [Wikipedia - Bezier_curve](#)

Examples

```
>> f = BezierFunction({{0, 0}, {0, 0.8}, {1, 1}}); FindRoot(0.5==Indexed(f(x), 2), {x,  
{x->0.361508}
```

Implementation status

- - partially implemented

Github

- [Implementation of BezierFunction](#)
-

BinaryDeserialize

```
BinaryDeserialize(byte-array)
```

deserialize the `byte-array` from WXF format into a Symja expression.

Examples

```
>> BinaryDeserialize(ByteArray({56,58,102,2,115,8,71,108,111,98,97,108,96,102,115,8,71,
f(g,2)
```

Related terms

[BinarySerialize](#), [ByteArray](#), [ByteArrayQ](#), [Export](#), [Import](#)

Implementation status

- - full supported

Github

- [Implementation of BinaryDeserialize](#)
-

BinaryDistance

```
BinaryDistance(u, v)
```

returns the binary distance between `u` and `v`. `0` if `u` and `v` are unequal. `1` if `u` and `v` are equal.

Examples

```
>> BinaryDistance({1, 2, 3, 4, 5}, {1, 2, 3, 4, 5})
1

>> BinaryDistance({1, 2, 3, 4, 5}, {1, 2, 3, 4, -5})
0

>> BinaryDistance({1, 2, 3, 4, 5}, {1, 2, 3, 4, 5.0})
0
```

Related terms

[FindClusters](#), [BrayCurtisDistance](#), [ChessboardDistance](#), [CanberraDistance](#), [CosineDistance](#), [EuclideanDistance](#), [ManhattanDistance](#), [SquaredEuclideanDistance](#)

Implementation status

- - full supported

Github

- [Implementation of BinaryDistance](#)
-

BinarySerialize

```
BinarySerialize(expr)
```

serialize the Symja `expr` into a byte array expression in WXF format.

Examples

```
>> BinarySerialize(f(g,2)) // Normal  
{56,58,102,2,115,8,71,108,111,98,97,108,96,102,115,8,71,108,111,98,97,108,96,103,67,2}
```

Related terms

[BinaryDeserialize](#), [ByteArray](#), [ByteArrayQ](#), [Export](#), [Import](#)

Implementation status

- - full supported

Github

- [Implementation of BinarySerialize](#)
-

BinCounts

```
BinCounts(list, widthOfBin)
```

count the number of elements, if `list`, is divided into successive bins with width `widthOfBin`.

```
BinCounts(list, {min, max, widthOfBin})
```

returns the count of elements in `list` that fall within each bin defined by the range from `min` to `max` with bins of width `widthOfBin`.

Examples

```
>> BinCounts({1,2,3,4,5},5)
{4,1}

>> BinCounts({1,2,3,4,5},10)
{5}

>> BinCounts[{4, 3, 1, 2, 4, 3, 6, 4}, {0, 10, 2}]
{2,5,1,0,0}
```

Implementation status

- - full supported

Github

- [Implementation of BinCounts](#)
-

Binomial

```
Binomial(n, k)
```

returns the binomial coefficient of the 2 integers `n` and `k`

For negative integers `k` this function will return `0` no matter what value is the other argument `n`.

See:

- [Wikipedia - Binomial coefficient](#)
- [Wikipedia - Combination](#)
- [John D. Cook - Binomial coefficients definition](#)
- [Kronenburg 2011 - The Binomial Coefficient for Negative Arguments](#)
- [Fungrim - Factorials and binomial coefficients](#)

Examples

```
>> Binomial(4,2)
6

>> Binomial(5, 3)
10
```

Implementation status

- - partially implemented

Github

- [Implementation of Binomial](#)
-

BinomialDistribution

```
BinomialDistribution(n, p)
```

returns the binomial distribution.

See:

- [Wikipedia - Binomial distribution](#)

Examples

The probability density function of the binomial distribution is

```
>> PDF(BinomialDistribution(n, p), x)
Piecewise({{(1-p)^(n-x)*p^x*Binomial(n,x), 0<=x<=n}}, 0)
```

The cumulative distribution function of the binomial distribution is

```
>> CDF(BinomialDistribution(n, p), x)
Piecewise({{BetaRegularized(1-p, n-Floor(x), 1+Floor(x)), 0<=x&& x<n}, {1, x>=n}}, 0)
```

The mean of the binomial distribution is

```
>> Mean(BinomialDistribution(n, p))
n*p
```

The standard deviation of the binomial distribution is

```
>> StandardDeviation(BinomialDistribution(n, p))
Sqrt(n*(1-p)*p)
```

The variance of the binomial distribution is

```
>> Variance(BinomialDistribution(n, p))
n*(1-p)*p
```

The random variates of a binomial distribution can be generated with function `RandomVariate`

```
>> RandomVariate(BinomialDistribution(10, 0.25), 10^1)
{1, 2, 1, 1, 4, 1, 1, 3, 2, 5}
```


Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of BinomialDistribution](#)
-

BipartiteGraphQ

```
BipartiteGraphQ(expr)
```

test if `expr` is a bipartite graph object.

See:

- [Wikipedia - Bipartite graph](#)
- [Wikipedia - Graph theory](#)

Examples

Implementation status

- - partially implemented

Github

- [Implementation of BipartiteGraphQ](#)
-

BitAnd

```
BitAnd(int1, int2, int3, ...)
```

returns the bitwise `AND` of the integer numbers `int1, int2, int3, ...`

Examples

```
>> Table(BitAnd(n,3), {n,-10,10})  
{2,3,0,1,2,3,0,1,2,3,0,1,2,3,0,1,2,3,0,1,2}
```

Implementation status

- - full supported

Github

- [Implementation of BitAnd](#)
-

BitClear

```
BitClear(i, n)
```

clears the `n`th bit in the integer `i`.

Examples

```
>> IntegerDigits(36,2)
{1,0,0,1,0,0}

>> BitClear(36,2)
32

>> IntegerDigits(32,2)
{1,0,0,0,0,0}

>>BitClear(36,-4)
BitClear(36,-4)
```

Implementation status

- - full supported

Github

- [Implementation of BitClear](#)
-

BitFlip

```
BitFlip(i, n)
```

flips the `n`th bit in the integer `i`.

Examples

```
>> IntegerDigits(36,2)
{1,0,0,1,0,0}

>> BitFlip(36,2)
32

>> IntegerDigits(32,2)
{1,0,0,0,0,0}
```

If `n` is negative the bits of the integer `i` are counted from the left.

```
>> BitFlip(36, -4)
32
```

Implementation status

- - full supported

Github

- [Implementation of BitFlip](#)
-

BitGet

```
BitGet(i, n)
```

gets the `n`th bit in the integer `i`.

Examples

```
>> IntegerDigits(36,2)
{1,0,0,1,0,0}

>> BitGet(36,2)
1

>> BitGet(36,-4)
BitGet(36,-4)
```

Implementation status

- - full supported

Github

- [Implementation of BitGet](#)
-

BitLength

`BitLength(x)`

gives the number of bits needed to represent the integer `x`. The sign of `x` is ignored.

Examples

```
>> BitLength(1023)
10

>> BitLength(100)
7

>> BitLength(-5)
3

>> BitLength(0)
0
```

Implementation status

- - full supported

Github

- [Implementation of BitLength](#)
-

BitNot

```
BitNot(integer-value)
```

returns the bitwise `NOT` of the integer number `integer-value`.

Examples

```
>> Table(BitNot(n), {n, -10, 10})  
{9, 8, 7, 6, 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11}
```

Implementation status

- - full supported

Github

- [Implementation of BitNot](#)
-

BitOr

```
BitOr(int1, int2, int3, ...)
```

returns the bitwise `OR` of the integer numbers `int1, int2, int3, ....`

Examples

```
>> Table(BitOr(n,3), {n,-10,10})  
{-9,-9,-5,-5,-5,-5,-1,-1,-1,-1,3,3,3,3,7,7,7,7,11,11,11}
```

Implementation status

- - full supported

Github

- [Implementation of BitOr](#)
-

BitSet

```
BitSet(i, b)
```

set the `b`th bit in the integer `i`.

Examples

```
>> IntegerDigits(32,2)
{1,0,0,0,0,0}

>> BitSet(32,1)
34

>> IntegerDigits(34,2)
{1,0,0,0,1,0}
```

Implementation status

- - full supported

Github

- [Implementation of BitSet](#)
-

BitXor

```
BitXor(int1, int2, int3, ...)
```

returns the bitwise `xOR` of the integer numbers `int1`, `int2`, `int3`,

Examples

```
>> Table(BitXor(n,3), {n,-10,10})  
{-11,-12,-5,-6,-7,-8,-1,-2,-3,-4,3,2,1,0,7,6,5,4,11,10,9}
```

Implementation status

- - full supported

Github

- [Implementation of BitXor](#)
-

Black

Black

RGB color value for the color black

Examples

```
>> Black  
RGBColor(0.0,0.0,0.0)
```

Block

```
Block({list_of_local_variables}, expr )
```

evaluates `expr` for the `list_of_local_variables`

Examples

```
>> $blk=Block({$i=10}, $i=$i+1; Return($i))  
11
```

The [A005132 Recaman's sequence](#) integer sequence

```
>> f(s_List) := Block({a = s[[-1]], len = Length@s}, Append(s, If(a > len && !MemberQ(s  
{0,1,3,6,2,7,13,20,12,21,11,22,10,23,9,24,8,25,43,62,42,63,41,18,42,17,43,16,44,15,45,1
```

Related terms

[Module](#), [With](#)

Implementation status

- - full supported

Github

- [Implementation of Block](#)
-

Blue

Blue

RGB color value for the color blue

Examples

```
>> Blue  
RGBColor(0.0,0.0,1.0)
```

Boole

```
Boole(expr)
```

returns 1 if `expr` evaluates to `True`; returns 0 if `expr` evaluates to `False`; and gives no result otherwise.

Examples

```
>> Boole(2 == 2)
1
>> Boole(7 < 5)
0
>> Boole(a == 7)
Boole(a==7)
```

Implementation status

- - full supported

Github

- [Implementation of Boole](#)
-

BooleanConvert

```
BooleanConvert(logical-expr)
```

convert the `logical-expr` to [disjunctive normal form](#)

```
BooleanConvert(logical-expr, "CNF")
```

convert the `logical-expr` to [conjunctive normal form](#)

```
BooleanConvert(logical-expr, "DNF")
```

convert the `logical-expr` to [disjunctive normal form](#)

Examples

```
>> BooleanConvert(Xor(x,y))
x&&!y||y&&!x

>> BooleanConvert(Xor(x,y), "CNF")
(x||y)&&(!x||!y)

>> BooleanConvert(Xor(x,y), "DNF")
x&&!y||y&&!x
```

Implementation status

- - full supported

Github

- [Implementation of BooleanConvert](#)
-

BooleanCountingFunction

```
BooleanCountingFunction(spec, vars)
```

Returns a boolean function (in disjunctive normal form) in the given `vars` which evaluates to `True` exactly when the number of `True` variables matches `spec`.

Examples

```
>> BooleanCountingFunction({2, 3}, {a, b, c, d})  
(a&&b&&!c)|| (a&&!b&&c)|| (a&&!b&&d)|| (!a&&b&&d)|| (!a&&c&&d)|| (b&&c&&!d)
```

BooleanFunction

```
BooleanFunction(n, number-of-variables)
```

create the `n`-th boolean function containing the `number-of-variables`. The `i`-th variable is represented by the `i`-th slot.

```
BooleanFunction({{b11, b12, ...}->True, {b21, b22, ...}->True, ...}, {v1, v2, ...})
```

create the boolean function by defining all possible combinations `{bx1, bx2, ...}` for obtaining `True` for the variable names `{v1, v2, ...}`

Examples

```
>> f=BooleanFunction(42,3);

>> f(True,False,True)
True

>> f(True,False,False)
False

>> BooleanConvert(f, "DNF")
(!#1&&#3)||(!#2&&#3)&

>> BooleanConvert(BooleanFunction({{False, False} -> False, {False, True} -> False, {True,
x&&y

>> BooleanConvert(BooleanFunction({{False, False} -> False, {False, True} -> True, {True,
x||y
```

Implementation status

- - full supported

Github

- [Implementation of BooleanFunction](#)
-

BooleaMaxterms

```
BooleaMaxterms({{b1,b2,...}}, {v1,v2,...})
```

create the conjunction of the variables `{v1,v2,...}`.

Examples

```
>> BooleanMaxterms({{1,1,1,1}}, {a, b, c,d})  
a||b||c||d  
  
>> BooleanMaxterms({{True,True,True,True}}, {a, b, c,d})  
a||b||c||d
```

Implementation status

- - full supported

Github

- [Implementation of BooleanMaxterms](#)
-

BooleanMinimize

```
BooleanMinimize(expr)
```

minimizes a boolean function with the [Quine McCluskey algorithm](#)

Examples

```
>> BooleanMinimize(x&& y || (!x)&& y)
y

>> BooleanMinimize((a&&!b) || (!a&&b) || (b&&!c) || (!b&&c))
a&&!b || !a&&c || b&&!c
```

Implementation status

- - full supported

Github

- [Implementation of BooleanMinimize](#)
-

BooleanMinterms

```
BooleanMinterms({{b1,b2,...}}, {v1,v2,...})
```

create the disjunction of the variables `{v1,v2,...}`.

Examples

```
>> BooleanMinterms({{1,1,1,1}}, {a, b, c,d})  
a&&b&&c&&d  
  
>> BooleanMinterms({{True,True,True,True}}, {a, b, c,d})  
a&&b&&c&&d
```

Implementation status

- - full supported

Github

- [Implementation of BooleanMinterms](#)
-

BooleanQ

```
BooleanQ(expr)
```

returns `True` if `expr` is either `True` or `False`.

Examples

```
>> BooleanQ(True)
True
>> BooleanQ(False)
True
>> BooleanQ(a)
False
>> BooleanQ(1 < 2)
True
>> BooleanQ("string")
False
>> BooleanQ(Together(x/y + y/x))
False
```

Github

- [Implementation of BooleanQ](#)
-

Booleans

Booleans

is the set of boolean values.

Examples

```
>> Solve(Xor(a, b, c, d) && (a || b) && ! (c || d), {a, b, c, d}, Booleans)
{{a->False,b->True,c->False,d->False},{a->True,b->False,c->False,d->False}}
```

Related terms

[Solve](#)

BooleanTable

```
BooleanTable(logical-expr, variables)
```

generate [truth values](#) from the `logical-expr`

Examples

```
>> BooleanTable(Implies(Implies(p, q), r), {p, q, r})
{True, False, True, True, True, False, True, False}

>> BooleanTable(Xor(p, q, r), {p, q, r})
{True, False, False, True, False, True, True, False}
```

Implementation status

- - full supported

Github

- [Implementation of BooleanTable](#)
-

BooleanVariables

```
BooleanVariables(logical-expr)
```

gives a list of the boolean variables that appear in the `logical-expr`.

Examples

```
>> BooleanVariables(Xor(p,q,r))  
{p,q,r}
```

Related terms

[Variables](#)

Implementation status

- - full supported

Github

- [Implementation of BooleanVariables](#)
-

BoxMatrix

```
BoxMatrix(radius, dimension)
```

gives a matrix of $2 \times \text{radius} + 1$ size inside a $\text{dimension} \times \text{dimension}$ matrix

Examples

BoxWhiskerChart

```
BoxWhiskerChart( )
```

plot a box whisker chart.

Examples

```
>> BoxWhiskerChart(RandomVariate(NormalDistribution(0, 1), 50))
```

Implementation status

- - experimental

Github

- [Implementation of BoxWhiskerChart](#)
-

BrayCurtisDistance

```
BrayCurtisDistance(u, v)
```

returns the Bray Curtis distance between `u` and `v`.

Examples

```
>> BrayCurtisDistance({-1, -1}, {10, 10})  
11/9
```

Implementation status

- - full supported

Github

- [Implementation of BrayCurtisDistance](#)
-

Break

Break()

exits a `For`, `While`, or `Do` loop.

Examples

```
>> n = 0
>> While(True, If(n>10, Break())); n=n+1)
>> n
11
```

Related terms

[Continue](#), [Do](#), [For](#), [While](#)

Implementation status

- - full supported

Github

- [Implementation of Break](#)
-

Brown

Brown

RGB color value for the color brown

Examples

```
>> Brown  
RGBColor(0.6,0.4,0.2)
```

ByteArray

```
ByteArray({list-of-byte-values})
```

converts the `list-of-byte-values` into a byte array. The argument in `ByteArray` should be a vector of unsigned byte values or a Base64-encoded string.

Examples

```
>> ByteArray({1,2,3})  
ByteArray[3 Bytes]
```

Related terms

[BinaryDeserialize](#), [BinarySerialize](#), [ByteArrayQ](#), [Export](#), [Import](#), [Normal](#)

Implementation status

- - full supported

Github

- [Implementation of ByteArray](#)
-

ByteArrayQ

```
ByteArrayQ(expr)
```

test if `expr` is a byte array object.

Examples

```
>> ByteArrayQ(ByteArray({1,2,3}))  
True
```

Related terms

[BinaryDeserialize](#), [BinarySerialize](#), [ByteArray](#), [Export](#), [Import](#)

Github

- [Implementation of ByteArrayQ](#)
-

ByteArrayToString

```
ByteArrayToString(byte-array)
```

decoding the specified `byte-array` using the default character set `UTF-8`.

See:

- [Wikipedia - Base64](#)

Examples

The example from Wikipedia:

```
>> ba1 = BaseEncode(StringToByteArray("Man is distinguished, not only by his reason, but
TWFuIGlzIGRpc3Rpbmd1aXNoZWQsIG5vdCBvbmx5IGJ5IGhpcyByZWZzb24sIGJ1dCBieSB0aGlzIHNpbmd1bGF

>> ba2 = BaseDecode(ba1)
ByteArray[269 Bytes]

>> ByteArrayToString(ba2)
Man is distinguished, not only by his reason, but by this singular passion from other a
```

Related terms

[BaseDecode](#), [BaseEncode](#), [StringToByteArray](#)

Implementation status

- - full supported

Github

- [Implementation of ByteArrayToString](#)
-

C

$C(n)$

represents the n -th constant in a solution to a differential equation.

Examples

```
>> DSolve({y'(x)==y(x)+2},y(x), x)
{{y(x)->-2+E^x*C(1)}}
```

Related terms

[DSolve](#)

CanberraDistance

```
CanberraDistance(u, v)
```

returns the canberra distance between `u` and `v`, which is a weighted version of the Manhattan distance.

See

- [Wikipedia - Canberra distance](#)

Examples

```
>> CanberraDistance({-1, -1}, {1, 1})  
2
```

Related terms

[FindClusters](#), [BinaryDistance](#), [BrayCurtisDistance](#), [ChessboardDistance](#), [CosineDistance](#), [EuclideanDistance](#), [ManhattanDistance](#), [SquaredEuclideanDistance](#)

Implementation status

- - full supported

Github

- [Implementation of CanberraDistance](#)
-

Cancel

```
Cancel(expr)
```

cancels out common factors in numerators and denominators.

Examples

```
>> Cancel(x / x ^ 2)
1/x
```

`cancel` threads over sums:

```
>> Cancel(x / x ^ 2 + y / y ^ 2)
1/x+1/y

>> Cancel(f(x) / x + x * f(x) / x ^ 2)
(2*f(x))/x
```

Implementation status

- - full supported

Github

- [Implementation of Cancel](#)
-

CarlsonRC

```
CarlsonRC(x, y)
```

returns the Carlson RC function..

See:

- [Wikipedia - Carlson symmetric form](#)

Implementation status

- - full supported

Github

- [Implementation of CarlsonRC](#)
-

CarlsonRD

```
CarlsonRD(x, y, z)
```

returns the Carlson RD function.

See:

- [Wikipedia - Carlson symmetric form](#)

Implementation status

- - full supported

Github

- [Implementation of CarlsonRD](#)
-

CarlsonRF

```
CarlsonRF(x, y, z)
```

returns the Carlson RF function.

See:

- [Wikipedia - Carlson symmetric form](#)

Implementation status

- - full supported

Github

- [Implementation of CarlsonRF](#)
-

CarlsonRG

```
CarlsonRG(x, y, z)
```

returns the Carlson RG function.

See:

- [Wikipedia - Carlson symmetric form](#)

Implementation status

- - full supported

Github

- [Implementation of CarlsonRG](#)
-

CarlsonRJ

```
CarlsonRJ(x, y, z, p)
```

returns the Carlson RJ function.

See:

- [Wikipedia - Carlson symmetric form](#)

Implementation status

- - full supported

Github

- [Implementation of CarlsonRJ](#)
-

CarmichaelLambda

```
CarmichaelLambda(n)
```

the Carmichael function of `n`

See:

- [Wikipedia - Carmichael function](#)

Examples

```
>> CarmichaelLambda(35)
12
```

Implementation status

- - partially implemented

Github

- [Implementation of CarmichaelLambda](#)
-

CartesianProduct

```
CartesianProduct(list1, list2)
```

returns the cartesian product for multiple lists.

See:

- [Wikipedia - Cartesian product](#)

Examples

```
>> CartesianProduct({1,2},{3,4})  
{{1,3},{1,4},{2,3},{2,4}}
```

Implementation status

- - full supported

Github

- [Implementation of CartesianProduct](#)
-

Cases

```
Cases(list, pattern)
```

returns the elements of `list` that match `pattern`.

```
Cases(list, pattern, ls)
```

returns the elements matching at levelspec `ls`.

```
Cases(list, pattern, Heads->True)
```

match including the head of the expression in the search.

Examples

```
>> Cases({a, 1, 2.5, "string"}, _Integer|_Real)
{1,2.5}

>> Cases(_Complex)[{1, 2I, 3, 4-I, 5}]
{I*2,4-I}

>> Cases(1, 2)
{}

>> Cases(f(1, 2), 2)
{2}

>> Cases(f(f(1, 2), f(2)), 2)
{}

>> Cases(f(f(1, 2), f(2)), 2, 2)
{2,2}

>> Cases(f(f(1, 2), f(2), 2), 2, Infinity)
{2,2,2}

>> Cases({1, f(2), f(3, 3, 3), 4, f(5, 5)}, f(x__) :> Plus(x))
{2,9,10}

>> Cases({1, f(2), f(3, 3, 3), 4, f(5, 5)}, f(x__) -> Plus(x))
{2, 3, 3, 3, 5, 5}
```

Related terms

[Pick](#), [Select](#)

Implementation status

- - full supported

Github

- [Implementation of Cases](#)
-

Catalan

Catalan

Catalan's constant

See:

- [Wikipedia - Catalan's constant](#)
- [Fungrim - Catalan's constant](#)

Examples

```
>> N(Catalan)
0.915965594177219

>> PolyGamma(1, 1/4)
8*Catalan+Pi^2
```

Implementation status

- - full supported

Github

- [Implementation of Catalan](#)
-

CatalanNumber

```
CatalanNumber(n)
```

returns the catalan number for the argument `n`.

See:

- [Wikipedia - Catalan number](#)

Examples

```
>> CatalanNumber(4)
14
```

Implementation status

- - full supported

Github

- [Implementation of CatalanNumber](#)
-

Catch

```
Catch(expr)
```

returns the value argument of the first `Throw(value)` generated in the evaluation of `expr`.

```
Catch(expr, form)
```

returns value from the first `Throw(value, tag)` for which `form` matches `tag`.

```
Catch(expr, form, f)
```

returns `f(value, tag)`.

Examples

Exit to the enclosing `catch` as soon as `Throw` is evaluated:

```
>> Catch(r; s; Throw(t); u; v)
t
```

Define a function that can "throw an exception":

```
>> f(x_) := If(x > 12, Throw(overflow), x!)
```

The result of `catch` is just what is thrown by `Throw`:

```
>> Catch(f(1) + f(15))
overflow

>> Catch(f(1) + f(4))
25
```

Implementation status

- - full supported

Github

- [Implementation of Catch](#)
-

Catenate

```
Catenate({l1, l2, ...})
```

concatenates the lists `l1, l2, ...`

Examples

```
>> Catenate({{1, 2, 3}, {4, 5}})
{1, 2, 3, 4, 5}
```

Implementation status

- - full supported

Github

- [Implementation of Catenate](#)
-

CauchyDistribution

```
CauchyDistribution(a,b)
```

returns the Cauchy distribution.

See:

- [Wikipedia - Cauchy distribution](#)

Examples

The probability density function of the Cauchy distribution is

```
>> PDF(CauchyDistribution(a, b), x)
1/(b*Pi*(1+(-a+x)^2/b^2))
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of CauchyDistribution](#)
-

CDF

```
CDF(distribution, value)
```

returns the cumulative distribution function of `value`.

```
PDF(distribution, {list} )
```

returns the cumulative distribution function of the values of `list`.

See:

- [Wikipedia - cumulative distribution function](#)

`CDF` can be applied to the following distributions:

[BernoulliDistribution](#), [BinomialDistribution](#), [DiscreteUniformDistribution](#),
[ErlangDistribution](#), [ExponentialDistribution](#), [FrechetDistribution](#), [GammaDistribution](#),
[GeometricDistribution](#), [GumbelDistribution](#), [HypergeometricDistribution](#),
[LogNormalDistribution](#), [NakagamiDistribution](#), [NormalDistribution](#),
[PoissonDistribution](#), [StudentTDistribution](#), [WeibullDistribution](#)

Examples

```
>> CDF(NormalDistribution(), -0.41)
0.3409

>> Table(CDF(NormalDistribution(0, s), x), {s, {.75, 1, 2}}, {x, -6, 6}) // N
{{0.0, 0.0, 0.0, 0.00003, 0.00383, 0.09121, 0.5, 0.90879, 0.99617, 0.99997, 1.0, 1.0, 1.0}, {0.0, 0.0
```

Related terms

[InverseCDF](#), [PDF](#)

Implementation status

- - full supported

Github

- [Implementation of CDF](#)
-

Ceiling

```
Ceiling(expr)
```

gives the first integer greater than or equal `expr`.

```
Ceiling(expr, a)
```

gives the first multiple of `a` greater than or equal to `expr`.

See:

- [Wikipedia - Floor and ceiling functions](#)

Examples

```
>> Ceiling(1/3)
1

>> Ceiling(-1/3)
0

>> Ceiling(1.2)
2

>> Ceiling(3/2)
2
```

For complex `expr`, take the ceiling of real and imaginary parts.

```
>> Ceiling(1.3 + 0.7*I)
2+I

>> Ceiling(10.4, -1)
10

>> Ceiling(-10.4, -1)
-11
```

Related terms

[IntegerPart](#), [Floor](#), [FractionalPart](#), [Round](#)

Implementation status

- - full supported

Github

- [Implementation of Ceiling](#)
-

CellularAutomaton

```
CellularAutomaton(rule-or-pure-function, initial-conndition, steps)
```

create a list of the evolution `steps` of the cellular automaton from the `rule-or-pure-function` specification for the `initial-condition`.

See:

- [Wikipedia - Cellular automaton](#)

Examples

```
>> CellularAutomaton(Xor(#, #5) &, {{1}, 0}, 3)
{{0,0,0,0,0,0,1,0,0,0,0,0,0}, {0,0,0,0,1,0,0,0,1,0,0,0,0}, {0,0,1,0,0,0,0,0,0,0,1,0,0}, {1,0,0,0,0,0,0,0,0,0,0,0,0}}

>> CellularAutomaton({1008, 2, 1, 2}, {{{1}, {1}}, 0}, 5)
{{0,0,0,1}, {0,0,1,0}, {0,0,0,0}, {0,1,0,0}, {0,0,1,0}, {1,0,0,1}}
```

CentralFeature

```
CentralFeature(list)
```

```
CentralFeature(list-of-rules)
```

returns the central feature of a `list` or a `list-of-rules`.

Option `DistanceFunction` can be used to define a distance function.

- `EuclideanDistance` is the `Automatic` default value
- `EditDistance` is the default value for list of strings
- `JaccardDissimilarity` is the default value for list of boolean values

See:

- [Wikipedia - Metric_space](#)

Examples

```
>> CentralFeature({{0,0}, {1,1}, {10,10}})
{1,1}
```

CentralMoment

```
CentralMoment(list, r)
```

gives the the `r`-th central moment (i.e. the `r`th moment about the mean) of `list`.

```
CentralMoment(dist, r)
```

gives the the `r`-th central moment of the distribution `dist`.

See:

- [Wikipedia - Central moment](#)

Examples

```
>> CentralMoment({1.1, 1.2, 1.4, 2.1, 2.4}, 4)
0.10085
```

Implementation status

- - experimental

Github

- [Implementation of CentralMoment](#)
-

CharacteristicPolynomial

```
CharacteristicPolynomial(matrix, var)
```

computes the characteristic polynomial of a `matrix` for the variable `var`.

See:

- [Wikipedia - Characteristic polynomial](#)
- [Wikipedia - Eigenvalues and Eigenvectors](#)
- [Wikipedia - Faddeev-LeVerrier algorithm](#)

Examples

```
>> CharacteristicPolynomial({{1, 2}, {42, 43}}, x)
-41-44*x+x^2
```

Related terms

[Eigensystem](#), [Eigenvalues](#), [Eigenvectors](#)

Implementation status

- - full supported

Github

- [Implementation of CharacteristicPolynomial](#)
-

CharacterRange

```
CharacterRange(min-character, max-character)
```

computes a list of character strings from `min-character` to `max-character`

Examples

The printable ASCII characters:

```
>> CharacterRange(" ", "~")  
{ , ! , " , # , $ , % , & , ' , ( , ) , * , + , , , - , . , / , 0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9 , : , ; , < , = , > , ? , @ , A , B , C , D , E , F , G , H , I , J ,
```

Implementation status

- - full supported

Github

- [Implementation of CharacterRange](#)
-

ChebyshevT

```
ChebyshevT(n, x)
```

returns the Chebyshev polynomial of the first kind $T_n(x)$.

See:

- [Wikipedia - Chebyshev polynomials](#)
- [Fungrim - Chebyshev polynomials](#)

Examples

```
>> ChebyshevT(8, x)
1-32*x^2+160*x^4-256*x^6+128*x^8
```

Implementation status

- - partially implemented

Github

- [Implementation of ChebyshevT](#)
-

ChebyshevU

```
ChebyshevU(n, x)
```

returns the Chebyshev polynomial of the second kind $u_n(x)$.

See:

- [Wikipedia - Chebyshev polynomials](#)
- [Fungrim - Chebyshev polynomials](#)

Examples

```
>> ChebyshevU(8, x)
1-40*x^2+240*x^4-448*x^6+256*x^8
```

Implementation status

- - partially implemented

Github

- [Implementation of ChebyshevU](#)
-

Check

```
Check(expr, failure)
```

evaluates `expr`, and returns the result, unless messages were generated, in which case `failure` will be returned.

Examples

```
>> Check(2^(3), err)
8
```

`0^(-42)` prints message: "Power: Infinite expression 1/0^42 encountered."

```
>> Check(0^(-42), failure)
failure
```

Implementation status

- - partially implemented

Github

- [Implementation of Check](#)
-

CheckAbort

```
CheckAbort(expr, failure-expr)
```

evaluates `expr`, and returns the result, unless `Abort` was called during the evaluation, in which case `failure-expr` will be returned.

Examples

```
>> CheckAbort(Abort(); -1, 41) + 1
42

>> CheckAbort(abc; -1, 41) + 1
0
```

Implementation status

- - partially implemented

Github

- [Implementation of CheckAbort](#)
-

ChessboardDistance

```
ChessboardDistance(u, v)
```

returns the chessboard distance (also known as Chebyshev distance) between `u` and `v`, which is the number of moves a king on a chessboard needs to get from square `u` to square `v`.

See:

- [Wikipedia - Chebyshev distance](#)
- [Wikipedia - Metric_space](#)

Examples

```
>> ChessboardDistance({-1, -1}, {1, 1})  
2
```

Related terms

[FindClusters](#), [BinaryDistance](#), [BrayCurtisDistance](#), [CanberraDistance](#), [CosineDistance](#), [EuclideanDistance](#), [ManhattanDistance](#), [SquaredEuclideanDistance](#)

Implementation status

- - full supported

Github

- [Implementation of ChessboardDistance](#)
-

ChineseRemainder

```
ChineseRemainder({a1, a2, a3,...}, {n1, n2, n3,...})
```

the chinese remainder function.

See:

- [Wikipedia - Chinese_remainder_theorem](#)
- [Rosetta Code - Chinese remainder theorem](#)

Examples

```
>> ChineseRemainder({0,3,4},{3,4,5})  
39  
  
>> ChineseRemainder({1,-15},{284407855036305,47})  
8532235651089151  
  
>> ChineseRemainder({-2,-17},{284407855036305,47})  
9669867071234368
```

Implementation status

- - partially implemented

Github

- [Implementation of ChineseRemainder](#)
-

CholeskyDecomposition

```
CholeskyDecomposition(matrix)
```

calculate the Cholesky decomposition of a hermitian, positive definite square `matrix`.

See:

- [Wikipedia - Cholesky decomposition](#)

Examples

```
>> CholeskyDecomposition({{11.0, 3.0}, {3.0, 5.0}})
{{3.3166247903554, 0.9045340337332909},
 {0.0, 2.04494943258218}}
```

Implementation status

- - full supported

Github

- [Implementation of CholeskyDecomposition](#)
-

Chop

```
Chop(numerical-expr)
```

replaces numerical values in the `numerical-expr` which are close to zero with symbolic value `0`.

```
Chop(numerical-expr, delta)
```

uses a tolerance of `delta`. The default tolerance is `10-10`.

Examples

```
>> Chop(0.000000000001)
0
```

Implementation status

- - full supported

Github

- [Implementation of Chop](#)
-

CirclePoints

```
CirclePoints(i)
```

gives the i points on the unit circle for a positive integer i .

Examples

```
>> CirclePoints(4)
{{1/Sqrt(2), -1/Sqrt(2)}, {1/Sqrt(2), 1/Sqrt(2)}, {-1/Sqrt(2), 1/Sqrt(2)}, {-1/Sqrt(2), -1/Sqr
```

Implementation status

- - full supported

Github

- [Implementation of CirclePoints](#)
-

Clear

```
Clear(symbol1, symbol2,...)
```

clears all values of the given symbols.

`clear` does not remove attributes, options, and default values associated with the symbols. Use `clearAll` to do so.

Examples

```
>> a=2
2

>> Definition(a)
{a=2}

>> Clear(a)
>> a
a
```

Implementation status

- - full supported

Github

- [Implementation of Clear](#)
-

ClearAll

```
ClearAll(symbol1, symbol2, ...)
```

clears all values and attributes associated with the given symbols.

Examples

```
>> ClearAll(x)
```

Implementation status

- - full supported

Github

- [Implementation of ClearAll](#)
-

ClearAttributes

```
ClearAttributes(symbol, attrib)
```

removes `attrib` from `symbol`'s attributes.

Examples

```
>> SetAttributes(f, Flat)

>> Attributes(f)
{Flat}

>> ClearAttributes(f, Flat)

>> Attributes(f)
{}
```

Attributes that are not even set are simply ignored:

```
>> ClearAttributes({f}, {Flat})
>> Attributes(f)
{}
```

Related terms

[Attributes](#), [Constant](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#), [SetAttributes](#)

Implementation status

- - full supported

Github

- [Implementation of ClearAttributes](#)
-

ClebschGordan

```
ClebschGordan({j1,m1},{j2,m2},{j3,m3})
```

get the Clebsch–Gordan coefficients. Clebsch–Gordan coefficients are numbers that arise in angular momentum coupling in quantum mechanic.

See:

- [Wikipedia - Clebsch-Gordan coefficients](#)
- [archive.org - Angular momentum in quantum mechanics](#)

Examples

```
>> ClebschGordan({3/2, -3/2}, {3/2, 3/2}, {1, 0})  
3/2*1/Sqrt(5)
```

Implementation status

- - partially implemented

Github

- [Implementation of ClebschGordan](#)
-

Clip

```
Clip(expr)
```

returns `expr` in the range `-1` to `1`. Returns `-1` if `expr` is less than `-1`. Returns `1` if `expr` is greater than `1`.

```
Clip(expr, {min, max})
```

returns `expr` in the range `min` to `max`. Returns `min` if `expr` is less than `min`. Returns `max` if `expr` is greater than `max`.

```
Clip(expr, {min, max}, {vMin, vMax})
```

returns `expr` in the range `min` to `max`. Returns `vMin` if `expr` is less than `min`. Returns `vMax` if `expr` is greater than `max`.

See

- [Wikipedia - Clipping \(signal processing\)](#)

Examples

```
>> Clip(Sin(Pi/7))
Sin(Pi/7)

>> Clip(Tan(E))
Tan(E)

>> Clip(Tan(2*E))
-1

>> Clip(Tan(-2*E))
1

>> Clip(x)
Clip(x)

>> Clip(Tan(2*E), {-1/2,1/2})
-1/2

>> Clip(Tan(-2*E), {-1/2,1/2})
1/2

>> Clip(Tan(E), {-1/2,1/2}, {a,b})
```


Tan(E)

```
>> Clip(Tan(2*E), {-1/2,1/2}, {a,b})
```

a

```
>> Clip(Tan(-2*E), {-1/2,1/2}, {a,b})
```

b

```
>> PiecewiseExpand(Clip(x))
```

```
Piecewise({{-1, x<-1}, {1, x>1}}, x)
```

Related terms

[Piecewise](#), [PiecewiseExpand](#)

Implementation status

- - full supported

Github

- [Implementation of Clip](#)
-

Close

```
Close(stream)
```

closes an input or output `stream`.

Examples

```
>> Close(StringToStream("123abc"))  
String
```

Implementation status

- - partially implemented

Github

- [Implementation of Close](#)
-

ClosenessCentrality

```
ClosenessCentrality(graph)
```

Computes the closeness centrality of each vertex of a `graph`.

See

- [Wikipedia - Closeness centrality](#)
- [Youtube - Betweenness centrality](#)

Examples

```
>> ClosenessCentrality(Graph({1, 2, 3, 4, 5}, {1<->2, 1<->3, 2<->3, 3<->4, 3<->5}))  
{0.666667, 0.666667, 1.0, 0.571429, 0.571429}
```

Implementation status

- - partially implemented

Github

- [Implementation of ClosenessCentrality](#)
-

Coefficient

```
Coefficient(polynomial, variable, exponent)
```

get the coefficient of $\text{variable}^{\text{exponent}}$ in polynomial .

See:

- [Wikipedia - Coefficient](#)

Examples

```
>> Coefficient(10(x^2)+2(y^2)+2*x, x, 2)
10

>> Coefficient(7*y^(3*w), y, 3*w)
7
```

In the next line `Coefficient` returns the coefficient of a particular term of a polynomial. In this case $(-210*c^2 * x^2*y*z^2)$ is a term of $(c*x-2*y+z)^7$ after it's expanded.

```
>> poly=(c*x-2*y+z)^7
(c*x-2*y+z)^7

>> Coefficient(poly, x^2*y*z^4)
-210*c^2
```

`CoefficientList` gets the same information as a list of coefficients. In the line below `Part` (`coeff, 3,2,5`) returns the coefficient of $x^{(3-1)}y^{(2-1)}z^{(5-1)}$. In general if we say `lst=CoefficientList(poly,{x1,x2,x3,...})` then `Part(lst, n1, n2, ,n3, ...)` will be the coefficient of $x_1^{(n1-1)}x_2^{(n2-1)}x_3^{(n3-1)}\dots$

```
>> coeff=CoefficientList(poly,{x,y,z}); Part(coeff, 3,2,5)
-210*c^2
```

The next line gives the coefficient of x^5 . As expected there is more than one term of poly with x^5 as a factor.

```
>> Coefficient(poly, x^5)
84*c^5*y^2-84*c^5*y*z+21*c^5*z^2
```

We can get the same result as the previous example if we use the next line.

```
>> Coefficient(poly, x, 5)
84*c^5*y^2-84*c^5*y*z+21*c^5*z^2
```

One can't get the result above directly from `CoefficientList`. Instead pieces of the above result are included in the result of `CoefficientList(poly, {x, y, z})`.

The line below can be used to get pieces of the result above. Specifically `coeff[[6]]` contains all coefficients of x^5 (including those that are zero).

```
>> coeff[[6]]
{{0, 0, 21*c^5, 0, 0, 0, 0, 0}, {0, -84*c^5, 0, 0, 0, 0, 0, 0}, {84*c^5, 0, 0, 0, 0, 0, 0, 0}, {0, 0, 0, 0, 0, 0, 0, 0}}
```

- `Part(coeff, 6, 1, 3)` is $21*c^5$ the coefficient of $(x^{(6 - 1)} * y^{(1 - 1)} * z^{(3 - 1)}) = (x^5 * z^2)$ in poly.
- `Part(coeff, 6, 2, 2)` is $(-84*c^5)$ the coefficient of $(x^{(6 - 1)} * y^{(2 - 1)} * z^{(2 - 1)}) = (x^5 * y * z)$ in poly.
- `Part(coeff, 6, 3, 1)` is $(84*c^5)$ the coefficient of $(x^{(6 - 1)} * y^{(3 - 1)} * z^{(1 - 1)}) = (x^5 * y^2)$ in poly.

All other coefficients under `coeff[[6]]` are zero which agrees with the result of `Coefficient(poly, x^5)`.

Related terms

[CoefficientList](#), [Exponent](#)

Implementation status

- - full supported

Github

- [Implementation of Coefficient](#)
-

CoefficientArrays

```
CoefficientArrays(list-of-polynomials, list-of-variables)
```

returns the sparse arrays of coefficients of the `list-of-variables` for the `list-of-polynomials`.

See:

- [Wikipedia - Coefficient](#)
- [Wikipedia - Sparse_matrix](#)

Examples

```
>> CoefficientArrays(2*x + 3*y^2 + 4*z + 5, {x, y, z}) // Normal
{5, {2, 0, 4}, {{0, 0, 0}, {0, 3, 0}, {0, 0, 0}}}
```

CoefficientList

```
CoefficientList(polynomial, variable)
```

get the coefficient list of a `polynomial`.

See:

- [Wikipedia - Coefficient](#)

Examples

```
>> CoefficientList(a+b*x, x)
{a,b}

>> CoefficientList(a+b*x+c*x^2, x)
{a,b,c}

>> CoefficientList(a+c*x^2, x)
{a,0,c}

>> CoefficientList((x + y)^3, z)
{(x+y)^3}

>> CoefficientList(Series(2*x, {x, 0, 9}), x)
{0,2}
```

In the next line `Coefficient` returns the coefficient of a particular term of a polynomial. In this case `(-210*c^2 * x^2*y*z^2)` is a term of `(c*x-2*y+z)^7` after it's expanded.

```
>> poly=(c*x-2*y+z)^7
(c*x-2*y+z)^7

>> Coefficient(poly, x^2*y*z^4)
-210*c^2
```

`CoefficientList` gets the same information as a list of coefficients. In the line below `Part` (`coeff, 3,2,5`) returns the coefficient of `x^(3-1)*y^(2-1)*z^(5-1)`. In general if we say `lst=CoefficientList(poly,{x1,x2,x3,...})` then `Part(lst, n1, n2, ,n3, ...)` will be the coefficient of `x1^(n1-1)*x2^(n2-1)*x3^(n3-1)...`

```
>> coeff=CoefficientList(poly,{x,y,z}); Part(coeff, 3,2,5)
-210*c^2
```

The next line gives the coefficient of x^5 . As expected there is more than one term of poly with x^5 as a factor.

```
>> Coefficient(poly, x^5)
84*c^5*y^2-84*c^5*y*z+21*c^5*z^2
```

We can get the same result as the previous example if we use the next line.

```
>> Coefficient(poly, x, 5)
84*c^5*y^2-84*c^5*y*z+21*c^5*z^2
```

One can't get the result above directly from `CoefficientList`. Instead pieces of the above result are included in the result of `CoefficientList(poly, {x, y, z})`. The line below can be used to get pieces of the result above. Specifically `coeff[[6]]` contains all coefficients of x^5 (including those that are zero).

```
>> coeff[[6]]
{{0,0,21*c^5,0,0,0,0,0},{0,-84*c^5,0,0,0,0,0,0},{84*c^5,0,0,0,0,0,0,0},{0,0,0,0,0,0,0,0}}
```

- `Part(coeff,6,1,3)` is $21*c^5$ the coefficient of $(x^{(6-1)} * y^{(1-1)} * z^{(3-1)}) = (x^5 * z^2)$ in poly.
- `Part(coeff,6,2,2)` is $(-84*c^5)$ the coefficient of $(x^{(6-1)} * y^{(2-1)} * z^{(2-1)}) = (x^5 * y * z)$ in poly.
- `Part(coeff,6,3,1)` is $(84*c^5)$ the coefficient of $(x^{(6-1)} * y^{(3-1)} * z^{(1-1)}) = (x^5 * y^2)$ in poly.

All other coefficients under `coeff[[6]]` are zero which agrees with the result of `Coefficient(poly, x^5)`.

Related terms

[Coefficient](#), [CoefficientRules](#), [Exponent](#), [MonomialList](#)

Implementation status

- - full supported

Github

- [Implementation of CoefficientList](#)
-

CoefficientRules

```
CoefficientRules(polynomial, list-of-variables)
```

get the list of coefficient rules of a `polynomial`.

See:

- [Wikipedia - Coefficient](#)

Examples

```
>> CoefficientRules(x^3+3*x^2*y+3*x*y^2+y^3, {x,y})  
{{3,0}->1,{2,1}->3,{1,2}->3,{0,3}->1}
```

Non integer or negative exponents returns only one rule:

```
>> CoefficientRules(7*y^w, {y,z})  
{{0,0}->7*y^w}  
  
>> CoefficientRules(c*x^(-2)+a+b*x,x)  
{{0}->a+c/x^2+b*x}
```

Related terms

[Coefficient](#), [CoefficientList](#), [Exponent](#), [MonomialList](#)

Implementation status

- - full supported

Github

- [Implementation of CoefficientRules](#)
-

Cofactor

```
Cofactor(matrix, {i,j})
```

calculate the cofactor of the matrix

Examples

```
>> Cofactor({{6, 0, 4, 9, 5}, {1, 9, 3, 1, 2}, {5, 4, 5, 3, 8}, {3, 9, 8, 2, 5}, {4, 1,  
-30
```

Implementation status

- - partially implemented

Github

- [Implementation of Cofactor](#)
-

Collect

```
Collect(expr, variable)
```

collect subexpressions in `expr` which belong to the same `variable`.

```
Collect(expr, variable, head)
```

collect subexpressions in `expr` which belong to the same `variable` and apply `head` on these subexpressions.

Collect additive terms of an expression.

This function collects additive terms of an expression with respect to a list of expression up to powers with rational exponents. By the term symbol here are meant arbitrary expressions, which can contain powers, products, sums etc. In other words symbol is a pattern which will be searched for in the expression's terms.

Examples

```
>> Collect(a*x^2 + b*x^2 + a*x - b*x + c, x)
c+(a-b)*x+(a+b)*x^2

>> Collect(a*Exp(2*x) + b*Exp(2*x), Exp(2*x))
(a+b)*E^(2*x)

>> Collect(x^2 + y*x^2 + x*y + y + a*y, {x, y})
(1+a)*y+x*y+x^2*(1+y)
```

Implementation status

- - full supported

Github

- [Implementation of Collect](#)
-

CollinearPoints

```
CollinearPoints({{x1,y1},{x2,y2},{a,b},...})
```

returns true if the point $\{a,b\}$ is on the line defined by the first two points $\{x1,y1\}$, $\{x2,y2\}$.

```
CollinearPoints({{x1,y1,z1},{x2,y2,z2},{a,b,c},...})
```

returns true if the point $\{a,b,c\}$ is on the line defined by the first two points $\{x1,y1,z1\}$, $\{x2,y2,z2\}$.

See:

- [Wikipedia - Collinearity](#)
- [Youtube - Collinear Points in 3D \(Ch1 Pr18\)](#)

Examples

```
>> CollinearPoints({{1,2,3}, {3,8,1}, {7,20,-3}})
True
```

Implementation status

- - partially implemented

Github

- [Implementation of CollinearPoints](#)
-

Commonest

```
Commonest(dataValueList)
```

the mode of a list of data values is the value that appears most often.

```
Commonest(dataValueList, n)
```

return the `n` values that appears most often.

See

- [Wikipedia - Mode \(statistics\)](#)

Examples

```
>> Commonest({1, 3, 6, 6, 6, 6, 7, 7, 12, 12, 17})  
{6}
```

Given the list of data {1, 1, 2, 4, 4} the mode is not unique – the dataset may be said to be **bimodal**, while a set with more than two modes may be described as **multimodal**.

```
>> Commonest({1, 1, 2, 4, 4})  
{1, 4}
```

Related terms

[Counts](#), [Tally](#)

Implementation status

- - full supported

Github

- [Implementation of Commonest](#)
-

Compile

```
Compile(list-of-arguments, expression)
```

compile the `expression` into a Java function, which has the arguments defined in `list-of-arguments` and return the compiled result in an `CompiledFunction` expression.

Examples

Compile the expression into a `CompiledFunction` and assign it to `f`:

```
>> f=Compile({{x, _Real}}, E^3-Cos(Pi^2/x));  
  
>> f(1.4567)  
19.20421  
  
>> f = Compile({{n, _Integer}}, Module({p = Range(n), i, x, t}, Do(x = RandomInteger({1, i},  
  
>> f(4)
```

Related terms

[CompiledFunction](#), [CompilePrint](#), [OptimizeExpression](#)

Implementation status

- - supported on Java virtual machine

Github

- [Implementation of Compile](#)
-

CompiledFunction

```
CompiledFunction(...)
```

represents a binary Java coded function.

Related terms

[Compile](#), [CompilePrint](#)

Implementation status

- - supported on Java virtual machine

Github

- [Implementation of CompiledFunction](#)
-

CompilePrint

```
CompilePrint(list-of-arguments}, expression)
```

compile the `expression` into a Java function and return the corresponding Java source code function, which has the arguments defined in `list-of-arguments`. You have to run Symja from a Java Development Kit (JDK) to compile to Java binary code.

Examples

Compile the expression into a `CompiledFunction` and assign it to `f`:

```
>> f=CompilePrint({x, _Real}, E^3-Cos(Pi^2/x))
```

```
>> f = CompilePrint({{n, _Integer}}, Module({p = Range(n), i, x, t}, Do(x = RandomInteger(
```

Related terms

[Compile](#), [CompiledFunction](#), [OptimizeExpression](#)

Implementation status

- - full supported

Github

- [Implementation of CompilePrint](#)
-

Complement

```
Complement(set1, set2)
```

get the complement set from `set1` and `set2`.

See

- [Wikipedia - Complement \(set theory\)](#)

Examples

```
>> Complement({1,2,3},{2,3,4})
{1}

>> Complement({2,3,4},{1,2,3})
{4}
```

In the line below we get all elements that are in `{3, 2, 7, 5, 2, 2, 3, 4, 5, 6, 1}` but not in `{2, 3}` or `{4, 6, 27, 23}`. Of course this would work with lists of arbitrary expressions, not only numbers.

```
>> Complement({3, 2, 7, 5, 2, 2, 3, 4, 5, 6, 1}, {2, 3}, {4, 6, 27, 23})
{1,5,7}
```

Related terms

[Intersection](#), [Union](#)

Implementation status

- - full supported

Github

- [Implementation of Complement](#)
-

CompleteGraph

```
CompleteGraph(order)
```

returns the complete graph with `order` vertices.

See

- [Wikipedia - Complete graph](#)

Examples

```
>> CompleteGraph(4) // AdjacencyMatrix // Normal
{{0,1,1,1},
 {1,0,1,1},
 {1,1,0,1},
 {1,1,1,0}}
```

Implementation status

- - partially implemented

Github

- [Implementation of CompleteGraph](#)
-

CompleteKaryTree

```
CompleteKaryTree(level)
```

create a binary tree graph with `level` levels.

```
CompleteKaryTree(level, m)
```

create a `m`-ary tree graph with `level` levels. In graph theory, an `m`-ary tree (for nonnegative integers `m`) (also known as `n`-ary, `k`-ary, `k`-way or generic tree) is a graph in which each node has no more than `m` children. A binary tree is an important case where `m = 2`; similarly, a ternary tree is one where `m = 3`.

See:

- [Wikipedia - M-ary tree](#)

Examples

Implementation status

- - experimental

Github

- [Implementation of CompleteKaryTree](#)
-

Complex

Complex

is the head of complex numbers.

Complex(a, b)

constructs the complex number $a + i \cdot b$.

See

- [Wikipedia - Complex number](#)

Examples

```
>> Head(2 + 3*I)
Complex

>> Complex(1, 2/3)
1+I*2/3

>> Abs(Complex(3, 4))
5

>> -2 / 3 - I
-2/3-I

>> Complex(10, 0)
10

>> 0. + I
I*1.0

>> 1 + 0*I
1

>> Head(1 + 0*I)
Integer

>> Complex(0.0, 0.0)
0.0

>> 0.*I
0.0

>> 0. + 0.*I
```

```
0.0

>> 1. + 0.*I
1.0

>> 0. + 1.*I
I*1.0
```

Check nesting Complex

```
>> Complex(1, Complex(0, 1))
0

>> Complex(1, Complex(1, 0))
1+I

>> Complex(1, Complex(1, 1))
I
```

Related terms

[I](#), [Im](#), [Re](#)

Implementation status

- - full supported

Github

- [Implementation of Complex](#)
-

Complexes

Complexes

is the set of complex numbers.

See

- [Wikipedia - Complex number](#)
-

ComplexExpand

```
ComplexExpand(expr)
```

expands `expr`. All variable symbols in `expr` are assumed to be non complex numbers.

```
ComplexExpand(expr, {x1, x2, ...})
```

expands `expr` assuming that variables matching any of the `xi` are complex.

See

- [Wikipedia - List of trigonometric identities](#)
- [Wikipedia - Complex number](#)

Examples

Assume that both `x` and `y` are real:

```
>> ComplexExpand(Sin(x+I*y))  
Cosh(y)*Sin(x)+I*cos(x)*Sinh(y)
```

```
>> ComplexExpand(3^(I*x))  
Cos(1/2*x*Log(9))+I*SIN(1/2*x*Log(9))
```

Implementation status

- - partially implemented

Github

- [Implementation of ComplexExpand](#)
-

ComplexInfinity

ComplexInfinity

represents an infinite complex quantity of undetermined direction.

See

- [Wikipedia - Infinity](#)

Examples

```
>> 1 / ComplexInfinity
0

>> ComplexInfinity + ComplexInfinity
ComplexInfinity

>> ComplexInfinity * Infinity
ComplexInfinity

>> FullForm(ComplexInfinity)
DirectedInfinity()
```

Implementation status

- - full supported

Github

- [Implementation of ComplexInfinity](#)
-

ComplexPlot3D

```
ComplexPlot3D(expr, {z, min, max })
```

create a 3D plot of `expr` for the complex variable `z` in the range `{ Re(min), Re(max) }` to `{ Im(min), Im(max) }`

See

- [Wikipedia - Complex number](#)
- [Wikipedia - Complex plane](#)
- [Wikipedia - Domain coloring](#)

Examples

```
>> ComplexPlot3D(Gamma(z), {z, -4.9-4.9*I, 4.9+4.9*I}, PlotRange->{0, 8.0})
```

Implementation status

- - full supported

Github

- [Implementation of ComplexPlot3D](#)
-

ComposeList

```
ComposeList(list-of-symbols, variable)
```

creates a list of compositions of the symbols applied at the argument `x`.

Examples

```
>> ComposeList({f,g,h}, x)
{x, f(x), g(f(x)), h(g(f(x)))}
```

Implementation status

- - full supported

Github

- [Implementation of ComposeList](#)
-

ComposeSeries

```
ComposeSeries( series1, series2 )
```

substitute `series2` into `series1`

```
ComposeSeries( series1, series2, series3 )
```

return multiple series composed.

See:

- [Wikipedia - Taylor series](#)
- [Wikipedia - Big O notation](#)
- [Wikipedia - Formal power series](#)

Examples

```
>> ComposeSeries(SeriesData(x, 0, {1, 3}, 2, 4, 1), SeriesData(x, 0, {1, 1,0,0}, 0, 4,  
x^2+3*x^3+0(x)^4
```

Related terms

[Series](#), [InverseSeries](#), [SeriesCoefficient](#), [SeriesData](#)

Implementation status

- - partially implemented

Github

- [Implementation of ComposeSeries](#)
-

CompositeQ

CompositeQ(n)

returns `True` if `n` is a composite integer number.

For very large numbers, `CompositeQ` uses [probabilistic prime testing](#), so it might be wrong sometimes (i.e. a number might be non composite even though `CompositeQ` says it is).

See

- [Wikipedia - Prime number](#)

Examples

```
>> Select(Range(100), CompositeQ)
{4, 6, 8, 9, 10, 12, 14, 15, 16, 18, 20, 21, 22, 24, 25, 26, 27, 28, 30, 32, 33, 34, 35, 36, 38, 39, 40, 42,
44, 45, 46, 48, 49, 50, 51, 52, 54, 55, 56, 57, 58, 60, 62, 63, 64, 65, 66, 68, 69, 70, 72, 74, 75, 76, 77,
78, 80, 81, 82, 84, 85, 86, 87, 88, 90, 91, 92, 93, 94, 95, 96, 98, 99, 100}
```

`CompositeQ` has attribute `Listable`:

```
>> CompositeQ(Range(20))
{False, False, False, True, False, True, False, True, True, True, False, True, False, True, True, True}
```

Implementation status

- - full supported

Github

- [Implementation of CompositeQ](#)
-

Composition

```
Composition(sym1, sym2,...)[arg1, arg2,...]
```

creates a composition of the symbols applied at the arguments.

Examples

```
>> Composition(u, v, w)[x, y]  
u(v(w(x,y)))
```

Implementation status

- - full supported

Github

- [Implementation of Composition](#)
-

CompoundExpression

```
CompoundExpression(expr1, expr2, ...)
```

or

```
expr1; expr2; ...
```

evaluates its arguments in turn, returning the last result.

Implementation status

- - full supported

Github

- [Implementation of CompoundExpression](#)
-

Compress

```
Compress(expression)
```

the `Compress` function creates a compressed, string-based representation of any expression. The output string contains the compressed data of the serialized expression. This string can be stored or transmitted, and the original expression can be fully reconstructed using the `Uncompress` function.

Examples

```
>> str = Compress(Expand((a+b)^3))
H4sIAAAAAAAAAA/0uMM1bQVjDWSowz0kqCsLSS4oyArKQ4YwDZmlsNHQAAAA==

>> Uncompress(str)
a^3+3*a^2*b+3*a*b^2+b^3
```

Related terms

[Uncompress](#)

Implementation status

- - full supported

Github

- [Implementation of Compress](#)
-

Condition

```
Condition(pattern, expr)
```

or

```
pattern /; expr
```

places an additional constraint on `pattern` that only allows it to match if `expr` evaluates to `True`.

Examples

The controlling expression of a `Condition` can use variables from the pattern:

```
>> f(3) /. f(x_) /; x>0 -> t
t

>> f(-3) /. f(x_) /; x>0 -> t
f(-3)
```

`Condition` can be used in an assignment:

```
>> f(x_) := p(x) /; x>0
>> f(3)
p(3)

>> f(-3)
f(-3)
```

Implementation status

- - full supported

Github

- [Implementation of Condition](#)
-

ConditionalExpression

```
ConditionalExpression(expr, condition)
```

if `condition` evaluates to `True` return `expr`, if `condition` evaluates to `False` return `Undefined`. Otherwise return the `ConditionalExpression` unevaluated.

Examples

```
>> ConditionalExpression(a, True)
a

>> ConditionalExpression(a, False)
Undefined

>> Limit(x^(a/x), x->Infinity)
ConditionalExpression(1, Element(a, Reals))
```

Implementation status

- - full supported

Github

- [Implementation of ConditionalExpression](#)
-

Conjugate

Conjugate(z)

returns the complex conjugate of the complex number z .

See

- [Wikipedia - Complex number](#)
- [Wikipedia - Complex conjugation](#)

Examples

```
>> Conjugate(3 + 4*I)
3 - 4 I

>> Conjugate(3)
3

>> Conjugate(a + b * I)
-I*Conjugate(b)+Conjugate(a)

>> Conjugate({{1, 2 + I*4, a + I*b}, {I}})
{{1, 2-I*4, -I*Conjugate(b)+Conjugate(a)}, {-I}}

>> {Conjugate(Pi), Conjugate(E)}
{Pi, E}

>> Conjugate(1.5 + 2.5*I)
1.5+I*(-2.5)
```

Implementation status

- - full supported

Github

- [Implementation of Conjugate](#)
 - [Rule definitions of Conjugate](#)
-

ConjugateTranspose

```
ConjugateTranspose(matrix)
```

get the transposed `matrix` with conjugated matrix elements.

```
ConjugateTranspose(tensor, permutation-list)
```

transposes rows and columns with conjugated matrix elements in the `tensor` according to `permutation-list`. If `tensor` is a `SparseArray` the result will also be a `SparseArray`.

See:

- [Wikipedia - Transpose](#)
- [Wikipedia - Complex conjugation](#)

Implementation status

- - partially implemented

Github

- [Implementation of ConjugateTranspose](#)
-

ConnectedGraphQ

```
ConnectedGraphQ(graph)
```

returns `True` if the `graph` is strongly connected, which means that every vertex is reachable from every other vertex.

See

- [Wikipedia - Strongly connected component](#)

Examples

```
>> ConnectedGraphQ(Graph({1,2,3,4},{1<->2, 2<->3, 3<->4}))  
True
```

Implementation status

- - partially implemented

Github

- [Implementation of ConnectedGraphQ](#)
-

Constant

Constant

is an attribute that indicates that a symbol is a constant.

See:

- [Wikipedia - Constant \(mathematics\)](#)

Examples

Mathematical constants like `E` have attribute `Constant`:

```
>> Attributes(E)
{Constant}
```

Constant symbols cannot be used as variables in `solve` and related functions.

`E` is a constant symbol and not a valid variable.

```
>> Solve(x + E == 0, E)
Solve(E+x==0,E)
```

Related terms

[Attributes](#), [ClearAttributes](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#), [SetAttributes](#)

ConstantArray

```
ConstantArray(expr, n)
```

returns a list of `n` copies of `expr`.

Examples

```
>> ConstantArray(a, 3)
{a, a, a}

>> ConstantArray(a, {2, 3})
{{a, a, a}, {a, a, a}}
```

Implementation status

- - full supported

Github

- [Implementation of ConstantArray](#)
-

ContainsAll

```
ContainsAll(list1, list2)
```

returns `True` if `list1` contains all of the elements that appear in `list2`.

Examples

Implementation status

- - full supported

Github

- [Implementation of ContainsAll](#)
-

ContainsAny

```
ContainsAny(list1, list2)
```

returns `True` if `list1` contains one of the elements that appear in `list2`.

Examples

Implementation status

- - full supported

Github

- [Implementation of ContainsAny](#)
-

ContainsExactly

```
ContainsExactly(list1, list2)
```

returns `True` if `list1` contains exactly the elements that appear in `list2`.

Examples

Implementation status

- - full supported

Github

- [Implementation of ContainsExactly](#)
-

ContainsNone

```
ContainsNone(list1, list2)
```

returns `True` if `list1` contains no element that appear in `list2`.

Examples

Implementation status

- - full supported

Github

- [Implementation of ContainsNone](#)
-

ContainsOnly

```
ContainsOnly(list1, list2)
```

returns `True` if `list1` contains only elements that are elements in `list2`.

Examples

Implementation status

- - full supported

Github

- [Implementation of ContainsOnly](#)
-

Context

```
Context(symbol)
```

yields the name of the context where `symbol` is defined in.

```
Context()
```

return the current context.

Examples

```
>> $ContextPath  
{System`,Global`}  
  
>> Context(a)  
Global`  
  
>> Context(Sin)  
System`
```

Implementation status

- - full supported

Github

- [Implementation of Context](#)
-

Contexts

`Contexts()`

return a list of all contexts

`Context()`

return the current context.

Examples

```
>> Contexts[] // InputForm
{"Global`", "Rubi`", "System`"}
```

Implementation status

- - full supported

Github

- [Implementation of Contexts](#)
-

Continue

Continue()

continues with the next iteration in a `For`, `While`, or `Do` loop.

Examples

```
>> For(i=1, i<=8, i=i+1, If(Mod(i,2) == 0, Continue())); Print(i)
| 1
| 3
| 5
| 7
```

Related terms

[Break](#), [Do](#), [For](#), [While](#)

Implementation status

- - full supported

Github

- [Implementation of Continue](#)
-

ContinuedFraction

```
ContinuedFraction(number)
```

the complete continued fraction representation for a rational or quadratic irrational `number`.

```
ContinuedFraction(number, n)
```

generate the first `n` terms in the continued fraction representation of `number`.

See:

- [Wikipedia - Continued fraction](#)

Examples

```
>> FromContinuedFraction({2,3,4,5})  
157/68  
  
>> ContinuedFraction(157/68)  
{2,3,4,5}  
  
>> ContinuedFraction(45/16)  
{2,1,4,3}
```

For square roots of non-negative integer arguments `ContinuedFraction` determines the periodic part:

```
>> ContinuedFraction(Sqrt(13))  
{3, {1,1,1,1,6}}  
  
>> ContinuedFraction(Sqrt(919))  
{30, {3,5,1,2,1,2,1,1,1,2,3,1,19,2,3,1,1,4,9,1,7,1,3,6,2,11,1,1,1,29,1,1,1,11,2,6,3,1,7,
```

Related terms

[FromContinuedFraction](#), [QuadraticIrrationalQ](#)

Implementation status

- - full supported

Github

- [Implementation of ContinuedFraction](#)
-

Convergents

```
Convergents({n1, n2, ...})
```

return the list of convergents which represents the continued fraction list $\{n_1, n_2, \dots\}$.

See:

- [Wikipedia - Continued fraction](#)

Examples

```
>> Convergents({2,3,4,5})  
{2, 7/3, 30/13, 157/68}
```

Implementation status

- - full supported

Github

- [Implementation of Convergents](#)
-

CoordinateBoundingBox

```
CoordinateBoundingBox({{x1,y1,...},{x2,y2,...},{x3,y3,...},...})
```

calculate the bounding box of the points `{{x1,y1,...},{x2,y2,...},{x3,y3,...},...}`.

```
CoordinateBoundingBox({{x1,y1,...},{x2,y2,...},{x3,y3,...},...}, pad)
```

add a pad to the calculated bounding box of the points `{{x1,y1,...},{x2,y2,...},{x3,y3,...},...}`.

Examples

```
>> CoordinateBoundingBox({{0, 1}, {2, 3}, {3,4}, {2, 3}, {1,1}})
{{0,1},{3,4}}

>> CoordinateBoundingBox({{a,b,u}, {c,d,v}, {e,f,w}}, Scaled(1/4))
{{Min(a,c,e)+1/4*(-Max(a,c,e)+Min(a,c,e)),Min(b,d,f)+1/4*(-Max(b,d,f)+Min(b,d,f)),Min(u,1/4*(-Max(u,v,w)+Min(u,v,w)))}, {Max(a,c,e)+1/4*(Max(a,c,e)-Min(a,c,e)),Max(b,d,f)+1/4*(Max(b,d,f)-Min(b,d,f)),Max(u,v,w)+1/4*(Max(u,v,w)-Min(u,v,w))}}
```

Implementation status

- - partially implemented

Github

- [Implementation of CoordinateBoundingBox](#)
-

CoplanarPoints

```
CoplanarPoints({{x1,y1,z1},{x2,y2,z2},{x3,y3,z3},{a,b,c},...})
```

returns true if the point $\{a,b,c\}$ is on the plane defined by the first three points $\{x_1, y_1, z_1\}, \{x_2, y_2, z_2\}, \{x_3, y_3, z_3\}$.

See:

- [Wikipedia - Coplanarity](#)

Examples

```
>> CoplanarPoints( {{3,2,-5}, {-1,4,-3}, {-3,8,-5}, {-3,2,1}})
True

>> CoplanarPoints( {{0,-1,-1}, {4,5,1}, {3,9,4}, {-4,4,3}})
False
```

Implementation status

- - partially implemented

Github

- [Implementation of CoplanarPoints](#)
-

CoprimeQ

`CoprimeQ(x, y)`

tests whether `x` and `y` are coprime by computing their greatest common divisor.

See:

- [Wikipedia - Coprime](#)

Examples

```
>> CoprimeQ(7, 9)
True
>> CoprimeQ(-4, 9)
True
>> CoprimeQ(12, 15)
False
>> CoprimeQ(2, 3, 5)
True
>> CoprimeQ(2, 4, 5)
False
```

Implementation status

- - full supported

Github

- [Implementation of CoprimeQ](#)
-

Correlation

```
Correlation(a, b)
```

computes Pearson's correlation of two equal-sized vectors `a` and `b`.

See:

- [Wikipedia - Pearson correlation coefficient](#)

Examples

```
>> Correlation({a,b},{c,d})  
((a-b)*(Conjugate(c)-Conjugate(d)))/(Sqrt((a-b)*(Conjugate(a)-Conjugate(b)))*Sqrt((c-d)  
  
>> Correlation({10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5}, {8.04, 6.95, 7.58, 8.81, 8.33, 9  
0.81642
```

Implementation status

- - full supported

Github

- [Implementation of Correlation](#)
-

CorrelationDistance

```
CorrelationDistance(u, v)
```

returns the correlation distance between `u` and `v`.

Examples

```
>> CorrelationDistance({a, b}, {c, d})  
1+(-a*c+b*c+a*d-b*d)/(2*Sqrt(Abs(a+1/2*(-a-b))^2+Abs(1/2*(-a-b)+b)^2)*Sqrt(Abs(c+1/2*(-a-b))^2+Abs(1/2*(-c-b)+d)^2)
```

Related terms

[FindClusters](#), [BinaryDistance](#), [BrayCurtisDistance](#), [ChessboardDistance](#), [CanberraDistance](#), [CosineDistance](#), [EuclideanDistance](#), [ManhattanDistance](#), [SquaredEuclideanDistance](#)

Implementation status

- - full supported

Github

- [Implementation of CorrelationDistance](#)
-

Cos

`Cos(expr)`

returns the cosine of `expr` (measured in radians). `Cos(expr)` will evaluate automatically in the case `expr` is a multiple of `Pi`, `Pi/2`, `Pi/3`, `Pi/4` and `Pi/6`.

See:

- [Wikipedia - Trigonometric functions](#)
- [Wikipedia - Radian](#)
- [Wikipedia - Degree](#)

Examples

```
>> Cos(0)
1

>> Cos(3*Pi)
-1

>> Cos(1.5*Pi)
-1.8369701987210297E-16
```

Implementation status

- - full supported

Github

- [Implementation of Cos](#)
 - [Rule definitions of Cos](#)
-

Cosh

`Cosh(z)`

returns the hyperbolic cosine of `z`.

Examples

```
>> Cosh(0)
1
```

Implementation status

- - full supported

Github

- [Implementation of Cosh](#)
 - [Rule definitions of Cosh](#)
-

CoshIntegral

```
CoshIntegral(expr)
```

returns the hyperbolic cosine integral of `expr`.

See

- [Wikipedia - Trigonometric integral](#)

Examples

```
>> CoshIntegral(0)  
-Infinity
```

Implementation status

- - full supported

Github

- [Implementation of CoshIntegral](#)
-

CosineDistance

```
CosineDistance(u, v)
```

returns the cosine distance between `u` and `v`.

Examples

```
>> N(CosineDistance({7, 9}, {71, 89}))  
7.596457213221441E-5  
  
>> CosineDistance({a, b}, {c, d})  
1-(a*c+b*d)/(Sqrt(Abs(a)^2+Abs(b)^2)*Sqrt(Abs(c)^2+Abs(d)^2))
```

Related terms

[FindClusters](#), [BinaryDistance](#), [BrayCurtisDistance](#), [ChessboardDistance](#), [CanberraDistance](#), [EuclideanDistance](#), [ManhattanDistance](#), [SquaredEuclideanDistance](#)

Implementation status

- - full supported

Github

- [Implementation of CosineDistance](#)
-

CosIntegral

```
CosIntegral(expr)
```

returns the cosine integral of `expr`.

See

- [Wikipedia - Trigonometric integral](#)

Examples

```
>> CosIntegral(0)
-Infinity
```

Implementation status

- - full supported

Github

- [Implementation of CosIntegral](#)
-

Cot

`Cot(expr)`

the cotangent function.

See:

- [Wikipedia - Trigonometric functions](#)
- [Wikipedia - Radian](#)
- [Wikipedia - Degree](#)

Examples

```
>> Cot(Pi/4)
1

>> Cot(0)
ComplexInfinity

>> Cot(1.)
0.6420926159343308
```

Implementation status

- - full supported

Github

- [Implementation of Cot](#)
 - [Rule definitions of Cot](#)
-

Coth

```
Coth(z)
```

returns the hyperbolic cotangent of z .

See

- [Wikipedia - Hyperbolic function](#)

Examples

```
>> Coth(0)  
ComplexInfinity
```

Implementation status

- - full supported

Github

- [Implementation of Coth](#)
 - [Rule definitions of Coth](#)
-

Count

```
Count(list, pattern)
```

returns the number of times `pattern` appears in `list`.

```
Count(list, pattern, ls)
```

counts the elements matching at levelspec `ls`.

Examples

```
>> Count({3, 7, 10, 7, 5, 3, 7, 10}, 3)
2

>> Count({{a, a}, {a, a, a}, a}, a, {2})
5
```

Implementation status

- - full supported

Github

- [Implementation of Count](#)
-

CountDistinct

```
CountDistinct(list)
```

returns the number of distinct entries in `list`.

Examples

```
>> CountDistinct({3, 7, 10, 7, 5, 3, 7, 10})
4

>> CountDistinct({{a, a}, {a, a, a}, a, a})
3
```

Implementation status

- - full supported

Github

- [Implementation of CountDistinct](#)
-

Counts

```
Counts({elem1, elem2, elem3, ...})
```

count the number of each distinct element in the list {elem1, elem2, elem3, ...}
and return the result as an association <|elem1->counter1, ...|>.

Examples

```
>> Counts({1,2,3,4,5,6,7,8,9,7,5,4,5,6,7,3,2,1,3,4,5,2,2,2,3,3,3,3,3})  
<|1->2,2->5,3->8,4->3,5->4,6->2,7->3,8->1,9->1|>
```

Related terms

[Commonest](#), [Tally](#)

Implementation status

- - full supported

Github

- [Implementation of Counts](#)
-

Covariance

```
Covariance(a, b)
```

computes the covariance between the equal-sized vectors `a` and `b`.

See:

- [Wikipedia - Covariance](#)

Examples

```
>> Covariance({10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5}, {8.04, 6.95, 7.58, 8.81, 8.33, 9.5, 7.61, 9.06, 8.5, 8.1, 8.99})  
5.501  
  
>> Covariance({0.2, 0.3, 0.1}, {0.3, 0.3, -0.2})  
0.025
```

Implementation status

- - full supported

Github

- [Implementation of Covariance](#)
-

CreateUUID

```
CreateUUID( )
```

retrieve a type 4 (pseudo randomly generated) UUID. The UUID is generated using a cryptographically strong pseudo random number generator.

See:

- [Wikipedia: Universally unique identifier](#)

Examples

```
>> CreateUUID()  
a13f186a-e9a3-4636-9e97-d0b3cbe7ada3  
  
>> CreateUUID("test-")  
test-8029ae50-5db7-4bda-80e5-56eab144f5da
```

Implementation status

- - full supported

Github

- [Implementation of CreateUUID](#)
-

Cross

```
Cross(a, b)
```

computes the vector cross product of `a` and `b`.

See:

- [Wikipedia: Cross product](#)

Examples

```
>> Cross({x1, y1, z1}, {x2, y2, z2})  
{-y2*z1+y1*z2, x2*z1-x1*z2, -x2*y1+x1*y2}  
  
>> Cross({x, y})  
{-y, x}
```

The arguments are expected to be vectors of equal length, and the number of arguments are expected to be 1 less than their length.

```
>> Cross({1, 2}, {3, 4, 5})  
Cross({1, 2}, {3, 4, 5})
```

Implementation status

- - full supported

Github

- [Implementation of Cross](#)
-

Csc

`Csc(z)`

returns the cosecant of `z`.

See:

- [Wikipedia - Trigonometric functions](#)

Examples

```
>> Csc(0)
ComplexInfinity

>> Csc(1)
Csc(1)

>> Csc(1.)
1.1883951057781212
```

Implementation status

- - full supported

Github

- [Implementation of Csc](#)
 - [Rule definitions of Csc](#)
-

Csch

```
Csch(z)
```

returns the hyperbolic cosecant of z .

See

- [Wikipedia - Hyperbolic function](#)

Examples

```
>> Csch(0)  
ComplexInfinity
```

Implementation status

- - full supported

Github

- [Implementation of Csch](#)
 - [Rule definitions of Csch](#)
-

CubeRoot

`CubeRoot(n)`

finds the real-valued cube root of the given `n`.

Examples

```
>> CubeRoot(16)
2*2^(1/3)

>> CubeRoot(-5)
-5^(1/3)

>> CubeRoot(-510000)
-10*510^(1/3)

>> CubeRoot(-5.1)
-1.7213006207263157

>> CubeRoot(b)
b^(1/3)

>> CubeRoot(-0.5)
-0.793701
```

Related terms

[Surd](#)

Implementation status

- - full supported

Github

- [Implementation of CubeRoot](#)
-

Cuboid

```
Cuboid({xmin, ymin, zmin})
```

is a unit cube.

```
Cuboid({xmin, ymin, zmin}, {xmax, ymax, zmax})
```

represents a cuboid extending from `{xmin, ymin, zmin}` to `{xmax, ymax, zmax}`.

Examples

```
>> Graphics3D(Cuboid({0, 0, 1}))  
-Graphics3D-
```

Implementation status

- - full supported

Github

- [Implementation of Cuboid](#)
-

Curl

```
Curl({f1, f2}, {x1, x2})
```

returns the curl $D(f2, x1) - D(f1, x2)$

```
Curl({f1, f2, f3}, {x1, x2, x3})
```

returns the curl vector $\{D(f3, x2) - D(f2, x3), D(f1, x3) - D(f3, x1), D(f2, x1) - D(f1, x2)\}$

See:

- [Wikipedia - Curl \(mathematics\)](#)

Examples

```
>> Curl({y, -x}, {x, y})  
-2
```

```
>> Curl({f(u,v,w), f(v,w,u), f(w,u,v), f(x)}, {u,v,w})  
{-D(f(v,w,u),w)+D(f(w,u,v),v), -D(f(w,u,v),u)+D(f(u,v,w),w), -D(f(u,v,w),v)+D(f(v,w,u),u)}
```

Implementation status

- - partially implemented

Github

- [Implementation of Curl](#)
-

Cyan

Cyan

RGB color value for the color cyan

Examples

```
>> Cyan  
RGBColor(0.0,1.0,1.0)
```

CycleGraph

```
CycleGraph(order)
```

returns the cycle graph with `order` vertices.

See

- [Wikipedia - Cycle graph](#)

Examples

```
>> CycleGraph(4) // AdjacencyMatrix // Normal
{{0,1,0,1},
 {1,0,1,0},
 {0,1,0,1},
 {1,0,1,0}}
```

Implementation status

- - partially implemented

Github

- [Implementation of CycleGraph](#)
-

Cycles

```
Cycles(a, b)
```

expression for defining canonical cycles of a permutation.

See:

- [Wikipedia: Cyclic permutation](#)

Examples

The singletons {2} and {5} are deleted:

```
>> PermutationCycles({4,2,7,6,5,8,1,3})  
Cycles({{1,4,6,8,3,7}})
```

Related terms

[FindPermutation](#), [PermutationCycles](#), [PermutationCyclesQ](#), [PermutationList](#), [PermutationListQ](#), [PermutationReplace](#), [Permutations](#), [Permute](#)

Implementation status

- - full supported

Github

- [Implementation of Cycles](#)
-

Cyclotomic

```
Cyclotomic(n, x)
```

returns the Cyclotomic polynomial $c_n(x)$.

See:

- [Wikipedia - Cyclotomic polynomial](#)

Examples

```
>> Cyclotomic(25, x)
1+x^5+x^10+x^15+x^20
```

The case of the 105-th cyclotomic polynomial is interesting because 105 is the lowest integer that is the product of three distinct odd prime numbers and this polynomial is the first one that has a coefficient other than 1 , 0 or 1 :

```
>> Cyclotomic(105, x)
1+x+x^2-x^5-x^6-2*x^7-x^8-x^9+x^12+x^13+x^14+x^15+x^16+x^17-x^20-x^22-x^24-x^26-x^28+x^
```

Implementation status

- - full supported

Github

- [Implementation of Cyclotomic](#)
-

Cylinder

```
Cylinder({{x1, y1, z1}, {x2, y2, z2}})
```

represents a cylinder of radius 1.

```
Cylinder({{x1, y1, z1}, {x2, y2, z2}}, r)
```

is a cylinder of radius r starting at $\{x_1, y_1, z_1\}$ and ending at $\{x_2, y_2, z_2\}$.

```
Cylinder({{x1, y1, z1}, {x2, y2, z2}, ... }, r)
```

is a collection of cylinders of radius r

Examples

```
>> Graphics3D(Cylinder({{0, 0, 0}, {1, 1, 1}}, 1))  
-Graphics3D-  
  
>> Graphics3D({Yellow, Cylinder({{-1, 0, 0}, {1, 0, 0}, {0, 0, Sqrt(3)}}, {1, 1, Sqrt(3)}  
-Graphics3D-
```

Implementation status

- - full supported

Github

- [Implementation of Cylinder](#)
-

D

```
D(f, x)
```

gives the partial derivative of `f` with respect to `x`.

```
D(f, x, y, ...)
```

differentiates successively with respect to `x`, `y`, etc.

```
D(f, {x,n})
```

gives the multiple derivative of order `n`.

```
D(f, {{x1, x2, ...}})
```

gives the vector derivative of `f` with respect to `x1`, `x2`, etc.

Note: the upper case identifier `D` is different from the lower case identifier `d`.

See:

- [Wikipedia - Differentiation rules](#)
- [Wikipedia - Derivative](#)
- [Wikipedia - General Leibniz rule](#)

Examples

First-order derivative of a polynomial:

```
>> D(x^3 + x^2, x)
2*x+3*x^2
```

Second-order derivative:

```
>> D(x^3 + x^2, {x, 2})
2+6*x
```

Trigonometric derivatives:

```
>> D(Sin(Cos(x)), x)
-Cos(Cos(x))*Sin(x)

>> D(Sin(x), {x, 2})
-Sin(x)

>> D(Cos(t), {t, 2})
-Cos(t)
```

Unknown variables are treated as constant:

```
>> D(y, x)
0

>> D(x, x)
1

>> D(x + y, x)
1
```

Derivatives of unknown functions are represented using 'Derivative':

```
>> D(f(x), x)
f'(x)

>> D(f(x, x), x)
Derivative(0,1)[f][x,x]+Derivative(1,0)[f][x,x]
```

Chain rule:

```
>> D(f(2*x+1, 2*y, x+y), x)
2*Derivative(1,0,0)[f][1+2*x,2*y,x+y]+Derivative(0,0,1)[f][1+2*x,2*y,x+y]

>> D(f(x^2, x, 2*y), {x,2}, y) // Expand
2*Derivative(0,2,1)[f][x^2,x,2*y]+4*Derivative(1,0,1)[f][x^2,x,2*y]+8*x*Derivative(
1,1,1)[f][x^2,x,2*y]+8*x^2*Derivative(2,0,1)[f][x^2,x,2*y]
```

Compute the gradient vector of a function:

```
>> D(x ^ 3 * Cos(y), {{x, y}})
{3*x^2*Cos(y), -x^3*Sin(y)}
```

Hesse matrix:

```

>> D(Sin(x) * Cos(y), {{x,y}, 2})
{{-Cos(y)*Sin(x), -Cos(x)*Sin(y)}, {-Cos(x)*Sin(y), -Cos(y)*Sin(x)}}

>> D(2/3*Cos(x) - 1/3*x*Cos(x)*Sin(x) ^ 2,x)//Expand
1/3*x*Sin(x)^3-1/3*Sin(x)^2*Cos(x)-2/3*Sin(x)-2/3*x*Cos(x)^2*Sin(x)

>> D(f(#1), {#1,2})
f''(#1)

>> D((#1&)(t), {t,4})
0

>> Attributes(f) = {HoldAll}; Apart(f''(x + x))
f''(2*x)

>> Attributes(f) = {}; Apart(f''(x + x))
f''(2*x)

>> D({#^2}, #)
{2*#1}

```

Related terms

[Derivative](#), [Diff](#), [Div](#), [DSolve](#), [Grad](#), [Integrate](#), [Laplacian](#), [Limit](#), [ND](#), [NIntegrate](#)

Implementation status

- - partially implemented

Github

- [Implementation of D](#)
 - [Rule definitions of D](#)
-

Dataset

```
Dataset( association )
```

create a `Dataset` object from the `association`

Dataset uses:

- [Github - JTablesaw - Java dataframe and visualization library](#)

Examples

```
>> dset=Dataset@<|101 -> <|"t" -> 42, "r" -> 7.5|>, 102 -> <|"t" -> 42, "r" -> 7.5|>, 1
```

		t		r	
101		42		7.5	
102		42		7.5	
103		42		7.5	

Related terms

[SemanticImport](#), [SemanticImportString](#)

DateObject

```
DateObject()
```

return the current date

```
DateObject({YYYY, MM, DD})
```

return the date object

Examples

```
>> DateObject({2016, 8, 1})  
2016-08-01T00:00
```

Implementation status

- - experimental

Github

- [Implementation of DateObject](#)
-

DateString

```
DateString()
```

return the current date as string

```
DateString({YYYY, MM, DD})
```

return the date as string

Examples

```
>> DateString(3155673600)
Sat 01 Jan 2000 00:00:00
```

Implementation status

- - experimental

Github

- [Implementation of DateString](#)
-

DateValue

```
DateValue("date-time-string")
```

return the current date in the specified date-time form

```
DateValue(date-object, {"date-time-str1", "date-time-str2", ...})
```

return the date object as a list in the specified date-time forms

Examples

```
>> DateValue(DateObject({2016, 8, 1}), {"Year", "Month", "Day"})  
{2016, 8, 1}
```

Implementation status

- - experimental

Github

- [Implementation of DateValue](#)
-

DeBruijnSequence

```
DeBruijnSequence(list, order)
```

```
DeBruijnSequence(string, order)
```

returns the de Briujn sequence of order `order`.

See

- [Wikipedia - De Bruijn sequence](#)

Examples

Decrement

`Decrement(x)`

`x--`

decrements `x` by `1`, returning the original value of `x`.

Examples

```
>> a = 5
>> a--
5

>> a
4
```

Implementation status

- - full supported

Github

- [Implementation of Decrement](#)
-

DedekindNumber

`DedekindNumber(n)`

returns the n th Dedekind number. Currently $0 \leq n \leq 9$ can be computed, otherwise the function returns unevaluated.

See

- [Wikipedia - Dedekind number](#)
- [OEIS - A000372](#)

Examples

```
>> Table(DedekindNumber(i), {i, 0, 11})  
{2, 3, 6, 20, 168, 7581, 7828354, 2414682040998, 56130437228687557907788, 2863865776682984111284
```

Implementation status

- - partially implemented

Github

- [Implementation of DedekindNumber](#)
-

Default

```
Default(symbol)
```

`Default` returns the default value associated with the `symbol` for a pattern default `_` expression.

```
Default(symbol, position)
```

`Default` returns the default value associated with the `symbol` for a pattern default `_` expression at `position`.

Examples

```
>> Default(test) = 1
1

>> test(x_., y_.) = {x, y}
{x,y}

>> test(a)
{a,1}

>> test( )
{1,1}
```

Implementation status

- - full supported

Github

- [Implementation of Default](#)
-

DefaultValues

DefaultValues(symbol)

DefaultValues returns the default values associated with the symbol.

Examples

```
>> Default(tst)=42
42

>> DefaultValues(tst)
{HoldPattern(Default(tst)):>42}

>> Default(xx1)=1;Default(xx2)=3;Default(xx3)=5;

>> DefaultValues /@ Names("xx*")
{{HoldPattern(Default(xx1)):>1},{HoldPattern(Default(xx3)):>5},{HoldPattern(Default(xx2))>
```

Defer

```
Defer(expr)
```

`Defer` doesn't evaluate `expr` and didn't appear in the output

Examples

```
>> Defer(3*2)
3*2
```

Implementation status

- - partially implemented

Github

- [Implementation of Defer](#)
-

Definition

```
Definition(symbol)
```

prints values and rules associated with `symbol`.

Examples

```
>> Definition(ArcSinh)
Attributes(ArcSinh)={Listable, NumericFunction, Protected}

ArcSinh(I/Sqrt(2))=I*1/4*Pi

ArcSinh(Undefined)=Undefined

ArcSinh(Infinity)=Infinity

ArcSinh(I*Infinity)=Infinity

ArcSinh(I)=I*1/2*Pi

ArcSinh(0)=0

ArcSinh(I*1/2)=I*1/6*Pi

ArcSinh(I*1/2*Sqrt(3))=I*1/3*Pi

ArcSinh(ComplexInfinity)=ComplexInfinity
```

```
>> a=2
2

>> Definition(a)
{a=2}
```

Implementation status

- - full supported

Github

- [Implementation of Definition](#)
-

Degree constant

Degree

the constant `Degree` converts angles from degree to `Pi/180` radians.

See:

- [Wikipedia - Degree \(angle\)](#)

Examples

```
>> Sin(30*Degree)
1/2
```

Degree has the value of $\text{Pi} / 180$

```
>> Degree == Pi / 180
True

>> Cos(Degree(x))
Cos(Degree(x))

>> N(Degree)
0.017453292519943295
```

Implementation status

- - full supported

Github

- [Implementation of Degree](#)
-

Delete

```
Delete(expr, n)
```

deletes the element at position `n` in `expr`. The position is counted from the end if `n` is negative.

```
Delete(expr, {m,n, ...})
```

deletes the element at position `{m, n, ...}`.

Examples

```
>> Delete({a, b, c, d}, 3)
{a,b,d}

>> Delete({a, b, c, d}, -2)
{a,b,d}
```

Implementation status

- - partially implemented

Github

- [Implementation of Delete](#)
-

DeleteCases

```
DeleteCases(list, pattern)
```

returns the elements of `list` that do not match `pattern`.

Examples

```
>> DeleteCases({a, 1, 2.5, "string"}, _Integer|_Real)
{a, "string"}

>> DeleteCases({a, b, 1, c, 2, 3}, _Symbol)
{1, 2, 3}
```

Implementation status

- - full supported

Github

- [Implementation of DeleteCases](#)
-

DeleteDuplicates

```
DeleteDuplicates(list)
```

deletes duplicates from `list`.

Examples

```
>> DeleteDuplicates({1, 7, 8, 4, 3, 4, 1, 9, 9, 2, 1})  
{1, 7, 8, 4, 3, 9, 2}
```

```
>> DeleteDuplicates({3, 2, 1, 2, 3, 4}, Less)  
{3, 2, 1}
```

Implementation status

- - full supported

Github

- [Implementation of DeleteDuplicates](#)
-

DeleteDuplicatesBy

```
DeleteDuplicatesBy(list, predicate)
```

deletes duplicates from `list`, for which the `predicate` returns `True`.

Examples

Implementation status

- - full supported

Github

- [Implementation of DeleteDuplicatesBy](#)
-

Denominator

```
Denominator(expr)
```

gives the denominator in `expr`. Denominator collects expressions with negative exponents.

See

- [Wikipedia - Fraction \(mathematics\)](#)

Examples

```
>> Denominator(a / b)
b
>> Denominator(2 / 3)
3
>> Denominator(a + b)
1
```

Implementation status

- - full supported

Github

- [Implementation of Denominator](#)
-

DensityHistogram

```
DensityHistogram( list-of-pair-values )
```

plot a density histogram for a `list-of-pair-values`

Examples

```
>> DensityHistogram({{1, 1}, {2, 2}, {3, 3}, None, {3, 3}, {5, 5}, f(), {2, 2}, {1, 1},
```

Implementation status

- - experimental

Github

- [Implementation of DensityHistogram](#)
-

Depth

```
Depth(expr)
```

gets the depth of `expr`.

```
Depth(expr, Heads->True)
```

gets the depth of `expr` with head expressions included.

The depth of an expression is defined as one plus the maximum number of `Part` indices required to reach any part of `expr`, except for heads.

Examples

```
>> Depth(x)
1

>> Depth(x + y)
2

>> Depth({{{{x}}}})
5
```

Complex numbers are atomic, and hence have depth 1:

```
>> Depth(1 + 2*I)
1
```

Generally `Depth` ignores heads.

```
>> Depth(f(a, b)[c])
2
```

But if you include the option `Heads->True`, the heads depth is included.

```
Depth(f(a, b)[c], Heads->True)
3
```

Implementation status

- - full supported

Github

- [Implementation of Depth](#)
-

Derivative

```
Derivative(n)[f]
```

represents the n -th derivative of the function f .

```
Derivative(n1, n2, n3, ...)[f]
```

represents a multivariate derivative.

See:

- [Wikipedia - Differentiation rules](#)
- [Wikipedia - Derivative](#)

Examples

```
>> Derivative(1)[Sin]
Cos(#1)&

>> Derivative(3)[Sin]
-Cos(#1)&

>> Derivative(2)[# ^ 3&]
6*(#1&)
```

`Derivative` can be entered using `'`:

```
>> Sin'(x)
Cos(x)

>> (# ^ 4&)'
12*(#1^2&)

>> f'(x) // FullForm
"Derivative(1)[f][x]"
```

The 0 th derivative of any expression is the expression itself:

```
>> Derivative(0,0,0)[a+b+c]
a+b+c
```

Unknown derivatives:

```
>> Derivative(2, 1)[h]
Derivative(2,1)[h]

>> Derivative(2, 0, 1, 0)[h(g)]
Derivative(2,0,1,0)[h(g)]
```

Related terms

[D](#), [DSolve](#), [Integrate](#), [Limit](#), [ND](#), [NIntegrate](#)

Implementation status

- - partially implemented

Github

- [Implementation of Derivative](#)
 - [Rule definitions of Derivative](#)
-

DesignMatrix

```
DesignMatrix(m, f, x)
```

returns the design matrix.

Examples

```
>> DesignMatrix({{2, 1}, {3, 4}, {5, 3}, {7, 6}}, x, x)
{{1,2},{1,3},{1,5},{1,7}}

>> DesignMatrix({{2, 1}, {3, 4}, {5, 3}, {7, 6}}, f(x), x)
{{1,f(2)},{1,f(3)},{1,f(5)},{1,f(7)}}

>> data = {{0, 1}, {1, 3}, {2, 4}, {3, 7}}; basis = {1, x}; m = DesignMatrix(data, basis)
FittedModel[0.9*#1+1.9*#2]
```

Related terms

[FittedModel](#), [LinearModelFit](#)

Implementation status

- - full supported

Github

- [Implementation of DesignMatrix](#)
-

Det

```
Det(matrix)
```

computes the determinant of the `matrix`.

See:

- [Wikipedia: Determinant](#)
- [Youtube - The determinant | Essence of linear algebra, chapter 6](#)
- [Youtube - Cramer's rule, explained geometrically | Essence of linear algebra, chapter 12](#)

Examples

```
>> Det({{1, 1, 0}, {1, 0, 1}, {0, 1, 1}})
-2
```

Symbolic determinant:

```
>> Det({{a, b, c}, {d, e, f}, {g, h, i}})
-c*e*g+b*f*g+c*d*h-a*f*h-b*d*i+a*e*i
```

Implementation status

- - full supported

Github

- [Implementation of Det](#)
-

Diagonal

```
Diagonal(matrix)
```

computes the diagonal vector of the `matrix`.

```
Diagonal(matrix, n)
```

computes the diagonal vector of the `n`-th diagonal above or below the main diagonal.

Examples

```
>> Diagonal({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}})
{1, 5, 9}

>> Diagonal({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}, -1)
{4, 8}
```

Implementation status

- - full supported

Github

- [Implementation of Diagonal](#)
-

DiagonalMatrix

```
DiagonalMatrix(list)
```

gives a matrix with the values in `list` on its diagonal and zeroes elsewhere.

Examples

```
>> DiagonalMatrix({1, 2, 3})  
{{1, 0, 0}, {0, 2, 0}, {0, 0, 3}}  
  
>> MatrixForm(%)  
  1   0   0  
  0   2   0  
  0   0   3  
  
>> DiagonalMatrix(a + b)  
DiagonalMatrix(a + b)
```

Implementation status

- - full supported

Github

- [Implementation of DiagonalMatrix](#)
-

DiagonalMatrixQ

```
DiagonalMatrixQ(matrix)
```

```
DiagonalMatrixQ(matrix, diagonal)
```

returns `True` if all elements of the `matrix` are `0` except the elements on the `diagonal`.

Examples

```
>> DiagonalMatrixQ({{a, 0, 0}, {b, 0, 0}, {0, 0, c}})
False

>> DiagonalMatrixQ({{0, a, 0, 0}, {0, 0, b, 0}, {0, 0, 0, c}}, 1)
True

>> DiagonalMatrixQ({{0, a, 0, 0}, {0, 0, b, 0}, {0, 0, c, 0}}, -1)
False

>> DiagonalMatrixQ({{0, 0, 0, 0}, {a, 0, 0, 0}, {0, b, 0, 0}}, -1)
True
```

Implementation status

- - full supported

Github

- [Implementation of DiagonalMatrixQ](#)
-

DialogInput

```
DialogInput()
```

if the file system is enabled, the user can input a string in a dialog box.

```
DialogInput("info-string")
```

if the file system is enabled, additionally show the `info-string` in the dialog box.

Examples

Convert the input with `ToExpression` into a Symja expression

```
>> ToExpression(DialogInput("::"))^3
```

DiceDissimilarity

```
DiceDissimilarity(u, v)
```

returns the Dice dissimilarity between the two boolean 1-D lists `u` and `v`, which is defined as $(c_{tf} + c_{ft}) / (2 * c_{tt} + c_{ft} + c_{tf})$, where `n` is `len(u)` and `cij` is the number of occurrences of `u(k)=i` and `v(k)=j` for `k < n`.

Examples

```
>> DiceDissimilarity({1, 0, 1, 1, 0, 1, 1}, {0, 1, 1, 0, 0, 0, 1})  
1/2
```

Implementation status

- - full supported

Github

- [Implementation of DiceDissimilarity](#)
-

DifferenceDelta

```
DifferenceDelta(f(x), x)
```

generates a forward difference $f(x+1) - f(x)$

```
DifferenceDelta(f(x), {x, n, h})
```

generates a higher-order forward difference for integers n .

See:

- [Wikipedia - Finite difference - Higher-order differences](#)

Examples

```
>> DifferenceDelta(b(a), a)
-b(a)+b(1+a)

>> DifferenceDelta(b(a), {a, 5, c})
-b(a)+5*b(a+c)-10*b(a+2*c)+10*b(a+3*c)-5*b(a+4*c)+b(a+5*c)
```

Implementation status

- - full supported

Github

- [Implementation of DifferenceDelta](#)
-

DifferenceQuotient

```
DifferenceQuotient(f, {var, h})
```

gives the difference quotient $(f(\text{var}+h)-f(\text{var}))/h$ of the expression `f` with respect to `var` and step-size `h`.

```
DifferenceQuotient(f, {var,n,h})
```

gives the multiple difference quotient of the expression `f` with respect to `var` and step-size `h`.

See:

- [Wikipedia - Difference quotient](#)

Examples

```
>> DifferenceQuotient(x^2, {x, h})  
h+2*x  
  
>> DifferenceQuotient(Sin(x), {x, h})  
(-Sin(x)+Sin(h+x))/h
```

Implementation status

- - partially implemented

Github

- [Implementation of DifferenceQuotient](#)
-

DifferenceRoot

`DifferenceRoot(equation)`

operator for generating a holonomic sequence defined by a linear difference equation.

See:

- [Wikipedia - Holonomic_function](#)

Examples

The sequence of Catalan numbers:

```
>> dr=DifferenceRoot(Function({y,n},{(-4*n-2)*y(n)+(n+2)*y(n+1)==0,y(0)==1}));  
  
>> dr(10)  
16796  
  
>> CatalanNumber(10)  
16796
```

DigitCharacter

DigitCharacter

represents the digits 0-9.

Examples

```
>> StringMatchQ("1", DigitCharacter)
True

>> StringMatchQ("a", DigitCharacter)
False

>> StringMatchQ("12", DigitCharacter)
False

>> StringMatchQ("123245", DigitCharacter..)
True
```

DigitCount

```
DigitCount(n)
```

returns a list of the number of integer digits for `n` for `radix` 10.

```
DigitCount(n, radix)
```

returns a list of the number of integer digits for `n` for `radix`.

```
DigitCount(n, radix, k)
```

returns a the number of integer digit `k` in `n` for `radix`.

See

- [Wikipedia - Radix](#)

Examples

OEIS integer sequence [A098949](#):

```
>> Select(Range(1000), DigitCount( # )[[1]] == 0 && DigitCount( # )[[3]] == 0 && DigitC  
{9, 29, 49, 69, 89, 90, 92, 94, 96, 98, 99, 209, 229, 249, 269, 289, 290, 292, 294, 296, 298, 299, 409,  
429, 449, 469, 489, 490, 492, 494, 496, 498, 499, 609, 629, 649, 669, 689, 690, 692, 694, 696, 698,  
699, 809, 829, 849, 869, 889, 890, 892, 894, 896, 898, 899, 900, 902, 904, 906, 908, 909, 920, 922,  
924, 926, 928, 929, 940, 942, 944, 946, 948, 949, 960, 962, 964, 966, 968, 969, 980, 982, 984, 986,  
988, 989, 990, 992, 994, 996, 998, 999}
```

Implementation status

- - full supported

Github

- [Implementation of DigitCount](#)
-

DigitQ

```
DigitQ(str)
```

returns `True` if `str` is a string which contains only digits.

Examples

```
>> DigitQ("1234")
True
```

Implementation status

- - full supported

Github

- [Implementation of DigitQ](#)
-

Dimensions

```
Dimensions(expr)
```

returns a list of the dimensions of the expression `expr`.

Examples

A vector of length 3:

```
>> Dimensions({a, b, c})  
{3}
```

A 3x2 matrix:

```
>> Dimensions({{a, b}, {c, d}, {e, f}})  
{3, 2}
```

Ragged arrays are not taken into account:

```
>> Dimensions({{a, b}, {b, c}, {c, d, e}})  
{3}
```

The expression can have any head:

```
>> Dimensions[f[f[a, b, c]]]  
{1, 3}  
  
>> Dimensions({})  
{0}  
  
>> Dimensions({{}})  
{1, 0}
```

Implementation status

- - full supported

Github

- [Implementation of Dimensions](#)
-

DiracDelta

```
DiracDelta(x)
```

`DiracDelta` function returns 0 for all real numbers x where $x \neq 0$.

Examples

```
>> DiracDelta(-42)
0
```

`DiracDelta` doesn't evaluate for 0:

```
>> DiracDelta(0)
DiracDelta(0)
```

Implementation status

- - partially implemented

Github

- [Implementation of DiracDelta](#)
-

DirectedEdge

```
DirectedEdge(a, b)
```

is a directed edge from vertex `a` to vertex `b` in a `graph` object.

See:

- [Wikipedia - Directed graph](#)
- [Wikipedia - Graph theory](#)

Examples

```
>> Graph({1 -> 2, 2 -> 3, 1 -> 3, 4 -> 2})
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [\[UndirectedEdge\]UndirectedEdge.md](#)), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

DirectedInfinity

```
DirectedInfinity(z)
```

represents an infinite multiple of the complex number `z`.

```
DirectedInfinity()
```

is the same as `ComplexInfinity`.

See

- [Wikipedia - Infinity](#)

Examples

```
>> DirectedInfinity(1)
Infinity

>> DirectedInfinity()
ComplexInfinity

>> DirectedInfinity(1 + I)
DirectedInfinity((1+I)/Sqrt(2))

>> 1 / DirectedInfinity(1 + I)
0
```

Indeterminate expression `-Infinity + Infinity` encountered.

```
>> DirectedInfinity(1) + DirectedInfinity(-1)
Indeterminate

>>DirectedInfinity(1+I)+DirectedInfinity(2+I)
DirectedInfinity((1+I)/Sqrt(2))+DirectedInfinity((2+I)/Sqrt(5))

>>DirectedInfinity(Sqrt(3))
Infinity
```

Implementation status

- - full supported

Github

- [Implementation of DirectedInfinity](#)
-

DirichletBeta

DirichletBeta(x)

DirichletBeta function returns the Dirichlet beta function.

See

- [Wikipedia - Dirichlet beta function](#)

Examples

```
>> Table(DirichletBeta(x), {x, -4, 4}) // N  
{2.5, 0.0, -0.5, 0.0, 0.5, 0.785398, 0.915966, 0.968946, 0.988945}
```

Implementation status

- - full supported

Github

- [Implementation of DirichletBeta](#)
-

DirichletEta

```
DirichletEta(x)
```

`DirichletEta` function returns the Dirichlet eta function.

See

- [Wikipedia - Dirichlet eta function](#)

Examples

```
>> Table(DirichletEta(x), {x, -4, 4}) // N  
{0.0, -0.125, 0.0, 0.25, 0.5, 0.693147, 0.822467, 0.901543, 0.947033}
```

Implementation status

- - full supported

Github

- [Implementation of DirichletEta](#)
-

DirichletLambda

`DirichletLambda(z)`

returns the Dirichlet lambda function.

Examples

```
>> Table(DirichletLambda(x), {x, -4, 4})  
{0, -7/120, 0, 1/12, 0, ComplexInfinity, Pi^2/8, 7/8*Zeta(3), Pi^4/96}
```

Implementation status

- - full supported

Github

- [Implementation of DirichletLambda](#)
-

DiscreteDelta

```
DiscreteDelta(n1, n2, n3, ...)
```

`DiscreteDelta` function returns `1` if all the `ni` are `0`. Returns `0` otherwise.

Examples

```
>> DiscreteDelta(0, 0, 0.0)
1
```

Implementation status

- - full supported

Github

- [Implementation of DiscreteDelta](#)
-

DiscretePlot

```
DiscretePlot( expr, {x, nmax} )
```

plots `expr` with `x` ranging from `1` to `nmax`.

```
DiscretePlot( expr, {x, nmin, nmax} )
```

plots `expr` with `x` ranging from `nmin` to `nmax`.

```
DiscretePlot( expr, {x, nmin, nmax, delta} )
```

plots `expr` with `x` ranging from `nmin` to `nmax` usings steps `delta`.

Note: This feature is available in the console app and in the web interface.

Examples

In the console apps, this command shows an HTML page with a JavaScript list plot control.

```
>> DiscretePlot(Sin(x), {x, 0, 4 Pi}, PlotRange->{{0, 4 Pi}, {0, 1.5}})
```

```
>> DiscretePlot(Tan(x), {x, -6, 6})
```

Related terms

[ListPlot](#), [ListLogPlot](#)

Implementation status

- - experimental

Github

- [Implementation of DiscretePlot](#)
-

DiscreteRatio

```
DiscreteRatio(f(var), var)
```

`DiscreteRatio` computes $f(\text{var}+1)/f(\text{var})$.

```
DiscreteRatio(f(var), {var, n, step})
```

`DiscreteRatio` computes the multiple discrete ratio with step `step`.

Examples

```
>> DiscreteRatio(k!, k)
1+k
```

Implementation status

- - partially implemented

Github

- [Implementation of DiscreteRatio](#)
-

DiscreteShift

```
DiscreteShift(f(var), {var, shift})
```

`DiscreteShift` computes the shift `f(var+shift)`.

```
DiscreteShift(f(var), {var, shift, step})
```

`DiscreteShift` computes the shift `f(var+shift*step)`.

Examples

```
>> DiscreteShift(n^2, n)
(1+n)^2
```

Implementation status

- - partially implemented

Github

- [Implementation of DiscreteShift](#)
-

DiscreteUniformDistribution

```
DiscreteUniformDistribution({min, max})
```

returns a discrete uniform distribution.

See:

- [Wikipedia - Discrete uniform distribution](#)

Examples

The variance of the discrete uniform distribution is

```
>> Variance(DiscreteUniformDistribution({l, r}))  
1/12*(-1+(1-l+r)^2)
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of DiscreteUniformDistribution](#)
-

Discriminant

```
Discriminant(poly, var)
```

computes the discriminant of the polynomial `poly` with respect to the variable `var`.

See:

- [Wikipedia - Discriminant](#)

Examples

```
>> Discriminant(a*x^2+b*x+c, x)
b^2-4*a*c
```

Implementation status

- - partially implemented

Github

- [Implementation of Discriminant](#)
-

Dispatch

```
Dispatch({rule1, rule2, ...})
```

create a dispatch map for a list of rules.

Examples

```
>> rul=Dispatch({"a\" -> 1, \"b\" -> 2, \"c\" -> 3})
Dispatch({a->1,b->2,c->3})

>> {"a", b, "c", d, e} /. rul
{1,b,3,d,e}

>> Dispatch({"a\" -> 1, \"b\" -> 2, \"c\" -> 3}) // Normal // InputForm", //
{"a"->1,"b"->2,"c"->3}
```

Related terms

[ReplaceAll](#), [ReplaceRepeated](#)

Implementation status

- - full supported

Github

- [Implementation of Dispatch](#)
-

Distribute

```
Distribute(f(x1, x2, x3,...))
```

distributes `f` over `Plus` appearing in any of the `xi`.

See:

- [Wikipedia - Distributive property](#)

Examples

```
>> Distribute((a+b)*(x+y+z))  
a*x+a*y+a*z+b*x+b*y+B*z
```

Implementation status

- - full supported

Github

- [Implementation of Distribute](#)
-

Div

```
Div({f1, f2, f3,...},{x1, x2, x3,...})
```

compute the divergence.

See:

- [Wikipedia - Divergence](#)

Examples

```
>> Div({x^2, y^3},{x, y})  
2*x+3*y^2
```

Implementation status

- - partially implemented

Github

- [Implementation of Div](#)
-

Divide

```
Divide(a, b)
```

```
a / b
```

represents the division of `a` by `b`.

Examples

```
>> 30 / 5
```

```
6
```

```
>> 1 / 8
```

```
1/8
```

```
>> Pi / 4
```

```
Pi / 4
```

Use `N` or a decimal point to force numeric evaluation:

```
>> Pi / 4.0
```

```
0.7853981633974483
```

```
>> N(1 / 8)
```

```
0.125
```

Nested divisions:

```
>> a / b / c
```

```
a/(b*c)
```

```
>> a / (b / c)
```

```
(a*c)/b
```

```
>> a / b / (c / (d / e))
```

```
(a*d)/(b*c*e)
```

```
>> a / (b ^ 2 * c ^ 3 / e)
```

```
(a*e)/(b^2*c^3)
```

```
>> 1 / 4.0
```

```
0.25
```

```
>> 10 / 3 // FullForm
```

```
Rational(10,3)
```

```
>> a / b // FullForm  
Times(a, Power(b, -1))
```

Implementation status

- - full supported

Github

- [Implementation of Divide](#)
-

DivideBy

```
DivideBy(x, dx)
```

```
x /= dx
```

is equivalent to `x = x / dx`.

Examples

```
>> a = 10
>> a /= 2
5

>> a
5
```

Implementation status

- - full supported

Github

- [Implementation of DivideBy](#)
-

DivideSides

```
DivideSides(compare-expr, value)
```

divides all elements of the `compare-expr` by `value`. `compare-expr` can be `True`, `False` or a comparison expression with head `Equal`, `Unequal`, `Less`, `LessEqual`, `Greater`, `GreaterEqual`.

Examples

```
>> DivideSides(a==a, x)
True

>> DivideSides(a==b, x)
a/x==b/x
```

Implementation status

- - full supported

Github

- [Implementation of DivideSides](#)
-

Divisible

```
Divisible(n, m)
```

returns `True` if `n` could be divide by `m`.

See:

- [Wikipedia - Divisor](#)

Examples

```
>> Divisible(42,7)
True
```

Implementation status

- - full supported

Github

- [Implementation of Divisible](#)
-

Divisors

Divisors(n)

returns all integers that divide the integer `n`.

See:

- [Wikipedia - Divisor](#)

Examples

```
>> Divisors(990)
{1, 2, 3, 5, 6, 9, 10, 11, 15, 18, 22, 30, 33, 45, 55, 66, 90, 99, 110, 165, 198, 330, 495, 990}

>> Divisors(341550071728321)
{1, 10670053, 32010157, 341550071728321}

>> Divisors(2010)
{1, 2, 3, 5, 6, 10, 15, 30, 67, 134, 201, 335, 402, 670, 1005, 2010}
```

Implementation status

- - full supported

Github

- [Implementation of Divisors](#)
-

DivisorSigma

```
DivisorSigma(k, n)
```

returns the sum of the k -th powers of the divisors of n .

See:

- [Wikipedia - Divisor function](#)

Examples

```
>> DivisorSigma(0,12)
6

>> DivisorSigma(1,12)
28
```

Implementation status

- - partially implemented

Github

- [Implementation of DivisorSigma](#)
-

DivisorSum

```
DivisorSum(n, head)
```

returns the sum of the divisors of `n`. The `head` is applied to each divisor.

```
DivisorSum(n, head, condition)
```

The `condition` checks, if the divisor should be summed up.

See:

- [Wikipedia - Divisor sum identities](#)

Examples

Calculate the [OEIS - sequence A002791](#):

```
>> a(n_) := DivisorSum(n, #^2 &, # < 5 &) + 4 * DivisorSum(n, # &, # > 4 &); Array(a, 7  
{1, 5, 10, 21, 21, 38, 29, 53, 46, 65, 45, 102, 53, 89, 90, 117, 69, 146, 77, 161, 122, 137, 93, 230,  
121, 161, 154, 217, 117, 278, 125, 245, 186, 209, 189, 354, 149, 233, 218, 353, 165, 374, 173, 329,  
306, 281, 189, 486, 225, 365, 282, 385, 213, 470, 285, 473, 314, 353, 237, 662, 245, 377, 410, 501,  
333, 566, 269, 497, 378, 569}
```

Implementation status

- - partially implemented

Github

- [Implementation of DivisorSum](#)
-

Do

```
Do(expr, {max})
```

evaluates `expr` `max` times.

```
Do(expr, {i, max})
```

evaluates `expr` `max` times, substituting `i` in `expr` with values from `1` to `max`.

```
Do(expr, {i, min, max})
```

starts with `i = max`.

```
Do(expr, {i, min, max, step})
```

uses a step size of `step`.

```
Do(expr, {i, {i1, i2, ...}})
```

uses values `i1`, `i2`, ... for `i`.

```
Do(expr, {i, imin, imax}, {j, jmin, jmax}, ...)
```

evaluates `expr` for each `j` from `jmin` to `jmax`, for each `i` from `imin` to `imax`, etc.

Examples

```
>> Do(Print(i), {i, 2, 4})
| 2
| 3
| 4

>> Do(Print({i, j}), {i,1,2}, {j,3,5})
| {1, 3}
| {1, 4}
| {1, 5}
| {2, 3}
| {2, 4}
| {2, 5}
```

You can use `Break()` and `Continue()` inside `Do`:

```
>> Do(If(i > 10, Break(), If(Mod(i, 2) == 0, Continue())); Print(i)), {i, 5, 20})
| 5
| 7
| 9

>> Do(Print("hi"), {1+1})
| hi
| hi
```

The [A005132 Recaman's sequence](#) integer sequence

```
>> a = {1}; Do( If( a[ [ -1 ] ] - n > 0 && Position( a, a[ [ -1 ] ] - n ) == {}, a = Ap
{1,3,6,2,7,13,20,12,21,11,22,10,23,9,24,8,25,43,62,42,63,41,18,42,17,43,16,44,15,45,14,
```

Related terms

[Break](#), [Continue](#), [For](#), [While](#)

Implementation status

- - full supported

Github

- [Implementation of Do](#)
-

Dot

```
Dot(x, y)
```

```
x . y
```

computes products of vectors, matrices, and tensors.

Note: the `Dot` operator has the attribute `Flat` ([associative property](#)) but not `Orderless` ([commutative property](#)).

See:

- [Wikipedia - Matrix multiplication](#)
- [YouTube - Matrix multiplication as composition | Essence of linear algebra, chapter 4](#)

Examples

Scalar product of vectors:

```
>> {a, b, c} . {x, y, z}
a*x+b*y+c*z
```

Product of matrices and vectors:

```
>> {{a, b}, {c, d}} . {x, y}
{a*x+b*y, c*x+d*y}
```

Matrix product:

```
>> {{a, b}, {c, d}} . {{r, s}, {t, u}}
{{a*r+b*t, a*s+b*u}, {c*r+d*t, c*s+d*u}}

>> a . b
a.b
```

Wrong dimensions print an error message:

```
{{a, b}, {c, d}} . {{x, y}}
```

Dot: Tensors `{{a,b},{c,d}}` and `{{x,y}}` have incompatible shapes.

Related terms

[Flat](#), [MatrixPower](#), [Orderless](#), [Times](#)

Implementation status

- - full supported

Github

- [Implementation of Dot](#)
-

DownValues

`DownValues(symbol)`

prints the down-value rules associated with `symbol`.

Examples

```
>> f(1)=3
3

>> f(x_):=x^3

>> DownValues(f)
{HoldPattern(f(1)):>3, HoldPattern(f(x_)):>x^3}
```

Implementation status

- - full supported

Github

- [Implementation of DownValues](#)
-

Drop

```
Drop(expr, n)
```

returns `expr` with the first `n` leaves removed.

Examples

```
>> Drop({a, b, c, d}, 3)
{d}

>> Drop({a, b, c, d}, -2)
{a,b}

>> Drop({a, b, c, d, e}, {2, -2})
{a,e}
```

Drop a submatrix:

```
>> A = Table(i*10 + j, {i, 4}, {j, 4})
{{11,12,13,14},{21,22,23,24},{31,32,33,34},{41,42,43,44}}

>> Drop(A, {2, 3}, {2, 3})
{{11,14},{41,44}}

>> Drop(Range(10), {-2, -6, -3})
{1,2,3,4,5,7,8,10}

>> Drop(Range(10), {10, 1, -3})
{2, 3, 5, 6, 8, 9}
```

Cannot drop positions `-5` through `-2` in `{1, 2, 3, 4, 5, 6}`.

```
>> Drop(Range(6), {-5, -2, -2})
Drop({1, 2, 3, 4, 5, 6}, {-5, -2, -2})
```

Implementation status

- - full supported

Github

- [Implementation of Drop](#)
-

DSolve

```
DSolve(equation, f(var), var)
```

attempts to solve a linear differential `equation` for the function `f(var)` and variable `var`.

See:

- [Wikipedia - Ordinary differential equation](#)

Examples

```
>> DSolve({y'(x)==y(x)+2},y(x), x)
{{y(x)->-2+E^x*C(1)}}

>> DSolve({y'(x)==y(x)+2,y(0)==1},y(x), x)
{{y(x)->-2+3*E^x}}

>> DSolve(y''(x) == 0, y(x), x)
{{y(x)->C(1)+x*C(2)}}

>> DSolve(y''(x) == y(x), y(x), x)
{{y(x)->C(1)/E^x+E^x*C(2)}}

>> DSolve(y''(x) == y(x), y, x)
{{y->Function({x},C(1)/E^x+E^x*C(2))}}
```

DSolve can also solve basic PDE

```
>> DSolve(D(f(x, y), x)/f(x, y) + 3*D(f(x, y), y) / f(x, y) == 2, f, {x, y})
{{f->Function({x,y},E^(2*x)*C(1)[-3*x+y])}}

>> DSolve(D(f(x, y), x)*x + D(f(x, y), y)*y == 2, f(x, y), {x, y})
{{f(x,y)->2*Log(x)+C(1)[y/x]}}

>> DSolve(D(y(x, t), t) + 2*D(y(x, t), x) == 0, y(x, t), {x, t})
{{y(x,t)->C(1)[1/2*(2*t-x)]}}
```

Related terms

[DSolveValue](#), [Factor](#), [FindRoot](#), [NRoots](#), [Solve](#)

Implementation status

- - partially implemented

Github

- [Implementation of DSolve](#)
-

DSolveValue

```
DSolveValue(equation, f(var), var)
```

attempts to solve a linear differential `equation` for the function `f(var)` and variable `var`.

See:

- [Wikipedia - Ordinary differential equation](#)

Examples

```
>> DSolveValue({y'(x)==y(x)+2}, y(x), x)
-2+E^x*C(1)

>> DSolveValue({y'(x)==y(x)+2, y(0)==1}, y(x), x)
2+3*E^x
```

Related terms

[DSolve](#), [Factor](#), [FindRoot](#), [NRoots](#), [Solve](#)

Dt

```
Dt(f, x)
```

gives the total derivative of f with respect to x .

```
Dt(f, x, y, ...)
```

differentiates successively with respect to x , y , etc.

```
Dt(f, {x, n})
```

gives the multiple derivative of order n .

See:

- [Wikipedia - Differentiation rules](#)
- [Wikipedia - Derivative](#)
- [Wikipedia - General Leibniz rule](#)

Examples

First-order derivative of a polynomial:

```
>> Dt(x^4, {x, 3})  
24*x  
  
>> Dt(x * y * z, x, Constants -> {y, z})  
y*z
```

Related terms

[Derivative](#), [D](#), [DSolve](#), [Grad](#), [Integrate](#), [Laplacian](#), [Limit](#), [ND](#), [NIntegrate](#)

E

E

Euler's constant E

Note: the upper case identifier `E` is different from the lower case identifier `e`.

See

- [Wikipedia - E \(mathematical constant\)](#)
- [Fungrim - ConstE](#)

Examples

```
>> Exp(1)
E

>> N(E)
2.718281828459045
```

Related terms

[EulerGamma](#), [Pi](#)

Implementation status

- - full supported

Github

- [Implementation of E](#)
-

Echo

```
Echo(expr)
```

prints the `expr` to the default output stream and returns `expr`.

```
Echo(expr, label)
```

prints `label` before printing `expr`.

```
Echo(expr, label, head)
```

prints `label` before printing `head(expr)` and returns `expr`.

Examples

```
>> {Echo(f(x,y)), Print(g(a,b))}  
{f(x,y), Null}
```

prints

```
f(x,y)  
g(a,b)
```

and returns

```
{f(x,y), Null}
```

Implementation status

- - full supported

Github

- [Implementation of Echo](#)
-

EchoFunction

```
EchoFunction()[expr]
```

operator form of the `Echo` function. Print the `expr` to the default output stream and return `expr`.

```
EchoFunction(head)[expr]
```

prints `head(expr)` and returns `expr`.

```
EchoFunction(label, head)[expr]
```

prints `label` before printing `head(expr)` and returns `expr`.

Examples

```
>> {EchoFunction()[f(x,y)], Print(g(a,b))}  
{f(x,y), Null}
```

prints

```
f(x,y)  
g(a,b)
```

and returns

```
{f(x,y), Null}
```

Implementation status

- - full supported

Github

- [Implementation of EchoFunction](#)
-

EdgeCount

```
EdgeCount(graph)
```

return the number of edges of the `graph`

See:

- [Wikipedia - Graph theory](#)

Examples

```
>> EdgeCount(CompleteGraph(4))  
6
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexCount](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of EdgeCount](#)
-

EdgeList

```
EdgeList(graph)
```

convert the `graph` into a list of edges.

See:

- [Wikipedia - Graph theory](#)

Examples

```
>> EdgeList(Graph({1 -> 2, 2 -> 3, 1 -> 3, 4 -> 2}))  
{1->2, 2->3, 1->3, 4->2}
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of EdgeList](#)
-

EdgeQ

```
EdgeQ(graph, edge)
```

test if `edge` is an edge in the `graph` object.

See:

- [Wikipedia - Graph theory](#)

Examples

```
>> EdgeQ(Graph({1 -> 2, 2 -> 3, 1 -> 3, 4 -> 2}), 2 -> 3)
True

>> EdgeQ(Graph({1 -> 2, 2 -> 3, 1 -> 3, 4 -> 2}), 2 -> 4)
False
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of EdgeQ](#)
-

EdgeRules

EdgeRules(graph)

convert the `graph` into a list of rules. All edge types (undirected, directed) are represented by a rule `lhs->rhs`.

See:

- [Wikipedia - Graph theory](#)

Examples

```
>> EdgeRules(Graph({1 \[UndirectedEdge] 2, 2 \[UndirectedEdge] 3, 3 \[UndirectedEdge] 1, 1->2, 2->3, 3->1})
```

Related terms

[EdgeList](#), [GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of EdgeRules](#)
-

EditDistance

```
EditDistance(a, b)
```

returns the Levenshtein distance of `a` and `b`, which is defined as the minimum number of insertions, deletions and substitutions on the constituents of `a` and `b` needed to transform one into the other.

See:

- [Wikipedia - Levenshtein distance](#)

Examples

```
>> EditDistance("kitten", "kitchen")
2

>> EditDistance("abc", "ac")
1
>> EditDistance("abc", "acb")
2

>> EditDistance("azbc", "abxyc")
3
```

The `IgnoreCase` option makes `EditDistance` ignore the case of letters:

```
>> EditDistance("time", "Thyme")
3

>> EditDistance("time", "Thyme", IgnoreCase -> True)
2
```

Implementation status

- - full supported

Github

- [Implementation of EditDistance](#)
-

EffectiveInterest

```
EffectiveInterest(i, n)
```

returns an effective interest rate object.

See:

- [Wikipedia - Annuity](#)

Examples

```
>> TimeValue(Annuity(100, 12), EffectiveInterest(.01, 0.25), 12)
1268.515
```

Related terms

[Annuity](#), [AnnuityDue](#), [TimeValue](#)

Implementation status

- - full supported

Github

- [Implementation of EffectiveInterest](#)
-

Eigensystem

```
Eigensystem(matrix)
```

return the numerical eigensystem of the `matrix` as a list `{eigenvalues, eigenvectors}`

See

- [Wikipedia - Eigenvalues and Eigenvectors](#)
- [Youtube - Eigenvectors and eigenvalues | Essence of linear algebra, chapter 14](#)

Examples

```
>> Eigensystem({{1,0,0},{0,1,0},{0,0,1}})
{{1.0,1.0,1.0},{1.0,0.0,0.0},{0.0,1.0,0.0},{0.0,0.0,1.0}}
```

Note: Symjas implementation of the `Eigenvectors` function adds zero vectors when the geometric multiplicity of the eigenvalue is smaller than its algebraic multiplicity (hence the regular eigenvector matrix should be non-square). With these additional null vectors, the `Eigenvectors` result matrix becomes square. This happens for example with the following square matrix:

```
>> Eigensystem({{1,0,0},{-2,1,0},{0,0,1}})
{{1.0,1.0,1.0},{-2.50055*10^-13,1.0,0.0},{0.0,0.0,1.0},{0.0,0.0,0.0}}
```

Its characteristic polynomial is $(1.0 - \lambda)^3$, hence it has one eigen value $\lambda = 1.0$ with algebraic multiplicity 3. However, this eigenvalue leads to only two eigenvectors $v_1 = \{0.0, 1.0, 0.0\}$ and $v_2 = \{0.0, 0.0, 1.0\}$, hence its geometric multiplicity is only 2, not 3. So we add a third zero vector $v_3 = \{0.0, 0.0, 0.0\}$.

Related terms

[Eigenvalues](#), [Eigenvectors](#), [CharacteristicPolynomial](#)

Implementation status

- - partially implemented

Github

- [Implementation of Eigensystem](#)
-

Eigenvalues

```
Eigenvalues(matrix)
```

get the numerical eigenvalues of the `matrix`.

See

- [Wikipedia - Eigenvalues and Eigenvectors](#)
- [Youtube - Eigenvectors and eigenvalues | Essence of linear algebra, chapter 14](#)

Examples

```
>> Eigenvalues({{1,0,0},{0,1,0},{0,0,1}})
{1.0,1.0,1.0}
```

Note: Symjas implementation of the `Eigenvectors` function adds zero vectors when the geometric multiplicity of the eigenvalue is smaller than its algebraic multiplicity (hence the regular eigenvector matrix should be non-square). With these additional null vectors, the `Eigenvectors` result matrix becomes square. This happens for example with the following square matrix:

```
>> Eigenvectors({{1,0,0},{-2,1,0},{0,0,1}})
{{-2.50055*10^-13,1.0,0.0},{0.0,0.0,1.0},{0.0,0.0,0.0}}

>> Eigenvalues({{1,0,0},{-2,1,0},{0,0,1}})
{1.0,1.0,1.0}
```

Its characteristic polynomial is $(1.0 - \lambda)^3$, hence it has one eigen value $\lambda = 1.0$ with algebraic multiplicity 3. However, this eigenvalue leads to only two eigenvectors $v_1 = \{0.0, 1.0, 0.0\}$ and $v_2 = \{0.0, 0.0, 1.0\}$, hence its geometric multiplicity is only 2, not 3. So we add a third zero vector $v_3 = \{0.0, 0.0, 0.0\}$.

Related terms

[Eigensystem](#), [Eigenvectors](#), [CharacteristicPolynomial](#)

Implementation status

- - partially implemented

Github

- [Implementation of Eigenvalues](#)
-

Eigenvectors

```
Eigenvectors(matrix)
```

get the numerical eigenvectors of the `matrix`.

See

- [Wikipedia - Eigenvalues and Eigenvectors](#)
- [Youtube - Eigenvectors and eigenvalues | Essence of linear algebra, chapter 14](#)

Examples

```
>> Eigenvectors({{1,0,0},{0,1,0},{0,0,1}})
{{1.0,0.0,0.0},{0.0,1.0,0.0},{0.0,0.0,1.0}}
```

Note: Symjas implementation of the `Eigenvectors` function adds zero vectors when the geometric multiplicity of the eigenvalue is smaller than its algebraic multiplicity (hence the regular eigenvector matrix should be non-square). With these additional null vectors, the `Eigenvectors` result matrix becomes square. This happens for example with the following square matrix:

```
>> Eigenvectors({{1,0,0},{-2,1,0},{0,0,1}})
{{-2.50055*10^-13,1.0,0.0},{0.0,0.0,1.0},{0.0,0.0,0.0}}

>> Eigenvalues({{1,0,0},{-2,1,0},{0,0,1}})
{1.0,1.0,1.0}
```

Its characteristic polynomial is $(1.0 - \lambda)^3$, hence it has one eigen value $\lambda = 1.0$ with algebraic multiplicity 3. However, this eigenvalue leads to only two eigenvectors $v_1 = \{0.0, 1.0, 0.0\}$ and $v_2 = \{0.0, 0.0, 1.0\}$, hence its geometric multiplicity is only 2, not 3. So we add a third zero vector $v_3 = \{0.0, 0.0, 0.0\}$.

Related terms

[Eigensystem](#), [Eigenvalues](#), [CharacteristicPolynomial](#)

Implementation status

- - partially implemented

Github

- [Implementation of Eigenvectors](#)
-

Element

```
Element(symbol, domain)
```

assume (or test) that the `symbol` is in the domain `domain`.

See:

- [Wikipedia - Domain of a function](#)

Examples

```
>> Refine(Sin(k*Pi), Element(k, Integers))  
0  
  
>> Refine(D(Abs(x),x), Element(x, Reals))  
x/Abs(x)
```

Implementation status

- - full supported

Github

- [Implementation of Element](#)
-

ElementData

```
ElementData("name", "property")
```

gives the value of the property for the chemical specified by name.

```
ElementData(n, "property")
```

gives the value of the property for the nth chemical element.

`ElementData` uses data from [Wikipedia - List of data references for chemical elements](#)

Examples

```
>> ElementData(74)
"Tungsten"

>> ElementData("He", "AbsoluteBoilingPoint")
4.22

>> ElementData("Carbon", "IonizationEnergies")
{1086.5, 2352.6, 4620.5, 6222.7, 37831, 47277.0}

>> ElementData(16, "ElectronConfigurationString")
"[Ne] 3s2 3p4"

>> ElementData(73, "ElectronConfiguration")
{{2},{2,6},{2,6,10},{2,6,10,14},{2,6,3},{2}}

>> ListPlot(Table(ElementData(z, "AtomicRadius"), {z, 118}))
```

Some properties are not appropriate for certain elements:

```
>> ElementData("He", "ElectroNegativity")
Missing(NotApplicable)
```

Some data is missing:

```
>> ElementData("Tc", "SpecificHeat")
Missing(NotAvailable)
```

All the known properties:

```
>> ElementData("Properties")
{"Abbreviation", "AbsoluteBoilingPoint", "AbsoluteMeltingPoint", "AtomicNumber", "AtomicRad
"BoilingPoint", "BrinellHardness", "BulkModulus", "CovalentRadius", "CrustAbundance", "Densi
"ElectronAffinity", "ElectronConfiguration", "ElectronConfigurationString", "ElectronShell
"Group", "IonizationEnergies", "LiquidDensity", "MeltingPoint", "MohsHardness", "Name", "Peri
"ShearModulus", "SpecificHeat", "StandardName", "ThermalConductivity", "VanDerWaalsRadius",
"YoungModulus"}
```

Implementation status

- - full supported

Github

- [Implementation of ElementData](#)
-

Eliminate

```
Eliminate(list-of-equations, list-of-variables)
```

attempts to eliminate the variables from the `list-of-variables` in the `list-of-equations`.

See:

- [Wikipedia - System of linear equations - Elimination of variables](#)

Examples

```
>> Eliminate({x==2+y, y==z}, y)
x==2+z
```

Related terms

[DSolve](#), [GroebnerBasis](#), [FindRoot](#), [NRoots](#), [Roots](#), [Solve](#)

Implementation status

- - partially implemented

Github

- [Implementation of Eliminate](#)
 - [Rule definitions of Eliminate](#)
-

EllipticE

EllipticE(z)

returns the complete elliptic integral of the second kind.

See

- [Wikipedia - Elliptic integral - Complete elliptic integral of the second kind](#)
- [Fungrim - Legendre elliptic integrals](#)

Examples

```
>> EllipticE(5/4,1)
Sin(5/4)
```

Implementation status

- - full supported

Github

- [Implementation of EllipticE](#)
-

EllipticF

```
EllipticF(z)
```

returns the incomplete elliptic integral of the first kind.

See

- [Wikipedia - Elliptic integral - Incomplete elliptic integral of the first kind](#)
- [Fungrim - Legendre elliptic integrals](#)

Examples

```
>> EllipticF(17/2*Pi, m)
17*EllipticK(m)
```

Implementation status

- - full supported

Github

- [Implementation of EllipticF](#)
-

EllipticK

EllipticK(z)

returns the complete elliptic integral of the first kind.

See

- [Wikipedia - Elliptic integral - Complete elliptic integral of the first kind](#)
- [Fungrim - Legendre elliptic integrals](#)

Examples

```
>> Table(EllipticK(x+I), {x, -1.0, 1.0, 1/4})  
{1.26549+I*0.16224, 1.30064+I*0.18478, 1.33866+I*0.21305, 1.37925+I*0.24904, 1.42127+I*0.29
```

Implementation status

- - full supported

Github

- [Implementation of EllipticK](#)
-

EllipticPi

```
EllipticPi(n,m)
```

returns the complete elliptic integral of the third kind.

```
EllipticPi(n,m,z)
```

returns the complete elliptic integral of the third kind.

See

- [Wikipedia - Elliptic integral - Complete elliptic integral of the third kind](#)
- [Fungrim - Legendre elliptic integrals](#)

Examples

```
>> EllipticPi(n,Pi/2,x)  
EllipticPi(n,x)
```

Implementation status

- - full supported

Github

- [Implementation of EllipticPi](#)
-

End

```
End( )
```

end a context definition started with `Begin`

Examples

```
>> Begin("mytest`")

>> Context()
mytest`

>> $ContextPath
{System`,Global`}

>> End()
mytest`

>> Context()
Global`

>> $ContextPath
{System`,Global`}
```

Implementation status

- - full supported

Github

- [Implementation of End](#)
-

EndPackage

```
EndPackage( )
```

end a package definition

Examples

```
>> BeginPackage("Test`")

>> $ContextPath
{Test`,System`}

>> EndPackage( )

>> $ContextPath
{Test`,System`,Global`}
```

Implementation status

- - full supported

Github

- [Implementation of EndPackage](#)
-

Entropy

```
Entropy(list)
```

return the base E (Shannon) information entropy of the elements in `list`.

```
Entropy(b, list)
```

return the base b (Shannon) information entropy of the elements in `list`.

See:

- [Wikipedia - Entropy \(information theory\)](#)

Examples

```
>> Entropy({a, b, b})  
2/3*Log(3/2)+Log(3)/3  
  
>> Entropy({a, b, b, c, c, c, d})  
3/7*Log(7/3)+2/7*Log(7/2)+2/7*Log(7)  
  
>> Entropy(b, {a, c, c})  
2/3*Log(3/2)/Log(b)+Log(3)/(3*Log(b))
```

Related terms

[Commonest](#), [Counts](#), [E](#), [Tally](#)

Implementation status

- - full supported

Github

- [Implementation of Entropy](#)
-

Equal

```
Equal(x, y)
```

```
x == y
```

yields `True` if `x` and `y` are known to be equal, or `False` if `x` and `y` are known to be unequal.

```
lhs == rhs
```

represents the equation `lhs = rhs`.

See

- [Wikipedia - Computer algebra - Equality](#)

Examples

```
>> a==a
```

```
True
```

```
>> a==b
```

```
a == b
```

```
>> 1==1
```

```
True
```

Lists are compared based on their elements:

```
>> {{1}, {2}} == {{1}, {2}}
```

```
True
```

```
>> {1, 2} == {1, 2, 3}
```

```
False
```

Symbolic constants are compared numerically:

```
>> E > 1
```

```
True
```

```
>> Pi == 3.14
```

```
False
```

```
>> Pi ^ E == E ^ Pi
```


False

```
>> N(E, 3) == N(E)
```

True

```
>> {1, 2, 3} < {1, 2, 3}
```

```
{1, 2, 3} < {1, 2, 3}
```

```
>> E == N(E)
```

True

```
>> {Equal(Equal(0, 0), True), Equal(0, 0) == True}
```

```
{True, True}
```

```
>> {Mod(6, 2) == 0, Mod(6, 4) == 0, (Mod(6, 2) == 0) == (Mod(6, 4) == 0), (Mod(6, 2) ==
```

```
{True,False,False,True}
```

```
>> a == a == a
```

True

```
>> {Equal(), Equal(x), Equal(1)}
```

```
{True, True, True}
```

Related terms

[SameQ](#), [Unequal](#), [UnsameQ](#)

Implementation status

- - full supported

Github

- [Implementation of Equal](#)
-

Equivalent

```
Equivalent(arg1, arg2, ...)
```

Equivalence relation. `Equivalent(A, B)` is `True` iff `A` and `B` are both `True` or both `False`. Returns `True` if all of the arguments are logically equivalent. Returns `False` otherwise. `Equivalent(arg1, arg2, ...)` is equivalent to `(arg1 && arg2 && ...)` `|| (!arg1 && !arg2 && ...)`.

See

- [Wikipedia - Logical equivalence](#)

Examples

```
>> Equivalent(True, True, False)
False

>> Equivalent(x, x && True)
True
```

If all expressions do not evaluate to `True` or `False`, `Equivalent` returns a result in symbolic form:

```
>> Equivalent(a, b, c)
Equivalent(a,b,c)
```

Otherwise, `Equivalent` returns a result in DNF

```
>> Equivalent(a, b, True, c)
a && b && c
>> Equivalent()
True
>> Equivalent(a)
True
```

Implementation status

- - full supported

Github

- [Implementation of Equivalent](#)

Erf

`Erf(z)`

returns the error function of `z`.

See

- [Wikipedia - Error function](#)
- [Fungrim - Error functions](#)

Examples

`Erf(z)` is an odd function:

```
>> Erf(-x)
-Erf(x)

>> Erf(1.0)
0.8427007929497151

>> Erf(0)
0

>> {Erf(0, x), Erf(x, 0)}
{Erf(x), -Erf(x)}
```

Implementation status

- - partially implemented

Github

- [Implementation of Erf](#)
-

Erfc

Erfc(z)

returns the complementary error function of `z`.

See

- [Wikipedia - Error function - Complementary error function](#)
- [Fungrim - Error functions](#)

Examples

`Erf(z)` is an odd function:

```
>> Erfc(-x) / 2
1/2*(2-Erfc(x))

>> Erfc(1.0)
0.15729920705028488

>> Erfc(0)
1
```

Implementation status

- - partially implemented

Github

- [Implementation of Erfc](#)
-

Erfi

`Erfi(z)`

returns the imaginary error function of `z`.

See

- [Wikipedia - Error function](#)
- [Fungrim - Error functions](#)

Examples

```
>> Erfi(-42*I*x)
-I*Erf(42*x)
```

Implementation status

- - partially implemented

Github

- [Implementation of Erfi](#)
-

ErlangDistribution

```
ErlangDistribution({k, lambda})
```

returns a Erlang distribution.

See:

- [Wikipedia - Erlang distribution](#)

Examples

The variance of the Erlang distribution is

```
>> Variance(ErlangDistribution(n, m))  
n/m^2
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of ErlangDistribution](#)
-

EuclideanDistance

```
EuclideanDistance(u, v)
```

returns the euclidean distance between `u` and `v`.

See

- [Wikipedia - Euclidean distance](#)
- [Wikipedia - Metric_space](#)

Examples

```
>> EuclideanDistance({-1, -1}, {1, 1})  
2*Sqrt(2)  
  
>> EuclideanDistance({a, b}, {c, d})  
Sqrt(Abs(a-c)^2+Abs(b-d)^2)
```

Related terms

[FindClusters](#), [BinaryDistance](#), [BrayCurtisDistance](#), [ChessboardDistance](#), [CanberraDistance](#), [CosineDistance](#), [ManhattanDistance](#), [SquaredEuclideanDistance](#)

Implementation status

- - full supported

Github

- [Implementation of EuclideanDistance](#)
-

EulerE

```
EulerE(n)
```

gives the euler number E_n .

```
EulerE(n, x)
```

gives the Eulerian polynomial.

See

- [Wikipedia - Eulerian polynomials](#)
- [Wikipedia - Euler number](#)

Examples

```
>> EulerE(6)
-61

>> EulerE(10, z)
-155*z+255*z^3-126*z^5+30*z^7-5*z^9+z^10
```

Github

- [Implementation of EulerE](#)
-

EulerGamma

EulerGamma

Euler-Mascheroni constant

See

- [Wikipedia - Euler-Mascheroni constant](#)
- [Fungrim - Euler's constant](#)

Examples

```
>> N(EulerGamma)
0.577216
```

Related terms

[E](#), [Glaisher](#), [Khinchin](#), [Pi](#)

Implementation status

- - full supported

Github

- [Implementation of EulerGamma](#)
-

EulerianGraphQ

```
EulerianGraphQ(graph)
```

returns `True` if `graph` is an eulerian graph, and `False` otherwise.

See:

- [Wikipedia - Eulerian_pat](#)

Examples

```
>> EulerianGraphQ(Graph({1 -> 2, 2 -> 3, 3 -> 1, 1 -> 3, 3 -> 4, 4 -> 1}))
True
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of EulerianGraphQ](#)
-

EulerPhi

```
EulerPhi(n)
```

compute Euler's totient function.

See:

- [Wikipedia - Euler's totient function](#)
- [Fungrim - Totient function](#)

Examples

```
>> EulerPhi(10)
4
```

Implementation status

- - partially implemented

Github

- [Implementation of EulerPhi](#)
-

Evaluate

```
Evaluate(expr)
```

the `Evaluate` function will be executed even if the function attributes `HoldFirst`, `HoldRest`, `HoldAll` are set for the function head.

Implementation status

- - full supported

Github

- [Implementation of Evaluate](#)
-

EvenQ

```
EvenQ(x)
```

returns `True` if `x` is even, and `False` otherwise.

```
EvenQ(x, GaussianIntegers->True)
```

returns `True` if `x` is even and a Gaussian integer number, and `False` otherwise.

See

- [Wikipedia - Parity \(mathematics\)](#)

Examples

```
>> EvenQ(4)
True

>> EvenQ(-3)
False

>> EvenQ(n)
False

>> EvenQ(2+4*I, GaussianIntegers->True)
True

>> EvenQ(1+I, GaussianIntegers->True)
False
```

Related terms

[Odd](#)

Implementation status

- - full supported

Github

- [Implementation of EvenQ](#)
-

ExactNumberQ

```
ExactNumberQ(expr)
```

returns `True` if `expr` is an exact number, and `False` otherwise.

Examples

```
>> ExactNumberQ(10)
True

>> ExactNumberQ(4.0)
False

>> ExactNumberQ(n)
False

>> ExactNumberQ(1+I)
True

>> ExactNumberQ(1 + 1. * I)
False
```

Github

- [Implementation of ExactNumberQ](#)
-

Except

```
Except(c)
```

represents a pattern object that matches any expression except those matching `c`.

```
Except(c, p)
```

represents a pattern object that matches `p` but not `c`.

Examples

```
>> Cases({x, a, b, x, c}, Except(x))  
{a,b,c}
```

```
>> Cases({a, 0, b, 1, c, 2, 3}, Except(1, _Integer))  
{0,2,3}
```

`Except` can also be used for string expressions:

```
>> StringReplace("Hello world!", Except(LetterCharacter) -> "")  
Helloworld
```

Exp

`Exp(z)`

the exponential function E^z .

See

- [Wikipedia - Exponential function](#)
- [Fungrim - Exponential function](#)

Examples

```
>> Exp(1)
E

>> Exp(10.0)
22026.465794806703

>> Exp(x) //FullForm
"Power(E, x)"
```

Implementation status

- - full supported

Github

- [Implementation of Exp](#)
-

Expand

Expand(expr)

expands out positive rational powers and products of sums in `expr`.

See:

- [Wikipedia - Multinomial theorem](#)

Examples

```
>> Expand((x + y) ^ 3)
x^3+3*x^2*y+3*x*y^2+y^3

>> Expand((a + b) (a + c + d))
a^2+a*b+a*c+b*c+a*d+b*d

>> Expand((a + b) (a + c + d) (e + f) + e a a)
2*a^2*e+a*b*e+a*c*e+b*c*e+a*d*e+b*d*e+a^2*f+a*b*f+a*c*f+b*c*f+a*d*f+b*d*f

>> Expand((a + b) ^ 2 * (c + d))
a^2*c+2*a*b*c+b^2*c+a^2*d+2*a*b*d+b^2*d

>> Expand((x + y) ^ 2 + x y)
x^2+3*x*y+y^2

>> Expand(((a + b) (c + d)) ^ 2 + b (1 + a))
a^2*c^2+2*a*b*c^2+b^2*c^2+2*a^2*c*d+4*a*b*c*d+2*b^2*c*d+a^2*d^2+2*a*b*d^2+b^2*d^2+b(1+a)
```

`Expand` expands out rational powers by expanding the `Floor()` part of the rational powers number:

```
>> Expand((x + 3)^(5/2)+(x + 1)^(3/2))
Sqrt(1+x)+x*Sqrt(1+x)+9*Sqrt(3+x)+6*x*Sqrt(3+x)+x^2*Sqrt(3+x)
```

`Expand` expands items in lists and rules:

```
>> Expand({4 (x + y), 2 (x + y) -> 4 (x + y)})
{4*x+4*y, 2*(x+y)->4*(x+y)}
```

`Expand` does not change any other expression.

```
>> Expand(Sin(x*(1 + y)))  
Sin(x*(1+y))  
  
>> a*(b*(c+d)+e) // Expand  
a*b*c+a*b*d+a*e  
  
>> (y^2)^(1/2)/(2x+2y)//Expand  
Sqrt(y^2)/(2*x+2*y)  
  
>> 2(3+2x)^2/(5+x^2+3x)^3 // Expand  
18/(5+3*x+x^2)^3+(24*x)/(5+3*x+x^2)^3+(8*x^2)/(5+3*x+x^2)^3
```

Related terms

[ExpandAll](#), [ExpandDenominator](#), [ExpandNumerator](#)

Implementation status

- - partially implemented

Github

- [Implementation of Expand](#)
-

ExpandAll

```
ExpandAll(expr)
```

expands out all positive integer powers and products of sums in `expr`.

Examples

```
>> ExpandAll((a + b) ^ 2 / (c + d)^2)
(a^2+2*a*b+b^2)/(c^2+2*c*d+d^2)
```

`ExpandAll` descends into sub expressions

```
>> ExpandAll((a + Sin(x*(1 + y)))^2)
a^2+Sin(x+x*y)^2+2*a*Sin(x+x*y)
```

`ExpandAll` also expands heads

```
>> ExpandAll(((1 + x)(1 + y))[x])
(1+x+y+x*y)[x]
```

Related terms

[Expand](#), [ExpandDenominator](#), [ExpandNumerator](#)

Implementation status

- - partially implemented

Github

- [Implementation of ExpandAll](#)
-

ExpandDenominator

```
ExpandDenominator(expr)
```

expands the denominator of `expr`.

Examples

```
>> ExpandDenominator((x+y)*(x-y)/((x+1)*y))  
((x-y)*(x+y))/(y+x*y)
```

Related terms

[Expand](#), [ExpandAll](#), [ExpandNumerator](#)

Implementation status

- - full supported

Github

- [Implementation of ExpandDenominator](#)
-

ExpandNumerator

```
ExpandNumerator(expr)
```

expands the numerator of `expr`.

Examples

```
>> ExpandNumerator((x+y)*(x-y)/((x+1)*y))  
(x^2-y^2)/((1+x)*y)
```

Related terms

[Expand](#), [ExpandAll](#), [ExpandDenominator](#)

Implementation status

- - full supported

Github

- [Implementation of ExpandNumerator](#)
-

Expectation

```
Expectation(pure-function, data-set)
```

returns the expected value of the `pure-function` for the given `data-set`.

See

- [Wikipedia - Expected value](#)

Examples

```
>> Expectation((#^3)&, {a,b,c})  
1/3*(a^3+b^3+c^3)  
  
>> Expectation(2*x+3,Distributed(x,{a,b,c,d}))  
1/4*(12+2*a+2*b+2*c+2*d)
```

Implementation status

- - experimental

Github

- [Implementation of Expectation](#)
-

ExpIntegralE

```
ExpIntegralE(n, expr)
```

returns the exponential integral $E_n(\text{expr})$ of expr .

See

- [Wikipedia - Exponential integral](#)

Examples

```
>> ExpIntegralE(0, z)
1/(E^z*z)
```

Implementation status

- - full supported

Github

- [Implementation of ExpIntegralE](#)
-

ExpIntegralEi

```
ExpIntegralEi(expr)
```

returns the exponential integral $Ei(expr)$ of $expr$.

See

- [Wikipedia - Exponential integral](#)

Examples

```
>> ExpIntegralEi(0)  
-Infinity
```

Implementation status

- - full supported

Github

- [Implementation of ExpIntegralEi](#)
-

Exponent

```
Exponent(polynomial, x)
```

gives the maximum power with which *x* appears in the expanded form of *polynomial*.

See

- [Wikipedia - Exponentiation](#)

Examples

```
>> Exponent(1+x^2+a*x^3, x)
3
```

Related terms

[Coefficient](#), [CoefficientList](#)

Implementation status

- - full supported

Github

- [Implementation of Exponent](#)
-

ExponentialDistribution

```
ExponentialDistribution(lambda)
```

returns an exponential distribution.

See:

- [Wikipedia - Exponential distribution](#)

Examples

The variance of the exponential distribution is

```
>> Variance(ExponentialDistribution(n))  
1/n^2
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of ExponentialDistribution](#)
-

Export

```
Export("path-to-filename", expression, "WXF")
```

if the file system is enabled, export the `expression` in WXF format to the "path-to-filename" file.

```
Export("path-to-filename", expression, "Table")
```

if the file system is enabled, export the `expression` in table format to the "path-to-filename" file.

```
Export("path-to-filename", graph, "GraphML")
```

or

```
Export("path-to-filename", graph, "DOT")
```

if the file system is enabled, export the `graph` in `GraphML` or `DOT` format to the "path-to-filename" file.

Examples

Export a graph:

```
>> Export("c:\\temp\\dotgraph.dot", Graph({1 \[DirectedEdge] 2, 2 \[DirectedEdge] 3, 3 \[DirectedEdge] 1}, c:\\temp\\dotgraph.dot))
```

Related terms

[BinaryDeserialize](#), [BinarySerialize](#), [ByteArray](#), [ByteArrayQ](#), [Import](#)

Implementation status

- - partially implemented

Github

- [Implementation of Export](#)
-

ExportString

```
ExportString(string, export-format)
```

export the `string` in `export-format`.

Supported formats: Base64, ExpressionJSON, JSON, Table

Examples

Export a string in base 64 format:

```
>> ExportString("Hello world", "Base64")
SGVsbG8gd29ybGQ=

>> ExportString({2.1+I*3.4}, "ExpressionJSON")
["List", ["Complex", 2.1, 3.4]]
```

Related terms

[Export](#), [Import](#), [ImportString](#)

Implementation status

- - partially implemented

Github

- [Implementation of ExportString](#)
-

ExtendedGCD

```
ExtendedGCD(n1, n2, ...)
```

computes the extended greatest common divisor of the given integers.

See:

- [Wikipedia: Extended Euclidean algorithm](#)
- [Wikipedia: Bézout's identity](#)

Examples

```
>> ExtendedGCD(10, 15)
{5, {-1, 1}}
```

ExtendedGCD works with any number of arguments:

```
>> ExtendedGCD(10, 15, 7)
{1, {-3, 3, -2}}
```

Compute the greatest common divisor and check the result:

```
>> numbers = {10, 20, 14};

>> {gcdVal, factors} = ExtendedGCD(Sequence @@ numbers)
{2, {3, 0, -2}}

>> Plus @@ (numbers * factors)
2
```

Implementation status

- - partially implemented

Github

- [Implementation of ExtendedGCD](#)
-

Extract

```
Extract(expr, list)
```

extracts parts of `expr` specified by `list`.

```
Extract(expr, {list1, list2, ...})
```

extracts a list of parts.

Examples

`Extract(expr, i, j, ...)` is equivalent to `Part(expr, {i, j, ...})`.

```
>> Extract(a + b + c, {2})
b

>> Extract({{a, b}, {c, d}}, {{1}, {2, 2}})
{{a,b},d}
```

Implementation status

- - full supported

Github

- [Implementation of Extract](#)
-

Factor

```
Factor(expr)
```

factors the polynomial expression `expr`

```
Factor(expr, GaussianIntegers->True)
```

for gaussian integers you can set the option `GaussianIntegers->True`

```
Factor(expr, Modulus->p)
```

to treat integers modulo an integer number `p`, you can set the option `Modulus->p`.

See:

- [Wikipedia - Factorization of polynomials](#)
- [Wikipedia - Quadratic formula](#)
- [Wikipedia - Cubic equation](#)
- [Wikipedia - Quartic function](#)
- [Wikipedia - Gaussian integer](#)

Examples

```
>> Factor(1+2*x+x^2, x)
(1+x)^2

>> Factor(x^4-1, GaussianIntegers->True)
(x-1)*(x+1)*(x-I)*(x+I)

>> Factor(x^3 + 3*x^2*y + 3*x*y^2 + y^3)
(x+y)^3
```

Factor can also be used with equations:

```
>> Factor(x*a == x*b+x*c)
a*x==(b+c)*x
```

and lists:

```
>> Factor({x + x^2, 2*x + 2*y + 2})
{x*(1+x), 2*(1+x+y)}
```


Implementation status

- - full supported

Github

- [Implementation of Factor](#)
-

Factorial

`Factorial(n)`

$n!$

returns the factorial number of the integer `n`

See

- [Wikipedia - Factorial](#)
- [Fungrim - Factorials and binomial coefficients](#)

Examples

```
>> Factorial(3)
6

>> 4!
24

>> 10.5!
1.1899423083962249E7

>> !a! //FullForm
Not(Factorial(a))
```

Implementation status

- - full supported

Github

- [Implementation of Factorial](#)
-

Factorial2

```
Factorial2(n)
```

```
n!!
```

returns the double factorial number of the integer `n` as `n*(n-2)*(n-4)...`.

See

- [Wikipedia - Double Factorial](#)

Examples

```
>> Factorial2(3)
3

>> Factorial2(9)
945

>> Factorial2(10)
3840
```

Implementation status

- - full supported

Github

- [Implementation of Factorial2](#)
-

FactorialPower

FactorialPower(v , n)

The `FactorialPower` implements the falling factorial. The falling factorial (sometimes called the descending factorial, falling sequential product, or lower factorial) is defined as the polynomial $v*(v-1)*(v-2)*\dots*(v-n+1)$.

See

- [Wikipedia - Falling and rising factorials](#)

Examples

```
>> FactorialPower(v,4) // FunctionExpand  
(-3+v)*(-2+v)*(-1+v)*v
```

Implementation status

- - experimental

Github

- [Implementation of FactorialPower](#)
-

FactorInteger

```
FactorInteger(n)
```

returns the factorization of `n` as a list of factors and exponents.

```
FactorInteger(n, GaussianIntegers->True)
```

for gaussian integers you can set the option `GaussianIntegers->True`

See

- [Wikipedia - Integer factorization](#)
- [Wikipedia - Prime number](#)

Examples

```
>> FactorInteger(990)
{{2, 1}, {3, 2}, {5, 1}, {11, 1}}

>> FactorInteger(341550071728321)
{{10670053, 1}, {32010157, 1}}

>> factors = FactorInteger(2010)
{{2, 1}, {3, 1}, {5, 1}, {67, 1}}
```

To get back the original number:

```
>> Times @@ Power @@@ factors
2010
```

`FactorInteger` factors rational numbers using negative exponents:

```
>> FactorInteger(2010 / 2011)
{{2, 1}, {3, 1}, {5, 1}, {67, 1}, {2011, -1}}
```

Gaussian integers:

```
>> FactorInteger(10+30*I, GaussianIntegers->True)
{{-1, 1}, {1+I, 3}, {1+I*2, 1}, {2+I, 2}}

>> FactorInteger(11+14*I, GaussianIntegers->True)
{{11+I*14, 1}}
```

Implementation status

- - full supported

Github

- [Implementation of FactorInteger](#)
-

FactorSquareFree

```
FactorSquareFree(polynomial)
```

factor the polynomial expression `polynomial` square free.

Implementation status

- - full supported

Github

- [Implementation of FactorSquareFree](#)
-

FactorSquareFreeList

```
FactorSquareFreeList(polynomial)
```

get the square free factors of the polynomial expression `polynomial`.

Implementation status

- - full supported

Github

- [Implementation of FactorSquareFreeList](#)
-

FactorTerms

```
FactorTerms(poly)
```

pulls out any overall numerical factor in `poly`.

Examples

```
>> FactorTerms(3+3/4*x^3+12/17*x^2, x)
3/68*(17*x^3+16*x^2+68)
```

Implementation status

- - full supported

Github

- [Implementation of FactorTerms](#)
-

FactorTermsList

```
FactorTermsList(poly)
```

pulls out any overall numerical factor in `poly` and returns the result in a list.

Examples

```
>> FactorTermsList(3+3/4*x^3+12/17*x^2)
{3/68, 68+16*x^2+17*x^3}
```

Implementation status

- - full supported

Github

- [Implementation of FactorTermsList](#)
-

False

```
False
```

the constant `False` represents the boolean value **false**

See

- [Wikipedia - Truth value](#)

Examples

```
>> Pi < E  
False
```

Github

- [Implementation of False](#)
-

Fibonacci

```
Fibonacci(n)
```

returns the Fibonacci number of the integer `n`

```
Fibonacci(n, var)
```

returns the `n`th Fibonacci polynomial for variable `var`.

See

- [Wikipedia - Fibonacci number](#)
- [Wikipedia - Fibonacci polynomials](#)
- [Fungrim - Fibonacci numbers](#)

Examples

```
>> Fibonacci(0)
0

>> Fibonacci(1)
1

>> Fibonacci(10)
55

>> Fibonacci(-52)
-32951280099

>> Fibonacci(200)
280571172992510140037611932413038677189525

>> Fibonacci(50, x)
25*x+2600*x^3+80730*x^5+1184040*x^7+10015005*x^9+54627300*x^11+206253075*x^13+
565722720*x^15+1166803110*x^17+1855967520*x^19+2319959400*x^21+2310789600*x^23+
1852482996*x^25+1203322288*x^27+635745396*x^29+273438880*x^31+95548245*x^33+
26978328*x^35+6096454*x^37+1086008*x^39+148995*x^41+15180*x^43+1081*x^45+48*x^47+x^49
```

Implementation status

- - full supported

Github

- [Implementation of Fibonacci](#)
-

FileHash

```
FileHash(file)
```

computes an MD5 hash for the contents of the specified `file`. The `FileHash` function computes a cryptographic hash for the contents of a `file`. It is useful for verifying file integrity and detecting changes.

```
FileHash(file, hash-type)
```

computes a hash of the specified `hash-type` (e.g., "SHA-256") for the file.

Examples

```
>> FileHash(StringToStream("Hello World"), "MD5")
235328152096874191772633713977838157797
```

Related terms

[Hash](#)

Implementation status

- - full supported

Github

- [Implementation of FileHash](#)
-

FileNames

```
FileNames( )
```

returns a list with the filenames in the current working folder..

```
FileNames(string-pattern)'
```

returns a list with the filenames in the current working folder that matches with `string-pattern`.

Examples

```
>> FileNames()  
  
>> FileNames("*.m")
```

FilePrint

```
FilePrint(file)
```

prints the raw contents of `file`.

Examples

```
>> f = FileNameJoin({$TemporaryDirectory, \"test_open.txt\"})  
/tmp/test_open.txt  
  
>> str = OpenWrite(f);  
  
>> Write(str, x^2 - y^2)  
  
>> Close(str);
```

`FilePrint` prints the file content:

```
>> FilePrint(f)
```

as:

```
x^2 - y^2
```

Implementation status

- - partially implemented

Github

- [Implementation of FilePrint](#)
-

FilterRules

```
FilterRules(list-of-option-rules, list-of-rules)
```

or

```
FilterRules(list-of-option-rules, list-of-symbols)
```

filter the `list-of-option-rules` by `list-of-rules` or `list-of-symbols`.

Examples

```
>> Options(f) = {a -> 1, b -> 2}
{a->1,b->2}

>> FilterRules({b -> 3, MaxIterations -> 5}, Options(f))
{b->3}
```

Related terms

[Options](#)

Implementation status

- - full supported

Github

- [Implementation of FilterRules](#)
-

FindClusters

```
FindClusters(list-of-data-points, k)
```

Clustering algorithm based on David Arthur and Sergei Vassilvitski k-means++ algorithm. Create `k` number of clusters to split the `list-of-data-points` into.

```
FindClusters(list-of-data-points, eps, minPts, Method->"DBSCAN", DistanceFunction->Eucl
```

Use the **density-based spatial clustering of applications with noise (DBSCAN)** algorithm as clustering `Method`. `eps` is the maximum radius of the neighborhood to be considered. `minPts` is the minimum number of points needed for a cluster. The `DistanceFunction` defines the function which should be used to measure the distance between 2 points.

See:

- [Wikipedia - K-means++](#)
- [Wikipedia - K-means clustering](#)
- [Wikipedia - DBSCAN](#)

Examples

```
>> FindClusters({1, 2, 3, 4, 5, 6, 7, 8, 9})
{{1.0,2.0,3.0},{6.0,7.0,8.0,9.0},{4.0,5.0}}

>> FindClusters({{83.08303244924173,58.83387754182331},{45.05445510940626,23.4696426496
{{73.5319,34.89615},{73.28498,33.96861},{73.45828,33.92584},{73.96579,35.73191},{74.00
```

Related terms

[BinaryDistance](#), [BrayCurtisDistance](#), [ChessboardDistance](#), [CanberraDistance](#), [CosineDistance](#), [EuclideanDistance](#), [ManhattanDistance](#), [SquaredEuclideanDistance](#)

Implementation status

- - full supported

Github

- [Implementation of FindClusters](#)
-

FindCycle

```
FindCycle(graph)
```

Find a cycle in the given `graph`.

Examples

```
>> FindCycle({2 -> 1, 1 -> 4, 3 -> 2, 2 -> 5, 6 -> 3, 5 -> 4, 4 -> 7, 6 -> 5, 8 -> 5, 6  
{8->5,5->4,4->7,7->10,10->11,11->8}}}
```

Implementation status

- - partially implemented

Github

- [Implementation of FindCycle](#)
-

FindEulerianCycle

```
FindEulerianCycle(graph)
```

find an eulerian cycle in the `graph`.

See

- [Wikipedia - Eulerian path](#)

Examples

```
>> FindEulerianCycle(Graph({1 -> 2, 2 -> 3, 3 -> 4, 4 -> 1}))  
{4->1,1->2,2->3,3->4}  
  
>> FindEulerianCycle(Graph({1 -> 2, 2 -> 3, 3 -> 4, 3 -> 1}))  
{}
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindShortestTour](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of FindEulerianCycle](#)
-

FindFit

```
FindFit(list-of-data-points, function, parameters, variable)
```

solve a least squares problem using the Levenberg-Marquardt algorithm.

See:

- [Wikipedia - Levenberg–Marquardt algorithm](#)

Examples

```
>> FindFit({{15.2,8.9},{31.1,9.9},{38.6,10.3},{52.2,10.7},{75.4,11.4}}, a*Log(b*x), {a,
{a->1.54503,b->20.28258}

>> FindFit({{1,1},{2,4},{3,9},{4,16}}, a+b*x+c*x^2, {a, b, c}, x)
{a->0.0,b->0.0,c->1.0}
```

The default initial guess in the following example for the parameters `{a,w,f}` is `{1.0, 1.0, 1.0}`. These initial values give a bad result:

```
>> FindFit(Table({t, 3*Sin(3*t + 1)}), {t, -3, 3, 0.1}), a* Sin(w*t + f), {a,w,f}, t)
{a->0.6688,w->1.49588,f->3.74845}
```

The initial guess `{2.0, 1.0, 1.0}` gives a much better result:

```
>> FindFit(Table({t, 3*Sin(3*t + 1)}), {t, -3, 3, 0.1}), a* Sin(w*t + f), {{a, 2}, {w,1}
{a->3.0,w->3.0,f->1.0}
```

You can omit `1.0` in the parameter list because it's the default value:

```
>> FindFit(Table({t, 3*Sin(3*t + 1)}), {t, -3, 3, 0.1}), a* Sin(w*t + f), {{a, 2}, w, f}
{a->3.0,w->3.0,f->1.0}
```

Related terms

[Fit](#), [FittedModel](#), [LinearModelFit](#)

Implementation status

- - full supported

Github

- [Implementation of FindFit](#)
-

FindGeneratingFunction

```
FindGeneratingFunction({i1, i2, i3, ...}, var)
```

searches for a unary generating function applied to the variable `var`, where the series coefficients equals `{i1, i2, i3, ...}`.

Note: The algorithm of this function currently only calculates the Pade approximant to find a generating function.

See

- [Wikipedia - Generating function](#)
- [Wikipedia - Pade approximant](#)

Examples

```
>> FindGeneratingFunction({1, 2, 3, 4}, x)
1/(1-2*x+x^2)
```

Implementation status

- - full supported

Github

- [Implementation of FindGeneratingFunction](#)
-

FindGraphIsomorphism

```
FindGraphIsomorphism(graph1, graph2)
```

returns an isomorphism between `graph1` and `graph2` if it exists. Return an empty list if no isomorphism exists.

See:

- [Wikipedia - Graph isomorphism](#)

Examples

```
>> FindGraphIsomorphism(Graph({1, 2, 3, 4}, {1<->2, 1<->4, 2<->3, 3<->4}), Graph({1, 2, 3, 4}, {1<->2, 2<->3, 3<->1, 4<->4|>})
```

Related terms

[IsomorphicGraphQ](#), [GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of FindGraphIsomorphism](#)
-

FindHamiltonianCycle

```
FindHamiltonianCycle(graph)
```

find an hamiltonian cycle in the `graph`.

See

- [Wikipedia - Hamiltonian path](#)
- [Wikipedia - Hamiltonian path problem](#)

Examples

```
>> FindHamiltonianCycle( {1 -> 2, 2 -> 3, 3 -> 4, 4 -> 1} )  
{1->2,2->3,3->4,4->1}
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindShortestTour](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of FindHamiltonianCycle](#)
-

FindInstance

```
FindInstance(equations, vars)
```

attempts to find one solution which solves the `equations` for the variables `vars`.

Examples

```
>> FindInstance({x^2==4,x+y^2==6}, {x,y})  
{{x->-2,y->-2*Sqrt(2)}}
```

Related terms

[Solve](#)

Implementation status

- - partially implemented

Github

- [Implementation of FindInstance](#)
-

FindLinearRecurrence

```
FindLinearRecurrence(list)
```

compute a minimal linear recurrence which returns list.

See

- [Wikipedia - Recurrence relation](#)
- [OEIS - Recurrence - Linear recurrences with constant coefficients](#)

Examples

```
>> FindLinearRecurrence({1, 2, 4, 7, 28, 128, 582, 2745, 13021, 61699, 292521, 1387138}  
{5, -2, 6, -11}
```

Implementation status

- - partially implemented

Github

- [Implementation of FindLinearRecurrence](#)
-

FindMaximum

```
FindMaximum(f, {x, xstart})
```

searches for a local numerical maximum of f for the variable x and the start value $xstart$.

```
FindMaximum(f, {x, xstart}, Method->methodName)
```

searches for a local numerical maximum of f for the variable x and the start value $xstart$, with one of the following method names:

```
FindMaximum(f, {{x, xstart},{y, ystart},...})
```

searches for a local numerical maximum of the multivariate function f for the variables $x, y, \dots$ and the corresponding start values $xstart, ystart, \dots$.

See

- [Wikipedia - Mathematical optimization](#)
- [Wikipedia - Rosenbrock function](#)

"Powell"

Implements the [Powell](#) optimizer.

This is the default method, if no `Method` is set.

"ConjugateGradient"

Implements the [Non-linear conjugate gradient](#) optimizer.

This is a derivative based method and the functions must be symbolically differentiable.

"SequentialQuadratic"

Implements the [Sequential Quadratic Programming](#) optimizer.

This is a derivative, multivariate based method and the functions must be symbolically differentiable.

"BOBYQA"

Implements [Powell's BOBYQA](#) optimizer (Bound Optimization BY Quadratic Approximation).

The "BOBYQA" method falls back to "CMAES" if the objective function has dimension 1.

"CMAES"

Implements the [Covariance Matrix Adaptation Evolution Strategy \(CMA-ES\)](#) optimizer.

Examples

```
>> FindMaximum(Sin(x), {x, 0.5})  
{1.0, {x->1.5708}}  
  
>> FindMaximum({(1-x)^2+100*(y-x^2)^2, x >= -2.0 && 2.0 >= x && y >= -0.5 && 1.5 >= y},  
{2034.0, {x->-2.0, y->-0.5}}}
```

Related terms

[FindMinimum](#), [FindRoot](#), [NRoots](#), [Solve](#)

Implementation status

- - partially implemented

Github

- [Implementation of FindMaximum](#)
-

FindMinimum

```
FindMinimum(f, {x, xstart})
```

searches for a local numerical minimum of `f` for the variable `x` and the start value `xstart`.

```
FindMinimum(f, {x, xstart}, Method->methodName)
```

searches for a local numerical minimum of `f` for the variable `x` and the start value `xstart`, with one of the following method names:

```
FindMinimum(f, {{x, xstart},{y, ystart},...})
```

searches for a local numerical minimum of the multivariate function `f` for the variables `x, y, ...` and the corresponding start values `xstart, ystart, ...`.

See

- [Wikipedia - Mathematical optimization](#)
- [Wikipedia - Rosenbrock function](#)

"Powell"

Implements the [Powell](#) optimizer.

This is the default method, if no `Method` is set.

"ConjugateGradient"

Implements the [Non-linear conjugate gradient](#) optimizer.

This is a derivative based method and the functions must be symbolically differentiable.

"SequentialQuadratic"

Implements the [Sequential Quadratic Programming](#) optimizer.

This is a derivative, multivariate based method and the functions must be symbolically differentiable.

"BOBYQA"

Implements [Powell's BOBYQA](#) optimizer (Bound Optimization BY Quadratic Approximation).

The "BOBYQA" method falls back to "CMAES" if the objective function has dimension 1.

"CMAES"

Implements the [Covariance Matrix Adaptation Evolution Strategy \(CMA-ES\)](#) optimizer.

Examples

```
>> FindMinimum(Sin(x), {x, 0.5})  
{-1.0, {x->-1.5708}}  
  
>> FindMinimum(Sin(x)*Sin(2*y), {{x, 2}, {y, 2}}, Method -> "ConjugateGradient")  
{-1.0, {x->1.5708, y->2.35619}}
```

Related terms

[FindMaximum](#), [FindRoot](#), [NRoots](#), [Solve](#)

Implementation status

- - partially implemented

Github

- [Implementation of FindMinimum](#)
-

FindPermutation

```
FindPermutation(list1, list2)
```

create a `Cycles({{...},{...}, ...})` permutation expression, for two lists whose arguments are the same but may be differently arranged.

See

- [Wikipedia - Permutation](#)

Examples

```
>> FindPermutation(CharacterRange("a","d"), {"a","d","c","b"})  
Cycles({{2,4}})
```

Related terms

[Cycles](#), [FindPermutation](#), [PermutationCycles](#), [PermutationCyclesQ](#), [PermutationList](#), [PermutationListQ](#), [PermutationReplace](#), [Permutations](#), [Permute](#)

Implementation status

- - full supported

Github

- [Implementation of FindPermutation](#)
-

FindRoot

```
FindRoot(f, {x, xmin, xmax})
```

searches for a numerical root of f for the variable x , in the range $xmin$ to $xmax$.

```
FindRoot(f, {x, xmin, xmax}, MaxIterations->maxiter)
```

searches for a numerical root of f for the variable x , with $maxiter$ iterations. The default maximum iteration is 100 .

```
FindRoot(f, {x, xmin, xmax}, AccuracyGoal->n)
```

searches for a numerical root of f for the variable x , with accuracy 10^{-n}

```
FindRoot(f, {x, xmin, xmax}, Method->methodName)
```

searches for a numerical root of f for the variable x , with one of the method names listed below.

```
FindRoot({f(x1,x2,...), g(x1,x2,...), ...}, {{x1, initialValue1}, {x2, initialValue2},
```

searches a multivariate root with Newton's iteration method for a differentiable, multivariate, vector-valued function.

See

- [Wikipedia - Zero of a function](#)
- [Wikipedia - Root-finding algorithm](#)
- [Wikipedia - Newton's method - k_variables, _k_functions](#)
- [Wikipedia - Laguerre's method](#)

Brent

Implements the Brent algorithm for finding zeros of real univariate functions (`BracketingNthOrderBrentSolver`). The function should be continuous but not necessarily smooth. The solve method returns a zero x of the function f in the given interval $[xmin, xmax]$.

This is the default method, if no `methodName` is given.

Newton

Implements Newton's method for finding zeros of real univariate functions. The function should be continuous but not necessarily smooth.

Bisection

Implements the bisection algorithm for finding zeros of univariate real functions. The function should be continuous but not necessarily smooth.

Muller

Implements the Muller's Method for root finding of real univariate functions. For reference, see Elementary Numerical Analysis, ISBN 0070124477, chapter 3. Muller's method applies to both real and complex functions, but here we restrict ourselves to real functions. Muller's original method would have function evaluation at complex point. Since our $f(x)$ is real, we have to find ways to avoid that. Bracketing condition is one way to go: by requiring bracketing in every iteration, the newly computed approximation is guaranteed to be real. Normally Muller's method converges quadratically in the vicinity of a zero, however it may be very slow in regions far away from zeros. For example, `FindRoot(Exp(x)-1 == 0, {x, -50, 100}, Method->Muller)`. In such case we use bisection as a safety backup if it performs very poorly. The formulas here use divided differences directly.

Ridders

Implements the Ridders' Method for root finding of real univariate functions. For reference, see C. Ridders, A new algorithm for computing a single root of a real continuous function, IEEE Transactions on Circuits and Systems, 26 (1979), 979 - 980. The function should be continuous but not necessarily smooth.

Secant

Implements the Secant method for root-finding (approximating a zero of a univariate real function). The solution that is maintained is not bracketed, and as such convergence is not guaranteed.

RegulaFalsi

Implements the Regula Falsi or False position method for root-finding (approximating a zero of a univariate real function). It is a modified Secant method. The Regula Falsi method is included for completeness, for testing purposes, for educational purposes, for comparison to other algorithms, etc. It is however not intended to be used for actual problems, as one of the bounds often remains fixed, resulting in very slow convergence. Instead, one of the well-known modified Regula Falsi algorithms can be used (Illinois or Pegasus). These two algorithms solve the fundamental issues of the original Regula Falsi algorithm, and greatly out-performs it for most, if not all, (practical) functions. Unlike the Secant method, the Regula Falsi guarantees convergence, by maintaining a bracketed solution. Note however, that due to the finite/limited

precision of Java's double type, which is used in this implementation, the algorithm may get stuck in a situation where it no longer makes any progress. Such cases are detected and result in a `ConvergenceException` exception being thrown. In other words, the algorithm theoretically guarantees convergence, but the implementation does not. The Regula Falsi method assumes that the function is continuous, but not necessarily smooth.

Illinois

Implements the Illinois method for root-finding (approximating a zero of a univariate real function). It is a modified Regula Falsi method. Like the Regula Falsi method, convergence is guaranteed by maintaining a bracketed solution. The Illinois method however, should converge much faster than the original Regula Falsi method. Furthermore, this implementation of the Illinois method should not suffer from the same implementation issues as the Regula Falsi method, which may fail to convergence in certain cases. The Illinois method assumes that the function is continuous, but not necessarily smooth.

Pegasus

Implements the Pegasus method for root-finding (approximating a zero of a univariate real function). It is a modified Regula Falsi method. Like the Regula Falsi method, convergence is guaranteed by maintaining a bracketed solution. The Pegasus method however, should converge much faster than the original Regula Falsi method. The Pegasus method should converge faster than the Illinois method, another Regula Falsi-based method. The Pegasus method assumes that the function is continuous, but not necessarily smooth.

Examples

```
>> FindRoot(Exp(x)==Pi^3,{x,-1,10}, Method->Bisection)
{x->3.434189647436142}

>> FindRoot(Sin(x), {x, -0.5, 0.5})
{x->0.0}
```

Using Newton's method for finding the root of a differentiable, multivariate, vector-valued function.

```
>> FindRoot({2*x1+x2==E^(-x1), -x1+2*x2==E^(-x2)},{x1, 0.0},{x2, 1.0}})
{x1->0.197594,x2->0.425514}

>> FindRoot({Exp(-Exp(-(x1+x2)))-x2*(1+x1^2), x1*cos(x2)+x2*sin(x1)-0.5},{x1, 0.0},{x2
{x1->0.353247,x2->0.606082}}
```

Related terms

[Factor](#), [Eliminate](#), [NRoots](#), [Solve](#)

Implementation status

- - partially implemented

Github

- [Implementation of FindRoot](#)
-

FindSequenceFunction

```
FindSequenceFunction({i1, i2, i3, ...})
```

searches for a unary integer function, which generates the integer sequence {i1, i2, i3, ...}.

```
FindSequenceFunction({i1, i2, i3, ...}, var)
```

searches for a unary integer function, which generates the integer sequence {i1, i2, i3, ...} applied to the variable var

See

- [Wikipedia - Integer sequence](#)
- [Wikipedia - On-Line Encyclopedia of Integer Sequences](#)

Examples

```
>> FindSequenceFunction({1*3, 2*3, 6*3, 24*3, 120*3, 720*3, 5040*3} ,n)
3*n!

>> FindSequenceFunction({6,18,54,162},n)
2*3^n");

>> FindSequenceFunction({7,9,11,13,15})
5+2*#1&
```

Implementation status

- - experimental

Github

- [Implementation of FindSequenceFunction](#)
-

FindShortestPath

```
FindShortestPath(graph, source, destination)
```

find a shortest path in the `graph` from `source` to `destination`.

See

- [Wikipedia - Pathfinding](#)
- [Wikipedia - Shortest path problem](#)

Examples

```
>> FindShortestPath(Graph({1 -> 2, 2 -> 4, 1 -> 3, 3 -> 2, 3 -> 4},{EdgeWeight->{3.0,1  
{1,3,2,4}}
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestTour](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of FindShortestPath](#)
-

FindShortestTour

```
FindShortestTour({{p11, p12}, {p21, p22}, {p31, p32}, ...})
```

find a shortest tour in the `graph` with minimum `EuclideanDistance`.

See

- [Wikipedia - Travelling salesman problem](#)

Examples

```
>> FindShortestTour({{1,2},{2,3},{3,1}})
{Sqrt(2)+2*Sqrt(5),{1,3,2,1}}
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of FindShortestTour](#)
-

FindSpanningTree

```
FindSpanningTree(graph)
```

find the minimum spanning tree in the `graph`.

See

- [Wikipedia - Minimum spanning tree](#)

Examples

```
>> FindSpanningTree(Graph({a, b, c, d, e, f}, {a<->b, a<->d, b<->c, b<->d, b<->e, c<->e, c<->f, d<->
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindShortestTour](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of FindSpanningTree](#)
-

FindVertexCover

```
FindVertexCover(graph)
```

algorithm to find a vertex cover for a `graph`. A vertex cover is a set of vertices that touches all the edges in the graph.

See

- [Wikipedia - Vertex cover](#)

Examples

```
>> FindVertexCover({1<->2, 1<->3, 2<->3, 3<->4, 3<->5, 3<->6})  
{3,1}
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindShortestPath](#), [FindSpanningTree](#), [FindShortestTour](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of FindVertexCover](#)
-

FiniteAbelianGroupCount

```
FiniteAbelianGroupCount(order)
```

returns the number of finite Abelian groups of order `order`.

See

- [Wikipedia - Finitely generated abelian group](#)

Examples

FiniteGroupCount

```
FiniteGroupCount(order)
```

returns the number of finite groups of order `order`.

See

- [Wikipedia - Finite group](#)

Examples

First

```
First(expr)
```

returns the first element in `expr`.

Examples

`First(expr)` is equivalent to `expr[[1]]`.

```
>> First({a, b, c})
a

>> First(a + b + c)
a
```

Nonatomic expression expected.

```
>> First(x)
First(x)
```

Implementation status

- - full supported

Github

- [Implementation of First](#)
-

FirstCase

```
FirstCase({arg1, arg2, ...}, pattern-matcher)
```

returns the first of the elements `argi` for which `pattern-matcher` is matching.

```
FirstCase({arg1, arg2, ...}, pattern-matcher, default)
```

if `pattern-matcher` is not matching, the `default` value will be returned.

Examples

Implementation status

- - full supported

Github

- [Implementation of FirstCase](#)
-

FirstPosition

```
FirstPosition(expression, pattern-matcher)
```

returns the first subexpression of `expression` for which `pattern-matcher` is matching.

```
FirstPosition(expression, pattern-matcher, default)
```

if `pattern-matcher` is not matching, the `default` value will be returned.

Examples

Implementation status

- - full supported

Github

- [Implementation of FirstPosition](#)
-

Fit

```
Fit(list-of-data-points, terms-list, variable)
```

solve a least squares problem using the Levenberg-Marquardt algorithm.

See:

- [Wikipedia - Levenberg–Marquardt algorithm](#)

Examples

```
>> Fit({{1,1},{2,4},{3,9},{4,16}}, {1,x,x^2}, x) // Chop  
x^2
```

Related terms

[FindFit](#), [FittedModel](#), [LinearModelFit](#)

Implementation status

- - full supported

Github

- [Implementation of Fit](#)
-

FittedModel

```
FittedModel( )
```

`FittedModel` holds the model generated with `LinearModelFit`

See:

- [Wikipedia - Linear regression](#)

Examples

Related terms

[FindFit](#), [Fit](#), [LinearModelFit](#)

Implementation status

- - partially implemented

Github

- [Implementation of FittedModel](#)
-

FiveNum

```
FiveNum({dataset})
```

the Tuckey five-number summary is a set of descriptive statistics that provide information about a `dataset`. It consists of the five most important sample percentiles:

1. the sample minimum (smallest observation)
2. the lower quartile or first quartile
3. the median (the middle value)
4. the upper quartile or third quartile
5. the sample maximum (largest observation)

See:

- [Wikipedia - Five-number summary](#)

Examples

```
>> FiveNum({0, 0, 1, 2, 63, 61, 27, 13})  
{0, 1/2, 15/2, 44, 63}
```

Related terms

[Median](#), [Quantile](#), [Quartiles](#)

Implementation status

- - full supported

Github

- [Implementation of FiveNum](#)
-

FixedPoint

```
FixedPoint(f, expr)
```

starting with `expr`, iteratively applies `f` until the result no longer changes.

```
FixedPoint(f, expr, n)
```

performs at most `n` iterations.

See:

- [Wikipedia - Fixed-point iteration](#)
- [Wikipedia - Newton's method](#)

Examples

```
>> FixedPoint(Cos, 1.0)
0.7390851332151607

>> FixedPoint(#+1 &, 1, 20)
21

>> FixedPoint(f, x, 0)
x
```

Non-negative integer expected.

```
>> FixedPoint(f, x, -1)
FixedPoint(f, x, -1)

>> FixedPoint(Cos, 1.0, Infinity)
0.739085
```

Related terms

[FixedPointList](#), [Nest](#), [NestWhile](#)

Implementation status

- - full supported

Github

- [Implementation of FixedPoint](#)
-

FixedPointList

```
FixedPointList(f, expr)
```

starting with `expr`, iteratively applies `f` until the result no longer changes, and returns a list of all intermediate results.

```
FixedPointList(f, expr, n)
```

performs at most `n` iterations.

See:

- [Wikipedia - Fixed-point iteration](#)
- [Wikipedia - Newton's method](#)

Examples

```
>> FixedPointList(Cos, 1.0, 4)
{1.0, 0.5403023058681398, 0.8575532158463934, 0.6542897904977791, 0.7934803587425656}
```

Observe the convergence of Newton's method for approximating square roots:

```
>> newton(n_) := FixedPointList(.5(# + n/#) &, 1.);
>> newton(9)
{1.0, 5.0, 3.4, 3.023529411764706, 3.00009155413138, 3.0000000001396984, 3.0, 3.0}
```

Get the "hailstone" sequence of a number:

```
>> collatz(1) := 1;
>> collatz(x_ ? EvenQ) := x / 2;
>> collatz(x_) := 3*x + 1;
>> FixedPointList(collatz, 14)
{14, 7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1, 1}
```

```
>> FixedPointList(f, x, 0)
{x}
```

Non-negative integer expected.

```
>> FixedPointList(f, x, -1)
FixedPointList(f, x, -1)
```

```
>> Last(FixedPointList(Cos, 1.0, Infinity))  
0.7390851332151607
```

Related terms

[FixedPoint](#), [Nest](#), [NestWhile](#)

Implementation status

- - full supported

Github

- [Implementation of FixedPointList](#)
-

Flat

Flat

is an attribute that specifies that nested occurrences of a function should be automatically flattened.

See

- [Wikipedia - Associative property](#)

Examples

A symbol with the `Flat` attribute represents an associative mathematical operation:

```
>> SetAttributes(f, Flat)
>> f(a, f(b, c))
f(a, b, c)
```

`Flat` is taken into account in pattern matching:

```
>> f(a, b, c) /. f(a, b) -> d
f(d, c)

>> SetAttributes({u, v}, Flat)
>> u(x_) := {x}
>> u()
u()

>> u(a)
{a}
```

Iteration limit of 1000 exceeded.

```
>> u(a, b)
```

Iteration limit of 1000 exceeded.

```
>> u(a, b, c)

>> v(x_) := x
>> v()
v()
```

```
>> v(a)
a
```

Iteration limit of 1000 exceeded.

```
>> v(a, b)
v(a, b)
```

Iteration limit of 1000 exceeded.

```
>> v(a, b, c)
```

Related terms

[Constant](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#)

Flatten

```
Flatten(expr)
```

flattens out nested lists in `expr`.

```
Flatten(expr, n)
```

stops flattening at level `n`.

```
Flatten(expr, n, h)
```

flattens expressions with head `h` instead of 'List'.

Examples

```
>> Flatten({{a, b}, {c, {d}, e}, {f, {g, h}}})
{a, b, c, d, e, f, g, h}
>> Flatten({{a, b}, {c, {e}, e}, {f, {g, h}}}, 1)
{a, b, c, {e}, e, f, {g, h}}
>> Flatten(f(a, f(b, f(c, d))), e, Infinity, f)
f(a, b, c, d, e)
>> Flatten({{a, b}, {c, d}}, {{2}, {1}})
{{a, c}, {b, d}}
>> Flatten({{a, b}, {c, d}}, {{1, 2}})
{a, b, c, d}
```

Flatten also works in irregularly shaped arrays

```
>> Flatten({{1, 2, 3}, {4}, {6, 7}, {8, 9, 10}}, {{2}, {1}})
{{1, 4, 6, 8}, {2, 7, 9}, {3, 10}}

>> Flatten({{{111, 112, 113}, {121, 122}}, {{211, 212}, {221, 222, 223}}}, {{3}, {1}, {2}})
{{{111, 121}, {211, 221}}, {{112, 122}, {212, 222}}, {{113}, {223}}}

>> Flatten({{{1, 2, 3}, {4, 5}}, {{6, 7}, {8, 9, 10}}}, {{3}, {1}, {2}})
{{{1, 4}, {6, 8}}, {{2, 5}, {7, 9}}, {{3}, {10}}}

>> Flatten({{{1, 2, 3}, {4, 5}}, {{6, 7}, {8, 9, 10}}}, {{2}, {1, 3}})
{{1, 2, 3, 6, 7}, {4, 5, 8, 9, 10}}

>> Flatten({{1, 2}, {3,4}}, {1, 2})
{1, 2, 3, 4}
```


Levels to be flattened together in `{{-1, 2}}` should be lists of positive integers.

```
>> Flatten({{1, 2}, {3, 4}}, {{-1, 2}})
Flatten({{1, 2}, {3, 4}}, {{-1, 2}}, List)
```

Level 2 specified in `{{1}, {2}}` exceeds the levels, 1, which can be flattened together in `{a, b}`.

```
>> Flatten({a, b}, {{1}, {2}})
Flatten({a, b}, {{1}, {2}}, List)
```

Check `n` completion

```
>> m = {{{1, 2}, {3}}, {{4}, {5, 6}}}
>> Flatten(m, {2})
{{{1, 2}, {4}}, {{3}, {5, 6}}}
>> Flatten(m, {{2}})
{{{1, 2}, {4}}, {{3}, {5, 6}}}
>> Flatten(m, {{2}, {1}})
{{{1, 2}, {4}}, {{3}, {5, 6}}}
>> Flatten(m, {{2}, {1}, {3}})
{{{1, 2}, {4}}, {{3}, {5, 6}}}
```

Level 4 specified in `{{2}, {1}, {3}, {4}}` exceeds the levels, 3, which can be flattened together in `{{{1, 2}, {3}}, {{4}, {5, 6}}}`.

```
>> Flatten(m, {{2}, {1}, {3}, {4}})
Flatten({{{{1, 2}, {3}}, {{4}, {5, 6}}}, {{2}, {1}, {3}, {4}}, List)

>> m{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

>> Flatten(m, {1})
{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}

>> Flatten(m, {2})
{{1, 4, 7}, {2, 5, 8}, {3, 6, 9}}
```

Implementation status

- - full supported

Github

- [Implementation of Flatten](#)
-

FlattenAt

```
FlattenAt(expr, position)
```

flattens out nested lists at the given `position` in `expr`.

Examples

```
>> FlattenAt(f(a, g(b,c), {d, e}, {f}), -2)
f(a,g(b,c),d,e,{f})

>> FlattenAt(f(a, g(b,c), {d, e}, {f}), 4)
f(a,g(b,c),{d,e},f)
```

Implementation status

- - full supported

Github

- [Implementation of FlattenAt](#)
-

Floor

```
Floor(expr)
```

gives the smallest integer less than or equal `expr`.

```
Floor(expr, a)
```

gives the smallest multiple of `a` less than or equal to `expr`.

See

- [Wikipedia - Floor and ceiling functions](#)

Examples

```
>> Floor(1/3)
0

>> Floor(-1/3)
-1

>> Floor(10.4)
10

>> Floor(10/3)
3

>> Floor(10)
10

>> Floor(21, 2)
20

>> Floor(2.6, 0.5)
2.5

>> Floor(-10.4)
-11
```

For complex `expr`, take the floor of real an imaginary parts.

```
>> Floor(1.5 + 2.7*I)
1+I*2
```

For negative `a`, the smallest multiple of `a` greater than or equal to `expr` is returned.

```
>> Floor(10.4, -1)
11

>> Floor(-10.4, -1)
-10
```

Related terms

[IntegerPart](#), [Ceiling](#), [FractionalPart](#), [Round](#)

Implementation status

- - full supported

Github

- [Implementation of Floor](#)
-

Fold

```
Fold[f, x, {a, b}]
```

returns `f[f[x, a], b]`, and this nesting continues for lists of arbitrary length.

Examples

```
>> Fold(test, t1, {a, b, c, d})  
test(test(test(test(t1,a),b),c),d)
```

Implementation status

- - full supported

Github

- [Implementation of Fold](#)
-

FoldList

```
FoldList[f, x, {a, b}]
```

returns `{x, f[x, a], f[f[x, a], b]}`

Examples

[A002110 Primorial numbers](#): product of first n primes.

```
>> FoldList(Times, 1, Prime(Range(20)))  
{1, 2, 6, 30, 210, 2310, 30030, 510510, 9699690, 223092870, 6469693230, 200560490130,  
7420738134810, 304250263527210, 13082761331670030, 614889782588491410,  
32589158477190044730, 1922760350154212639070, 117288381359406970983270,  
7858321551080267055879090, 557940830126698960967415390}
```

Implementation status

- - full supported

Github

- [Implementation of FoldList](#)
-

For

```
For(start, test, incr, body)
```

evaluates `start`, and then iteratively `body` and `incr` as long as test evaluates to `True`.

```
For(start, test, incr)
```

evaluates only `incr` and no `body`.

```
For(start, test)
```

runs the loop without any `body`.

See:

- [Wikipedia - For loop](#)

Examples

Compute the factorial of 10 using `For`:

```
>> n := 1
>> For(i=1, i<=10, i=i+1, n = n * i)
>> n
3628800

>> n == 10!
True

>> n := 1
>> For(i=1, i<=10, i=i+1, If(i > 5, Return(i)); n = n * i)
6

>> n
120
```

Related terms

[Break](#), [Continue](#), [Do](#), [While](#)

Implementation status

- - full supported

Github

- [Implementation of For](#)
-

Fourier

```
Fourier(vector-of-complex-numbers)
```

Discrete Fourier transform of a `vector-of-complex-numbers`. Fourier transform is restricted to vectors with length of power of 2.

See

- [Wikipedia - Discrete Fourier transform](#)

Examples

```
>> Fourier({1 + 2*I, 3 + 11*I})
{2.82843+I*9.19239, -1.41421+I*(-6.36396)}

>> Fourier({1,2,0,0})
{1.5,0.5+I*1.0, -0.5,0.5+I*(-1.0)}
```

The first argument is restricted to vectors with a length of power of 2.

```
>> Fourier({1,2,0,0,7})
Fourier({1,2,0,0,7})
```

Related terms

[InverseFourier](#)

Implementation status

- - full supported

Github

- [Implementation of Fourier](#)
-

FourierDCTMatrix

```
FourierDCTMatrix(n)
```

gives a discrete cosine transform matrix with the dimension (n, n) and method `DCT-2`

```
FourierDCTMatrix(n, methodNumber)
```

gives a discrete cosine transform matrix with the dimension (n, n) and method `DCT-methodNumber`

See

- [Wikipedia - Discrete cosine transform](#)

Examples

```
>> FourierDCTMatrix(3, 1)
{{1/2, 1/2, 1/2},
 {1, 0, -1},
 {1/2, -1/2, 1/2}}
```

Implementation status

- - partially implemented

Github

- [Implementation of FourierDCTMatrix](#)
-

FourierDSTMatrix

```
FourierDSTMatrix(n)
```

gives a discrete sine transform matrix with the dimension (n, n) and method `DST-2`

```
FourierDSTMatrix(n, methodNumber)
```

gives a discrete sine transform matrix with the dimension (n, n) and method `DST-methodNumber`

See

- [Wikipedia - Discrete sine transform](#)

Examples

```
>> FourierDSTMatrix(3, 1)
{{1/2, 1/Sqrt(2), 1/2},
 {1/Sqrt(2), 0, -1/Sqrt(2)},
 {1/2, -1/Sqrt(2), 1/2}}
```

Implementation status

- - partially implemented

Github

- [Implementation of FourierDSTMatrix](#)
-

FourierMatrix

```
FourierMatrix(n)
```

gives a fourier matrix with the dimension `n`.

Examples

```
>> FourierMatrix(4)
{{1/2, 1/2, 1/2, 1/2},
 {1/2, I*1/2, -1/2, -I*1/2},
 {1/2, -1/2, 1/2, -1/2},
 {1/2, -I*1/2, -1/2, I*1/2}}
```

Implementation status

- - full supported

Github

- [Implementation of FourierMatrix](#)
-

FractionalPart

```
FractionalPart(number)
```

get the fractional part of a `number` .

See

- [Wikipedia - Fractional part](#)

Examples

```
>> FractionalPart(1.5)  
0.5
```

Related terms

[IntegerPart](#), [Ceiling](#), [Floor](#), [Round](#)

Implementation status

- - full supported

Github

- [Implementation of FractionalPart](#)
-

FrechetDistribution

```
FrechetDistribution(a,b)
```

returns a Frechet distribution.

See:

- [Wikipedia - Frechet distribution](#)

Examples

The variance of the Frechet distribution is

```
>> Variance(FrechetDistribution(n, m))  
Piecewise({{m^2*(Gamma(1-2/n)-Gamma(1-1/n)^2),n>2}},Infinity)
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of FrechetDistribution](#)
-

FreeQ

```
FreeQ(expr, x)
```

returns `True` if `expr` does not contain the expression `x`.

Examples

```
>> FreeQ(y, x)
True

>> FreeQ(a+b+c, a+b)
False

>> FreeQ({1, 2, a^(a+b)}, Plus)
False

>> FreeQ(a+b, x_+y_+z_)
True

>> FreeQ(a+b+c, x_+y_+z_)
False

>> FreeQ(x_+y_+z_)(a+b)
True
```

Implementation status

- - full supported

Github

- [Implementation of FreeQ](#)
-

FrobeniusNumber

```
FrobeniusNumber({a1, ..., aN})
```

returns the Frobenius number of the nonnegative integers $\{a_1, \dots, a_N\}$

The Frobenius problem, also known as the postage-stamp problem or the money-changing problem, is an integer programming problem that seeks nonnegative integer solutions to $x_1 a_1 + \dots + x_N a_N = M$ where a_i and M are positive integers. In particular, the Frobenius number $\text{FrobeniusNumber}(\{a_1, \dots, a_N\})$, is the largest M so that this equation fails to have a solution.

See:

- [Wikipedia - Coin problem](#)

Examples

```
>> FrobeniusNumber({1000, 1476, 3764, 4864, 4871, 7773})
47350

>> FrobeniusNumber({4,6,8})
Infinity
```

Related terms

[FrobeniusSolve](#), [IntegerPartitions](#)

Implementation status

- - partially implemented

Github

- [Implementation of FrobeniusNumber](#)
-

FrobeniusSolve

```
FrobeniusSolve({a1, ..., aN}, M)
```

get a list of solutions for the Frobenius equation given by the list of integers $\{a_1, \dots, a_N\}$ and the non-negative integer M .

The Frobenius problem, also known as the postage-stamp problem or the money-changing problem, is an integer programming problem that seeks nonnegative integer solutions to $x_1 a_1 + \dots + x_N a_N = M$ where a_i and M are positive integers. In particular, the Frobenius number $\text{FrobeniusNumber}(\{a_1, \dots, a_N\})$, is the largest M so that this equation fails to have a solution.

See:

- [Wikipedia - Coin problem](#)
- [Wikipedia - Partition \(number theory\)](#)
- [Wikipedia - Diophantine equation](#)

Examples

Solve the diophantine equation $2x + 3y + 4z == 29$:

```
>> FrobeniusSolve({2, 3, 4}, 29)
{{0, 3, 5}, {0, 7, 2}, {1, 1, 6}, {1, 5, 3}, {1, 9, 0}, {2, 3, 4}, {2, 7, 1}, {3, 1, 5}, {3, 5, 2}, {4, 3, 3}, {4, 7, 0}, {5, 1, 4}, {5, 5, 1}, {6, 3, 2}, {7, 1, 3}, {7, 5, 0}, {8, 3, 1}, {9, 1, 2}, {10, 3, 0}, {11, 1, 1}, {13, 1, 0}}
```

Related terms

[FrobeniusNumber](#), [IntegerPartitions](#)

Implementation status

- - partially implemented

Github

- [Implementation of FrobeniusSolve](#)
-

FromCharCode

```
FromCharCode({ch1, ch2, ...})
```

converts the `ch1, ch2, ...` character codes into a string of corresponding characters.

Examples

```
>> FromCharCode({97,45,51})  
a-3
```

Related terms

[ToCharCode](#), [ToUnicode](#)

Implementation status

- - full supported

Github

- [Implementation of FromCharCode](#)
-

FromContinuedFraction

```
FromContinuedFraction({n1, n2, ...})
```

reconstructs a number from the list of its continued fraction terms `{n1, n2, ...}`.

See

- [Wikipedia - Continued fraction](#)

Examples

```
>> FromContinuedFraction({2,3,4,5})  
157/68  
  
>> ContinuedFraction(157/68)  
{2,3,4,5}  
  
>> FromContinuedFraction({3, 7, 15, 1, 292, 1, 1, 1, 2, 1})  
1146408/364913  
  
>> FromContinuedFraction(Range(5))  
225/157
```

Related terms

[ContinuedFraction](#)

Implementation status

- - full supported

Github

- [Implementation of FromContinuedFraction](#)
-

FromDigits

```
FromDigits(list)
```

creates an expression from the list of digits for radix `10`.

```
FromDigits(list, radix)
```

creates an expression from the list of digits for the given `radix`.

```
FromDigits(string)
```

creates an expression from the characters in the `string` for radix `10`.

```
FromDigits(string, radix)
```

creates an expression from the characters in the `string` for radix `10`.

Examples

```
>> FromDigits("789abc")
790122

>> FromDigits("789abc", 16)
790122

>> FromDigits({1,1,1,1,0,1,1}, 2)
123
```

The [A023391](#) $a(n+1) = a(n)$ converted to base 9 from base 8 (written in base 10) integer sequence

```
>> NestList(FromDigits(IntegerDigits(#, 8), 9) &, 8, 50)
{8, 9, 10, 11, 12, 13, 14, 15, 16, 18, 20, 22, 24, 27, 30, 33, 37, 41, 46, 51, 57, 64, 81, 100, 121, 145, "181, 22
```

Implementation status

- - full supported

Github

- [Implementation of FromDigits](#)

FromLetterNumber

```
FromLetterNumber(number)
```

get the corresponding characters from the English alphabet.

```
FromLetterNumber(number, language-string)
```

get the corresponding characters from the languages alphabet.

Examples

```
>> FromLetterNumber(-2, "Dutch")
ij

>> FromLetterNumber(26, "Dutch")
ij

>> FromLetterNumber(1)
a
```

Related terms

[Alphabet](#), [LetterNumber](#)

Implementation status

- - full supported

Github

- [Implementation of FromLetterNumber](#)
-

FromPolarCoordinates

```
FromPolarCoordinates({r, t})
```

return the cartesian coordinates for the polar coordinates $\{r, t\}$.

```
FromPolarCoordinates({r, t, p})
```

return the cartesian coordinates for the polar coordinates $\{r, t, p\}$.

See

- [Wikipedia - Polar coordinate system](#)
- [Wikipedia - Cartesian coordinate system](#)

Examples

```
>> FromPolarCoordinates({r, t})  
{r*cos(t), r*sin(t)}  
  
>> FromPolarCoordinates({r, t, p})  
{r*cos(t), r*cos(p)*sin(t), r*sin(p)*sin(t)}
```

Related terms

[ToPolarCoordinates](#)

Implementation status

- - partially implemented

Github

- [Implementation of FromPolarCoordinates](#)
-

FromRomanNumeral

```
FromRomanNumeral(roman-number-string)
```

converts the given `roman-number-string` to an integer number.

See

- [Wikipedia - Roman numerals](#)
- [Unicode Number Forms - Archaic Roman Numerals](#)

Examples

```
>> FromRomanNumeral("MDCLXVI")  
1666
```

Zeros are represented by `N`

```
>> FromRomanNumeral("N")  
0
```

Related terms

[RomanNumeral](#)

Implementation status

- - partially implemented

Github

- [Implementation of FromRomanNumeral](#)
-

FromSphericalCoordinates

```
FromSphericalCoordinates({r, t, p})
```

returns the cartesian coordinates for the spherical coordinates $\{r, t, p\}$.

See

- [Wikipedia - Spherical coordinate system](#)
- [Wikipedia - Polar coordinate system](#)
- [Wikipedia - Cartesian coordinate system](#)

Examples

```
>> FromSphericalCoordinates( {r, t, p} )  
{r*cos(p)*sin(t), r*sin(p)*sin(t), r*cos(t)}
```

Implementation status

- - partially implemented

Github

- [Implementation of FromSphericalCoordinates](#)
-

FullDefinition

```
FullDefinition(symbol)
```

prints value and rule definitions associated with `symbol` and dependent symbols without attribute `Protected` recursively.

Examples

```
>> FullDefinition(ArcSinh)
Attributes(ArcSinh)={Listable,NumericFunction,Protected}

ArcSinh(I/Sqrt(2))=I*1/4*Pi

ArcSinh(Undefined)=Undefined

ArcSinh(Infinity)=Infinity

ArcSinh(I*Infinity)=Infinity

ArcSinh(I)=I*1/2*Pi

ArcSinh(0)=0

ArcSinh(I*1/2)=I*1/6*Pi

ArcSinh(I*1/2*Sqrt(3))=I*1/3*Pi

ArcSinh(ComplexInfinity)=ComplexInfinity
```

```
>> a(x_):=b(x,y);b(u_,v_):={{u,v},a}

>> FullDefinition(a)
a(x_):=b(x,y)

b(u_,v_):={{u,v},a}
```

Implementation status

- - full supported

Github

- [Implementation of FullDefinition](#)
-

FullForm

```
FullForm(expression)
```

shows the internal representation of the given `expression`.

Examples

FullForm shows the difference in the internal expression representation for the `Times` operator:

```
>> FullForm(x(x+1))  
x(Plus(1, x))  
  
>> FullForm(x*(x+1))  
Times(x, Plus(1, x))
```

Related terms

[LeafCount](#)

Implementation status

- - full supported

Github

- [Implementation of FullForm](#)
-

FullSimplify

```
FullSimplify(expr)
```

works like `Simplify` but additionally tries some `FunctionExpand` rule transformations to simplify `expr`.

```
FullSimplify(expr, option1, option2, ...)
```

full simplifies `expr` with some additional options set

- Assumptions - use assumptions to simplify the expression
- ComplexFunction - use this function to determine the "weight" of an expression.

Examples

```
>> FullSimplify(Cos(n*ArcCos(x)) == ChebyshevT(n, x))  
True
```

Related terms

[Simplify](#)

Implementation status

- - partially implemented

Github

- [Implementation of FullSimplify](#)
-

Function

```
Function(body)
```

or

```
body &
```

represents a pure function with parameters `#1`, `#2`....

```
Function({x1, x2, ...}, body)
```

or

```
(body &)[x1, x2, ...]
```

represents a pure function with parameters `x1`, `x2`....

```
Function({x1, x2, ...}, body, attr)
```

assume that the function has the attributes `attr`.

See:

- [Wikipedia - Pure function](#)
- [Wikipedia - Lambda calculus](#)

Examples

```
>> f := # ^ 2 &
>> f(3)
9

>> #^3& /@ {1, 2, 3}
{1,8,27}

>> #1+#2&[4, 5]
9
```

You can use `Function` with named parameters:

```
>> Function({x, y}, x * y)[2, 3]  
6
```

Parameters are renamed, when necessary, to avoid confusion:

```
>> Function({x}, Function({y}, f(x, y)))[y]
```

Related terms

[Slot](#), [SlotSequence](#)

Implementation status

- - full supported

Github

- [Implementation of Function](#)
-

FunctionExpand

`FunctionExpand(expression)`

expands the special function `expression`. `FunctionExpand` expands simple nested radicals.

See

- [Wikipedia - Nested radical](#)
- [Wikipedia - Trigonometric constants expressed in real radicals](#)

Examples

```
>> FunctionExpand(Beta(10, b))
362880/(b*(1+b)*(2+b)*(3+b)*(4+b)*(5+b)*(6+b)*(7+b)*(8+b)*(9+b))

>> FunctionExpand(Sqrt(5 + 2*Sqrt(6)))
Sqrt(2)+Sqrt(3)
```

Implementation status

- - partially implemented

Github

- [Implementation of FunctionExpand](#)
 - [Rule definitions of FunctionExpand](#)
-

FunctionURL

```
FunctionURL(built-in-symbol)
```

returns the GitHub URL of the `built-in-symbol` implementation in the [Symja GitHub repository](#).

Examples

Get the GitHub URL of the `NIntegrate` function implementation:

```
>> FunctionURL(NIntegrate)
https://github.com/axkr/symja_android_library/blob/master/symja_android_library/mathecl
```

Implementation status

- - experimental

Github

- [Implementation of FunctionURL](#)
-

Gamma

`Gamma(z)`

is the gamma function on the complex number `z`.

`Gamma(z, x)`

is the upper incomplete gamma function.

`Gamma(z, x0, x1)`

is equivalent to `Gamma(z, x0) - Gamma(z, x1)`.

See

- [Wikipedia - Gamma function](#)
- [Fungrim - Gamma function](#)
- [Wikipedia - Incomplete gamma function](#)

Examples

```
>> Gamma(8)
5040

>> Gamma(1/2)
Sqrt(Pi)

>> Gamma(2.2)
1.1018024908797128
```

Implementation status

- - full supported

Github

- [Implementation of Gamma](#)
 - [Rule definitions of Gamma](#)
-

GammaDistribution

```
GammaDistribution(a,b)
```

returns a gamma distribution.

See:

- [Wikipedia - Gamma distribution](#)

Examples

The variance of the gamma distribution is

```
>> Variance(GammaDistribution(n, m))  
m^2*n
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of GammaDistribution](#)
-

Gather

```
Gather(list, test)
```

gathers leaves of `list` into sub lists of items that are the same according to `test`.

```
Gather(list)
```

gathers leaves of `list` into sub lists of items that are the same.

Examples

The order of the items inside the sub lists is the same as in the original list.

```
>> Gather({1, 7, 3, 7, 2, 3, 9})
{{1}, {7, 7}, {3, 3}, {2}, {9}}

>> Gather({1/3, 2/6, 1/9})
{{1/3, 1/3}, {1/9}}
```

Implementation status

- - full supported

Github

- [Implementation of Gather](#)
-

GatherBy

```
GatherBy(list, f)
```

gathers leaves of `list` into sub lists of items whose image under `f` identical.

```
GatherBy(list, {f, g, ...})
```

gathers leaves of `list` into sub lists of items whose image under `f` identical. Then, gathers these sub lists again into sub sub lists, that are identical under `g`.

Examples

```
>> GatherBy({{1, 3}, {2, 2}, {1, 1}}, Total)
{{{1,3},{2,2}},{{1,1}}}
```

```
>> GatherBy({"xy", "abc", "ab"}, StringLength)
{{xy,ab},{abc}}
```

```
>> GatherBy({{2, 0}, {1, 5}, {1, 0}}, Last)
{{{2,0},{1,0}},{{1,5}}}
```

```
>> GatherBy({{1, 2}, {2, 1}, {3, 5}, {5, 1}, {2, 2, 2}}, {Total, Length})
{{{1,2},{2,1}},{{3,5}},{{5,1}},{{2,2,2}}}
```

Implementation status

- - full supported

Github

- [Implementation of GatherBy](#)
-

GCD

```
GCD(n1, n2, ...)
```

computes the greatest common divisor of the given integers.

See

- [Wikipedia - Greatest common divisor](#)
- [Fungrim - Greatest common divisor](#)

Examples

```
>> GCD(20, 30)
10
>> GCD(10, y)
GCD(10, y)
```

GCD is Listable:

```
>> GCD(4, {10, 11, 12, 13, 14})
{2, 1, 4, 1, 2}
```

Related terms

[LCM](#)

Implementation status

- - full supported

Github

- [Implementation of GCD](#)
-

GegenbauerC

`GegenbauerC(n, a, x)`

returns the GegenbauerC polynomial.

See:

- [Wikipedia - Gegenbauer polynomials](#)

Examples

```
>> GegenbauerC(2, a, z)
-a+2*a*(1+a)*z^2

>> GegenbauerC(8, z)
1/4-8*z^2+40*z^4-64*z^6+32*z^8

>> GegenbauerC(3, 1 + I)
-22/3+I*10/3
```

Implementation status

- - full supported

Github

- [Implementation of GegenbauerC](#)
-

GeoDistance

```
GeoDistance({latitude1, longitude1}, {latitude2, longitude2})
```

returns the geodesic distance between `{latitude1, longitude1}` and `{latitude2, longitude2}`.

See

- [Wikipedia - Geodesics on an ellipsoid](#)
- [Wikipedia - Geographical_distance](#)

Examples

Calculate the distance between Oslo and Berlin in miles:

```
>> GeoDistance({59.914, 10.752}, {52.523, 13.412})  
521.4298968444851[mi]
```

Calculate the distance between Oslo and Berlin in kilometers:

```
>> UnitConvert(GeoDistance({59.914, 10.752}, {52.523, 13.412}), "km")  
839.1600759072911[km]
```

Implementation status

- - full supported

Github

- [Implementation of GeoDistance](#)
-

GeometricDistribution

```
GeometricDistribution(p)
```

returns a geometric distribution.

See:

- [Wikipedia - Geometric distribution](#)

Examples

The variance of the geometric distribution is

```
>> Variance(GeometricDistribution(n))  
(1-n)/n^2
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of GeometricDistribution](#)
-

GeometricMean

```
GeometricMean({a, b, c,...})
```

returns the geometric mean of `{a, b, c,...}`.

See:

- [Wikipedia - Geometric mean](#)

Examples

```
>> GeometricMean({1, 2, 3, 4,5, 6, 7})  
2^(4/7)*3^(2/7)*35^(1/7)
```

Related terms

[ArithmeticGeometricMean](#), [HarmonicMean](#), [Mean](#), [Median](#)

Implementation status

- - full supported

Github

- [Implementation of GeometricMean](#)
-

Get

```
Get("path-to-package-file-name")
```

or

```
<<"path-to-package-file-name"
```

load the package defined in `path-to-package-file-name`.

This function doesn't work in the web interface. A file system has to be available to load a package.

Examples

Load the package `VectorAnalysis.m` from file system:

```
<<"VectorAnalysis.m"
```

Implementation status

- - partially implemented

Github

- [Implementation of Get](#)
-

Glaisher

Glaisher

Glaisher constant.

The `glaisher` constant is named after mathematicians James Whitbread Lee Glaisher and Hermann Kinkelin. Its approximate value is: `1.2824271291...`

See

- [Wikipedia - Glaisher-Kinkelin constant](#)

Implementation status

- - full supported

Github

- [Implementation of Glaisher](#)
-

GoldbachList

```
GoldbachList(even-number)
```

return the list of Goldbach prime pairs for the `even-number`

```
GoldbachList(even-number, max-results)
```

return `max-results` number of pairs.

See

- [Wikipedia - Goldbach's conjecture](#)

Examples

```
>> GoldbachList(64)
{{3, 61}, {5, 59}, {11, 53}, {17, 47}, {23, 41}}

>> GoldbachList(1000000000000000000, 1)
{{11, 999999999999999989}}
```

Implementation status

- - partially implemented

Github

- [Implementation of GoldbachList](#)
-

GoldenAngle

GoldenAngle

is the golden angle $\text{Pi} * (3 - \text{Sqrt}(5))$.

See

- [Wikipedia - Golden angle](#)

Examples

```
>> N(GoldenAngle)
2.39996
```

Related terms

[GoldenRatio](#)

Implementation status

- - full supported

Github

- [Implementation of GoldenAngle](#)
-

GoldenRatio

```
GoldenRatio
```

is the golden ratio $(1+\sqrt{5})/2$.

See

- [Wikipedia - Golden ratio](#)
- [Fungrim - Golden ratio](#)

Examples

```
>> N(GoldenRatio)
1.618033988749895
```

Related terms

[GoldenAngle](#), [E](#), [EulerGamma](#)

Implementation status

- - full supported

Github

- [Implementation of GoldenRatio](#)
-

Grad

```
Grad(function, list-of-variables)
```

gives the gradient of the function.

```
Grad({f1, f2, ...}, {v1, v2, ...})
```

returns the Jacobian matrix for the vector of functions.

See:

- [Wikipedia - Gradient](#)
- [Wikipedia - Jacobian matrix](#)

Examples

```
>> Grad(2*x+3*y^2-Sin(z), {x, y, z})  
{2, 6*y, -Cos(z)}
```

Create a Jacobian matrix:

```
>> Grad({f(x, y), g(x, y)}, {x, y})  
{ {Derivative(1, 0)[f][x, y], Derivative(0, 1)[f][x, y]}, {Derivative(1, 0)[g][x, y], Derivative(0, 1)[g][x, y]} }
```

Example from Wikipedia where the Jacobian matrix doesn't need to be squared:

```
>> Grad({x1, 5*x3, 4*x2^2-2*x3, x3*Sin[x1]}, {x1, x2, x3})  
{ {1, 0, 0}, {0, 0, 5}, {0, 8*x2, -2}, {x3*Cos(x1), 0, Sin(x1)} }
```

Implementation status

- - partially implemented

Github

- [Implementation of Grad](#)
-

Graph

```
Graph({edge1, ..., edgeN})
```

create a graph from the given edges `edge1, ..., edgeN`.

See:

- [Wikipedia - Graph](#)
- [Wikipedia - Graph theory](#)

Examples

A directed graph:

```
>> Graph({1 -> 2, 2 -> 3, 3 -> 4, 4 -> 1})
```

An undirected graph:

```
>> Graph({1 <-> 2, 2 <-> 3, 3 <-> 4, 4 <-> 1})
```

An undirected weighted graph:

```
>> Graph({1 <-> 2, 2 <-> 3, 3 <-> 4, 4 <-> 1}, {EdgeWeight->{2.0, 3.0, 4.0, 5.0}})
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Github

- [Implementation of Graph](#)
-

GraphCenter

```
GraphCenter(graph)
```

compute the `graph` center. The center of a `graph` is the set of vertices of `graph` eccentricity equal to the `graph` radius.

See:

- [Wikipedia - Graph center](#)
- [Wikipedia - Graph theory](#)

Related terms

[GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of GraphCenter](#)
-

GraphComplement

```
GraphComplement(graph)
```

returns the graph complement of `graph`.

Implementation status

- - partially implemented

Github

- [Implementation of GraphComplement](#)
-

GraphDiameter

```
GraphDiameter(graph)
```

return the diameter of the `graph`.

See:

- [Wikipedia - Distance \(graph theory\) - Related concepts](#)
- [Wikipedia - Graph theory](#)

Related terms

[GraphCenter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of GraphDiameter](#)
-

GraphDifference

```
GraphDifference(graph1, graph2)
```

returns the graph difference of `graph1`, `graph2`.

Implementation status

- - partially implemented

Github

- [Implementation of GraphDifference](#)
-

GraphDisjointUnion

```
GraphDisjointUnion(graph1, graph2, graph3,...)
```

returns the disjoint graph union of `graph1`, `graph2`, `graph3`,...

Implementation status

- - partially implemented

Github

- [Implementation of GraphDisjointUnion](#)
-

Graphics

```
Graphics(primitives, options)
```

represents a two-dimensional graphic.

Examples

```
>> Graphics(Table({EdgeForm({GrayLevel(0, 0.5)}), Hue((-11+q+10*r)/72, 1, 1, 0.6), Disk  
-Graphics-
```

Graphics3D

```
Graphics3D(primitives, options)
```

represents a three-dimensional graphic.

Examples

```
>> Graphics3D(Polygon({{0,0,0}, {0,1,1}, {1,0,0}}))  
-Graphics3D-  
  
>> Graphics3D(Sphere())  
-Graphics3D-
```

GraphIntersection

```
GraphIntersection(graph1, graph2, graph3,...)
```

returns the graph intersection of `graph1`, `graph2`, `graph3`,...

Implementation status

- - partially implemented

Github

- [Implementation of GraphIntersection](#)
-

GraphPeriphery

```
GraphPeriphery(graph)
```

compute the `graph` periphery. The periphery of a `graph` is the set of vertices of graph eccentricity equal to the graph diameter.

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - full supported

Github

- [Implementation of GraphPeriphery](#)
-

GraphPower

```
GraphPower(graph, n)
```

the function uses Dijkstra's algorithm (i.e. [FindShortestPath](#)) to find the shortest path between each pair of vertices. If the length of the shortest path is less than or equal to `n`, it adds an edge between the vertices in the new graph. The result is a new graph that is the power of the original graph.

Examples

```
>> GraphPower({1 -> 2, 2 -> 3, 3 -> 4, 4 -> 5, 5->6}, 2)
Graph({1,2,3,4,5,6},{1->2,1->3,2->3,2->4,3->4,3->5,4->5,4->6,5->6})
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - full supported

Github

- [Implementation of GraphPower](#)
-

GraphQ

```
GraphQ(expr)
```

test if `expr` is a graph object.

See:

- [Wikipedia - Graph](#)
- [Wikipedia - Graph theory](#)

Examples

```
>> GraphQ(Graph({1 -> 2, 2 -> 3, 1 -> 3, 4 -> 2}) )
True

>> GraphQ( Sin(x) )
False

>> GraphQ( Graph({1->2, 2->3, 3->1}, Edgeweight->{5.061,2.282,5.086}) )
True
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#), [WeightedGraphQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of GraphQ](#)
-

GraphRadius

```
GraphRadius(graph)
```

return the radius of the `graph`.

- [Wikipedia - Distance \(graph theory\) - Related concepts](#)

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of GraphRadius](#)
-

GraphUnion

```
GraphUnion(graph1, graph2, graph3,...)
```

returns the graph union of `graph1`, `graph2`, `graph3`,...

Implementation status

- - partially implemented

Github

- [Implementation of GraphUnion](#)
-

Gray

Gray

RGB color value for the color gray

Examples

```
>> Gray  
RGBColor(0.5,0.5,0.5)
```

Greater

```
Greater(x, y)
```

```
x > y
```

yields `True` if `x` is known to be greater than `y`.

```
lhs > rhs
```

represents the inequality `lhs > rhs`.

Examples

```
>> Pi>0
```

```
True
```

```
>> {Greater(), Greater(x), Greater(1)}
```

```
{True, True, True}
```

Implementation status

- - full supported

Github

- [Implementation of Greater](#)
-

GreaterEqual

```
GreaterEqual(x, y)
```

```
x >= y
```

yields `True` if `x` is known to be greater than or equal to `y`.

```
lhs >= rhs
```

represents the inequality `lhs >= rhs`.

Examples

```
>> x>=x
```

```
True
```

```
>> {GreaterEqual(), GreaterEqual(x), GreaterEqual(1)}  
{True, True, True}
```

Implementation status

- - full supported

Github

- [Implementation of GreaterEqual](#)
-

GreaterEqualThan

```
GreaterEqualThan(rhs)
```

operator applied to an expr `lhs` (`GreaterEqualThan(rhs)[lhs]`) returns `GreaterEqual(lhs, rhs)`.

Examples

GreaterThan

```
GreaterThan(rhs)
```

operator applied to an expr `lhs` (`GreaterThan(rhs)[lhs]`) returns `Greater(lhs, rhs)`.

Examples

Green

Green

RGB color value for the color green

Examples

```
>> Green  
RGBColor(0.0,1.0,0.0)
```

GridGraph

```
GridGraph({v1,v2})
```

returns the grid graph with $v1 \times v2$ vertices.

See

- [Wikipedia - Lattice graph](#)

Examples

```
>> GridGraph({3,4}) // AdjacencyMatrix // MatrixForm
{{0,1,0,1,0,0,0,0,0,0,0,0},
 {1,0,1,0,1,0,0,0,0,0,0,0},
 {0,1,0,0,0,1,0,0,0,0,0,0},
 {1,0,0,0,1,0,1,0,0,0,0,0},
 {0,1,0,1,0,1,0,1,0,0,0,0},
 {0,0,1,0,1,0,0,0,1,0,0,0},
 {0,0,0,1,0,0,0,1,0,1,0,0},
 {0,0,0,0,1,0,1,0,1,0,1,0},
 {0,0,0,0,0,1,0,1,0,0,0,1},
 {0,0,0,0,0,0,1,0,0,0,1,0},
 {0,0,0,0,0,0,0,1,0,1,0,1},
 {0,0,0,0,0,0,0,0,1,0,1,0}}
```

Implementation status

- - partially implemented

Github

- [Implementation of GridGraph](#)
-

GroebnerBasis

```
GroebnerBasis({polynomial-list},{variable-list})
```

returns a Gröbner basis for the `polynomial-list` and `variable-list`.

See:

- [Wikipedia - Gröbner basis](#)

Examples

```
>> GroebnerBasis({x*y-2*y, 2*y^2-x^2}, {y, x})  
{-2*x^2+x^3, -2*y+x*y, -x^2+2*y^2}  
  
>> GroebnerBasis({x*y-2*y, 2*y^2-x^2}, {x, y})  
{-2*y+y^3, -2*y+x*y, x^2-2*y^2}
```

Implementation status

- - full supported

Github

- [Implementation of GroebnerBasis](#)
-

GroupBy

```
GroupBy(list, head)
```

return an association where the elements of `list` are grouped by `head(element)`

```
GroupBy(assoc, head)
```

return an association where the rules of `assoc` are grouped by `head(rule-value)`

Examples

A hierachical `GroupBy`

```
>> expr = {{a}, {a, b}, {a, c}, {a, b, c, d}, {a, b, c, f}, {b, c}, {b, d}};

>> hg = Normal @ GroupBy(# /. {} -> Nothing , First -> Rest, Function(x, hg(x, #2))) /

>> hg(expr, func)
{a->{b->c->{func(d), func(f)}, func(c)}, b->{func(c), func(d)}}
```

Implementation status

- - full supported

Github

- [Implementation of GroupBy](#)
-

Gudermannian

```
Gudermannian(expr)
```

computes the gudermannian function.

See:

- [Wikipedia - Gudermannian function](#)

Examples

Implementation status

- - full supported

Github

- [Implementation of Gudermannian](#)
 - [Rule definitions of Gudermannian](#)
-

GumbelDistribution

```
GumbelDistribution(a, b)
```

returns a Gumbel distribution.

See:

- [Wikipedia - Gumbel distribution](#)

Examples

The variance of the Gumbel distribution is

```
>> Variance(GumbelDistribution(n, m))  
1/6*m^2*Pi^2
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of GumbelDistribution](#)
-

HamiltonianGraphQ

```
HamiltonianGraphQ(graph)
```

returns `True` if `graph` is an hamiltonian graph, and `False` otherwise.

See:

- [Wikipedia - Hamiltonian path](#)
- [Wikipedia - Hamiltonian path problem](#)

Examples

```
>> HamiltonianGraphQ(Graph({1 -> 2, 2 -> 3, 3 -> 4, 4 -> 1}))  
True
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of HamiltonianGraphQ](#)
-

HammingDistance

```
HammingDistance(a, b)
```

returns the Hamming distance of `a` and `b`, i.e. the number of different elements.

See:

- [Wikipedia - Hamming distance](#)
- [Youtube - Hamming codes and error correction](#)
- [Youtube - Hamming codes part 2, the elegance of it all](#)

Examples

```
>> HammingDistance("time", "dime")  
1
```

The `IgnoreCase` option makes `EditDistance` ignore the case of letters:

```
>> HammingDistance("TIME", "dime", IgnoreCase -> True)  
1
```

Implementation status

- - full supported

Github

- [Implementation of HammingDistance](#)
-

HankelH1

```
HankelH1(n, x)
```

returns Hankel function of the first kind.

Examples

Implementation status

- - full supported

Github

- [Implementation of HankelH1](#)
-

HankelH2

```
HankelH2(n, x)
```

returns Hankel function of the second kind.

Examples

Implementation status

- - full supported

Github

- [Implementation of HankelH2](#)
-

HankelMatrix

```
HankelMatrix(n)
```

gives a Hankel matrix with the dimension `n`.

```
HankelMatrix(vector)
```

gives a Hankel matrix with the elements of the `vector`.

```
HankelMatrix(vector1, vector2)
```

gives a Hankel matrix with the combined elements of the `vector1` and `vector2`.

Examples

```
>> HankelMatrix({f, a, b, c, d},{d, y, z})  
{f, a, b},  
 {a, b, c},  
 {b, c, d},  
 {c, d, y},  
 {d, y, z}
```

Implementation status

- - full supported

Github

- [Implementation of HankelMatrix](#)
-

HarmonicMean

```
HarmonicMean({a, b, c, ...})
```

returns the harmonic mean of `{a, b, c, ...}`.

See:

- [Wikipedia - Harmonic mean](#)

Examples

```
>> HarmonicMean({1, 2, 3, 4, 5, 6, 7})  
980/363
```

Related terms

[ArithmeticGeometricMean](#), [GeometricMean](#), [Mean](#), [Median](#)

Implementation status

- - full supported

Github

- [Implementation of HarmonicMean](#)
-

HarmonicNumber

```
HarmonicNumber(n)
```

returns the n th harmonic number.

See

- [Wikipedia - Harmonic number](#)

Examples

```
>> Table(HarmonicNumber(n), {n, 8})  
{1, 3/2, 11/6, 25/12, 137/60, 49/20, 363/140, 761/280}
```

Implementation status

- - full supported

Github

- [Implementation of HarmonicNumber](#)
-

Hash

```
Hash(expression)
```

the `Hash` function computes a hash value for any `expression`. It can generate both non-cryptographic integer hash codes and cryptographic hashes.

```
Hash(expression, hash-type)
```

computes a hash of the specified `hash-type` (e.g., "SHA-256").

Examples

Related terms

[FileHash](#)

Implementation status

- - full supported

Github

- [Implementation of Hash](#)
-

Haversine

Haversine(z)

returns the haversine function of z .

See:

- [Wikipedia - Haversine formula](#)

Examples

```
>> Haversine(1.5)
0.4646313991661485

>> Haversine(0.5 + 2 * I)
-1.150818666457047+I*0.8694047522371576
```

Implementation status

- - full supported

Github

- [Implementation of Haversine](#)
-

Head

```
Head(expr)
```

returns the head of the expression or atom `expr`.

```
Head(expr, newHead)
```

returns `newHead(Head(expr))`.

Examples

```
>> Head(a * b)
Times

>> Head(6)
Integer

>> Head(6+I)
Complex

>> Head(6.0)
Real

>> Head(6.0+I)
Complex

>> Head(3/4)
Rational

>> Head(x)
Symbol
```

Implementation status

- - full supported

Github

- [Implementation of Head](#)
-

HeavisideTheta

```
HeavisideTheta(expr1, expr2, ..., exprN)
```

returns `1` if all `expr1, expr2, ..., exprN` are positive and `0` if one of the `expr1, expr2, ... exprN` is negative. `HeavisideTheta(0)` returns unevaluated as `HeavisideTheta(0)`.

Examples

```
>> HeavisideTheta[42]  
1
```

Implementation status

- - full supported

Github

- [Implementation of HeavisideTheta](#)
-

HermiteDecomposition

```
HermiteDecomposition(matrix)
```

calculate the Hermite-decomposition as a list `{u,r}` of a square `matrix`.

See:

- [Wikipedia - Hermite normal form](#)

Examples

```
>> HermiteDecomposition({{ 5, 2, -1, 4, 0}, {3,8,2,0,6}, {1,-4,5,2,1}, {0,3,7,9,2}, {6,  
{{{-298,6,-69,5,257},{-51,1,-12,1,44},{-457,9,-106,8,394},{-270,5,-63,5,233},{703,14,-1
```

Implementation status

- - full supported

Github

- [Implementation of HermiteDecomposition](#)
-

HermiteH

`HermiteH(n, x)`

returns the Hermite polynomial $H_n(x)$.

See:

- [Wikipedia - Hermite polynomials](#)

Examples

```
>> HermiteH(8, x)
1680-13440*x^2+13440*x^4-3584*x^6+256*x^8

>> HermiteH(3, 1 + I)
-28+I*4
```

Implementation status

- - partially implemented

Github

- [Implementation of HermiteH](#)
-

HermitianMatrixQ

```
HermitianMatrixQ(m)
```

returns `True` if `m` is a hermitian matrix.

See

- [Wikipedia - Hermitian matrix](#)

Examples

```
>> HermitianMatrixQ({{2, 2 + I, 4}, {2-I, 3, I}, {4, -I, 1}})
True
```

Implementation status

- - full supported

Github

- [Implementation of HermitianMatrixQ](#)
-

HessenbergDecomposition

```
HessenbergDecomposition(matrix)
```

calculate the Hessenberg-decomposition as a list `{p, h}` of a square `matrix`.

See:

- [Wikipedia - Hessenberg-Verfahren](#)

Examples

```
>> {p, h} = HessenbergDecomposition({{2.0, 5, 8, 7}, {5, 2, 2, 8}, {7, 5, 6, 6}, {5, 4, 4, 8}}), //  
{{{1.0, 0.0, 0.0, 0.0}, {0.0, -0.502519, -0.475751, -0.721897}, {0.0, -0.703526, -0.260306, 0.6612
```

Implementation status

- - full supported

Github

- [Implementation of HessenbergDecomposition](#)
-

HexidecimalCharacter

HexidecimalCharacter

represents the characters 0-9, a-f and A-F.

Examples

```
>> StringMatchQ( #, HexidecimalCharacter) & /@ {"a", "1", "A", "x", "H", " ", "."}
{True, True, True, False, False, False, False}
```

HilbertMatrix

```
HilbertMatrix(n)
```

gives the hilbert matrix with `n` rows and columns.

```
HilbertMatrix({n,m})
```

gives the hilbert matrix with `n` rows and `m` columns.

See

- [Wikipedia - Hilbert matrix](#)

Examples

```
>> HilbertMatrix(2)
{{1, 1/2},
 {1/2, 1/3}}
```

Implementation status

- - full supported

Github

- [Implementation of HilbertMatrix](#)
-

Histogram

```
Histogram(list-of-values)
```

plots a histogram for a `list-of-values`

Examples

```
>> Histogram({1, 2, 3, None, 3, 5, f(), 2, 1, 5,4,3,2,foo, 2, 3})
```

Implementation status

- - experimental

Github

- [Implementation of Histogram](#)
-

HodgeDual

```
HodgeDual(tensor, dimensions, slots)
```

`HodgeDual` evaluates the Hodge star of a tensor.

See

- [Wikipedia - Hodge star operator](#)

Examples

```
>> m3 = {{{{1,0},{0,1},{0,0}},{0,0},{0,1},{1,0}}}; Normal@HodgeDual(m3, 3, {2})
{{{0,0},{0,0}},{0,0},{1,0}},{0,-1},{0,-1}}}, {{{0,0},{-1,0}},{0,0},{0,0}},{1,0},{0,
```

Hold

```
Hold(expr)
```

`Hold` doesn't evaluate `expr`. `Hold` evaluates `UpValues` for its arguments.

`HoldComplete` doesn't evaluate `UpValues`.

Examples

```
>> Hold(3*2)
Hold(3*2)

>> Attributes(Hold)
{HoldAll, Protected}
```

Related terms

[HoldComplete](#), [HoldForm](#), [HoldPattern](#), [ReleaseHold](#), [Unevaluated](#)

Implementation status

- - full supported

Github

- [Implementation of Hold](#)
-

HoldAll

HoldAll

is an attribute specifying that all arguments of a function should be left unevaluated.

Related terms

[Attributes](#), [ClearAttributes](#), [Constant](#), [Flat](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#), [SetAttributes](#)

HoldAllComplete

HoldAllComplete

is an attribute specifying that all arguments of a function should be left completely unevaluated and `Sequence` expressions shouldn't be flattened out.

Related terms

[Attributes](#), [ClearAttributes](#), [Constant](#), [Flat](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#), [SetAttributes](#)

HoldComplete

```
HoldComplete(expr)
```

`HoldComplete` doesn't evaluate `expr`. `Hold` evaluates `UpValues` for its arguments.

`HoldComplete` doesn't evaluate `UpValues`.

Examples

```
>> HoldComplete(3*2)
HoldComplete(3*2)

>> Attributes(HoldComplete)
{HoldAllComplete, Protected, SequenceHold}
```

Related terms

[Hold](#), [HoldForm](#), [HoldPattern](#), [ReleaseHold](#), [Unevaluated](#)

Implementation status

- - full supported

Github

- [Implementation of HoldComplete](#)
-

HoldFirst

HoldFirst

is an attribute specifying that the first argument of a function should be left unevaluated.

Related terms

[Attributes](#), [ClearAttributes](#), [Constant](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#), [SetAttributes](#)

HoldForm

```
HoldForm(expr)
```

`HoldForm` doesn't evaluate `expr` and didn't appear in the output.

Examples

```
>> HoldForm(3*2)
3*2
```

Related terms

[Hold](#), [HoldComplete](#), [HoldPattern](#), [ReleaseHold](#), [Unevaluated](#)

Implementation status

- - full supported

Github

- [Implementation of HoldForm](#)
-

HoldPattern

```
HoldPattern(expr)
```

`HoldPattern` doesn't evaluate `expr` for pattern-matching.

Examples

One might be very surprised that the following line evaluates to `True`!

```
>> MatchQ(And(x, y, z), Times(p__))
True
```

When the line above is evaluated `Times(p__)` evaluates to `(p__)` before Symja checks to see if the pattern matches. `MatchQ` then determines if `And(x,y,z)` matches the pattern `(p__)` and it does because `And(x,y,z)` is itself a sequence of one.

Now the next line also evaluates to `True` because both `( And(p__ ) )` and `( Times(p__ ) )` evaluate to `( p__ )`.

```
>> Times(p__)==And(p__)
True
```

In the examples above prevent the patterns from evaluating, by wrapping them with `HoldPattern` as in the following lines.

```
>> MatchQ(And(x, y, z), HoldPattern(Times(p__)))
False

>> HoldPattern(Times(p__))==HoldPattern(And(p__))
False
```

In the next lines `HoldPattern` is used to ensure the head `(And)` is changed to `(List)`. The two examples that follow have the same effect, but the use of `HoldPattern` isn't needed.

```
>> And(x, y, z)/.HoldPattern(And(a__)) ->List(a)
{x,y,z}

>> And(x, y, z)/.And->List
{x,y,z}

>> And(x, y, z)/.And(a_,b__)->List(a,b)
{x,y,z}
```

Related terms

[Hold](#), [HoldComplete](#), [HoldForm](#), [ReleaseHold](#), [Unevaluated](#)

Implementation status

- - full supported

Github

- [Implementation of HoldPattern](#)
-

HoldRest

HoldRest

is an attribute specifying that all but the first argument of a function should be left unevaluated.

Related terms

[Attributes](#), [ClearAttributes](#), [Constant](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#), [SetAttributes](#)

HornerForm

```
HornerForm(polynomial)
```

Generate the horner scheme for a univariate *polynomial*.

```
HornerForm(polynomial, x)
```

Generate the horner scheme for a univariate *polynomial* in *x*.

See:

- [Wikipedia - Horner scheme](#)
- [Rosetta Code - Horner's rule for polynomial evaluation](#)

Examples

```
>> HornerForm(3+4*x+5*x^2+33*x^6+x^8)
3+x*(4+x*(5+(33+x^2)*x^4))

>> HornerForm(a+b*x+c*x^2, x)
a+x*(b+c*x)
```

Implementation status

- - full supported

Github

- [Implementation of HornerForm](#)
-

HurwitzLerchPhi

```
HurwitzLerchPhi(z, s, a)
```

returns the Hurwitz-Lerch transcendent function.

See:

- [Wikipedia - Lerch zeta function](#)

Examples

```
>> HurwitzLerchPhi(-1, -1, 1)
1/4
```

Related terms

[HurwitzZeta](#), [LerchPhi](#), [PolyLog](#), [Zeta](#)

Implementation status

- - experimental

Github

- [Implementation of HurwitzLerchPhi](#)
 - [Rule definitions of HurwitzLerchPhi](#)
-

HurwitzZeta

```
HurwitzZeta(s, a)
```

returns the Hurwitz zeta function.

See:

- [Wikipedia - Hurwitz zeta function](#)
- [Fungrim - Hurwitz zeta function](#)

Examples

```
>> HurwitzZeta(3,1/2)  
7*Zeta(3)
```

Related terms

[Zeta](#)

Implementation status

- - partially implemented

Github

- [Implementation of HurwitzZeta](#)
-

HyperCubeGraph

HyperCubeGraph(order)

the hypercube graph Q_n is the graph formed from the vertices and edges of an n -dimensional hypercube. For instance, the cube graph Q_3 is the graph formed by the 8 vertices and 12 edges of a three-dimensional cube.

See

- [Wikipedia - Hypercube graph](#)

Examples

```
>> HypercubeGraph(4) // AdjacencyMatrix // Normal
{{0,1,1,0,1,0,0,0,1,0,0,0,0,0,0,0},
 {1,0,0,1,0,1,0,0,0,1,0,0,0,0,0,0},
 {1,0,0,1,0,0,1,0,0,0,1,0,0,0,0,0},
 {0,1,1,0,0,0,0,1,0,0,0,1,0,0,0,0},
 {1,0,0,0,0,1,1,0,0,0,0,0,1,0,0,0},
 {0,1,0,0,1,0,0,1,0,0,0,0,0,1,0,0},
 {0,0,1,0,1,0,0,1,0,0,0,0,0,0,1,0},
 {0,0,0,1,0,1,1,0,0,0,0,0,0,0,0,1},
 {1,0,0,0,0,0,0,0,0,1,1,0,1,0,0,0},
 {0,1,0,0,0,0,0,0,1,0,0,1,0,1,0,0},
 {0,0,1,0,0,0,0,0,1,0,0,1,0,0,1,0},
 {0,0,0,1,0,0,0,0,0,1,1,0,0,0,0,1},
 {0,0,0,0,1,0,0,0,1,0,0,0,0,1,1,0},
 {0,0,0,0,0,1,0,0,0,1,0,0,1,0,0,1},
 {0,0,0,0,0,0,1,0,0,0,1,0,1,0,0,1},
 {0,0,0,0,0,0,0,1,0,0,0,1,0,1,1,0}}
```

Implementation status

- - partially implemented

Github

- [Implementation of HypercubeGraph](#)
-

Hyperfactorial

`Hyperfactorial(n)`

returns the hyper factorial number of the integer `n`. The hyperfactorial of a positive integer `n` is the product of the numbers of the form `xx` from `11` to `nn`.

See

- [Wikipedia - Hyperfactorial](#)

Examples

```
>> Hyperfactorial(-7)
-40310784000000
```

Implementation status

- - partially implemented

Github

- [Implementation of Hyperfactorial](#)
-

Hypergeometric0F1

```
Hypergeometric0F1(b, z)
```

return the `Hypergeometric0F1` function

See:

- [Wikipedia - Generalized hypergeometric function](#)
- [Wikipedia - Bessel function - Relation to hypergeometric series](https://en.wikipedia.org/wiki/Bessel_function#Relation_to_hypergeometric_series)https://en.wikipedia.org/wiki/Bessel_function#Relation_to_hypergeometric_series)
- [Fungrim - Confluent hypergeometric functions](#)

Implementation status

- - full supported

Github

- [Implementation of Hypergeometric0F1](#)
-

Hypergeometric1F1

```
Hypergeometric1F1(a, b, z)
```

return the `Hypergeometric1F1` function

See:

- [Wikipedia - Confluent hypergeometric function](#)
- [Wikipedia - Generalized hypergeometric function](#)
- [Fungrim - Confluent hypergeometric functions](#)

Implementation status

- - full supported

Github

- [Implementation of Hypergeometric1F1](#)
 - [Rule definitions of Hypergeometric1F1](#)
-

Hypergeometric2F1

```
Hypergeometric2F1(a, b, c, z)
```

return the `Hypergeometric2F1` function

See:

- [Wikipedia - Hypergeometric function](#)
- [Wikipedia - Generalized hypergeometric function](#)
- [Fungrim - Gauss hypergeometric function](#)

Implementation status

- - full supported

Github

- [Implementation of Hypergeometric2F1](#)
 - [Rule definitions of Hypergeometric2F1](#)
-

HypergeometricDistribution

```
HypergeometricDistribution(n, s, t)
```

returns a hypergeometric distribution.

See:

- [Wikipedia - Hypergeometric distribution](#)

Examples

The variance of the hypergeometric distribution is

```
>> Variance(HypergeometricDistribution(n, ns, nt))  
(n*ns*(1-ns/nt)*(-n+nt))/((-1+nt)*nt)
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of HypergeometricDistribution](#)
-

HypergeometricPFQ

```
HypergeometricPFQ({a,...}, {b,...}, c)
```

return the `HypergeometricPFQ` function

See:

- [Wikipedia - Generalized hypergeometric function](#)

Implementation status

- - experimental

Github

- [Implementation of HypergeometricPFQ](#)
 - [Rule definitions of HypergeometricPFQ](#)
-

HypergeometricU

```
HypergeometricU(a, b, z)
```

return the Tricomi confluent hypergeometric function `HypergeometricU` fu

See:

- [Wikipedia - Confluent hypergeometric function](#)
- [Fungrim - Confluent hypergeometric functions](#)

Examples

```
>> HypergeometricU[1,4,8]  
41/256  
  
>> HypergeometricU(0, b, z)  
1
```

Implementation status

- - experimental

Github

- [Implementation of HypergeometricU](#)
 - [Rule definitions of HypergeometricU](#)
-

I

I

Imaginary unit - internally converted to the complex number $0+1*i$. `I` represents the imaginary number `Sqrt(-1)`. `I^2` will be evaluated to `-1`.

Note: the upper case identifier `I` is different from the lower case identifier `i`.

See

- [Wikipedia - Imaginary unit](#)
- [Fungrim - Imaginary unit](#)

Examples

```
>> I^2
-1

>> (3+I)*(3-I)
10
```

Related terms

[Complex](#), [Im](#), [Re](#)

Implementation status

- - full supported

Github

- [Implementation of I](#)
-

Identity

```
Identity(x)
```

is the identity function, which returns `x` unchanged.

See

- [Wikipedia - Identity function](#)

Examples

```
>> Identity(5)  
5
```

Implementation status

- - full supported

Github

- [Implementation of Identity](#)
-

IdentityMatrix

```
IdentityMatrix(n)
```

gives the identity matrix with `n` rows and columns.

```
IdentityMatrix({n,m})
```

gives the identity matrix with `n` rows and `m` columns.

See

- [Wikipedia - Identity matrix](#)

Examples

```
>> IdentityMatrix(3)
{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}
```

Implementation status

- - full supported

Github

- [Implementation of IdentityMatrix](#)
-

If

```
If(cond, pos, neg)
```

returns `pos` if `cond` evaluates to `True`, and `neg` if it evaluates to `False`.

```
If(cond, pos, neg, other)
```

returns `other` if `cond` evaluates to neither `True` nor `False`.

```
If(cond, pos)
```

returns `Null` if `cond` evaluates to `False`.

Examples

```
>> If(1<2, a, b)
a
```

If the second branch is not specified, `Null` is taken:

```
If(1<2, a) a
```

```
If(False, a) //FullForm Null
```

You might use comments (inside ``(*`` and ``*)``) to make the branches of ``If`` more readable

```
If(a, (then) b, (else) c);
```

```
### Implementation status
```

```
* &#x2705; - full supported
```

```
### Github
```

* [Implementation of If](https://github.com/axkr/symja_android_library/blob/master/symj

Im

`Im(z)`

returns the imaginary component of the complex number `z`.

See

- [Wikipedia - Complex number](#)

Examples

```
>> Im(3+4*I)
4

>> Im(0.5 + 2.3*I)
2.3
```

Related terms

[Complex](#), [Re](#), [ReIm](#)

Implementation status

- - full supported

Github

- [Implementation of Im](#)
-

Implies

```
Implies(arg1, arg2)
```

Logical implication.

`Implies(A, B)` is equivalent to `!A || B`. `Implies(expr1, expr2)` evaluates each argument in turn, returning `True` as soon as the first argument evaluates to `False`. If the first argument evaluates to `True`, `Implies` returns the second argument.

See

- [Wikipedia - Logical consequence](#)

Examples

```
>> Implies(False, a)
True
>> Implies(True, a)
a
```

If an expression does not evaluate to `True` or `False`, `Implies` returns a result in symbolic form:

```
>> Implies(a, Implies(b, Implies(True, c)))
Implies(a, Implies(b, c))
```

Implementation status

- - full supported

Github

- [Implementation of Implies](#)
-

Import

```
Import("path-to-filename", "WXF")
```

if the file system is enabled, import an expression in WXF format from the "path-to-filename" file.

```
Import("path-to-filename", "Table")
```

if the file system is enabled, import an expression in table format from the "path-to-filename" file.

```
Import("path-to-filename", "GraphML")
```

or

```
Import("path-to-filename", "DOT")
```

if the file system is enabled, import a graph in `GraphML` or `DOT` format from the "path-to-filename" file.

Examples

Import a graph:

```
>> Import("c:\\temp\\dotgraph.graphml", "GraphML")  
Graph({1, 2, 3}, {(1,2), (2,3), (3,1)})
```

Related terms

[BinaryDeserialize](#), [BinarySerialize](#), [ByteArray](#), [ByteArrayQ](#), [Export](#)

ImportString

```
ImportString(string, import-format)
```

import the `string` from `import-format`.

Supported formats: Base64, ExpressionJSON, JSON, Table

Examples

Import a string from base 64 format:

```
>> ImportString("SGVsbG8gd29ybGQ=", "Base64")  
Hello world
```

Related terms

[Export](#), [Import](#), [ExportString](#)

Implementation status

- - partially implemented

Github

- [Implementation of ImportString](#)
-

In

`In(k)`

gives the `k`th line of input.

Examples

```
>> x = 1
1

>> x = x + 1
2

>> Do(In(2), {3})

>> x
5

>> In(-1)
5

>> Definition(In)
Attributes(In)={Listable,NHoldFirst,Protected}
In(1):=x=1
In(2):=x=x+1
In(3):=Do(In(2), {3})
In(4):=x
In(5):=In(-1)
```

Implementation status

- - partially implemented

Github

- [Implementation of In](#)
-

Increment

```
Increment(x)
```

```
x++
```

increments `x` by `1`, returning the original value of `x`.

Examples

```
>> a = 2;  
>> a++  
2  
  
>> a  
3
```

Grouping of 'Increment', 'PreIncrement' and 'Plus':

```
>> +++++a+++++2//Hold//FullForm  
Hold(Plus(PreIncrement(PreIncrement(Increment(Increment(a)))), 2))
```

Implementation status

- - full supported

Github

- [Implementation of Increment](#)
-

Indeterminate

Indeterminate

represents an indeterminate result.

Note: Java's `Double.NaN`, a constant holding a Not-a-Number (NaN) value of type `double`, will be converted to `Indeterminate` internally.

See:

- [Wikipedia - IEEE_754 - NaNs](#)

Examples

```
>> 0^0
Indeterminate

>> Tan(Indeterminate)
Indeterminate
```

InexactNumberQ

```
InexactNumberQ(expr)
```

returns `True` if `expr` is not an exact number, and `False` otherwise.

Examples

```
>> InexactNumberQ(a)
False

>> InexactNumberQ(3.0)
True

>> InexactNumberQ(2/3)
False
```

`InexactNumberQ` can be applied to complex numbers:

```
>> InexactNumberQ(4.0+I)
True
```

Github

- [Implementation of InexactNumberQ](#)
-

Infinity

Infinity

represents an infinite real quantity.

See

- [Wikipedia - Infinity](#)

Examples

```
>> 1 / Infinity
0

>> Infinity + 100
Infinity
```

Use `Infinity` in sum and limit calculations:

```
>> Sum(1/x^2, {x, 1, Infinity})
Pi ^ 2 / 6

>> FullForm(Infinity)
DirectedInfinity(1)

>> (2 + 3.5*I) / Infinity
0.0

>> Infinity + Infinity
Infinity
```

Indeterminate expression `0 Infinity` encountered.

```
>> Infinity / Infinity
Indeterminate
```

Implementation status

- - full supported

Github

- [Implementation of Infinity](#)

Inner

```
Inner(f, x, y, g)
```

computes a generalized inner product of `x` and `y`, using a multiplication function `f` and an addition function `g`.

See

- [Wikipedia - Inner product space](#)

Examples

```
>> Inner(f, {a, b}, {x, y}, g)
g(f(a, x), f(b, y))
```

`Inner` can be used to compute a dot product:

```
>> Inner(Times, {a, b}, {c, d}, Plus) == {a, b} . {c, d}
True
```

The inner product of two boolean matrices:

```
>> Inner(And, {{False, False}, {False, True}}, {{True, False}, {True, True}}, Or)
{{False, False}, {True, True}}
```

Implementation status

- - full supported

Github

- [Implementation of Inner](#)
-

Input

```
Input()
```

if the file system is enabled, the user can input an expression. After input this expression will be evaluated immediately.

```
Input("string")
```

if the file system is enabled, show the string in the console and allow the user input.

Examples

```
>> InputString("::")^3
```

Related terms

[InputStringImport](#)

InputForm

```
InputForm(expr)
```

print the `expr` as if it should be inserted by the user for evaluation.

Examples

Print the string with quotes:

```
>> "a string" // InputForm  
"a string"
```

Implementation status

- - full supported

Github

- [Implementation of InputForm](#)
-

InputString

```
InputString()
```

if the file system is enabled, the user can input a string.

```
InputString("string")
```

if the file system is enabled, additionally show the `string` in the console.

Examples

Convert the input with `ToExpression` into a Symja expression

```
>> ToExpression(InputString("::"))^3
```

Related terms

[InputImport](#)

Insert

```
Insert(list, elem, n)
```

inserts `elem` at position `n` in `list`. When `n` is negative, the position is counted from the end.

Examples

```
>> Insert({a,b,c,d,e}, x, 3)
{a,b,x,c,d,e}

>> Insert({a,b,c,d,e}, x, -2)
{a,b,c,d,x,e}
```

Implementation status

- - full supported

Github

- [Implementation of Insert](#)
-

InstanceOf

```
InstanceOf[java-object, "class-name"]
```

return the result of the Java expression `java-object instanceof class`.

Note: the Java specific functions which call Java native classes are only available in the MMA mode in a local installation. All symbol and method names have to be case sensitive.

Examples

```
>> loc = JavaNew["java.util.Locale", "US"]
JavaObject[class java.util.Locale]

>> InstanceOf[loc, "java.util.Locale"]
True

>> InstanceOf[loc, "java.io.Serializable"]
True
```

Related terms

[JavaClass](#), [JavaNew](#), [JavaObject](#), [JavaObjectQ](#), [LoadJavaClass](#), [SameObjectQ](#)

Implementation status

- - supported on Java virtual machine

Github

- [Implementation of InstanceOf](#)
-

Integer

Integer

is the head of integers.

Examples

```
>> Head(5)
```

```
Integer
```

```
>> {a, b} = {2^10000, 2^10000 + 1}; {a == b, a < b, a <= b}  
{False, True, True}
```

IntegerDigits

```
IntegerDigits(n, base)
```

returns a list of integer digits for `n` under `base`.

```
IntegerDigits(n, base, padLeft)
```

pads the result list on the left with maximum `padLeft` zeros.

See

- [Wikipedia - Radix](#)

Examples

```
>> IntegerDigits(123)
{1,2,3}

>> IntegerDigits(-123)
{1,2,3}

>> IntegerDigits(123, 2)
{1,1,1,1,0,1,1}

>> IntegerDigits(123, 2, 10)
{0,0,0,1,1,1,1,0,1,1}

>> IntegerDigits({123,456,789}, 2, 10)
{{0,0,0,1,1,1,1,0,1,1},{0,1,1,1,0,0,1,0,0,0},{1,1,0,0,0,1,0,1,0,1}}
```

The [A018900 Sum of two distinct powers of 2](#) integer sequence

```
>> Select(Range(1000), (Count(IntegerDigits(#, 2), 1)==2)&)
{3,5,6,9,10,12,17,18,20,24,33,34,36,40,48,65,66,68,72,80,96,129,130,132,136,144,160,192}
```

Implementation status

- - full supported

Github

- [Implementation of IntegerDigits](#)
-

IntegerExponent

```
IntegerExponent(n, b)
```

gives the highest exponent of `b` that divides `n`.

Examples

```
>> IntegerExponent(16, 2)
4

>> IntegerExponent(-510000)
4

>> IntegerExponent(10, b)
IntegerExponent(10, b)
```

Implementation status

- - full supported

Github

- [Implementation of IntegerExponent](#)
-

IntegerLength

```
IntegerLength(x)
```

gives the number of digits in the base-10 representation of `x`.

```
IntegerLength(x, b)
```

gives the number of base-`b` digits in `x`.

See

- [Wikipedia - Radix](#)

Examples

```
>> IntegerLength(123456)
6

>> IntegerLength(10^10000)
10001

>> IntegerLength(-10^1000)
1001
```

`IntegerLength` with base 2:

```
>> IntegerLength(8, 2)
4
```

Check that `IntegerLength` is correct for the first 100 powers of 10:

```
>> IntegerLength /@ (10 ^ Range(100)) == Range(2, 101)
True
```

The base must be greater than 1:

```
>> IntegerLength(3, -2)
IntegerLength(3, -2)
```

'0' is a special case:


```
>> IntegerLength(0)
0

>> IntegerLength /@ (10 ^ Range(100) - 1) == Range(1, 100)
True
```

Implementation status

- - full supported

Github

- [Implementation of IntegerLength](#)
-

IntegerName

```
IntegerName(integer-number)
```

gives the spoken number string of `integer-number` in language `English`.

```
IntegerName(integer-number, "language")
```

gives the spoken number string of `integer-number` in language `language`.

See

- [Wikipedia - Integer](#)

Examples

```
>> IntegerName(0)
zero

>> IntegerName(42)
forty-two

>> IntegerName(-42)
minus forty-two

>> IntegerName(-123007,"German")
minus einhundertdreiundzwanzigtausendsieben

>> IntegerName(123007,"Spanish")
ciento veintitrés mil siete
```

Github

- [Implementation of IntegerName](#)
-

IntegerPart

```
IntegerPart(expr)
```

for real `expr` return the integer part of `expr`.

See

- [Wikipedia - Floor and ceiling functions](#)

Examples

```
>> IntegerPart(2.4)
2

>> IntegerPart(-9/4)
-2
```

Related terms

[Ceiling](#), [Floor](#), [FractionalPart](#), [Round](#)

Implementation status

- - full supported

Github

- [Implementation of IntegerPart](#)
-

IntegerPartitions

```
IntegerPartitions(n)
```

returns all partitions of the integer `n`.

```
IntegerPartitions(n, k)
```

lists the possible ways to partition `n` into smaller integers, using up to `k` elements.

```
IntegerPartitions(n, {lower, upper}, {listOfIntegers})
```

lists the possible ways to partition `n` with the numbers in `{listOfIntegers}`, using between `lower` and `upper` number of elements.

See

- [Wikipedia - Partition \(number theory\)](#)
- [Wikipedia - Coin problem](#)
- [Wikipedia - Diophantine equation](#)

Examples

```
>> IntegerPartitions(3)
{{3},{2,1},{1,1,1}}

>> IntegerPartitions(10,2)
{{10},{9,1},{8,2},{7,3},{6,4},{5,5}}
```

The "McNugget partitions" [OEIS - Number of partitions of n into parts 6, 9 or 20](#).

```
>> Table(Length(IntegerPartitions(i, All, {6, 9, 20})), {i,0, 100, 1})
{1,0,0,0,0,0,1,0,0,1,0,0,1,0,0,1,0,0,2,0,1,1,0,0,2,0,1,2,0,1,2,0,1,2,0,1,3,0,2,2,
1,1,3,0,2,3,1,2,3,1,2,3,1,2,4,1,3,3,2,2,5,1,3,4,2,3,5,2,3,5,2,3,6,2,4,5,3,3,7,2,
5,6,3,4,7,3,5,7,3,5,8,3,6,7,4,5,9,3,7,8,5}
```

Related terms

[FrobeniusNumber](#), [FrobeniusSolve](#)

Implementation status

- - full supported

Github

- [Implementation of IntegerPartitions](#)
-

IntegerQ

```
IntegerQ(expr)
```

returns `True` if `expr` is an integer, and `False` otherwise.

Examples

```
>> IntegerQ(3)
4

>> IntegerQ(Pi)
False
```

Github

- [Implementation of IntegerQ](#)
-

Integers

Integers

is the set of integer numbers.

Examples

```
>> Solve({x^2 + 2 y^3 == 3681, x > 0, y > 0}, {x, y}, Integers)
{{x->15, y->12}, {x->41, y->10}, {x->57, y->6}}
```

Integrate

```
Integrate(f, x)
```

integrates f with respect to x . The result does not contain the additive integration constant.

```
Integrate(f, {x,a,b})
```

computes the definite integral of f with respect to x from a to b .

See:

- [Wikipedia: Integral](#)
- [Wikipedia: Antiderivative](#)
- [Rubi: Rule-based Integration](#)

Examples

```
>> Integrate(x^2, x)
x^3/3

>> Integrate(Tan(x) ^ 5, x)
-Log(Cos(x))-Tan(x)^2/2+Tan(x)^4/4
```

Related terms

[D](#), [DSolve](#), [Int](#), [Limit](#), [ND](#), [NIntegrate](#)

Implementation status

- - partially implemented

Github

- [Implementation of Integrate](#)
-

InterpolatingFunction

```
InterpolatingFunction(data-list)
```

get the representation for the given `data-list` as piecewise `InterpolatingPolynomials`.

See:

- [Wikipedia - Newton Polynomial](#)

Examples

```
>> ipf=InterpolatingFunction({{0, 17}, {1, 3}, {2, 5}, {3, 4}, {4, 3}, {5, 0}, {6,23}})
InterpolatingFunction({{0,17},{1,3},{2,5},{3,4},{4,3},{5,0},{6,23}})

>> ipf(19/4)
-19/32
```

Related terms

[InterpolatingPolynomial](#), [NDSolve](#)

Implementation status

- - partially implemented

Github

- [Implementation of InterpolatingFunction](#)
-

InterpolatingPolynomial

```
InterpolatingPolynomial(data-list, symbol)
```

get the polynomial representation for the given `data-list`.

Newton polynomial interpolation, is the interpolation polynomial for a given set of data points in the Newton form. The Newton polynomial is sometimes called Newton's divided differences interpolation polynomial because the coefficients of the polynomial are calculated using divided differences.

See:

- [Wikipedia - Newton Polynomial](#)

Examples

```
>> InterpolatingPolynomial({{1, 7}, {3, 11}, {5, 27}}, x)
(3/2*x-5/2)*(x-1)+7
```

Related terms

[InterpolatingFunction](#)

Implementation status

- - partially implemented

Github

- [Implementation of InterpolatingPolynomial](#)
-

Interpolation

```
Interpolation(data-list)
```

return an `InterpolationFunction` for the `data-list`.

```
Interpolation(data-list, point)
```

return the value for a given `point` interpolating the `data-list`.

Examples

```
>> ipf=Interpolation(Table({x, Exp(4/(1+x^2))}, {x, 0, 3, 0.5})); ipf(2.5)
1.73624

>> Interpolation(Table({x, Exp(4/(1+x^2))}, {x, 0, 3, 0.5}), 2.5)
1.73624

>> Exp(4/(1 + 2.5^2))
1.73624
```

Related terms

[InterpolatingFunction](#)

Implementation status

- - partially implemented

Github

- [Implementation of Interpolation](#)
-

InterquartileRange

```
InterquartileRange(list)
```

returns the interquartile range (IQR), which is between upper and lower quartiles,
 $IQR = Q3 - Q1$.

```
InterquartileRange(distribution)
```

returns the interquartile range (IQR) of the `distribution`.

See

- [Wikipedia - Interquartile range](#)

`InterquartileRange` can be applied to the following distributions:

[BernoulliDistribution](#), [CauchyDistribution](#), [ErlangDistribution](#), [ExponentialDistribution](#),
[FrechetDistribution](#), [GammaDistribution](#), [GumbelDistribution](#), [LogNormalDistribution](#),
[NakagamiDistribution](#), [NormalDistribution](#), [StudentTDistribution](#), [WeibullDistribution](#)

Examples

```
>> InterquartileRange({7, 7, 31, 31, 47, 75, 87, 115, 116, 119, 119, 155, 177})  
88
```

Related terms

[FiveNum](#), [Quantile](#), [Quartiles](#)

Implementation status

- - full supported

Github

- [Implementation of InterquartileRange](#)
 - [Rule definitions of InterquartileRange](#)
-

Interrupt

```
Interrupt( )
```

Interrupt an evaluation and returns `$Aborted`.

Examples

`Print(test1)` prints string: "test1"

```
>> Print(test1); Interrupt(); Print(test2)
$Aborted
```

Implementation status

- - full supported

Github

- [Implementation of Interrupt](#)
-

Intersection

```
Intersection(set1, set2, ...)
```

get the intersection set from `set1` and `set2`

See

- [Wikipedia - Intersection \(set theory\)](#)

Examples

```
>> Intersection({a, a, b, c}, {b, a})  
{a, b}
```

Related terms

[Complement](#), [Union](#)

Implementation status

- - full supported

Github

- [Implementation of Intersection](#)
-

Interval

```
Interval({a, b})
```

represents the closed interval from `a` to `b`.

See

- [Wikipedia - Interval arithmetic](#)
- [Wikipedia - Interval \(mathematics\)](#)

Examples

```
>> Interval({1, 6}) * Interval({0, 2})
Interval({0,12})

>> Interval({1.5, 6}) * Interval({0.1, 2.7})
Interval({0.15,16.2})

>> Sign(Interval({-43, -42}))
-1

>> Im(Interval({-Infinity, Infinity}))
0

>> ArcCot(Interval({-1, Infinity}))
Interval({-Pi/2, -Pi/4},{0,Pi/2})
```

Related terms

[IntervalIntersection](#), [IntervalMemberQ](#), [IntervalUnion](#)

Implementation status

- - full supported

Github

- [Implementation of Interval](#)
-

IntervalComplement

```
IntervalComplement(interval_1, interval_2)
```

compute the complement of the intervals `interval_1 \ interval_2`. The intervals must be of structure `IntervalData` (closed/opened ends of interval) and not of structure `Interval` (only closed ends)

See:

- [Wikipedia - Complement \(set theory\) Relative complement](#)
- [Wikipedia - Interval \(mathematics\)](#)

Examples

```
>> IntervalComplement(IntervalData({1/2, LessEqual, LessEqual, 3}), IntervalData({0, Less, Less, 1}), IntervalData({1, Less, Less, E}))
```

Related terms

[IntervalData](#), [IntervalIntersection](#), [IntervalMemberQ](#)

Implementation status

- - full supported

Github

- [Implementation of IntervalComplement](#)
-

IntervalData

```
IntervalData({a, leftEnd, rightEnd, b})
```

represents the open/closed ends interval from `a` to `b`. `leftEnd` and `rightEnd` must have the value `Less` for representing an open ended interval or `LessEqual` for representing a closed ended interval.

See

- [Wikipedia - Interval arithmetic](#)
- [Wikipedia - Interval \(mathematics\)](#)

Examples

```
>> IntervalData({1, Less, Less, 6}) * IntervalData({0, Less, Less, 2})  
IntervalData({0, Less, Less, 12})
```

Related terms

[IntervalComplement](#), [IntervalIntersection](#), [IntervalMemberQ](#), [IntervalUnion](#)

Implementation status

- - full supported

Github

- [Implementation of IntervalData](#)
-

IntervalIntersection

```
IntervalIntersection(interval_1, interval_2, ...)
```

compute the intersection of the intervals `interval_1, interval_2, ...`

See:

- [Wikipedia - Interval arithmetic](#)
- [Wikipedia - Interval \(mathematics\)](#)

Examples

```
>> IntervalIntersection(Interval({1, 2}, {3, 4}, {5, 7}, {8, 8.5}), Interval({1.5, 3.5}  
Interval({1.5,2},{3,3.5},{5,6}))
```

Related terms

[Interval](#), [IntervalMemberQ](#), [IntervalUnion](#)

Implementation status

- - full supported

Github

- [Implementation of IntervalIntersection](#)
-

IntervalMemberQ

```
IntervalMemberQ(interval, intervalOrRealNumber)
```

returns `True`, if `intervalOrRealNumber` is completely surrounded by `interval`

See

- [Wikipedia - Interval arithmetic](#)
- [Wikipedia - Interval \(mathematics\)](#)

Examples

```
>> IntervalMemberQ(Interval({4,10}), Interval({2*Pi, 3*Pi}))
True

>> IntervalMemberQ(Interval({4,10}), Interval({2*Pi, 4*Pi}))
False
```

Related terms

[Interval](#), [IntervalIntersection](#), [IntervalUnion](#)

Implementation status

- - full supported

Github

- [Implementation of IntervalMemberQ](#)
-

IntervalUnion

```
IntervalUnion(interval_1, interval_2, ...)
```

compute the union of the intervals `interval_1, interval_2, ...`

See:

- [Wikipedia - Interval arithmetic](#)
- [Wikipedia - Interval \(mathematics\)](#)

Examples

```
>> IntervalUnion(Interval({1, 2}, {3, 4}, {5, 7}, {8, 8.5}), Interval({1.5, 3.5}, {4.1, 4.2}), Interval({1,4},{4.1,7},{8,8.5},{9,10}))
```

Related terms

[Interval](#), [IntervalIntersection](#), [IntervalMemberQ](#)

Implementation status

- - full supported

Github

- [Implementation of IntervalUnion](#)
-

Inverse

```
Inverse(matrix)
```

computes the inverse of the `matrix`.

See:

- [Wikipedia - Invertible matrix](#)
- [Youtube - Inverse matrices, column space and null space | Essence of linear algebra, chapter 7](#)

Examples

```
>> Inverse({{1, 2, 0}, {2, 3, 0}, {3, 4, 1}})
{{-3,2,0},
 {2, -1,0},
 {1, -2,1}}
```

The matrix `{{1, 0}, {0, 0}}` is singular.

```
>> Inverse({{1, 0}, {0, 0}})
Inverse({{1, 0}, {0, 0}})

>> Inverse({{1, 0, 0}, {0, Sqrt(3)/2, 1/2}, {0, -1 / 2, Sqrt(3)/2}})
{{1,0,0},
 {0,Sqrt(3)/2, -1/2},
 {0,1/2,1/(1/(2*Sqrt(3))+Sqrt(3)/2)}}
```

Implementation status

- - full supported

Github

- [Implementation of Inverse](#)
-

InverseCDF

```
InverseCDF(dist, q)
```

returns the inverse cumulative distribution for the distribution `dist` as a function of `q`

```
InverseCDF(dist, {x1, x2, ...})
```

returns the inverse CDF evaluated at `{x1, x2, ...}`

```
InverseCDF(dist)
```

returns the inverse CDF as a pure function.

Examples

```
>> InverseCDF[NormalDistribution[0, 1], x]  
ConditionalExpression(-Sqrt(2)*InverseErfc(2*x), 0<=x<=1)
```

Related terms

[CDF](#), [PDF](#)

Implementation status

- - full supported

Github

- [Implementation of InverseCDF](#)
-

InverseErf

`InverseErf(z)`

returns the inverse error function of `z`.

See

- [Wikipedia - Error_function - Inverse functions](#)

Examples

`InverseErf(z)` is an odd function:

```
>> InverseErf /@ {-1, 0, 1}
{-Infinity, 0, Infinity}
```

'`InverseErf($z$)`' only returns numeric values for '`-1 <= $z$ <= 1`':

```
>> InverseErf /@ {0.9, 1.0, 1.1}
{1.1630871536766743, Infinity, InverseErf(1.1)}
```

Implementation status

- - partially implemented

Github

- [Implementation of InverseErf](#)
-

InverseErfc

`InverseErfc(z)`

returns the inverse complementary error function of `z`.

See

- [Wikipedia - Error_function - Inverse functions](#)

Examples

```
>> InverseErfc /@ {0, 1, 2}
{Infinity, 0, -Infinity}
```

Implementation status

- - partially implemented

Github

- [Implementation of InverseErfc](#)
-

InverseFourier

```
InverseFourier(vector-of-complex-numbers)
```

Inverse discrete Fourier transform of a `vector-of-complex-numbers`. Fourier transform is restricted to vectors with length of power of 2.

See

- [Wikipedia - Discrete Fourier transform - Inverse transform](#)

Examples

```
>> InverseFourier({2.82843+I*9.19239, -1.41421+I*(-6.36396)})  
{1.0+I*2.0, 3.0+I*11.0}  
  
>> InverseFourier({1.5, 0.5+I*1.0, -0.5, 0.5+I*(-1.0)})  
{1.0, 2.0+I*2.22045*10^-16, 0.0, I*(-2.22045*10^-16)}
```

The first argument is restricted to vectors with a length of power of 2.

```
>> InverseFourier({1, 2, 0, 0, 7})  
InverseFourier({1, 2, 0, 0, 7})
```

Related terms

[Fourier](#)

Implementation status

- - full supported

Github

- [Implementation of InverseFourier](#)
-

InverseFunction

```
InverseFunction(head)
```

returns the inverse function for the symbol `head`.

Examples

```
>> InverseFunction(Sin)  
ArcSin
```

Implementation status

- - partially implemented

Github

- [Implementation of InverseFunction](#)
-

InverseGammaDistribution

```
InverseGammaDistribution(a,b)
```

returns a inverse gamma distribution.

See:

- [Wikipedia - Inverse-gamma distribution](#)

Examples

The variance of the inverse gamma distribution is

```
>> Variance(InverseGammaDistribution(n, m))  
Piecewise({{m^2/((1 - n)^2*(-2 + n)),n>2}},Indeterminate)
```

Related terms

[CDF](#), [GammaDistribution](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

InverseGudermannian

```
InverseGudermannian(expr)
```

computes the inverse gudermannian function.

See:

- [Wikipedia - Gudermannian function](#)

Examples

Implementation status

- - full supported

Github

- [Implementation of InverseGudermannian](#)
-

InverseHaversine

```
InverseHaversine(z)
```

returns the inverse haversine function of z .

See

- [Wikipedia - Versine](#)

Examples

```
>> InverseHaversine(0.5)
1.5707963267948968

>> InverseHaversine(1 + 2.5 * I)
1.764589463349828+I*2.3309746530493127
```

Implementation status

- - full supported

Github

- [Implementation of InverseHaversine](#)
-

InverseLaplaceTransform

```
InverseLaplaceTransform(f, s, t)
```

returns the inverse laplace transform.

See:

- [Wikipedia - Laplace transform](#)

Examples

```
>> InverseLaplaceTransform(3/(s-1)+(2*s)/(s^2+4), s, t)  
3*E^t+2*Cos(2*t)
```

Implementation status

- - experimental

Github

- [Implementation of InverseLaplaceTransform](#)
 - [Rule definitions of InverseLaplaceTransform](#)
-

InverseSeries

```
InverseSeries( series )
```

return the inverse series.

See:

- [Wikipedia - Taylor series](#)
- [Wikipedia - Big O notation](#)
- [Wikipedia - Formal power series](#)

Examples

```
>> InverseSeries(Series(Sin(x), {x, 0, 7}))  
x+x^3/6+3/40*x^5+5/112*x^7+O(x)^8
```

Related terms

[ComposeSeries](#), [Series](#), [SeriesCoefficient](#), [SeriesData](#)

Implementation status

- - experimental

Github

- [Implementation of InverseSeries](#)
-

InverseZTransform

```
InverseZTransform(x, z, n)
```

returns the inverse Z-Transform of x .

See:

- [Wikipedia - Z-transform](#)

Examples

```
>> InverseZTransform(f(z)+ g(z)+h(z), z, n)  
InverseZTransform(f(z), n, z)+InverseZTransform(g(z), n, z)+InverseZTransform(h(z), n, z)
```

Implementation status

- - experimental

Github

- [Implementation of InverseZTransform](#)
 - [Rule definitions of InverseZTransform](#)
-

IsomorphicGraphQ

```
IsomorphicGraphQ(graph1, graph2)
```

returns `True` if an isomorphism exists between `graph1` and `graph2`. Return `False` in all other cases.

See:

- [Wikipedia - Graph isomorphism](#)

Examples

```
>> IsomorphicGraphQ(Graph({1, 2, 3, 4}, {1<->2, 1<->4, 2<->3, 3<->4}), Graph({1, 2, 3, 4}, {1<->3, 1<->4, 2<->3, 3<->4}), True)
```

Related terms

[FindGraphIsomorphism](#), [GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of IsomorphicGraphQ](#)
-

JaccardDissimilarity

```
JaccardDissimilarity(u, v)
```

returns the Jaccard-Needham dissimilarity between the two boolean 1-D lists `u` and `v`, which is defined as $(c_{tf} + c_{ft}) / (c_{tt} + c_{ft} + c_{tf})$, where n is `len(u)` and c_{ij} is the number of occurrences of `u(k)=i` and `v(k)=j` for `k < n`.

Examples

```
>> JaccardDissimilarity({1, 0, 1, 1, 0, 1, 1}, {0, 1, 1, 0, 0, 0, 1})  
2/3
```

Implementation status

- - full supported

Github

- [Implementation of JaccardDissimilarity](#)
-

JacobiAmplitude

```
JacobiAmplitude(x, m)
```

returns the amplitude $\text{am}(x, m)$ for Jacobian elliptic function.

See

- [Wikipedia - Jacobi elliptic functions](#)
- [NIST - Jacobi's Amplitude \(am\) Function](#)

Implementation status

- - full supported

Github

- [Implementation of JacobiAmplitude](#)
-

JacobiCD

```
JacobiCD(x, m)
```

returns the Jacobian elliptic function `cd(x, m)`.

See

- [Wikipedia - Jacobi elliptic functions](#)
- [NIST - Jacobian elliptic functions](#)

Examples

```
>> JacobiCD(10.0, 1/3)  
-0.945268
```

Implementation status

- - full supported

Github

- [Implementation of JacobiCD](#)
-

JacobiCN

```
JacobiCN(x, m)
```

returns the Jacobian elliptic function `cn(x, m)`.

See

- [Wikipedia - Jacobi elliptic functions](#)
- [NIST - Jacobian elliptic functions](#)

Examples

```
>> JacobiCN(10.0, 1/3)
-0.92107
```

Implementation status

- - full supported

Github

- [Implementation of JacobiCN](#)
-

JacobiDN

```
JacobiDN(x, m)
```

returns the Jacobian elliptic function `dn(x, m)`.

See

- [Wikipedia - Jacobi elliptic functions](#)
- [NIST - Jacobian elliptic functions](#)

Examples

```
>> JacobiDN(10.0, 1/3)
0.974401
```

Implementation status

- - full supported

Github

- [Implementation of JacobiDN](#)
-

JacobiMatrix

```
JacobiMatrix(matrix, var)
```

creates a Jacobian matrix.

Examples

```
>> JacobiMatrix({f(u),f(v),f(w),f(x)}, {u,v,w})  
{f'(u),0,0},{0,f'(v),0},{0,0,f'(w)},{0,0,0}}
```

Implementation status

- - full supported

Github

- [Implementation of JacobiMatrix](#)
-

JacobiP

```
JacobiP(n, a, b, z)
```

returns the Jacobi polynomial.

See

- [Wikipedia - Jacobi polynomials](#)

Examples

```
>> JacobiP(2, a, b, z)
1/4*(1/2*(1+b)*(2+b)*(1-z)^2+(2+a)*(2+b)*(-1+z)*(1+z)+1/2*(1+a)*(2+a)*(1+z)^2)
```

Implementation status

- - partially implemented

Github

- [Implementation of JacobiP](#)
-

JacobiSC

```
JacobiSC(x, m)
```

returns the Jacobian elliptic function `sc(x, m)`.

See

- [Wikipedia - Jacobi elliptic functions](#)
- [NIST - Jacobian elliptic functions](#)

Examples

```
>> JacobiSC(10.0, 1/3)
-0.422766
```

Implementation status

- - full supported

Github

- [Implementation of JacobiSC](#)
 - [Rule definitions of JacobiSC](#)
-

JacobiSD

```
JacobiSD(x, m)
```

returns the Jacobian elliptic function `sd(x, m)`.

See

- [Wikipedia - Jacobi elliptic functions](#)
- [NIST - Jacobian elliptic functions](#)

Examples

```
>> JacobiSD(10.0, 1/3)
0.399627
```

Implementation status

- - full supported

Github

- [Implementation of JacobiSD](#)
-

JacobiSN

```
JacobiSN(x, m)
```

returns the Jacobian elliptic function `sn(x, m)`.

See

- [Wikipedia - Jacobi elliptic functions](#)
- [NIST - Jacobian elliptic functions](#)

Examples

```
>> JacobiSN(10.0, 1/3)  
0.389397
```

Implementation status

- - full supported

Github

- [Implementation of JacobiSN](#)
-

JacobiSymbol

```
JacobiSymbol(m, n)
```

calculates the Jacobi symbol.

See

- [Wikipedia - Jacobi symbol](#)
- [Wikipedia - Legendre symbol](#)

Examples

```
>> JacobiSymbol(1001, 9907)
-1
```

The `JacobiSymbol(a, n)` is a generalization of the Legendre symbol that allows for a composite second (bottom) argument `n`, although `n` must still be odd and positive.

```
>> JacobiSymbol(12345, 331)
-1
```

Implementation status

- - partially implemented

Github

- [Implementation of JacobiSymbol](#)
-

JavaClass

```
JavaClass[class-name]
```

a `JavaClass` expression can be created with the `LoadJavaClass` function and wraps a Java `java.lang.Class` object. All static method names are assigned to a context which will be created by the last part of the class name.

Note: the Java specific functions which call Java native classes are only available in the MMA mode in a local installation. All symbol and method names have to be case sensitive.

Examples

```
>> clazz= LoadJavaClass["java.lang.Math"]
JavaClass[java.lang.Math]

>> Math`sin[0.5]
0.479426

>> Math`sinh[0.6]
0.636654
```

Related terms

[InstanceOf](#), [JavaNew](#), [JavaObject](#), [JavaObjectQ](#), [JavaShow](#), [LoadJavaClass](#), [SameObjectQ](#)

JavaForm

```
JavaForm(expr)
```

returns the Symja Java form of the `expr`. In Java you can use the created Symja expressions.

```
JavaForm(expr, Float)
```

or

```
JavaForm(expr, Float->True)
```

returns the `java.lang.Math` form of the `expr`.

See:

- [docs.oracle.com - java.lang.Math](https://docs.oracle.com/javase/7/docs/api/java/lang/Math.html)

Examples

JavaForm can add the `F.` prefix for class `org.matheclipse.core.expression.F` if you set `Prefix->True`:

```
>> JavaForm(D(sin(x)*cos(x),x), Prefix->True)
"F.Plus(F.Sqr(F.Cos(F.x)),F.Negate(F.Sqr(F.Sin(F.x))))"

>> JavaForm(I/2*E^((-I)*x)-I/2*E^(I*x))
"Plus(Times(CC(0L,1L,1L,2L),Power(E,Times(CNI,x))),Times(CC(0L,1L,-1L,2L),Power(E,Times
```

JavaForm evaluates its argument before creating the Java form:

```
>> JavaForm(D(sin(x)*cos(x),x))
"Plus(Sqr(Cos(x)),Negate(Sqr(Sin(x))))"
```

You can use `Hold` to suppress the evaluation:

```
>> JavaForm(Hold(D(sin(x)*cos(x),x)))
"D(Times(Sin(x),Cos(x)),x)"

>> JavaForm(Hold(D(sin(x)*cos(x),x)), prefix->True)
"F.D(F.Times(F.Sin(F.x),F.Cos(F.x)),F.x)"
```

Generate output for `java.lang.Math` `double` or `float` expressions:

```
>> JavaForm(E^3-Cos(Pi^2/x), Float)
Math.pow(Math.E,3)-Math.cos(Math.pow(Math.PI,2)/x)
```

Implementation status

- - partially implemented

Github

- [Implementation of JavaForm](#)
-

JavaObject

```
JavaObject[class className]
```

a `JavaObject` can be created with the `JavaNew` function.

Note: the Java specific functions which call Java native classes are only available in the MMA mode in a local installation. All symbol and method names have to be case sensitive.

Examples

```
>> loc = JavaNew["java.util.Locale", "US"]  
JavaObject[class java.util.Locale]  
  
>> ds = JavaNew["java.text.DecimalFormatSymbols", loc]  
JavaObject[class java.text.DecimalFormatSymbols]  
  
>> dm = JavaNew["java.text.DecimalFormat", "#.00", ds]  
JavaObject[class java.text.DecimalFormat]
```

Calls the `format` method of the `dm` Java object.

```
>> dm@format[0.815] // InputForm  
".81"
```

Related terms

[InstanceOf](#), [JavaClass](#), [JavaObject](#), [JavaObjectQ](#), [JavaShow](#), [LoadJavaClass](#), [SameObjectQ](#)

Implementation status

- - supported on Java virtual machine

Github

- [Implementation of JavaNew](#)
-

JavaNew

```
JavaNew["class-name"]
```

create a `JavaObject` from the `class-name` default constructor.

```
JavaNew["class-name", arg1, arg2, ...]
```

create a `JavaObject` from the `class-name` constructor matching the arguments
`arg1, arg2, ...`

Note: the Java specific functions which call Java native classes are only available in the MMA mode in a local installation. All symbol and method names have to be case sensitive.

Examples

```
>> loc = JavaNew["java.util.Locale", "US"]
JavaObject[class java.util.Locale]

>> ds = JavaNew["java.text.DecimalFormatSymbols", loc]
JavaObject[class java.text.DecimalFormatSymbols]

>> dm = JavaNew["java.text.DecimalFormat", "#.00", ds]
JavaObject[class java.text.DecimalFormat]
```

Calls the `format` method of the `dm` Java object.

```
>> dm@format[0.815] // InputForm
".81"
```

Related terms

[InstanceOf](#), [JavaClass](#), [JavaNew](#), [JavaObjectQ](#), [JavaShow](#), [LoadJavaClass](#), [SameObjectQ](#)

Implementation status

- supported on Java virtual machine

Github

- [Implementation of JavaObject](#)
-

JavaObjectQ

```
JavaObjectQ[java-object]
```

return `True` if `java-object` is a `JavaObject` expression.

Note: the Java specific functions which call Java native classes are only available in the MMA mode in a local installation. All symbol and method names have to be case sensitive.

Examples

```
>> loc = JavaNew["java.util.Locale", "US"]
JavaObject[class java.util.Locale]

>> JavaObjectQ[loc]
True
```

Related terms

[InstanceOf](#), [JavaClass](#), [JavaNew](#), [JavaObject](#), [JavaShow](#), [LoadJavaClass](#), [SameObjectQ](#)

Implementation status

- - supported on Java virtual machine

Github

- [Implementation of JavaObjectQ](#)
-

JavaShow

```
JavaShow[ java.awt.Window ]
```

show the `JavaObject` which has to be an instance of `java.awt.Window`.

Note: the Java specific functions which call Java native classes are only available in the MMA mode in a local installation. All symbol and method names have to be case sensitive.

Examples

```
>> frame= JavaNew["javax.swing.JFrame", "Simple JFrame Demo"];  
  
JavaShow[frame]
```

Related terms

[InstanceOf](#), [JavaClass](#), [JavaNew](#), [JavaObject](#), [JavaObjectQ](#), [LoadJavaClass](#), [SameObjectQ](#)

Join

```
Join(l1, l2)
```

concatenates the lists `l1` and `l2`.

Examples

`Join` concatenates lists:

```
>> Join({a, b}, {c, d, e})
{a,b,c,d,e}

>> Join({{a, b}, {c, d}}, {{1, 2}, {3, 4}})
{{a,b},{c,d},{1,2},{3,4}}
```

The concatenated expressions may have any head:

```
>> Join(a + b, c + d, e + f)
a+b+c+d+e+f
```

However, it must be the same for all expressions:

```
>> Join(a + b, c * d)
Join(a+b,c*d)

>> Join(x, y)
Join(x,y)

>> Join(x + y, z)
Join(x+y,z)

>> Join(x + y, y * z, a)
Join(x + y, y z, a)

>> Join(x, y + z, y * z)
Join(x,y+z,y*z)
```

The [A001597](#) Perfect powers: m^k where $m > 0$ and $k \geq 2$

```
>> Join({1}, Select(Range(1770), GCD@@FactorInteger(#)[[All, 2]]>1&))
{1,4,8,9,16,25,27,32,36,49,64,81,100,121,125,128,144,169,196,216,225,243,256,289,324,343,361,400,441,484,529,576,625,676,729,784,841,900,961,1024,1089,1156,1225,1296,1369,1444,1521,1600,1681,1764,1849,1936,2025,2116,2209,2304,2401,2500,2601,2704,2809,2916,3025,3136,3249,3364,3481,3600,3721,3844,3969,4096,4225,4356,4489,4624,4761,4900,5041,5184,5329,5476,5625,5776,5929,6084,6241,6400,6561,6724,6891,7064,7241,7424,7611,7804,7999,8196,8399,8596,8799,8996,9196,9396,9596,9796,9996}
```

Implementation status

- - full supported

Github

- [Implementation of Join](#)
-

JordanDecomposition

```
JordanDecomposition(matrix)
```

calculate the Jordan-decomposition as a list $\{s, j\}$ of a square `matrix` with the property `s.j.Inverse(s) == matrix`, where `s` is the similarity matrix and `j` is the Jordan normal form of the `matrix`.

See:

- [Wikipedia - Jordan normal form](#)

Examples

```
>> JordanDecomposition({{5,4,2,1},{0,1,-1,-1},{-1,-1,3,0},{1,1,-1,2}})
{{{ -1, 1, 1, 1}, {1, -1, 0, 0}, {0, 0, -1, 0}, {0, 1, 1, 0}}, {{1, 0, 0, 0}, {0, 2, 0, 0}, {0, 0, 4, 1}, {0, 0, 0, 4}}}
```

Related terms

[MatrixExp](#), [MatrixFunction](#), [MatrixLog](#), [SchurDecomposition](#)

JSForm

```
JSForm(expr)
```

returns the JavaScript form of the `expr`.

```
JSForm(expr, "Mathcell")
```

returns the JavaScript form of the `expr` with 'Mathcell' flavor output.

JSForm generates JavaScript output for the following JavaScript libraries:

- github.com/jsxgraph/jsxgraph
- github.com/paulmasson/math
- github.com/paulmasson/mathcell

This JavaScript flavour is also used in the `[Manipulate](Manipulate.md)` function.

See:

- developer.mozilla.org - Global Objects Math

Examples

Generate output for JavaScript floating-point arithmetic expressions:

```
>> JSForm(E^3-Cos(Pi^2/x))  
(20.085536923187664)-Math.cos((9.869604401089358)/x)
```

Generate output for MathCell and Math JavaScript libraries:

```
>> JSForm(4*EllipticE(x)+KleinInvariantJ(t)^3, "Mathcell")  
add(mul(4,ellipticE(x)),pow(kleinJ(t),3))
```

With `JSForm` you can display the generated JavaScript form of the `Manipulate` function

```
>> Manipulate(Plot(Sin(x)*Cos(1 + a*x), {x, 0, 2*Pi}), {a,0,10}) // JSForm
```

Related terms

[Manipulate](#)

Implementation status

- - partially implemented

Github

- [Implementation of JSForm](#)
-

KaryTree

`KaryTree(v)`

create a binary tree graph with `v` vertices.

`KaryTree(v, m)`

create a `m`-ary tree graph with `v` vertices. In graph theory, an `m`-ary tree (for nonnegative integers `m`) (also known as `n`-ary, `k`-ary, `k`-way or generic tree) is a graph in which each node has no more than `m` children. A binary tree is an important case where `m = 2`; similarly, a ternary tree is one where `m = 3`.

See:

- [Wikipedia - M-ary tree](#)

Examples

Implementation status

- - experimental

Github

- [Implementation of KaryTree](#)
-

Key

```
Key(key)
```

represents a `key` used to access a value in an association.

```
association[[Key(key)]]
```

determine the value associated to `key` in `association`.

Examples

```
>> <|1 -> a, 3 -> b|>[[Key(3)]]  
b
```

Related terms

[Association](#), [AssociationQ](#), [Counts](#), [Lookup](#), [Keys](#), [KeySort](#), [Values](#)

Implementation status

- - full supported

Github

- [Implementation of Key](#)
-

KeyDrop

```
KeyDrop(<|key1->value1, ...|>, {k1, k2, ...})
```

`KeyDrop` is a function used to remove specified keys `{k1, k2, ...}` and their associated values from an association. It is useful for simplifying or filtering associations by excluding unwanted keys.

Examples

```
>> productInfo = <|"Name" -> "Widget X", "Price" -> 19.99, "InStock" -> True, "SupplierID" -> "ABC"|>

>> KeyDrop(productInfo, {"SupplierID", "InStock"})
<|Name->Widget X, Price->19.99|>
```

Implementation status

- - full supported

Github

- [Implementation of KeyDrop](#)
-

Keys

```
Keys(association)
```

return a list of keys of the `association`.

Examples

```
>> Keys(<|ahey->avalue, bkey->bvalue, ckey->cvalue|>)  
{ahey, bkey, ckey}
```

Related terms

[Association](#), [AssociationQ](#), [Counts](#), [Lookup](#), [Key](#), [KeySort](#), [Values](#)

Implementation status

- - full supported

Github

- [Implementation of Keys](#)
-

KeySelect

```
KeySelect(<|key1->value1, ...|>, head)
```

returns an association of the elements for which `head(keyi)` returns `True`.

```
KeySelect({key1->value1, ...}, head)
```

returns an association of the elements for which `head(keyi)` returns `True`.

Examples

```
>> r = {beta -> 4, alpha -> 2, x -> 4, z -> 2, w -> 0.8};  
  
>> KeySelect(r, MatchQ(#[alpha|x]&)  
<|alpha->2,x->4|>
```

Implementation status

- - full supported

Github

- [Implementation of KeySelect](#)
-

KeySort

```
KeySort(<|key1->value1, ...|>)
```

sort the `<|key1->value1, ...|>` entries by the `key` values.

```
KeySort(<|key1->value1, ...|>, comparator)
```

sort the entries by the `comparator`.

Examples

```
>> KeySort(<|2 -> y, 3 -> z, 1 -> x|>)  
<|1->x,2->y,3->z|>  
  
>> KeySort(<|2 -> y, 3 -> z, 1 -> x|>, Greater)  
<|3->z,2->y,1->x|>
```

Implementation status

- - full supported

Github

- [Implementation of KeySort](#)
-

KeyTake

```
KeyTake(<|key1->value1, ...|>, {k1, k2, ...})
```

returns an association of the rules for which the `k1, k2, ...` are keys in the association.

```
KeyTake({key1->value1, ...}, {k1, k2, ...})
```

returns an association of the rules for which the `k1, k2, ...` are keys in the association.

Examples

```
>> r = {beta -> 4, alpha -> 2, x -> 4, z -> 2, w -> 0.8};  
  
>> KeyTake(r, {alpha, x})  
<|alpha->2, x->4|>
```

Implementation status

- - full supported

Github

- [Implementation of KeyTake](#)
-

KeyValueMap

```
KeyValueMap(head, association)
```

returns a list of the rules pairs `{head(k1, v1), head(k2, v2), ...}` for which the `k1, k2, ...` are the keys and the `v1, v2, ...` are the corresponding values in the association.

Examples

The operator form `KeyValueMap(f)` can be used:

```
>> KeyValueMap(f)[<|a -> 1, b -> 2, c -> 3|>]  
{f(a,1), f(b,2), f(c,3)}
```

KeyValuePattern

```
KeyValuePattern( {rule-pattern1, rule-pattern1,...})
```

`KeyValuePattern` is a pattern-matching construct used to identify elements in a collection (such as a list of associations) that match a specified set of key-value pairs. It is particularly useful for filtering data structures like lists of dictionaries or associations based on specific criteria.

Examples

```
>> peopleData = {<|"Name" -> "Alice", "Age" -> 30, "City" -> "New York"|>,<|"Name" -> "Bob", "Age" -> 25, "City" -> "Los Angeles"|>,<|"Name" -> "David", "Age" -> 25, "City" -> "New York"|>}

>> Cases(peopleData, KeyValuePattern({"Age" -> 25}))
{<|Name->Bob, Age->25, City->Los Angeles|>,<|Name->David, Age->25, City->New York|>}
```

Khinchin

Khinchin

Khinchin's constant

See:

- [Wikipedia:Khinchin's constant](#)

Examples

```
>> N(Khinchin)
2.6854520010653062
```

Implementation status

- - full supported

Github

- [Implementation of Khinchin](#)
-

KolmogorovSmirnovTest

```
KolmogorovSmirnovTest(data)
```

Computes the `p-value`, or *observed significance level*, of a one-sample [Wikipedia: Kolmogorov-Smirnov test](#) evaluating the null hypothesis that `data` conforms to the `NormalDistribution()`.

```
KolmogorovSmirnovTest(data, distribution)
```

Computes the `p-value`, or *observed significance level*, of a one-sample [Wikipedia: Kolmogorov-Smirnov test](#) evaluating the null hypothesis that `data` conforms to the (continuous) `distribution`.

```
KolmogorovSmirnovTest(data, distribution, "TestData")
```

Computes the `test statistic` and the `p-value`, or *observed significance level*, of a one-sample [Wikipedia: Kolmogorov-Smirnov test](#) evaluating the null hypothesis that `data` conforms to the (continuous) `distribution`.

Examples

```
>> data = { 0.53236606, -1.36750258, -1.47239199, -0.12517888, -1.24040594, 1.90357309,
>> KolmogorovSmirnovTest(data)
0.744855

>> KolmogorovSmirnovTest(data, NormalDistribution(), "TestData")
{0.0930213, 0.744855}
```

Implementation status

- - full supported

Github

- [Implementation of KolmogorovSmirnovTest](#)
-

KroneckerDelta

```
KroneckerDelta(arg1, arg2, ..., argN)
```

if all arguments `arg1` to `argN` are equal return `1`, otherwise return `0`.

See

- [Wikipedia - Kronecker delta](#)

Examples

```
>> KroneckerDelta(42)
0

>> KroneckerDelta(42, 42.0, 42)
1
```

Implementation status

- - full supported

Github

- [Implementation of KroneckerDelta](#)
-

KroneckerProduct

```
KroneckerProduct(t1, t2, ...)
```

Kronecker product of the tensors `t1`, `t2`,

See

- [Wikipedia - Kronecker product](#)

Examples

```
>> ta = {{1,4,-7}, {-2,3,3}}; tb = {{8,-9,-6,5},{1,-3,-4,7},{2,8,-8,-3},{1,2,-5,-1}};

>> KroneckerProduct(ta, tb) // MatrixForm
{{8,-9,-6,5,32,-36,-24,20,-56,63,42,-35},
 {1,-3,-4,7,4,-12,-16,28,-7,21,28,-49},
 {2,8,-8,-3,8,32,-32,-12,-14,-56,56,21},
 {1,2,-5,-1,4,8,-20,-4,-7,-14,35,7},
 {-16,18,12,-10,24,-27,-18,15,24,-27,-18,15},
 {-2,6,8,-14,3,-9,-12,21,3,-9,-12,21},
 {-4,-16,16,6,6,24,-24,-9,6,24,-24,-9},
 {-2,-4,10,2,3,6,-15,-3,3,6,-15,-3}}
```

Implementation status

- - partially implemented

Github

- [Implementation of KroneckerProduct](#)
-

Kurtosis

```
Kurtosis(list)
```

gives the Pearson measure of kurtosis for `list` (a measure of existing outliers).

```
Kurtosis(dist)
```

gives the Pearson measure of kurtosis for the distribution `dist`.

See

- [Wikipedia - Kurtosis](#)

Examples

```
>> Kurtosis({1.1, 1.2, 1.4, 2.1, 2.4})  
1.42098
```

Implementation status

- - full supported

Github

- [Implementation of Kurtosis](#)
-

LaguerreL

LaguerreL(n , x)

returns the Laguerre polynomial $L_n(x)$.

See

- [Wikipedia - Laguerre polynomials](#)

Examples

```
>> LaguerreL(8, x)
1-8*x+14*x^2-28/3*x^3+35/12*x^4-7/15*x^5+7/180*x^6-x^7/630+x^8/40320
```

Implementation status

- - partially implemented

Github

- [Implementation of LaguerreL](#)
-

LaplaceTransform

```
LaplaceTransform(f, t, s)
```

returns the laplace transform.

See:

- [Wikipedia - Laplace transform](#)

Examples

```
>> LaplaceTransform(t^2*Exp(2+3*t), t, s)
(-2*E^2)/(3-s)^3
```

Implementation status

- - experimental

Github

- [Implementation of LaplaceTransform](#)
 - [Rule definitions of LaplaceTransform](#)
-

Laplacian

```
Laplacian(function, {x1, x2, ... , xN})
```

returns the Laplace operator

```
Laplacian(function, {x1, x2, ... , xN}, metric-string)
```

returns the Laplace operator for the given `metric-string`. Possible metrics are `"Cartesian"`, `"Cylindrical"`, `"Polar"`, `"Spherical"`.

See:

- [Wikipedia - Laplace operator](#)

Examples

```
>> Laplacian(Sin(x^2 + y^2), {x, y}) // Simplify
4*(Cos(x^2+y^2)-x^2*Sin(x^2+y^2)-y^2*Sin(x^2+y^2))

>> Laplacian(Sin(x^2 + y^2), {x, y}, "Cartesian")
4*Cos(x^2+y^2)-4*x^2*Sin(x^2+y^2)-4*y^2*Sin(x^2+y^2)

>> Laplacian(Sin(r^2), {r, t}, "Polar")
4*Cos(r^2)-4*r^2*Sin(r^2)
```

Implementation status

- - partially implemented

Github

- [Implementation of Laplacian](#)
-

Last

```
Last(expr)
```

returns the last element in `expr`.

Examples

`Last(expr)` is equivalent to `expr[[-1]]`.

```
>> Last({a, b, c})  
c
```

Nonatomic expression expected.

```
>> Last(x)  
Last(x)
```

Implementation status

- - full supported

Github

- [Implementation of Last](#)
-

LCM

```
LCM(n1, n2, ...)
```

computes the least common multiple of the given integers.

See

- [Wikipedia - Least common multiple](#)

Examples

```
>> LCM(15, 20)
60
>> LCM(20, 30, 40, 50)
600
```

Related terms

[GCD](#)

Implementation status

- - full supported

Github

- [Implementation of LCM](#)
-

LeafCount

LeafCount(expr)

returns the total number of indivisible subexpressions in `expr`.

Examples

```
>> LeafCount(1 + x + y^a)
6

>> LeafCount(f(x, y))
3

>> LeafCount({1 / 3, 1 + I})
7

>> LeafCount(Sqrt(2))
5

>> LeafCount(100!)
1

>> LeafCount(f(1, 2)[x, y])
5

>> LeafCount(10+I)
3
```

Related terms

[FullForm](#)

Implementation status

- - full supported

Github

- [Implementation of LeafCount](#)
-

LeastSquares

```
LeastSquares(matrix, right)
```

solves the linear least-squares problem ' $\text{matrix} \cdot x = \text{right}$ '.

Examples

```
>> LeastSquares(Table(Complex(i,Rational(2 * i + 2 + j, 1 + 9 * i + j)),{i,0,3},{j,0,2}  
{-1577780898195/827587904419-I*11087326045520/827587904419,35583840059240/5793115330933
```

Implementation status

- - full supported

Github

- [Implementation of LeastSquares](#)
-

LegendreP

```
LegendreP(n, x)
```

returns the Legendre polynomial $P_n(x)$.

See

- [Wikipedia - Legendre polynomials](#)
- [Fungrim - Legendre polynomials](#)

Examples

```
>> LegendreP(4, x)
3/8-15/4*x^2+35/8*x^4
```

Implementation status

- - partially implemented

Github

- [Implementation of LegendreP](#)
-

LegendreQ

`LegendreQ(n, x)`

returns the Legendre functions of the second kind $Q_n(x)$.

See

- [Wikipedia - Legendre polynomials](#)

Examples

```
>> Expand(LegendreQ(4, z))  
55/24*z-35/8*z^3-3/16*Log(1-z)+15/8*z^2*Log(1-z)-35/16*z^4*Log(1-z)+3/16*Log(1+z)-15/8*
```

Implementation status

- - partially implemented

Github

- [Implementation of LegendreQ](#)
-

Length

```
Length(expr)
```

returns the number of leaves in `expr`.

Examples

Length of a list:

```
>> Length({1, 2, 3})  
3
```

'Length' operates on the 'FullForm' of expressions:

```
>> Length(Exp(x))  
2  
  
>> FullForm(Exp(x))  
Power(E, x)
```

The length of atoms is 0:

```
>> Length(a)  
0
```

Note that rational and complex numbers are atoms, although their 'FullForm' might suggest the opposite:

```
>> Length(1/3)  
0  
  
>> FullForm(1/3)  
Rational(1, 3)
```

Implementation status

- - full supported

Github

- [Implementation of Length](#)
-

LengthWhile

```
LengthWhile({e1, e2, ...}, head)
```

returns the number of elements `ei` at the start of list for which `head(ei)` returns `True`

.

Examples

```
>> LengthWhile({1, 2, 3, 10, 5, 8, 42, 11}, # < 10 &)  
3
```

Implementation status

- - full supported

Github

- [Implementation of LengthWhile](#)
-

LerchPhi

```
LerchPhi(z, s, a)
```

returns the Lerch transcendent function.

See:

- [Wikipedia - Lerch transcendent](#)

Examples

```
>> LerchPhi(-1,1,0)", //  
-Log(2)
```

Related terms

[HurwitzLerchPhi[(HurwitzLerchPhi.md), [HurwitzZeta](#), [PolyLog](#), [Zeta](#)

Implementation status

- - partially implemented

Github

- [Implementation of LerchPhi](#)
-

Less

```
Less(x, y)
```

```
x < y
```

yields `True` if `x` is known to be less than `y`.

```
lhs < rhs
```

represents the inequality `lhs < rhs`.

Examples

```
>> 3<4
```

```
True
```

```
>> {Less(), Less(x), Less(1)}
```

```
{True, True, True}
```

Implementation status

- - full supported

Github

- [Implementation of Less](#)
-

LessEqual

```
LessEqual(x, y)
```

```
x <= y
```

yields `True` if `x` is known to be less than or equal `y`.

```
lhs <= rhs
```

represents the inequality `lhs` `rhs`.

Examples

```
>> 3<=4
```

```
True
```

```
>> {LessEqual(), LessEqual(x), LessEqual(1)}  
{True, True, True}
```

Implementation status

- - full supported

Github

- [Implementation of LessEqual](#)
-

LessEqualThan

```
LessEqualThan(rhs)
```

operator applied to an expr `lhs` (`LessEqualThan(rhs)[lhs]`) returns `LessEqual(lhs, rhs)`.

Examples

LessThan

```
LessThan(rhs)
```

operator applied to an expr `lhs` (`LessThan(rhs)[lhs]`) returns `Less(lhs, rhs)`.

Examples

LetterCharacter

LetterCharacter

represents letters..

Examples

```
>> StringMatchQ( #, LetterCharacter) & /@ {"a", "1", "A", " ", "."}
{True, False, True, False, False}
```

LetterCharacter also matches unicode characters.

```
>> StringMatchQ("\[Lambda]", LetterCharacter)
```

LetterCounts

```
LetterCounts(string)
```

count the number of each distinct character in the `string` and return the result as an association `<|char->counter1, ...|>`.

See

- [Wikipedia - The quick brown fox jumps over the lazy dog](#)

Examples

```
>> LetterCounts("The quick brown fox jumps over the lazy dog") // InputForm
<|"T"->1, " "->8, "a"->1, "b"->1, "c"->1, "d"->1, "e"->3, "f"->1, "g"->1, "h"->2, "i"->1, "j"->1,
"k"->1, "l"->1, "m"->1, "n"->1, "o"->4, "p"->1, "q"->1, "r"->2, "s"->1, "t"->1, "u"->2, "v"->1, "w"
```

Implementation status

- - full supported

Github

- [Implementation of LetterCounts](#)
-

LetterNumber

```
LetterNumber(character)
```

returns the position of the `character` in the English alphabet.

```
FromLetterNumber(character, language-string)
```

returns the position of the `character` in the languages alphabet.

Examples

```
>> LetterNumber("b")
2

>> LetterNumber("B")
2

>> LetterNumber("ij", "Dutch")
26

>> LetterNumber("dzs", "Hungarian")
8
```

Related terms

[Alphabet](#), [FromLetterNumber](#)

Implementation status

- - full supported

Github

- [Implementation of LetterNumber](#)
-

LetterQ

```
LetterQ(expr)
```

tests whether `expr` is a string, which only contains letters.

A character is considered to be a letter if its general category type, provided by the Java method `Character#getType()` is any of the following:

- `UPPERCASE_LETTER`
- `LOWERCASE_LETTER`
- `TITLECASE_LETTER`
- `MODIFIER_LETTER`
- `OTHER_LETTER`

Not all letters have case. Many characters are letters but are neither uppercase nor lowercase nor titlecase.

Examples

```
>> LetterQ("k")
True

>> LetterQ("7")", //
False
```

Implementation status

- - full supported

Github

- [Implementation of LetterQ](#)
-

Level

```
Level(expr, levelspec)
```

gives a list of all sub-expressions of `expr` at the level(s) specified by `levelspec`.

Level uses standard level specifications:

```
n
```

levels 1 through n

```
Infinity
```

all levels from level 1

```
{n}
```

level n only

```
{m, n}
```

levels m through n

Level 0 corresponds to the whole expression. A negative level `-n` consists of parts with depth n.

Examples

Level `-1` is the set of atoms in an expression:

```
>> Level(a + b ^ 3 * f(2 x ^ 2), {-1})
{a,b,3,2,x,2}

>> Level({{{{a}}}}, 3)
{{a},{a}},{{{a}}}}

>> Level({{{{a}}}}, -4)
{{{a}}}}

>> Level({{{{a}}}}, -5)
{}
```

```
>> Level(h0(h1(h2(h3(a)))), {0, -1})
{a, h3(a), h2(h3(a)), h1(h2(h3(a))), h0(h1(h2(h3(a))))}
```

Use the option `Heads -> True` to include heads:

```
>> Level({{{{a}}}}, 3, Heads -> True)
{List, List, List, {a}, {{a}}, {{{a}}}}

>> Level(x^2 + y^3, 3, Heads -> True)
{Plus, Power, x, 2, x^2, Power, y, 3, y^3}

>> Level(a ^ 2 + 2 * b, {-1}, Heads -> True)
{Plus, Power, a, 2, Times, 2, b}

>> Level(f(g(h))[x], {-1}, Heads -> True)
{f, g, h, x}

>> Level(f(g(h))[x], {-2, -1}, Heads -> True)
{f, g, h, g(h), x, f(g(h))[x]}
```

Implementation status

- - full supported

Github

- [Implementation of Level](#)
-

LevelQ

```
LevelQ(expr)
```

tests whether `expr` is a valid level specification.

Examples

```
>> LevelQ(2)
True

>> LevelQ({2, 4})
True

>> LevelQ(Infinity)
True

>> LevelQ(a + b)
False
```

Implementation status

- - full supported

Github

- [Implementation of LevelQ](#)
-

LeviCivitaTensor

LeviCivitaTensor(n)

returns the n -dimensional Levi-Civita tensor as sparse array. The Levi-Civita symbol represents a collection of numbers; defined from the sign of a permutation of the natural numbers $1, 2, \dots, n$, for some positive integer n .

See:

- [Wikipedia - Levi-Civita symbol](#)

Examples

```
>> LeviCivitaTensor(4) // Normal
{{{0,0,0,0},{0,0,0,0},{0,0,0,0},{0,0,0,0}},{0,0,0,0},{0,0,0,0},{0,0,0,1},{0,0,-1,0}},{0,0,0,-1},{0,0,0,0},{0,1,0,0}},{0,0,0,0},{0,0,1,0},{0,-1,0,0},{0,0,0,0}},{0,0,0,-1},{0,0,0,0},{0,0,1,0}},{0,0,0,0},{0,0,0,0},{0,0,0,0},{0,0,0,0}},{0,0,0,1},{0,0,0,0},{0,0,0,0},{-1,0,0,0}},{0,0,-1,0},{0,0,0,0},{1,0,0,0},{0,0,0,0}},{0,0,0,1},{0,0,0,0},{0,0,0,0},{0,-1,0,0}},{0,0,0,-1},{0,0,0,0},{1,0,0,0}},{0,0,0,0},{0,0,0,0},{0,0,0,0},{0,0,0,0}},{0,1,0,0},{-1,0,0,0},{0,0,0,0},{0,0,0,0}},{0,0,-1,0},{0,1,0,0},{0,0,0,0}},{0,0,1,0},{0,0,0,0},{-1,0,0,0},{0,0,0,0}},{0,-1,0,0},{1,0,0,0},{0,0,0,0},{0,0,0,0}},{0,0,0,0},{0,0,0,0},{0,0,0,0},{0,0,0,0}}}
```

Return the n -dimensional Levi-Civita tensor as a list by appending the symbol `List`

```
>> LeviCivitaTensor(3,List)
{{{0,0,0},{0,0,1},{0,-1,0}},{0,0,-1},{0,0,0},{1,0,0}},{0,1,0},{-1,0,0},{0,0,0}}}
```

Implementation status

- - partially implemented

Github

- [Implementation of LeviCivitaTensor](#)
-

LightBlue

LightBlue

RGB color value for the color light blue

Examples

```
>> LightBlue  
RGBColor(0.87,0.94,1.0)
```

LightBrown

LightBrown

RGB color value for the color light brown

Examples

```
>> LightBrown  
RGBColor(0.94,0.91,0.88)
```

LightCyan

LightCyan

RGB color value for the color light cyan

Examples

```
>> LightCyan  
RGBColor(0.9,1.0,1.0)
```

LightGray

LightGray

RGB color value for the color light gray

Examples

```
>> LightGray  
RGBColor(0.85,0.85,0.85)
```

LightGreen

LightGreen

RGB color value for the color light green

Examples

```
>> LightGreen  
RGBColor(0.0,1.0,0.0)
```

LightMagenta

LightMagenta

RGB color value for the color light magenta

Examples

```
>> LightMagenta  
RGBColor(1.0,0.9,1.0)
```

LightOrange

```
LightOrange
```

RGB color value for the color light orange

Examples

```
>> LightOrange  
RGBColor(1.0,0.9,0.8)
```

LightPink

LightPink

RGB color value for the color light pink

Examples

```
>> LightPink  
RGBColor(1.0, 0.925, 0.925)
```

LightPurple

LightPurple

RGB color value for the color light purple

Examples

```
>> LightPurple  
RGBColor(0.94, 0.88, 0.94)
```

LightRed

LightRed

RGB color value for the color light red

Examples

```
>> LightRed  
RGBColor(1.0,0.85,0.85)
```

LightYellow

LightYellow

RGB color value for the color light yellow

Examples

```
>> LightYellow  
RGBColor(1.0,1.0,0.85)
```

Limit

```
Limit(expr, x->x0)
```

gives the limit of `expr` as `x` approaches `x0`

```
Limit(expr, x->x0, Direction->-1)
```

gives the limit as `x` decreases in value approaching `x0` (`x` approaches `x0` "from the right" or "from above")

```
Limit(expr, x->x0, Direction->1)
```

gives the limit as `x` increases in value approaching `x0` (`x` approaches `x0` "from the left" or "from below")

See:

- [Wikipedia - Limit of a function](#)
- [Wikipedia - One-sided limit](#)
- [Wikipedia - L'Hôpital's rule](#)

Examples

```
>> Limit(7+Sin(x)/x, x->Infinity)
7

>> Limit(x^(-2/3), x->0)
Indeterminate

>> Limit(x^(-2/3), x->0 , Direction->-1)
Infinity
```

Related terms

[D](#), [DSolve](#), [Integrate](#), [ND](#), [NIntegrate](#)

Implementation status

- - partially implemented

Github

- [Implementation of Limit](#)
 - [Rule definitions of Limit](#)
-

LinearModelFit

```
LinearModelFit({{x11,x12,y1},{x21,x22,y2},...}, {Func1, func2,...}, {var1, var2,...})
```

Create a linear regression model from a matrix of observed value pairs `{xij, ..., yi}`.

```
LinearModelFit({y1,y2,y3,...}, expr, symbol)
```

Create a linear regression model from a vector of observed value pairs `{{1.0, y1}, {2.0, y2}, {3.0, y3}, ...}`.

```
LinearModelFit({design-matrix, response-vector})
```

Create a linear regression model from `design-matrix` and `response-vector`.

See:

- [Wikipedia - Linear regression](#)

Examples

```
>> nlm=LinearModelFit({{1, 1, 6.5}, {2, 1, 8.4}, {1, 2, 9.6}, {2, 2, 12.1}, {3, 2, 14.6}},  
FittedModel[1.7+1.6*a+2.9*b+0.3*a*b]  
  
>>nlm // Normal  
1.7+1.6*a+2.9*b+0.3*a*b
```

The following properties are supported for `FittedModel`:

```
>> nlm("BestFit")  
1.7+1.6*a+2.9*b+0.3*a*b");  
  
>> nlm("BestFitParameters")  
{1.7,1.6,2.9,0.3}");  
  
>> nlm("EstimatedVariance")  
0.06  
  
>> nlm("FitResiduals")  
{5.32907*10^-15,1.77636*10^-15,-0.1,0.2,-0.1}  
  
>> nlm("ParameterErrors")  
{1.15758,0.714143,0.663325,0.387298}
```

```
>> nlm("RSquared")
0.99989

>> LinearModelFit({ { 1, 3 }, { 2, 5 }, { 3, 7 }, { 4, 14 }, { 5, 11 } }, x, x) // Normal
FittedModel[0.5+2.5*x]
```

Related terms

[DesignMatrix](#), [Fit](#), [FindFit](#), [FittedModel](#)

Implementation status

- - full supported

Github

- [Implementation of LinearModelFit](#)
-

LinearProgramming

```
LinearProgramming(coefficientsOfLinearObjectiveFunction, constraintList, constraintRela
```

the `LinearProgramming` function provides an implementation of [George Dantzig's simplex algorithm](#) for solving linear optimization problems with linear equality and inequality constraints and implicit non-negative variables.

See

- [Wikipedia - Linear programming](#)

Examples

```
>> LinearProgramming({-2, 1, -5}, {{1, 2, 0},{3, 2, 0},{0,1,0},{0,0,1}}, {{6, -1},{12, -1},  
{4.0,0.0,1.0}}
```

solves the linear problem:

```
Minimize -2x + y - 5
```

with the constraints:

```
x + 2y <= 6  
3x + 2y <= 12  
x >= 0  
y >= 0
```

Related terms

[NMaximize](#), [NMinimize](#)

Implementation status

- - partially implemented

Github

- [Implementation of LinearProgramming](#)
-

LinearRecurrence

```
LinearRecurrence(list1, list2, n)
```

solve the linear recurrence and return the generated sequence of elements.

```
LinearRecurrence(list1, list2, {n})
```

solve the linear recurrence and return the n -th element of the list.

See

- [Wikipedia - Recurrence relation](#)
- [OEIS - Recurrence - Linear recurrences with constant coefficients](#)

Examples

The [A001608 Perrin](#) integer sequence

```
>> LinearRecurrence({0, 1, 1}, {3, 0, 2}, 50)
{3, 0, 2, 3, 2, 5, 5, 7, 10, 12, 17, 22, 29, 39, 51, 68, 90, 119, 158, 209, 277, 367, 486, 644, 853, 1130, 1497, 1995, 2689, 3592, 4755, 6227, 8170, 10743, 14097, 18402, 23847, 30712, 39307, 50955, 66982, 87999, 115443, 150915, 196117, 253750, 325522, 414247, 523029, 665000, 844019, 1075000, 1374019, 1747000, 2209000, 2775000, 3460000, 4290000, 5290000, 6390000, 7700000, 9250000, 10990000, 12950000, 15090000, 17470000, 20090000, 22900000, 26000000, 29400000, 33100000, 37000000, 41100000, 45400000, 49900000, 54600000, 59500000, 64600000, 69900000, 75400000, 81100000, 87000000, 93100000, 99400000, 105900000, 112600000, 119500000, 126600000, 133900000, 141400000, 149100000, 157000000, 165100000, 173400000, 181900000, 190600000, 199500000, 208600000, 217900000, 227400000, 237100000, 247000000, 257100000, 267400000, 277900000, 288600000, 299500000, 310600000, 321900000, 333400000, 345100000, 357000000, 369100000, 381400000, 393900000, 406600000, 419500000, 432600000, 445900000, 459400000, 473100000, 487000000, 501100000, 515400000, 529900000, 544600000, 559500000, 574600000, 589900000, 605400000, 621300000, 637400000, 653700000, 670200000, 686900000, 703800000, 720900000, 738200000, 755700000, 773400000, 791300000, 809400000, 827700000, 846200000, 864900000, 883800000, 902900000, 922200000, 941700000, 961400000, 981300000, 1001400000, 1021700000, 1042200000, 1062900000, 1083800000, 1104900000, 1126200000, 1147700000, 1169400000, 1191300000, 1213400000, 1235700000, 1258200000, 1280900000, 1303800000, 1326900000, 1350200000, 1373700000, 1397400000, 1421300000, 1445400000, 1469700000, 1494200000, 1518900000, 1543800000, 1568900000, 1594200000, 1619700000, 1645400000, 1671300000, 1697400000, 1723700000, 1750200000, 1776900000, 1803800000, 1830900000, 1858200000, 1885700000, 1913400000, 1941300000, 1969400000, 1997700000, 2026200000, 2054900000, 2083800000, 2112900000, 2142200000, 2171700000, 2201400000, 2231300000, 2261400000, 2291700000, 2322200000, 2352900000, 2383800000, 2414900000, 2446200000, 2477700000, 2509400000, 2541300000, 2573400000, 2605700000, 2638200000, 2670900000, 2703800000, 2736900000, 2770200000, 2803700000, 2837400000, 2871300000, 2905400000, 2939700000, 2974200000, 3008900000, 3043800000, 3078900000, 3114200000, 3149700000, 3185400000, 3221300000, 3257400000, 3293700000, 3330200000, 3366900000, 3403800000, 3440900000, 3478200000, 3515700000, 3553400000, 3591300000, 3629400000, 3667700000, 3706200000, 3744900000, 3783800000, 3822900000, 3862200000, 3901700000, 3941300000, 3981100000, 4021100000, 4061300000, 4101700000, 4142200000, 4182900000, 4223800000, 4264900000, 4306200000, 4347700000, 4389400000, 4431300000, 4473400000, 4515700000, 4558200000, 4600900000, 4643800000, 4686900000, 4730200000, 4773700000, 4817400000, 4861300000, 4905400000, 4949700000, 4994200000, 5038900000, 5083800000, 5128900000, 5174200000, 5219700000, 5265400000, 5311300000, 5357400000, 5403700000, 5450200000, 5496900000, 5543800000, 5590900000, 5638200000, 5685700000, 5733400000, 5781300000, 5829400000, 5877700000, 5926200000, 5974900000, 6023800000, 6072900000, 6122200000, 6171700000, 6221300000, 6271100000, 6321100000, 6371300000, 6421700000, 6472200000, 6522900000, 6573800000, 6624900000, 6676200000, 6727700000, 6779400000, 6831300000, 6883400000, 6935700000, 6988200000, 7040900000, 7093800000, 7146900000, 7199200000, 7251700000, 7304400000, 7357300000, 7410400000, 7463700000, 7517200000, 7570900000, 7624800000, 7678900000, 7733200000, 7787700000, 7842400000, 7897300000, 7952400000, 8007700000, 8063200000, 8118900000, 8174800000, 8230900000, 8287200000, 8343700000, 8400400000, 8457300000, 8514400000, 8571700000, 8629200000, 8686900000, 8744800000, 8802900000, 8861200000, 8919700000, 8978400000, 9037300000, 9096400000, 9155700000, 9215200000, 9274900000, 9334800000, 9394900000, 9455200000, 9515700000, 9576400000, 9637300000, 9698400000, 9759700000, 9821200000, 9882900000, 9944800000, 10006900000, 10069200000, 10131700000, 10194400000, 10257300000, 10320400000, 10383700000, 10447200000, 10510900000, 10574800000, 10638900000, 10703200000, 10767700000, 10832400000, 10897300000, 10962400000, 11027700000, 11093200000, 11158900000, 11224800000, 11290900000, 11357200000, 11423700000, 11490400000, 11557300000, 11624400000, 11691700000, 11759200000, 11826900000, 11894800000, 11962900000, 12031200000, 12099700000, 12168400000, 12237300000, 12306400000, 12375700000, 12445200000, 12514900000, 12584800000, 12654900000, 12725200000, 12795700000, 12866400000, 12937300000, 13008400000, 13079700000, 13151200000, 13222900000, 13294800000, 13366900000, 13439200000, 13511700000, 13584400000, 13657300000, 13730400000, 13803700000, 13877200000, 13950900000, 14024800000, 14098900000, 14173200000, 14247700000, 14322400000, 14397300000, 14472400000, 14547700000, 14623200000, 14698900000, 14774800000, 14850900000, 14927200000, 15003700000, 15080400000, 15157300000, 15234400000, 15311700000, 15389200000, 15466900000, 15544800000, 15622900000, 15701200000, 15779700000, 15858400000, 15937300000, 16016400000, 16095700000, 16175200000, 16254900000, 16334800000, 16414900000, 16495200000, 16575700000, 16656400000, 16737300000, 16818400000, 16899700000, 16981200000, 17062900000, 17144800000, 17226900000, 17309200000, 17391700000, 17474400000, 17557300000, 17640400000, 17723700000, 17807200000, 17890900000, 17974800000, 18058900000, 18143200000, 18227700000, 18312400000, 18397300000, 18482400000, 18567700000, 18653200000, 18738900000, 18824800000, 18910900000, 18997200000, 19083700000, 19170400000, 19257300000, 19344400000, 19431700000, 19519200000, 19606900000, 19694800000, 19782900000, 19871200000, 19959700000, 20048400000, 20137300000, 20226400000, 20315700000, 20405200000, 20494900000, 20584800000, 20674900000, 20765200000, 20855700000, 20946400000, 21037300000, 21128400000, 21219700000, 21311200000, 21402900000, 21494800000, 21586900000, 21679200000, 21771700000, 21864400000, 21957300000, 22050400000, 22143700000, 22237200000, 22330900000, 22424800000, 22518900000, 22613200000, 22707700000, 22802400000, 22897300000, 22992400000, 23087700000, 23183200000, 23278900000, 23374800000, 23470900000, 23567200000, 23663700000, 23760400000, 23857300000, 23954400000, 24051700000, 24149200000, 24246900000, 24344800000, 24442900000, 24541200000, 24639700000, 24738400000, 24837300000, 24936400000, 25035700000, 25135200000, 25234900000, 25334800000, 25434900000, 25535200000, 25635700000, 25736400000, 25837300000, 25938400000, 26039700000, 26141200000, 26242900000, 26344800000, 26446900000, 26549200000, 26651700000, 26754400000, 26857300000, 26960400000, 27063700000, 27167200000, 27270900000, 27374800000, 27478900000, 27583200000, 27687700000, 27792400000, 27897300000, 28002400000, 28107700000, 28213200000, 28318900000, 28424800000, 28530900000, 28637200000, 28743700000, 28850400000, 28957300000, 29064400000, 29171700000, 29279200000, 29386900000, 29494800000, 29602900000, 29711200000, 29819700000, 29928400000, 30037300000, 30146400000, 30255700000, 30365200000, 30474900000, 30584800000, 30694900000, 30805200000, 30915700000, 31026400000, 31137300000, 31248400000, 31359700000, 31471200000, 31582900000, 31694800000, 31806900000, 31919200000, 32031700000, 32144400000, 32257300000, 32370400000, 32483700000, 32597200000, 32710900000, 32824800000, 32938900000, 33053200000, 33167700000, 33282400000, 33397300000, 33512400000, 33627700000, 33743200000, 33858900000, 33974800000, 34090900000, 34207200000, 34323700000, 34440400000, 34557300000, 34674400000, 34791700000, 34909200000, 35026900000, 35144800000, 35262900000, 35381200000, 35499700000, 35618400000, 35737300000, 35856400000, 35975700000, 36095200000, 36214900000, 36334800000, 36454900000, 36575200000, 36695700000, 36816400000, 36937300000, 37058400000, 37179700000, 37301200000, 37422900000, 37544800000, 37666900000, 37789200000, 37911700000, 38034400000, 38157300000, 38280400000, 38403700000, 38527200000, 38650900000, 38774800000, 38898900000, 39023200000, 39147700000, 39272400000, 39397300000, 39522400000, 39647700000, 39773200000, 39898900000, 40024800000, 40150900000, 40277200000, 40403700000, 40530400000, 40657300000, 40784400000, 40911700000, 41039200000, 41166900000, 41294800000, 41422900000, 41551200000, 41679700000, 41808400000, 41937300000, 42066400000, 42195700000, 42325200000, 42454900000, 42584800000, 42714900000, 42845200000, 42975700000, 43106400000, 43237300000, 43368400000, 43499700000, 43631200000, 43762900000, 43894800000, 44026900000, 44159200000, 44291700000, 44424400000, 44557300000, 44690400000, 44823700000, 44957200000, 45090900000, 45224800000, 45358900000, 45493200000, 45627700000, 45762400000, 45897300000, 46032400000, 46167700000, 46303200000, 46438900000, 46574800000, 46710900000, 46847200000, 46983700000, 47120400000, 47257300000, 47394400000, 47531700000, 47669200000, 47806900000, 47944800000, 48082900000, 48221200000, 48359700000, 48498400000, 48637300000, 48776400000, 48915700000, 49055200000, 49194900000, 49334800000, 49474900000, 49615200000, 49755700000, 49896400000, 50037300000, 50178400000, 50319700000, 50461200000, 50602900000, 50744800000, 50886900000, 51029200000, 51171700000, 51314400000, 51457300000, 51600400000, 51743700000, 51887200000, 52030900000, 52174800000, 52318900000, 52463200000, 52607700000, 52752400000, 52897300000, 53042400000, 53187700000, 53333200000, 53478900000, 53624800000, 53770900000, 53917200000, 54063700000, 54210400000, 54357300000, 54504400000, 54651700000, 54799200000, 54946900000, 55094800000, 55242900000, 55391200000, 55539700000, 55688400000, 55837300000, 55986400000, 56135700000, 56285200000, 56434900000, 56584800000, 56734900000, 56885200000, 57035700000, 57186400000, 57337300000, 57488400000, 57639700000, 57791200000, 57942900000, 58094800000, 58246900000, 58399200000, 58551700000, 58704400000, 58857300000, 59010400000, 59163700000, 59317200000, 59470900000, 59624800000, 59778900000, 59933200000, 60087700000, 60242400000, 60397300000, 60552400000, 60707700000, 60863200000, 61018900000, 61174800000, 61330900000, 61487200000, 61643700000, 61800400000, 61957300000, 62114400000, 62271700000, 62429200000, 62586900000, 62744800000, 62902900000, 63061200000, 63219700000, 63378400000, 63537300000, 63696400000, 63855700000, 64015200000, 64174900000, 64334800000, 64494900000, 64655200000, 64815700000, 64976400000, 65137300000, 65298400000, 65459700000, 65621200000, 65782900000, 65944800000, 66106900000, 66269200000, 66431700000, 66594400000, 66757300000, 66920400000, 67083700000, 67247200000, 67410900000, 67574800000, 67738900000, 67903200000, 68067700000, 68232400000, 68397300000, 68562400000, 68727700000, 68893200000, 69058900000, 69224800000, 69390900000, 69557200000, 69723700000, 69890400000, 70057300000, 70224400000, 70391700000, 70559200000, 70726900000, 70894800000, 71062900000, 71231200000, 71399700000, 71568400000, 71737300000, 71906400000, 72075700000, 72245200000, 72414900000, 72584800000, 72754900000, 72925200000, 73095700000, 73266400000, 73437300000, 73608400000, 73779700000, 73951200000, 74122900000, 74294800000, 74466900000, 74639200000, 74811700000, 74984400000, 75157300000, 75330400000, 75503700000, 75677200000, 75850900000, 76024800000, 76198900000, 76373200000, 76547700000, 76722400000, 76897300000, 77072400000, 77247700000, 77423200000, 77598900000, 77774800000, 77950900000, 78127200000, 78303700000, 78480400000, 78657300000, 78834400000, 79011700000, 79189200000, 79366900000, 79544800000, 79722900000, 79901200000, 80079700000, 80258400000, 80437300000, 80616400000, 80795700000, 80975200000, 81154900000, 81334800000, 81514900000, 81695200000, 81875700000, 82056400000, 82237300000, 82418400000, 82599700000, 82781200000, 82962900000, 83144800000, 83326900000, 83509200000, 83691700000, 83874400000, 84057300000, 84240400000, 84423700000, 84607200000, 84790900000, 84974800000, 85158900000, 85343200000, 85527700000, 8
```

Implementation status

- - full supported

Github

- [Implementation of LinearRecurrence](#)
-

LinearSolve

```
LinearSolve(matrix, right)
```

solves the linear equation system `matrix . x = right` and returns one corresponding solution `x`.

See

- [Wikipedia - System of linear equations - Solving a linear system](#)

Examples

```
>> LinearSolve({{1, 1, 0}, {1, 0, 1}, {0, 1, 1}}, {1, 2, 3})  
{0,1,2}
```

Test the solution:

```
>> {{1, 1, 0}, {1, 0, 1}, {0, 1, 1}} . {0, 1, 2}  
{1,2,3}
```

If there are several solutions, one arbitrary solution is returned:

```
>> LinearSolve({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}, {1, 1, 1})  
{-1,1,0}
```

Infeasible systems are reported:

```
>> LinearSolve({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}, {1, -2, 3})  
  
LinearSolve({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}, {1, -2, 3})
```

Argument `{1, {2}}` at position 1 is not a non-empty rectangular matrix.

```
>> LinearSolve({1, {2}}, {1, 2})  
LinearSolve({1, {2}}, {1, 2})  
  
>> LinearSolve({{1, 2}, {3, 4}}, {1, {2}})  
LinearSolve({{1, 2}, {3, 4}}, {1, {2}})
```

Implementation status

- - full supported

Github

- [Implementation of LinearSolve](#)
-

List

```
List(e1, e2, ..., ei)
```

or

```
{e1, e2, ..., ei}
```

represents a list containing the elements `e1...ei`.

Examples

`List` is the head of lists:

```
>> Head({1, 2, 3})  
List
```

Lists can be nested:

```
>> {{a, b, {c, d}}}  
{{a, b, {c, d}}}
```

Github

- [Implementation of List](#)
-

Listable

Listable

is an attribute specifying that a function should be automatically applied to each element of a list.

Examples

```
>> SetAttributes(f, Listable)
>> f({1, 2, 3}, {4, 5, 6})
{f(1,4),f(2,5),f(3,6)}

>> f({1, 2, 3}, 4)
{f(1,4),f(2,4),f(3,4)}

>> {{1, 2}, {3, 4}} + {5, 6}
{{6,7},{9,10}}
```

Related terms

[Attributes](#), [ClearAttributes](#), [Constant](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#), [SetAttributes](#)

ListConvolve

```
ListConvolve(kernel-list, tensor-list)
```

create the convolution of the `kernel-list` with `tensor-list`.

Examples

```
>> ListConvolve({x, y}, {a, b, c, d, e, f})  
{b*x+a*y, c*x+b*y, d*x+c*y, e*x+d*y, f*x+e*y}
```

Implementation status

- - partially implemented

Github

- [Implementation of ListConvolve](#)
-

ListCorrelate

```
ListCorrelate(kernel-list, tensor-list)
```

create the correlation of the `kernel-list` with `tensor-list`.

Examples

```
>> ListCorrelate({{u, v}, {w, x}}, {{a,b,c,p}, {d,e,f,q}, {g, h, i,r}})
{{a*u+b*v+d*w+e*x, b*u+c*v+e*w+f*x, c*u+p*v+f*w+q*x}, {d*u+e*v+g*w+h*x, e*u+f*v+h*w+i*x, f*u
```

Implementation status

- - partially implemented

Github

- [Implementation of ListCorrelate](#)
-

ListLinePlot

```
ListLinePlot( { list-of-points } )
```

generate a JavaScript list line plot control for the `list-of-points`.

Note: This feature is available in the console app and in the web interface.

Examples

In the console apps, this command shows an HTML page with a JavaScript list line plot control.

```
>> Manipulate(ListLinePlot(Table({Sin(t), Cos(t*a)}, {t, 50})), {a,1,4,1})
```

With `JSForm` you can display the generated JavaScript form of the `Manipulate` function

```
>> Manipulate(ListLinePlot(Table({Sin(t), Cos(t*a)}, {t, 50})), {a,1,4,1}) // JSForm
```

Related terms

[JSForm](#) [Manipulate](#) [ParametricPlot](#) [Plot](#) [Plot3D](#)

Implementation status

- - partially implemented

Github

- [Implementation of ListLinePlot](#)
-

ListLinePlot3D

```
ListLinePlot3D( { list-of-lines } )
```

generate a JavaScript list plot 3D control for the `list-of-lines`.

Note: This feature is available in the console app and in the web interface.

Examples

In the console apps, this command shows an HTML page with a JavaScript list plot control.

```
>> ListLinePlot3D({{0,1,0}, {1,0,2}, {1,1,2}, {2,2,-3}, {2,20,3}})
```

Related terms

[JSForm](#), [ListPointPlot3D](#), [Manipulate](#), [ParametricPlot](#), [Plot](#), [Plot3D](#)

Implementation status

- - experimental

Github

- [Implementation of ListLinePlot3D](#)
-

ListLogLogPlot

```
ListLogLogPlot( { list-of-points } )
```

generate an image of a logarithmic X and logarithmic Y plot for the `list-of-points`.

Examples

```
>> ListLogLogPlot(RandomReal(1, 20)^5)
```

Related terms

[ArrayPlot](#), [ListPlot](#), [ListLogPlot](#), [Manipulate](#), [ParametricPlot](#), [Plot](#), [Plot3D](#)

Implementation status

- - experimental

Github

- [Implementation of ListLogLogPlot](#)
-

ListLogPlot

```
ListLogPlot( { list-of-points } )
```

generate an image of a logarithmic Y plot for the `list-of-points`.

Examples

```
>> ListLogPlot(RandomReal(1, 20)^3)
```

Related terms

[ArrayPlot](#), [ListPlot](#), [ListLogLogPlot](#), [Manipulate](#), [ParametricPlot](#), [Plot](#), [Plot3D](#)

Implementation status

- - experimental

Github

- [Implementation of ListLogPlot](#)
-

ListPlot

```
ListPlot( { list-of-points } )
```

generate a JavaScript list plot control for the `list-of-points`.

Note: This feature is available in the console app and in the web interface.

Examples

In the console apps, this command shows an HTML page with a JavaScript list plot control.

```
>> Manipulate(ListPlot(Table({Sin(t), Cos(t*a)}, {t, 100})), {a,1,4,1})
```

With `JSForm` you can display the generated JavaScript form of the `Manipulate` function

```
>> Manipulate(ListPlot(Table({Sin(t), Cos(t*a)}, {t, 100})), {a,1,4,1}) // JSForm
```

Related terms

[JSForm](#) [Manipulate](#) [ParametricPlot](#) [Plot](#) [Plot3D](#)

Implementation status

- - partially implemented

Github

- [Implementation of ListPlot](#)
-

ListPlot3D

```
ListPlot3D( { list-of-polygons } )
```

generate a JavaScript list plot 3D control for the `list-of-polygons`.

Implementation status

- - experimental

Github

- [Implementation of ListPlot3D](#)
-

ListPointPlot3D

```
ListPointPlot3D( { list-of-points } )
```

generate a JavaScript list plot 3D control for the `list-of-points`.

Note: This feature is available in the console app and in the web interface.

Examples

In the console apps, this command shows an HTML page with a JavaScript list plot control.

```
>> Manipulate(ListPointPlot3D(Table({Sin(t), Cos(t*a), Cos(t^2) }, {t, 500})), {a,1,4,1
```

With `JSForm` you can display the generated JavaScript form of the `Manipulate` function

```
>> Manipulate(ListPointPlot3D(Table({Sin(t), Cos(t*a), Cos(t^2) }, {t, 500})), {a,1,4,1
```

Related terms

[JSForm](#), [ListLinePlot3D](#), [Manipulate](#), [ParametricPlot](#), [Plot](#), [Plot3D](#)

Implementation status

- - experimental

Github

- [Implementation of ListPointPlot3D](#)
-

ListQ

```
ListQ(expr)
```

tests whether `expr` is a `List`.

Examples

```
>> ListQ({1, 2, 3})
True

>> ListQ({{1, 2}, {3, 4}})
True

>> ListQ(x)
False
```

Github

- [Implementation of ListQ](#)
-

LoadJavaClass

```
LoadJavaClass["class-name"]
```

loads the class with the specified `class-name` and return a `JavaClass` expression. All static method names are assigned to a context which will be created by the last part of the class name.

Note: the Java specific functions which call Java native classes are only available in the MMA mode in a local installation. All symbol and method names have to be case sensitive.

Examples

```
>> clazz= LoadJavaClass["org.jsoup.Jsoup"]
JavaClass[org.jsoup.Jsoup]

>> conn=Jsoup`connect["https://jsoup.org/"]
JavaObject[class org.jsoup.helper.HttpConnection]

>> doc=conn@get[ ];
```

Print the title of the HTML page.

```
>> Print[doc@title[ ]]
jsoup Java HTML Parser, with the best of HTML5 DOM methods and CSS selectors.
```

Related terms

[InstanceOf](#), [JavaClass](#), [JavaNew](#), [JavaObject](#), [JavaObjectQ](#)

Implementation status

- - supported on Java virtual machine

Github

- [Implementation of LoadJavaClass](#)
-

Log

`Log(z)`

returns the natural logarithm of `z`.

`Log(base, z)`

get the logarithm of `z` for base `base`.

For the natural logarithm functions `Ln(x)` or `Log(x)` the logarithms are taken with the natural base, `E`. To get a logarithm of a different base `b`, use `Log(b, x)`, which is essentially a short-hand for `Log(x)/Log(b)`. Other short-hands are `Log10(x)` for `Log(x)/Log(10)` and `Log2(x)` for `Log(x)/Log(2)`.

See

- [Wikipedia - Logarithm](#)
- [Fungrim - Natural logarithm](#)

Examples

```
>> Log(E)
1

>> Log({0, 1, E, E * E, E ^ 3, E ^ x})
{-Infinity,0,1,2,3,Log(E^x)}

>> Log(0.)
Indeterminate

>> Log(1000) / Log(10)
3

>> Log(1.4)
0.3364722366212129

>> Log(Exp(1.4))
1.3999999999999997

>> Log(-1.4)
0.3364722366212129+I*3.141592653589793
```

Implementation status

- - full supported

Github

- [Implementation of Log](#)
 - [Rule definitions of Log](#)
-

Log10

`Log10(z)`

returns the base-10 logarithm of `z`. `Log10(z)` will be converted to `Log(z)/Log(10)` in symbolic mode.

See

- [Wikipedia - Logarithm](#)

Examples

```
>> Log10(1000)
3

>> Log10({2., 5.})
{0.30102999566398114, 0.6989700043360186}

>> Log10(E ^ 3)
3/Log(10)
```

Implementation status

- - full supported

Github

- [Implementation of Log10](#)
-

Log2

`Log2(z)`

returns the base-2 logarithm of `z`. `Log2(z)` will be converted to `Log(z)/Log(2)` in symbolic mode.

See

- [Wikipedia - Logarithm](#)

Examples

```
>> Log2(4 ^ 8)
16

>> Log2(5.6)
2.48543

>> Log2(E ^ 2)
2/Log(2)
```

Implementation status

- - full supported

Github

- [Implementation of Log2](#)
-

LogGamma

`LogGamma(z)`

is the logarithmic gamma function on the complex number `z`.

See

- [Wikipedia - Gamma function](#)
- [Fungrim - Gamma function](#)

Examples

```
>> LogGamma(1/2)  
Log(Pi)/2
```

Implementation status

- - partially implemented

Github

- [Implementation of LogGamma](#)
-

LogIntegral

```
LogIntegral(expr)
```

returns the integral logarithm of `expr`.

See

- [Wikipedia - Logarithmic integral function](#)

Examples

```
>> LogIntegral(0)  
0
```

Implementation status

- - full supported

Github

- [Implementation of LogIntegral](#)
-

LogisticSigmoid

```
LogisticSigmoid(z)
```

returns the logistic sigmoid of `z`.

See

- [Wikipedia - Logistic function](#)

Examples

```
>> LogisticSigmoid(0.5)
0.6224593312018546

>> LogisticSigmoid(0.5 + 2.3*I)
1.0647505893884985+I*0.8081774171575826

>> LogisticSigmoid({-0.2, 0.1, 0.3})
{0.45016600268752216, 0.52497918747894, 0.574442516811659}

>> LogisticSigmoid(I*Pi)
LogisticSigmoid(I*Pi)
```

Implementation status

- - full supported

Github

- [Implementation of LogisticSigmoid](#)
-

LogNormalDistribution

```
LogNormalDistribution(m, s)
```

returns a log-normal distribution.

See

- [Wikipedia - Log-normal distribution](#)

Examples

The variance of the log-normal distribution is

```
>> Variance(LogNormalDistribution(m,s))  
(-1+E^s^2)*E^(2*m+s^2)
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of LogNormalDistribution](#)
-

Lookup

```
Lookup(association, key)
```

return the value in the `association` which is associated with the `key`. If no value is available return `Missing("KeyAbsent",key)`.

```
Lookup(association, key, defaultValue)
```

return the value in the `association` which is associated with the `key`. If no value is available return `defaultValue`.

Examples

```
>> Lookup(<|a -> 11, b -> 17|>, a)
11

>> Lookup(<|a -> 1, b -> 2|>, c)
Missing(KeyAbsent,c)

>> Lookup(<|a -> 1, b -> 2|>, c, 42)
42
```

Related terms

[Association](#), [AssociationQ](#), [Counts](#), [Keys](#), [KeySort](#), [Values](#)

Implementation status

- - full supported

Github

- [Implementation of Lookup](#)
-

LowerCaseQ

```
LowerCaseQ(str)
```

is `True` if the given `str` is a string which only contains lower case characters.

Implementation status

- - full supported

Github

- [Implementation of LowerCaseQ](#)
-

LowerTriangularize

```
LowerTriangularize(matrix)
```

create a lower triangular matrix from the given `matrix`.

```
LowerTriangularize(matrix, n)
```

create a lower triangular matrix from the given `matrix` below the `n`-th subdiagonal.

See

- [Wikipedia - Triangular matrix](#)

Examples

```
>> LowerTriangularize({{a,b,c,d}, {d,e,f,g}, {h,i,j,k}, {l,m,n,o}}, -1)
{{0,0,0,0}
 {d,0,0,0}
 {h,i,0,0}
 {l,m,n,0}}
```

Implementation status

- - full supported

Github

- [Implementation of LowerTriangularize](#)
-

LowerTriangularMatrixQ

```
LowerTriangularMatrixQ(matrix)
```

```
LowerTriangularMatrixQ(matrix, diagonal)
```

returns `True` if `matrix` is lower triangular.

Examples

Implementation status

- - full supported

Github

- [Implementation of LowerTriangularMatrixQ](#)
-

LucasL

LucasL(*n*)

gives the *n*th Lucas number.

LucasL(*n*, *var*)

returns the *n*th Lucas polynomial for variable *var*.

Lucas numbers satisfy a recurrence relation similar to that of the Fibonacci sequence, in which each term is the sum of the preceding two. They are generated by choosing the initial values

$\text{LucasL}(0) = 2$ and $\text{LucasL}(1) = 1$.

See

- [Wikipedia - Lucas number](#)
- [Wikipedia - Fibonacci polynomials](#)

Examples

```
>> LucasL(10)
123

>> LucasL(50, x)
2+625*x^2+32500*x^4+672750*x^6+7400250*x^8+50075025*x^10+227613750*x^12+
736618125*x^14+1767883500*x^16+3241119750*x^18+4639918800*x^20+5272635000*x^22+
4814145000*x^24+3562467300*x^26+2148789800*x^28+1059575660*x^30+427248250*x^32+
140512125*x^34+37469900*x^36+8021650*x^38+1357510*x^40+177375*x^42+17250*x^44+
1175*x^46+50*x^48+x^50
```

Implementation status

- - partially implemented

Github

- [Implementation of LucasL](#)
-

LUDecomposition

```
LUDecomposition(matrix)
```

calculate the LUP-decomposition of a square `matrix`.

See:

- [Wikipedia - LU decomposition](#)
- [Commons Math - Class FieldLUDecomposition](#)

Examples

```
>> LUDecomposition[{{1, 2, 3}, {3, 4, 11}, {13, 7, 8}}]  
{{{1,0,0},  
  {3,1,0},  
  {13,19/2,1}},  
{{1,2,3},  
 {0,-2,2},  
 {0,0,-50}}, {1,2,3}}
```

Implementation status

- - full supported

Github

- [Implementation of LUDecomposition](#)
-

MachineNumberQ

```
MachineNumberQ(expr)
```

returns `True` if `expr` is a machine-precision real or complex number.

Examples

```
>> MachineNumberQ(3.14159265358979324)
False

>> MachineNumberQ(1.5 + 2.3*I)
True

>> MachineNumberQ(2.71828182845904524 + 3.14159265358979324*I)
False

>> MachineNumberQ(1.5 + 3.14159265358979324*I)
True

>> MachineNumberQ(1.5 + 5 *I)
True
```

Github

- [Implementation of MachineNumberQ](#)
-

Magenta

Magenta

RGB color value for the color magenta

Examples

```
>> Magenta  
RGBColor(1.0,0.0,1.0)
```

MangoldtLambda

```
MangoldtLambda(n)
```

the von Mangoldt function of `n`

See:

- [Wikipedia - Von Mangoldt function](#)

Examples

```
>> MangoldtLambda({1, 2, 3, 4, 5, 6, 7, 8, 9})  
{0, Log(2), Log(3), Log(2), Log(5), 0, Log(7), Log(2), Log(3)}
```

Implementation status

- - partially implemented

Github

- [Implementation of MangoldtLambda](#)
-

ManhattanDistance

```
ManhattanDistance(u, v)
```

returns the Manhattan distance between `u` and `v`, which is the number of horizontal or vertical moves in the grid like Manhattan city layout to get from `u` to `v`.

See:

- [Wikipedia - Taxicab geometry](#)

Examples

```
>> ManhattanDistance({-1, -1}, {1, 1})  
4
```

Related terms

[FindClusters](#), [BinaryDistance](#), [BrayCurtisDistance](#), [ChessboardDistance](#), [CanberraDistance](#), [CosineDistance](#), [EuclideanDistance](#), [SquaredEuclideanDistance](#)

Implementation status

- - full supported

Github

- [Implementation of ManhattanDistance](#)
-

Manipulate

```
Manipulate(plot, {x, min, max})
```

generate a JavaScript control for the expression `plot` which can be manipulated by a range slider `{x, min, max}`.

```
Manipulate(formula, {x, min, max, step})
```

display generated `formulas` and define the `steps` in which the values for `x` should change.

Note: This feature is not available on all supported platforms.

Manipulate generates JavaScript output for the following JavaScript libraries:

- github.com/jsxgraph/jsxgraph
- github.com/paulmasson/math
- github.com/paulmasson/mathcell

Examples

In the console apps, this command shows an HTML page with a JavaScript plot control, where the value of the variable `a` can be manipulated by a slider in the range `[0..10]`.

```
>> Manipulate(Plot(Sin(x)*Cos(1 + a*x), {x, 0, 2*Pi}), {a,0,10})
```

A 3D surface plot control:

```
>> Manipulate(Plot3D(Sin(a*x*y), {x, -1.5, 1.5}, {y, -1.5, 1.5}), {a,1,5})
```

Display a slider for the generated formulas:

```
>> Manipulate(Factor(x^n + 1), {n, 1, 5, 1})
```

Display buttons to change the plotted function:

```
>> Manipulate(Plot(f(x), {x, 0, 2*Pi}), {f, {Sin, Cos, Tan, Cot}})
```

With `JSForm` you can display the generated JavaScript form of the `Manipulate` function

```
>> Manipulate(Plot(Sin(x)*Cos(1 + a*x), {x, 0, 2*Pi}), {a,0,10}) // JSForm
```

Related terms

[JSForm](#), [ParametricPlot](#), [Plot](#), [Plot3D](#)

Implementation status

- - full supported

Github

- [Implementation of Manipulate](#)
-

Map

```
Map(f, expr) or f /@ expr
```

applies `f` to each part on the first level of `expr`.

```
Map(f, expr, levelspec)
```

applies `f` to each level specified by `levelspec` of `expr`.

Examples

```
>> f /@ {1, 2, 3}
{f(1), f(2), f(3)}
>> #^2 & /@ {1, 2, 3, 4}
{1, 4, 9, 16}
```

Map `f` on the second level:

```
>> Map(f, {{a, b}, {c, d, e}}, {2})
{{f(a), f(b)}, {f(c), f(d), f(e)}}
```

Include heads:

```
>> Map(f, a + b + c, Heads->True)
f(Plus)[f(a), f(b), f(c)]
```

Level specification `a + b` is not of the form `n`, `{n}`, or `{m, n}`.

```
>> Map(f, expr, a+b, Heads->True)
Map(f, expr, a + b, Heads -> True)
```

Implementation status

- - full supported

Github

- [Implementation of Map](#)
-

MapApply

```
MapApply(head, expr)
```

or

```
head @@@ expr
```

is equivalent to `Apply(head, expr, {1})`.

Examples

```
>> h @@@ {{a, b}, {c, d}}  
{h(a,b),h(c,d)}
```

Implementation status

- - full supported

Github

- [Implementation of MapApply](#)
-

MapAt

```
MapAt(f, expr, n)
```

applies `f` to the element at position `n` in `expr`. If `n` is negative, the position is counted from the end.

```
MapAt(f, expr, {i, j, ...})
```

applies `f` to the part of `expr` at position `{i, j, ...}`.

```
MapAt(f, pos)
```

represents an operator form of `MapAt` that can be applied to an expression..

Examples

Map function `f` to the second element of a simple flat list:

```
>> MapAt(f, {a, b, c}, 2)
{a, f(b), c}
```

Above, we specified a simple integer value `2`. In general, the expression can be an arbitrary vector.

Using `MapAt` with `Function(0)`, we can zero a value or values in a vector:

```
>> MapAt(0&, {{0, 1}, {1, 0}}, {2, 1})
{{0, 1}, {0, 0}}
```

Use the operator form of `MapAt`:

```
>> MapAt(f, -1)[{a, b, c}]
{a, b, f(c)}
```

Implementation status

- - full supported

Github

- Implementation of MapAt
-

MapIndexed

```
MapIndexed(f, expr)
```

applies `f` to each part on the first level of `expr` and appending the elements position as a list in the second argument.

```
MapIndexed(f, expr, levelspec)
```

applies `f` to each level specified by `levelspec` of `expr` and appending the elements position as a list in the second argument.

Examples

```
>> MapIndexed(f, {{{{a, b}, {c, d}}}, {{{u, v}, {s, t}}}}, 2)
{f({f({{a,b},{c,d}},{1,1})),{1}},f({f({{u,v},{s,t}},{2,1})),{2}}}
```

Implementation status

- - full supported

Github

- [Implementation of MapIndexed](#)
-

MapThread

```
MapThread(f, {{a1, a2, ...}, {b1, b2, ...}, ...})
```

returns `{f(a1, b1, ...), f(a2, b2, ...), ...}`.

```
MapThread(f, {expr1, expr2, ...}, n)
```

applies `f` at level `n`.

Examples

```
>> MapThread(f, {{a, b, c}, {1, 2, 3}})
{f(a,1),f(b,2),f(c,3)}

>> MapThread(f, {{{a, b}, {c, d}}, {{e, f}, {g, h}}}, 2)
{{f(a, e), f(b, f)}, {f(c, g), f(d, h)}}
```

Non-negative machine-sized integer expected at position 3 in `MapThread(f, {{a, b}, {c, d}}, {1})`.

```
>> MapThread(f, {{a, b}, {c, d}}, {1})
MapThread(f, {{a, b}, {c, d}}, {1})
```

Object `{a, b}` at position `{2, 1}` in `MapThread(f, {{a, b}, {c, d}}, 2)` has only 1 of required 2 dimensions.

```
>> MapThread(f, {{a, b}, {c, d}}, 2)
MapThread(f, {{a, b}, {c, d}}, 2)
```

Incompatible dimensions of objects at positions `{2, 1}` and `{2, 2}` of `MapThread(f, {{a}, {b, c}})`; dimensions are 1 and 2.

```
>> MapThread(f, {{a}, {b, c}})
MapThread(f, {{a}, {b, c}})

>> MapThread(f, {})
{}

>> MapThread(f, {a, b}, 0)
f(a, b)
```

Object a at position {2, 1} in MapThread(f, {a, b}, 1) has only 0 of required 1 dimensions.

```
>> MapThread(f, {a, b}, 1)
MapThread(f, {a, b}, 1)
```

Implementation status

- - full supported

Github

- [Implementation of MapThread](#)
-

MatchingDissimilarity

```
MatchingDissimilarity(u, v)
```

returns the Matching dissimilarity between the two boolean 1-D lists `u` and `v`, which is defined as $(c_{tf} + c_{ft}) / n$, where `n` is `len(u)` and `cij` is the number of occurrences of `u(k)=i` and `v(k)=j` for `k<n`.

Examples

```
>> MatchingDissimilarity({1, 0, 1, 1, 0, 1, 1}, {0, 1, 1, 0, 0, 0, 1})  
4/7
```

Implementation status

- - full supported

Github

- [Implementation of MatchingDissimilarity](#)
-

MatchQ

```
MatchQ(expr, form)
```

tests whether `expr` matches `form`.

Examples

```
>> MatchQ(123, _Integer)
True

>> MatchQ(123, _Real)
False

>> MatchQ(_Integer)[123]
True
```

Implementation status

- - full supported

Github

- [Implementation of MatchQ](#)
-

MathMLForm

```
MathMLForm(expr)
```

returns the MathML form of the evaluated `expr`.

See

- [Wikipedia - MathML](#)

Examples

```
>> MathMLForm(D(Sin(x)*Cos(x), x))
```

Implementation status

- - partially implemented

Github

- [Implementation of MathMLForm](#)
-

MatrixExp

```
MatrixExp(matrix)
```

computes the matrix exponential of the square `matrix`.

See

- [Wikipedia - Matrix exponential](#)
- [Wikipedia - Logarithm of a matrix](#)

Examples

```
>> MatrixExp({{3.4, 1.2}, {0.001, -0.9}})
{{29.97054, 8.24991},
 {0.00687492, 0.408375}}
```

Related terms

[JordanDecomposition](#), [MatrixFunction](#), [MatrixLog](#), [SchurDecomposition](#)

MatrixForm

```
MatrixForm(matrix)
```

print a `matrix` or sparse array in matrix form

Examples

```
>> SparseArray({{1, 1} -> 1, {2, 2} -> 2, {3, 3} -> 3, {1, 3} -> 4}) // MatrixForm
```

```
>> A = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}; MatrixForm(A)
```

MatrixFunction

```
MatrixFunction(function-head, matrix)
```

computes the matrix function of the `matrix`.

See

- [Wikipedia - Analytic function of a matrix](#)
- [Wikipedia - Matrix exponential](#)

Examples

An example from Wikipedia:

```
>> mf=MatrixFunction(Gamma, {{1,3},{2,1}})
{{1/2*(Gamma(1-Sqrt(6))+Gamma(1+Sqrt(6))),1/2*Sqrt(3/2)*(-Gamma(1-Sqrt(6))+Gamma(1+Sqrt(6)))},
{1/2*Sqrt(3/2)*(Gamma(1-Sqrt(6))-Gamma(1+Sqrt(6))),1/2*(Gamma(1-Sqrt(6))+Gamma(1+Sqrt(6)))}}

>> N(mf)
{{2.81144,0.407999},{0.271999,2.81144}}
```

Related terms

[JordanDecomposition](#), [MatrixExp](#), [MatrixLog](#), [SchurDecomposition](#)

MatrixLog

`MatrixLog(matrix)`

computes the matrix logarithm of the square `matrix`.

See

- [Wikipedia - Logarithm of a matrix](#)
- [Wikipedia - Matrix exponential](#)

Examples

```
>> MatrixLog({{3.4, 1.2}, {0.001, -0.9}})
{{1.22377+I*0.00020385, 0.37081+I*(-0.87661)},
 {0.000309008+I*(-0.000730508), -0.104964+I*3.14139}}
```

Related terms

[JordanDecomposition](#), [MatrixExp](#), [MatrixFunction](#), [SchurDecomposition](#)

MatrixMinimalPolynomial

```
MatrixMinimalPolynomial(matrix, var)
```

computes the matrix minimal polynomial of a `matrix` for the variable `var`.

See:

- [Wikipedia - Minimal polynomial \(linear algebra\)](#)

Examples

```
>> MatrixMinimalPolynomial({{1, -1, -1}, {1, -2, 1}, {0, 1, -3}}, x)
-1+x+4*x^2+x^3
```

Implementation status

- - full supported

Github

- [Implementation of MatrixMinimalPolynomial](#)
-

MatrixPlot

```
MatrixPlot( matrix )
```

create a matrix plot.

Examples

```
>> MatrixPlot({{4, 2, 1}, {3, 0, -2}, {0, 0, -1}})
```

Implementation status

- - experimental

Github

- [Implementation of MatrixPlot](#)
-

MatrixPower

```
MatrixPower(matrix, n)
```

computes the `n`th power of a `matrix`

See

- [Wikipedia - Matrix multiplication - Powers of a matrix](#)

Examples

```
>> MatrixPower({{1, 2}, {1, 1}}, 10)
{{3363, 4756},
 {2378, 3363}}

>> MatrixPower({{1, 2}, {2, 5}}, -3)
{{169, -70},
 {-70, 29}}
```

Argument `{{1, 0}, {0}}` at position 1 is not a non-empty rectangular matrix.

```
>> MatrixPower({{1, 0}, {0}}, 2)
MatrixPower({{1, 0}, {0}}, 2)
```

Implementation status

- - full supported

Github

- [Implementation of MatrixPower](#)
-

MatrixQ

```
MatrixQ(m)
```

returns `True` if `m` is a list of equal-length lists.

```
MatrixQ(m, f)
```

only returns `True` if `f(x)` returns `True` for each element `x` of the matrix `m`.

See

- [Wikipedia - Matrix \(mathematics\)](#)
- [Youtube - Linear transformations and matrices | Essence of linear algebra, chapter 3](#)
- [Youtube - Matrix multiplication as composition | Essence of linear algebra, chapter 4](#)

Examples

```
>> MatrixQ({{1, 3}, {4.0, 3/2}}, NumberQ)
True
```

Implementation status

- - full supported

Github

- [Implementation of MatrixQ](#)
-

MatrixRank

```
MatrixRank(matrix)
```

returns the rank of `matrix`.

See

- [Wikipedia - Rank \(linear algebra\)](#)

Examples

```
>> MatrixRank({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}})
2
>> MatrixRank({{1, 1, 0}, {1, 0, 1}, {0, 1, 1}})
3
>> MatrixRank({{a, b}, {3 a, 3 b}})
1
```

Argument `{{1, 0}, {0}}` at position 1 is not a non-empty rectangular matrix.

```
>> MatrixRank({{1, 0}, {0}})
MatrixRank({{1, 0}, {0}})
```

Implementation status

- - full supported

Github

- [Implementation of MatrixRank](#)
-

Max

```
Max(e_1, e_2, ..., e_i)
```

returns the expression with the greatest value among the `e_i`.

Examples

Maximum of a series of numbers:

```
>> Max(4, -8, 1)
4
```

`Max` flattens lists in its arguments:

```
>> Max({1,2},3,{-3,3.5,-Infinity},{1/2})
3.5
```

`Max` with symbolic arguments remains in symbolic form:

```
>> Max(x, y)
Max(x,y)

>> Max(5, x, -3, y, 40)
Max(40,x,y)
```

With no arguments, `Max` gives `-Infinity`:

```
>> Max()
-Infinity

>> Max(x)
x
```

Implementation status

- - full supported

Github

- [Implementation of Max](#)
-

MaxFilter

```
MaxFilter(list, r)
```

filter which evaluates the `Max` of `list` for the radius `r`.

Examples

```
>> MaxFilter({1, 2, 3, 2, 1}, 1)
{2,3,3,3,2}
```

Implementation status

- - experimental

Github

- [Implementation of MaxFilter](#)
-

MaximalBy

```
MaximalBy(list, function-head)
```

get the elements from `list`, for which the applied `function-head` is maximal.

```
MaximalBy(list, function-head, n)
```

get the `n` largest elements from `list`, for which the applied `function-head` is maximal.

Maximize

```
Maximize(unary-function, variable)
```

returns the maximum of the unary function for the given `variable`.

See

- [Wikipedia - Derivative test](#)

Examples

```
>> Maximize(-x^4-7*x^3+2*x^2 - 42, x)
{-42-7*(-21/8-Sqrt(505)/8)^3+2*(21/8+Sqrt(505)/8)^2-(21/8+Sqrt(505)/8)^4, {x->-21/8-Sqrt
```

Print a message if no maximum can be found

```
>> Maximize(x^4+7*x^3-2*x^2 + 42, x)
{Infinity, {x->-Infinity}}
```

Related terms

[Minimize](#)

Implementation status

- - full supported

Github

- [Implementation of Maximize](#)
-

Mean

```
Mean(list)
```

returns the statistical mean of `list`.

See:

- [Wikipedia - Mean](#)
- [Wikipedia - Quantile - Relationship to the mean](#)

`Mean` can be applied to the following distributions:

[BernoulliDistribution](#), [BinomialDistribution](#), [DiscreteUniformDistribution](#),
[ErlangDistribution](#), [ExponentialDistribution](#), [FrechetDistribution](#), [GammaDistribution](#),
[GeometricDistribution](#), [GumbelDistribution](#), [HypergeometricDistribution](#),
[LogNormalDistribution](#), [NakagamiDistribution](#), [NormalDistribution](#),
[PoissonDistribution](#), [StudentTDistribution](#), [WeibullDistribution](#)

Examples

```
>> Mean({26, 64, 36})  
42  
  
>> Mean({1, 1, 2, 3, 5, 8})  
10/3  
  
>> Mean({a, b})  
1/2*(a+b)
```

The [mean](#) of the normal distribution is

```
>> Mean(NormalDistribution(m, s))  
m
```

Implementation status

- - full supported

Github

- [Implementation of Mean](#)
-

MeanFilter

```
MeanFilter(list, r)
```

filter which evaluates the `Mean` of `list` for the radius `r`.

Examples

```
>> MeanFilter({-3, 3, 6, 0, 0, 3, -3, -9}, 2)
{2, 3/2, 6/5, 12/5, 6/5, -9/5, -9/4, -3}
```

Implementation status

- - experimental

Github

- [Implementation of MeanFilter](#)
-

Median

```
Median(list)
```

returns the median of `list`.

See:

- [Wikipedia - Median](#)

`Median` can be applied to the following distributions:

[BernoulliDistribution](#), [BinomialDistribution](#), [DiscreteUniformDistribution](#),
[ErlangDistribution](#), [ExponentialDistribution](#), [FrechetDistribution](#), [GammaDistribution](#),
[GeometricDistribution](#), [GumbelDistribution](#), [HypergeometricDistribution](#),
[LogNormalDistribution](#), [NakagamiDistribution](#), [NormalDistribution](#),
[StudentTDistribution](#), [WeibullDistribution](#)

Examples

```
>> Median({26, 64, 36})  
36
```

For lists with an even number of elements, `Median` returns the mean of the two middle values:

```
>> Median({-11, 38, 501, 1183})  
539/2
```

Passing a matrix returns the medians of the respective columns:

```
>> Median({{100, 1, 10, 50}, {-1, 1, -2, 2}})  
{99/2, 1, 4, 26}  
  
>> Median(LogNormalDistribution(m,s))  
E^m
```

Related terms

[FiveNum](#), [Quantile](#), [Quartiles](#)

Implementation status

- - full supported

Github

- [Implementation of Median](#)
-

MedianFilter

```
MedianFilter(list, r)
```

filter which evaluates the `Median` of `list` for the radius `r`.

See

- [Wikipedia - Median filter](#)

Examples

```
>> MedianFilter({2,3,80,6}, 1)
{5/2,3,6,43}
```

Implementation status

- - experimental

Github

- [Implementation of MedianFilter](#)
-

MeijerG

```
MeijerG({{a(1),a(2),...,a(n)},{a(n+1),a(n+2),...,a(p)}},{b(1),b(2),...,b(m)},{b(m+1),b
```

return the `MeijerG` function. The G-function was introduced by Cornelis Simon Meijer (1936) as a very general function intended to include most of the known special functions as particular cases.

See:

- [Wikipedia - Meijer G-function](#)

Examples

```
>> MeijerG({{},{ }}, {{b1}, {}}, z)
z^b1/E^z
```

Implementation status

- - experimental

Github

- [Implementation of MeijerG](#)
-

MemberQ

```
MemberQ(list, pattern)
```

returns `True` if pattern matches any element of `list`, or `False` otherwise.

Examples

```
>> MemberQ({a, b, c}, b)
True
>> MemberQ({a, b, c}, d)
False
>> MemberQ({"a", b, f(x)}, _?NumericQ)
False
>> MemberQ(_List)({{}})
True
```

Implementation status

- - full supported

Github

- [Implementation of MemberQ](#)
-

Merge

```
Merge(list-of-rules-or-associations, function)
```

use the `function` to merge right-hand-side values with the left-hand-side key in the `list-of-rules-or-associations`.

Examples

```
>> Merge({<|x->1|>, x->2, x->3, {y->4}}, Identity)
<|x->{1,2,3},y->{4}|>
```

Implementation status

- - full supported

Github

- [Implementation of Merge](#)
-

MersennePrimeExponent

```
MersennePrimeExponent(n)
```

returns the n th mersenne prime exponent. $2^n - 1$ must be a prime number.
Currently $0 < n \leq 52$ can be computed, otherwise the function returns unevaluated.

See

- [Wikipedia - Mersenne prime](#)
- [Wikipedia - List of perfect numbers](#)

Examples

```
>> Table(MersennePrimeExponent(i), {i, 20})  
{2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107, 127, 521, 607, 1279, 2203, 2281, 3217, 4253, 4423}
```

Implementation status

- - partially implemented

Github

- [Implementation of MersennePrimeExponent](#)
-

MersennePrimeExponentQ

MersennePrimeExponentQ(n)

returns `True` if $2^n - 1$ is a prime number. Currently `0 <= n <= 52` can be computed in reasonable time.

See

- [Wikipedia - Mersenne prime](#)
- [Wikipedia - List of perfect numbers](#)

Examples

```
>> Select(Range(10000), MersennePrimeExponentQ)
{2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107, 127, 521, 607, 1279, 2203, 2281, 3217, 4253, 4423, 9689, 9941}
```

Implementation status

- - partially implemented

Github

- [Implementation of MersennePrimeExponentQ](#)
-

Message

```
Message(symbol::msg, expr1, expr2, ...)
```

displays the specified message, replacing placeholders in the message text with the corresponding expressions.

Examples

```
>> a::b = "Hello world!"  
Hello world!  
  
>> Message(a::b)  
a: Hello world!  
  
>> a::c := "Hello `1`, Mr 00`2`!"  
  
>> Message(a::c, "you", 3 + 4)  
a: Hello you, Mr 007!
```

Implementation status

- - full supported

Github

- [Implementation of Message](#)
-

MessageName

```
MessageName(symbol, msg)
```

or

```
symbol::msg
```

`symbol::msg` identifies a message. `MessageName` is the head of message IDs of the form `symbol::tag`.

Examples

The second parameter 'tag' is interpreted as a string.

```
>> FullForm(a::b)
MessageName(a, b)

>> FullForm(a::"b")
MessageName(a, "b")
```

Implementation status

- - full supported

Github

- [Implementation of MessageName](#)
-

Messages

`Messages(symbol)`

return all messages which are asociated to `symbol`.

Examples

```
>> a::hello:="Hello world"; a::james:="Hello `1`, Mr 00`2`!"  
  
>> Messages(a)  
{HoldPattern(a::james):>Hello `1`, Mr 00`2`!,HoldPattern(a::hello):>Hello world}
```

Implementation status

- - full supported

Github

- [Implementation of Messages](#)
-

Min

```
Min(e_1, e_2, ..., e_i)
```

returns the expression with the lowest value among the `e_i`.

Examples

Minimum of a series of numbers:

```
>> Max(4, -8, 1)
-8
```

`Min` flattens lists in its arguments:

```
>> Min({1,2},3,{-3,3.5,-Infinity},{1/2})
-Infinity
```

`Min` with symbolic arguments remains in symbolic form:

```
>> Min(x, y)
Min(x,y)

>> Min(5, x, -3, y, 40)
Min(-3,x,y)
```

With no arguments, `Min` gives `Infinity`:

```
>> Min()
Infinity

>> Min(x)
x
```

Implementation status

- - full supported

Github

- [Implementation of Min](#)
-

MinFilter

```
MinFilter(list, r)
```

filter which evaluates the `Min` of `list` for the radius `r`.

Implementation status

- - experimental

Github

- [Implementation of MinFilter](#)
-

MinimalBy

```
MinimalBy(list, function-head)
```

get the elements from `list`, for which the applied `function-head` is minimal.

```
MinimalBy(list, function-head, n)
```

get the `n` smallest elements from `list`, for which the applied `function-head` is minimal.

Minimize

```
Minimize(unary-function, variable)
```

returns the minimum of the unary function for the given `variable`.

See:

- [Wikipedia - Derivative test](#)

Examples

```
>> Minimize(x^4+7*x^3-2*x^2 + 42, x)
{42+7*(-21/8-Sqrt(505)/8)^3-2*(21/8+Sqrt(505)/8)^2+(21/8+Sqrt(505)/8)^4, {x->-21/8-Sqrt(
```

Related terms

[Maximize](#)

Implementation status

- - partially implemented

Github

- [Implementation of Minimize](#)
-

Minors

```
Minors(matrix)
```

returns the minors of the matrix.

```
Minors(matrix, n)
```

returns the minors of the matrix with the determinant of the minors of size $n \times n$

```
Minors(matrix, n, function)
```

returns the minors of the matrix with the function applied on the minors of size $n \times n$

See:

- [Wikipedia - Minor \(linear algebra\)](#)

Examples

```
>> Minors({{1,4,7},{3,0,5},{-1,9,11}})
{{-12,-16,20},{13,18,-19},{27,38,-45}}
```

Implementation status

- - full supported

Github

- [Implementation of Minors](#)
-

Minus

```
Minus(expr)
```

```
-expr
```

is the negation of `expr`.

Examples

```
>> -a //FullForm  
"Times(-1, a)"
```

`Minus` automatically distributes:

```
>> -(x - 2/3)  
2/3-x
```

`Minus` threads over lists:

```
>> -Range(10)  
{-1, -2, -3, -4, -5, -6, -7, -8, -9, -10}
```

Implementation status

- - full supported

Github

- [Implementation of Minus](#)
-

MissingQ

```
MissingQ(expr)
```

returns `True` if `expr` is a `Missing()` expression.

Examples

```
>> MissingQ(Missing("Test message"))  
True
```

Github

- [Implementation of MissingQ](#)
-

Mod

```
Mod(x, m)
```

returns `x` modulo `m`.

See

- [Wikipedia - Modulo operation](#)

Examples

```
>> Mod(14, 6)
2

>> Mod(-3, 4)
1

>> Mod(-3, -4)
-3
```

The argument 0 should be nonzero

```
>> Mod(5, 0)
Mod(5, 0)
```

Implementation status

- - full supported

Github

- [Implementation of Mod](#)
-

ModularInverse

```
ModularInverse(k, n)
```

returns the modular inverse $k^{-1} \bmod n$.

See

- [Wikipedia - Modular multiplicative inverse](#)

Examples

```
>> ModularInverse(3, 11)
4
```

Implementation status

- - full supported

Github

- [Implementation of ModularInverse](#)
-

Module

```
Module({list_of_local_variables}, expr )
```

evaluates `expr` for the `list_of_local_variables` by renaming local variables.

Examples

Print `11` to the console and return `10`:

```
>> xm=10;Module({xm=xm}, xm=xm+1;Print(xm));xm  
10
```

Related terms

[Block](#), [With](#)

Implementation status

- - full supported

Github

- [Implementation of Module](#)
-

MoebiusMu

MoebiusMu(expr)

calculate the Möbius function.

- $\text{MoebiusMu}(n) = 1$ if n is a square-free integer with an even number of prime factors.
- $\text{MoebiusMu}(n) = -1$ if n is a square-free integer with an odd number of prime factors.
- $\text{MoebiusMu}(n) = 0$ if n has a squared prime factor.

See:

- [Wikipedia - Möbius function](#)

Examples

```
>> MoebiusMu(30)
-1

>> FactorInteger(30)
{{2, 1}, {3, 1}, {5, 1}}
```

Implementation status

- - partially implemented

Github

- [Implementation of MoebiusMu](#)
-

MonomialList

```
MonomialList(polynomial, list-of-variables)
```

get the list of monomials of a `polynomial` expression, with respect to the `list-of-variables`.

See

- [Wikipedia - Monomial](#)

Examples

```
>> MonomialList((x + y)^3)
{x^3, 3*x^2*y, 3*x*y^2, y^3}
```

Implementation status

- - full supported

Github

- [Implementation of MonomialList](#)
-

Most

```
Most(expr)
```

returns `expr` with the last element removed.

`Most(expr)` is equivalent to `expr[[:-2]]`.

Examples

```
>> Most({a, b, c})
{a, b}

>> Most(a + b + c)
a+b
```

Nonatomic expressions are expected.

```
>> Most(x)
Most(x)
```

Implementation status

- - full supported

Github

- [Implementation of Most](#)
-

Multinomial

```
Multinomial(n1, n2, ...)
```

gives the multinomial coefficient $(n_1+n_2+\dots)!/(n_1! \ n_2! \ \dots)$.

See

- [Wikipedia: Multinomial coefficient](#)

Examples

```
>> Multinomial(2, 3, 4, 5)
2522520
```

```
>> Multinomial()
1
```

`Multinomial(n-k, k)` is equivalent to `Binomial(n, k)`.

```
>> Multinomial(2, 3)
10
```

Implementation status

- - full supported

Github

- [Implementation of Multinomial](#)
-

MultiplicativeOrder

```
MultiplicativeOrder(a, n)
```

gives the multiplicative order a modulo n .

See

- [Wikipedia: Multiplicative order](#)

Examples

The [A023394 Prime factors of Fermat numbers](#) integer sequence

```
>> Select(Prime(Range(500)), IntegerQ(Log(2, MultiplicativeOrder(2, # ))))&
{3, 5, 17, 257, 641}
```

Implementation status

- - partially implemented

Github

- [Implementation of MultiplicativeOrder](#)
-

MultiplySides

```
MultiplySides(compare-expr, value)
```

multiplies `value` with all elements of the `compare-expr`. `compare-expr` can be `True`, `False` or a comparison expression with head `Equal`, `Unequal`, `Less`, `LessEqual`, `Greater`, `GreaterEqual`.

Examples

```
>> MultiplySides(a==a, x)
True

>> MultiplySides(a==b, x)
a*x==b*x
```

Implementation status

- - full supported

Github

- [Implementation of MultiplySides](#)
-

N

`N(expr)`

gives the numerical value of `expr`.

`N(expr, precision)`

evaluates `expr` numerically with a precision of `prec` digits.

Note: the upper case identifier `N` is different from the lower case identifier `n`.

Examples

```
>> N(Pi)
3.141592653589793

>> N(Pi, 50)
3.1415926535897932384626433832795028841971693993751

>> N(1/7)
0.14285714285714285

>> N(1/7, 5)
1.4285714285714285714e-1
```

Related terms

[EvalF](#)

Implementation status

- - full supported

Github

- [Implementation of N](#)
-

NakagamiDistribution

```
NakagamiDistribution(m, o)
```

returns a Nakagami distribution.

See

- [Wikipedia - Nakagami distribution](#)

Examples

The variance of the Nakagami distribution is

```
>> Variance(NakagamiDistribution(n, m))  
m+(-m*Pochhammer(n,1/2)^2)/n
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of NakagamiDistribution](#)
-

Names

```
Names(string)
```

or

```
Names(pattern)
```

return the symbols from the context path matching the `string` or `pattern`.

Examples

```
>> sysnames = Names("System`*");

>> Select(sysnames, MemberQ(Attributes(#), OneIdentity) &) // InputForm
{"And", "Composition", "Dot", "GCD", "Intersection", "Join", "Max", "Min", "Or", "Plus", "Power",
```

Implementation status

- - full supported

Github

- [Implementation of Names](#)
-

Nand

```
Nand(arg1, arg2, ...)
```

Logical NAND function. It evaluates its arguments in order, giving `True` immediately if any of them are `False`, and `False` if they are all `True`.

See

- [Wikipedia - Sheffer stroke](#)

Examples

```
>> Nand(True, True, True)
False

>> Nand(True, False, a)
True
```

Implementation status

- - full supported

Github

- [Implementation of Nand](#)
-

ND

```
ND(function, x, value)
```

returns a numerical approximation of the partial derivative of the `function` for the variable `x` and the given `value`.

```
ND(function, {x, n} , value)
```

returns a numerical approximation of the partial derivative of order `n`.

See:

- [Wikipedia - Numerical differentiation](#)

Examples

```
>> ND(BesselY(10.0,x), x, 1)
1.20940*10^9

>> ND(Cos(x)^3, {x,2}, 1)
1.82226
```

Related terms

[D](#), [DSolve](#), [Integrate](#), [Limit](#), [NIntegrate](#)

Implementation status

- - partially implemented

Github

- [Implementation of ND](#)
-

NDSolve

```
NDSolve({equation-list}, functions, t)
```

attempts to solve the linear differential `equation-list` for the `functions` and the time-dependent-variable `t`. Returns an `InterpolatingFunction` function object.

See:

- [Wikipedia - Ordinary differential equation](#)

Examples

Example taken from [Tutorial — Differential Equations](#)

```
>> model=NDSolve({x'(t) == 10*(y(t) - x(t)), y'(t) == x(t)*(28 - z(t)) - y(t), z'(t) ==  
{x->InterpolatingFunction[Piecewise({{InterpolatingPolynomial({.....
```

Plot the interpolating function and the sine function.

```
>> Plot({Evaluate(z(t) /.model)}, {t, 0, 20})
```

Related terms

[DSolve](#), [InterpolatingFunction](#), [NRoots](#), [Solve](#)

Implementation status

- - partially implemented

Github

- [Implementation of NDSolve](#)
-

Nearest

```
Nearest(list-of-values, x-value)
```

returns the value from the `list-of-values` which is nearest to `x-value`. By default the `EuclideanDistance` is used. With the `DistanceFunction` option you can specify your own distance function.

Examples

```
>> Nearest({1, 2, 3, 5, 7, 11, 13, 17, 19}, 9)
{7, 11}
```

Related terms

[FindClusters](#), [BinaryDistance](#), [BrayCurtisDistance](#), [ChessboardDistance](#), [CanberraDistance](#), [CosineDistance](#), [EuclideanDistance](#), [ManhattanDistance](#), [NearestTo](#), [SquaredEuclideanDistance](#)

Implementation status

- - full supported

Github

- [Implementation of Nearest](#)
-

NearestTo

NearestTo(x-value)

returns the value from the `list-of-values` which is nearest to `x-value`. By default the `EuclideanDistance` is used. With the `DistanceFunction` option you can specify your own distance function.

Examples

```
>> NearestTo(9)[{1, 2, 3, 5, 7, 11, 13, 17, 19}]  
{7, 11}
```

Related terms

[FindClusters](#), [BinaryDistance](#), [BrayCurtisDistance](#), [ChessboardDistance](#), [CanberraDistance](#), [CosineDistance](#), [EuclideanDistance](#), [ManhattanDistance](#), [Nearest](#), [SquaredEuclideanDistance](#)

Implementation status

- - full supported

Github

- [Implementation of NearestTo](#)
-

Negative

Negative(x)

returns `True` if `x` is a negative real number.

Examples

```
>> Negative(0)
False

>> Negative(-3)
True

>> Negative(10/7)
False

>> Negative(1+2*I)
False

>> Negative(a + b)
Negative(a+b)

>> Negative(-E)
True

>> Negative(Sin({11, 14}))
{True, False}
```

Implementation status

- - full supported

Github

- [Implementation of Negative](#)
-

Nest

```
Nest(f, expr, n)
```

starting with `expr`, iteratively applies `f` `n` times and returns the final result.

Examples

```
>> Nest(f, x, 3)
f(f(f(x)))

>> Nest((1+#) ^ 2 &, x, 2)
(1+(1+x)^2)^2
```

Related terms

[FixedPoint](#), [FixedPointList](#), [NestWhile](#)

Implementation status

- - full supported

Github

- [Implementation of Nest](#)
-

NestList

```
NestList(f, expr, n)
```

starting with `expr`, iteratively applies `f` `n` times and returns a list of all intermediate results.

Examples

```
>> NestList(f, x, 3)
{x, f(x), f(f(x)), f(f(f(x)))}

>> NestList(2 # &, 1, 8)
{1, 2, 4, 8, 16, 32, 64, 128, 256}
```

Implementation status

- - full supported

Github

- [Implementation of NestList](#)
-

NestWhile

```
NestWhile(f, expr, test)
```

applies a function `f` repeatedly on an expression `expr`, until applying `test` on the result no longer yields `True`.

```
NestWhile(f, expr, test, m)
```

supplies the last `m` results to `test` (default value: 1).

```
NestWhile(f, expr, test, All)
```

supplies all results gained so far to `test`.

Examples

Divide by 2 until the result is no longer an integer:

```
>> NestWhile(#/2&, 10000, IntegerQ)  
625/2
```

Related terms

[FixedPoint](#), [FixedPointList](#), [Nest](#)

Implementation status

- - full supported

Github

- [Implementation of NestWhile](#)
-

NestWhileList

```
NestWhileList(f, expr, test)
```

applies a function `f` repeatedly on an expression `expr`, until applying `test` on the result no longer yields `True`. It returns a list of all intermediate results.

```
NestWhileList(f, expr, test, m)
```

supplies the last `m` results to `test` (default value: 1). It returns a list of all intermediate results.

```
NestWhileList(f, expr, test, All)
```

supplies all results gained so far to `test`. It returns a list of all intermediate results.

```
NestWhileList[f, expr, test, m, max, n]
```

applies `f` to `expr` until `test` does not return `True`. It returns a list of all intermediate results. `test` is a function that takes as its arguments the last `m` results. `max` denotes the maximum number of applications of `f` and `n` denotes that `f` should be applied another `n` times after `test` has terminated the recursion. If `n` is a negative integer, the last `-n` elements will be dropped.

Examples

Divide by 2 until the result is no longer an integer:

```
>> NestWhileList(#/2&, 10000, IntegerQ)
{10000, 5000, 2500, 1250, 625, 625/2}

>> NestWhileList(++1 &, 1, True &, 1, 4, 5)
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

Implementation status

- - full supported

Github

- [Implementation of NestWhileList](#)
-

NExpectation

```
NExpectation(pure-function, data-set)
```

returns the expected value of the `pure-function` for the given `data-set` numerically.

See

- [Wikipedia - Expected value](#)

Examples

```
>> NExpectation(x^2+7*x+8, Distributed(x, NormalDistribution()))  
9.0
```

Implementation status

- - experimental

Github

- [Implementation of NExpectation](#)
-

NextPrime

```
NextPrime(n)
```

gives the next prime after `n`.

```
NextPrime(n, k)
```

gives the `k`th prime after `n`.

Examples

```
>> NextPrime(10000)
10007
>> NextPrime(100, -5)
73
>> NextPrime(10, -5)
-2
>> NextPrime(100, 5)
113
>> NextPrime(5.5, 100)
563
>> NextPrime(5, 10.5)
NextPrime(5, 10.5)
```

Implementation status

- - full supported

Github

- [Implementation of NextPrime](#)
-

NHoldAll

NHoldAll

is an attribute that protects all arguments of a function from numeric evaluation.

Examples

```
>> N(f(2, 3))  
f(2.0,3.0)  
  
>> SetAttributes(f, NHoldAll)  
>> N(f(2, 3))  
f(2,3)
```

Related terms

[Attributes](#), [ClearAttributes](#), [Constant](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#), [SetAttributes](#)

NHoldFirst

NHoldFirst

is an attribute that protects the first argument of a function from numeric evaluation.

Related terms

[Attributes](#), [ClearAttributes](#), [Constant](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldRest](#), [Orderless](#), [SetAttributes](#)

NHoldRest

NHoldRest

is an attribute that protects all but the first argument of a function from numeric evaluation.

Related terms

[Attributes](#), [ClearAttributes](#), [Constant](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [Orderless](#), [SetAttributes](#)

NIntegrate

```
NIntegrate(f, {x,a,b})
```

computes the numerical univariate real integral of f with respect to x from a to b .

See:

- [Wikipedia - Numerical integration](#)
- [Wikipedia - Trapezoidal rule](#)
- [Wikipedia - Romberg's method](#)
- [Wikipedia - Riemann sum](#)
- [Wikipedia - Simpson's rule](#)
- [Wikipedia - Truncation error \(numerical integration\)](#)
- [Wikipedia - Gauss-Kronrod quadrature formula](#)

Examples

```
>> NIntegrate((x-1)*(x-0.5)*x*(x+0.5)*(x+1), {x,0,1})  
-0.0208333333333333
```

LegendreGauss is the default method for numerical integration

```
>> NIntegrate((x-1)*(x-0.5)*x*(x+0.5)*(x+1), {x,0,1}, Method->LegendreGauss)  
-0.0208333333333333
```

```
>> NIntegrate((x-1)*(x-0.5)*x*(x+0.5)*(x+1), {x,0,1}, Method->Simpson)  
-0.0208333320915699
```

```
>> NIntegrate((x-1)*(x-0.5)*x*(x+0.5)*(x+1), {x,0,1}, Method->Trapezoid)  
-0.0208333271245165
```

```
>> NIntegrate((x-1)*(x-0.5)*x*(x+0.5)*(x+1), {x,0,1}, Method->Romberg)  
-0.0208333333333333
```

```
>> NIntegrate(Exp(-x^2), {x, -Infinity, Infinity}, Method->GaussKronrod)  
1.772453850905516
```

```
>> NIntegrate(Cos(200*x), {x,0,1}, Method->GaussKronrod)  
-0.004366486486070
```

Other options include `MaxIterations` and `MaxPoints`

```
>> NIntegrate((x-1)*(x-0.5)*x*(x+0.5)*(x+1), {x,0,1}, Method->Trapezoid, MaxIterations-  
-0.0208333271245165
```

Integrate along a complex line:

```
>> NIntegrate(1.25+I*2.0+(-3.25+I*0.125)*x+(I*3.0)*x^2,{x, -1.75+I*4.0, 1.5+I*(-12.0)})  
-1427.4921875+I*(-709.06640625)
```

Related terms

[D](#), [DSolve](#), [Integrate](#), [Limit](#), [ND](#)

Implementation status

- - partially implemented

Github

- [Implementation of NIntegrate](#)
-

NMaximize

```
NMaximize({maximize_function, constraints}, variables_list)
```

the `NMaximize` function provides an implementation of [George Dantzig's simplex algorithm](#) for solving linear optimization problems with linear equality and inequality constraints and implicit non-negative variables.

See:

- [Wikipedia - Linear programming](#)

Examples

```
>> NMaximize({-2*x+y-5, x+2*y<=6 && 3*x + 2*y <= 12 }, {x, y})  
{-2.0, {x->0.0, y->3.0}}
```

solves the linear problem:

```
Maximize -2x + y - 5
```

with the constraints:

```
x + 2y <= 6  
3x + 2y <= 12  
x >= 0  
y >= 0
```

Related terms

[LinearProgramming](#), [NMinimize](#)

Implementation status

- - partially implemented

Github

- [Implementation of NMaximize](#)
-

NMinimize

```
NMinimize({maximize_function, constraints}, variables_list)
```

the `NMinimize` function provides an implementation of [George Dantzig's simplex algorithm](#) for solving linear optimization problems with linear equality and inequality constraints and implicit non-negative variables.

See:

- [Wikipedia - Linear programming](#)

Examples

```
>> NMinimize({-2*x+y-5, x+2*y<=6 && 3*x + 2*y <= 12}, {x, y})  
{-13.0, {x->4.0, y->0.0}}
```

solves the linear problem:

```
Minimize -2x + y - 5
```

with the constraints:

```
x  + 2y <= 6  
3x + 2y <= 12  
    x >= 0  
        y >= 0
```

Related terms

[LinearProgramming](#), [NMaximize](#)

Implementation status

- - partially implemented

Github

- [Implementation of NMinimize](#)
-

None

None

is a possible value for `span` and `quiet`.

NoneTrue

```
NoneTrue({expr1, expr2, ...}, test)
```

returns `True` if no application of `test` to `expr1`, `expr2`, ... evaluates to `True`.

```
NoneTrue(list, test, level)
```

returns `True` if no application of `test` to items of `list` at `level` evaluates to `True`.

```
NoneTrue(test)
```

gives an operator that may be applied to expressions.

Examples

```
>> NoneTrue({1, 3, 5}, EvenQ)
True

>> NoneTrue({1, 4, 5}, EvenQ)
False

>> NoneTrue({}, EvenQ)
True
```

Implementation status

- - full supported

Github

- [Implementation of NoneTrue](#)
-

NonNegative

```
NonNegative(x)
```

returns `True` if `x` is a positive real number or zero.

Examples

```
>> {Positive(0), NonNegative(0)}  
{False, True}
```

Implementation status

- - full supported

Github

- [Implementation of NonNegative](#)
-

NonPositive

```
NonPositive(x)
```

returns `True` if `x` is a negative real number or zero.

Examples

```
>> {Negative(0), NonPositive(0)}  
{False, True}
```

Implementation status

- - full supported

Github

- [Implementation of NonPositive](#)
-

Nor

```
Nor(arg1, arg2, ...)
```

Logical NOR function. It evaluates its arguments in order, giving `False` immediately if any of them are `True`, and `True` if they are all `False`.

See

- [Wikipedia - Logical NOR](#)

Examples

```
>> Nor(False, False, False)
True

>> Nor(False, True, a)
False
```

Implementation status

- - full supported

Github

- [Implementation of Nor](#)
-

Norm

```
Norm(v)
```

returns the norm of the vector `v`

```
Norm(m, l)
```

computes the `l`-norm of matrix `m` (currently only works for vectors!).

```
Norm(m)
```

computes the norm of matrix `m` (SVD)

```
Norm(m, "Frobenius")
```

computes the Frobenius norm of matrix `m`

See

- [Wikipedia - Norm \(mathematics\)](#)
- [Wikipedia - Matrix norm](#)

Examples

```
>> Norm({1, 2, 3, 4}, 2)
Sqrt(30)

>> Norm({10, 100, 200}, 1)
310

>> Norm({a, b, c})
Sqrt(Abs(a)^2+Abs(b)^2+Abs(c)^2)

>> Norm({-100, 2, 3, 4}, Infinity)
100

>> Norm(1 + I)
Sqrt(2)
```

The first Norm argument should be a number, vector, or matrix.

```
>> Norm({1, {2, 3}})
Norm({1, {2, 3}})

>> Norm({x, y})
Sqrt(Abs(x)^2+Abs(y)^2)

>> Norm({x, y}, p)
(Abs(x) ^ p + Abs(y) ^ p) ^ (1 / p)

>> Norm({{1,2}, {3,4}})
5.46499
```

The second argument of Norm, 0, should be a symbol, Infinity, or an integer or real number not less than 1 for vector p-norms; or 1, 2, Infinity, or "Frobenius" for matrix norms.

```
>> Norm({x, y}, 0)
Norm({x, y}, 0)
```

The second argument of Norm, 0.5, should be a symbol, Infinity, or an integer or real number not less than 1 for vector p-norms; or 1, 2, Infinity, or "Frobenius" for matrix norms.

```
>> Norm({x, y}, 0.5)
Norm({x, y}, 0.5)

>> Norm({})
Norm({})

>> Norm(0)
0
```

Implementation status

- - full supported

Github

- [Implementation of Norm](#)
-

Normal

```
Normal(expr)
```

converts a Symja expression `expr` into a normal expression.

Examples

Normalize a series expression:

```
>> Normal(SeriesData(x, 0, {1, 0, -1, -4, -17, -88, -549}, -1, 6, 1))  
1/x-x-4*x^2-17*x^3-88*x^4-549*x^5
```

Normalize a byte array expression:

```
>> BinarySerialize(f(g,2)) // Normal  
{56,58,102,2,115,8,71,108,111,98,97,108,96,102,115,8,71,108,111,98,97,108,96,103,67,2}
```

Normalize a sparse array:

```
>> s=SparseArray({11 -> a, 17 -> b})  
SparseArray(Number of elements: 2 Dimensions: {17} Default value: 0)  
  
>> Normal(s)  
{0,0,0,0,0,0,0,0,0,0,0,0,a,0,0,0,0,0,b}
```

Normalize an association:

```
>> assoc = AssociationThread({"U", "V"}, {1, 2})  
<|U->1,V->2|>  
  
>> Normal(assoc)  
{U->1,V->2}
```

Related terms

[Association](#), [ByteArray](#), [SeriesData](#), [SparseArray](#)

Implementation status

- - partially implemented

Github

- [Implementation of Normal](#)
-

NormalDistribution

```
NormalDistribution(m, s)
```

returns the normal distribution of mean `m` and sigma `s`.

```
NormalDistribution()
```

returns the standard normal distribution for `m = 0` and `s = 1`.

See

- [Wikipedia - Normal distribution](#)

Examples

The [probability density function](#) of the normal distribution is

```
>> PDF(NormalDistribution(m, s), x)
1/(Sqrt(2)*E^((-m+x)^2/(2*s^2))*Sqrt(Pi)*s)
```

The [cumulative distribution function](#) of the standard normal distribution is

```
>> CDF(NormalDistribution( ), x)
1/2*(2-Erfc(x/Sqrt(2)))
```

The [mean](#) of the normal distribution is

```
>> Mean(NormalDistribution(m, s))
m
```

The [standard deviation](#) of the normal distribution is

```
>> StandardDeviation(NormalDistribution(m, s))
s
```

The [variance](#) of the normal distribution is

```
>> Variance(NormalDistribution(m, s))
s^2
```

The [random variates](#) of a normal distribution can be generated with function `RandomVariate`

```
>> RandomVariate(NormalDistribution(2,3), 10^1)
{1.14364, 6.09674, 5.16495, 2.39937, -0.52143, -1.46678, 3.60142, -0.85405, 2.06373, -0.29795}
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of NormalDistribution](#)
-

Normalize

`Normalize(v)`

calculates the normalized vector `v` as `v/Norm(v)`.

`Normalize(z)`

calculates the normalized complex number `z`.

`Normalize(v, f)`

calculates the normalized vector `v` and use the function `f` as the norm. Default value is the predefined function `Norm`.

See:

- [Wikipedia - Unit vector](#)

Examples

```
>> Normalize({1, 1, 1, 1})
{1/2,1/2,1/2,1/2}

>> Normalize(1 + I)
(1+I)/Sqrt(2)

>> Normalize(0)
0

>> Normalize({0})
{0}

>> Normalize({})
{}

>> Normalize({x,y}, f)
{x/f({x,y}),y/f({x,y})}
```

Implementation status

- - full supported

Github

- [Implementation of Normalize](#)
-

Not

```
Not(expr)
```

or

```
!expr
```

Logical Not function (negation). Returns `True` if the statement is `False`. Returns `False` if the `expr` is `True`

See

- [Wikipedia - Negation](#)

Examples

```
>> !True
False
>> !False
True
>> !b
!b
```

Implementation status

- - full supported

Github

- [Implementation of Not](#)
-

Nothing

Nothing

during evaluation of a list with a `Nothing` element `{..., Nothing, ...}`, the symbol `Nothing` is removed from the arguments.

`Nothing(...)`

an function with head `Nothing` will be reduced to the symbol `Nothing`.

Examples

```
>> {a, Nothing, Nothing, d}
{a,d}

>> bar(a, Nothing, baz(Nothing), c)
bar(a,Nothing,baz(Nothing),c)
```

Implementation status

- - full supported

Github

- [Implementation of Nothing](#)
-

NRoots

```
NRoots(polynomial==0)
```

gives the numerical roots of a univariate polynomial `polynomial`.

Examples

```
>> NRoots(x^3-4*x^2+x+6==0)
{-1.0, 2.0, 3.0}
```

Related terms

[DSolve](#), [Eliminate](#), [GroebnerBasis](#), [FindRoot](#), [Reduce](#), [Roots](#), [Solve](#)

Implementation status

- - full supported

Github

- [Implementation of NRoots](#)
-

NSolve

```
NSolve(equations, vars)
```

attempts to solve `equations` for the variables `vars`.

```
NSolve(equations, vars, domain)
```

attempts to solve `equations` for the variables `vars` in the given `domain`.

Note: `NSolve` calls [Solve](#) in numeric mode.

Options

- `MaxRoots` the maximum number of roots, which should be returned

Examples

It's important to use the `==` operator to define the equations. If you have unintentionally assigned a value to the variables `x`, `y` with the `=` operator you have to call `Clear(x,y)` to clear the definitions for these variables.

```
>> NSolve({Sin(x)-11==y, x+y==-9}, {y,x})  
{x->1.1060601577062719,y->-10.106060157706272}
```

Related terms

[Solve](#), [Reduce](#), [Roots](#)

Implementation status

- - partially implemented

Github

- [Implementation of NSolve](#)
-

NSum

```
NSum(expr, {i, imin, imax})
```

evaluates the numerical approximated sum of `expr` with `i` ranging from `imin` to `imax`.

```
NSum(expr, {i, imin, imax, di})
```

`i` ranges from `imin` to `imax` in steps `di`

See

- [Wikipedia - Series \(mathematics\)](#)
- [Wikipedia - Summation](#)
- [Wikipedia - Convergence_tests](#)

Examples

```
>> NSum(k, {k, 1, 10})  
55
```

Implementation status

- - experimental

Github

- [Implementation of NSum](#)
-

Null

```
Null
```

is the implicit result of expressions that do not yield a result.

Examples

```
>> FullForm[a:=b]  
"Null"
```

Github

- [Implementation of Null](#)
-

NullSpace

```
NullSpace(matrix)
```

returns a list of vectors that span the nullspace of the `matrix`.

See:

- [Wikipedia - Kernel \(linear algebra\)](#)
- [Youtube - Inverse matrices, column space and null space | Essence of linear algebra, chapter 7](#)

Examples

```
>> NullSpace({{1, 0, -3, 0, 2, -8}, {0, 1, 5, 0, -1, 4}, {0, 0, 0, 1, 7, -9}, {0, 0, 0, 0, 0, 0}})
{{3, -5, 1, 0, 0, 0},
 {-2, 1, 0, -7, 1, 0},
 {8, -4, 0, 9, 0, 1}}
```

```
>> NullSpace({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}})
{{1, -2, 1}}
```

```
>> A = {{1, 1, 0}, {1, 0, 1}, {0, 1, 1}}
>> NullSpace(A)
{}
```

```
>> MatrixRank(A)
3
```

Argument `{1, {2}}` at position 1 is not a non-empty rectangular matrix.

```
>> NullSpace({1, {2}})
NullSpace({1, {2}})
```

Implementation status

- - full supported

Github

- [Implementation of NullSpace](#)
-

NumberLinePlot

```
NumberLinePlot( list-of-numbers )
```

or

```
NumberLinePlot( { list-of-numbers1, list-of-numbers2, ... } )
```

generates a JavaScript control, which plots a list of values along a line. for the `list-of-numbers`.

Note: This feature is available in the console app and in the web interface.

Examples

```
>> NumberLinePlot(Table(Prime(x), {x, 10}))
```

Implementation status

- - experimental

Github

- [Implementation of NumberLinePlot](#)
-

NumberQ

```
NumberQ(expr)
```

returns `True` if `expr` is an explicit number, and `False` otherwise.

Examples

```
>> NumberQ(3+I)
True

>> NumberQ(5!)
True

>> NumberQ(Pi)
False
```

Github

- [Implementation of NumberQ](#)
-

NumberString

NumberString

represents the characters in a number.

Examples

```
>> StringMatchQ("1234", NumberString)
True

>> StringMatchQ("1234.5", NumberString)
True

>> StringMatchQ("1.2'20", NumberString)
False
```

Numerator

`Numerator(expr)`

gives the numerator in `expr`. Numerator collects expressions with non negative exponents.

See

- [Wikipedia - Fraction \(mathematics\)](#)

Examples

```
>> Numerator(a / b)
a
>> Numerator(2 / 3)
2
>> Numerator(a + b)
a + b
```

Implementation status

- - full supported

Github

- [Implementation of Numerator](#)
-

NumericalOrder

```
NumericalOrder(a, b)
```

is 0 if `a` equals `b`. Is -1 or 1 according to numerical order of `a` and `b`.

See

- [Wikipedia - Order theory](#)

Examples

```
>> NumericalOrder(3,4)
1

>> NumericalOrder(4,3)
-1

>> NumericalOrder(Infinity,4)
-1

>> NumericalOrder(-Infinity,3/4)
1
```

Related terms

[NumericalSort](#), [Order](#), [Sort](#)

Implementation status

- - partially implemented

Github

- [Implementation of NumericalOrder](#)
-

NumericalSort

```
NumericalSort(list)
```

`NumericalSort(list)` is evaluated by calling `Sort(list, NumericalOrder)`.

See

- [Wikipedia - Order theory](#)

Examples

```
>> NumericalSort({1,2,3,Infinity,-Infinity,E,Pi,GoldenRatio,Degree})  
{-Infinity,Pi/180,1,GoldenRatio,2,E,3,Pi,Infinity}
```

Related terms

[NumericalOrder](#), [Order](#), [Sort](#)

Implementation status

- - partially implemented

Github

- [Implementation of NumericalSort](#)
-

NumericFunction

NumericFunction

is an attribute for a symbol `f` to denote that the result of `f(arg1, arg2, ...)` can be treated as a numeric value provided that each `argN` is a numeric value.

Examples

The `NumericFunction` is set for the `LogisticSigmoid` function:

```
>> Attributes(LogisticSigmoid)
{Listable, NumericFunction, Protected}

>> LogisticSigmoid(0.5)
0.6224593312018546

>> LogisticSigmoid({-2/10, 1/10, 3/10}) // N
{0.450166, 0.524979, 0.574443}
```

NumericQ

```
NumericQ(expr)
```

returns `True` if `expr` is an explicit numeric expression, and `False` otherwise.

Examples

```
>> NumericQ(E+Pi)
True

>> NumericQ(Sqrt(3))
True
```

Github

- [Implementation of NumericQ](#)
-

OddQ

```
OddQ(x)
```

returns `True` if `x` is odd, and `False` otherwise.

```
OddQ(x, GaussianIntegers->True)
```

returns `True` if `x` is odd and a Gaussian integer number, and `False` otherwise.

See

- [Wikipedia - Parity \(mathematics\)](#)

Examples

```
>> OddQ(-3)
True

>> OddQ(0)
False

>> OddQ(1+4*I, GaussianIntegers->True)
True

>> OddQ(2+4*I, GaussianIntegers->True)
False
```

Related terms

[EvenQ](#)

Implementation status

- - full supported

Github

- [Implementation of OddQ](#)
-

Off

```
off( )
```

switch off the interactive trace.

Examples

`on()` enables the trace of the evaluation steps.

```
>> On()  
On() --> Null  
  
>> D(Sin(x)+Cos(x), x)  
  
NotListQ(x) --> True  
  
D(x,x) --> 1  
  
D(x,x)*(-1)*Sin(x) --> (-1)*1*Sin(x)  
  
(-1)*1*Sin(x) --> -Sin(x)  
  
D(Cos(x),x) --> -Sin(x)  
  
NotListQ(x) --> True  
  
D(x,x) --> 1  
  
Cos(x)*D(x,x) --> 1*Cos(x)  
  
1*Cos(x) --> Cos(x)  
  
D(Sin(x),x) --> Cos(x)  
  
D(Cos(x)+Sin(x),x) --> -Sin(x)+Cos(x)  
  
Cos(x)-Sin(x)
```

`off()` disables the trace of the evaluation steps.

```
>> Off()  
  
>> D(Sin(x)+Cos(x), x)  
Cos(x)-Sin(x)
```

Related terms

[On](#)

Implementation status

- - partially implemented

Github

- [Implementation of Off](#)
-

On

```
On( )
```

switch on the interactive trace. The output is printed in the defined `out` stream.

```
On({head1, head2, ... })
```

switch on the interactive trace only for the `head`s defined in the list. The output is printed in the defined `out` stream.

```
On({head1, head2, ... }, Unique)
```

or

```
On(All, Unique)
```

switch on the interactive trace only for the defined `head` s. The output is printed only once for a combination of *unevaluated* input expression and *evaluated* output expression.

Examples

`on()` enables the trace of the evaluation steps.

```
>> On()  
On() --> Null  
  
>> D(Sin(x)+Cos(x), x)  
  
NotListQ(x) --> True  
  
D(x,x) --> 1  
  
D(x,x)*(-1)*Sin(x) --> (-1)*1*Sin(x)  
  
(-1)*1*Sin(x) --> -Sin(x)  
  
D(Cos(x),x) --> -Sin(x)  
  
NotListQ(x) --> True  
  
D(x,x) --> 1
```

```
Cos(x)*D(x,x) --> 1*Cos(x)
```

```
1*Cos(x) --> Cos(x)
```

```
D(Sin(x),x) --> Cos(x)
```

```
D(Cos(x)+Sin(x),x) --> -Sin(x)+Cos(x)
```

```
Cos(x)-Sin(x)
```

`off()` disables the trace of the evaluation steps.

```
>> Off()
```

```
>> D(Sin(x)+Cos(x), x)
```

```
Cos(x)-Sin(x)
```

Related terms

[Off](#)

Implementation status

- - partially implemented

Github

- [Implementation of On](#)
-

OneIdentity

OneIdentity

is an attribute assigned to a symbol, say `f`, indicating that `f(x)`, `f(f(x))`,... etc. are all equivalent to `x` in pattern matching.

Examples

`OneIdentity` affects pattern matching. It does not affect evaluation.

```
>> SetAttributes[f, OneIdentity]

>> a /. f(x_:0, u_) -> {u}
{a}
```

However, without a default argument, the pattern does not match:

```
>> a /. f(u_) -> {u}
a
```

`OneIdentity` does not affect evaluation:

```
>> f(a)
f(a)
```

OpenAppend

```
OpenAppend("file-name")
```

opens a file and returns an OutputStream to which writes are appended.

Examples

```
>> f = FileNameJoin({$TemporaryDirectory, \"test_open.txt\"})  
/tmp/test_open.txt  
  
>> str = OpenWrite(f);  
  
>> Write(str, x^2 - y^2)  
  
>> Close(str);
```

`FilePrint` prints the file content:

```
>> FilePrint(f)
```

as:

```
x^2 - y^2
```

```
>> str = OpenAppend(f);  
  
>> Write(str, x^2 + y^2)  
  
>> Close(str);
```

`FilePrint` prints the file content:

```
>> FilePrint(f)
```

as:

```
x^2 - y^2  
x^2 + y^2
```

Implementation status

- - partially implemented

Github

- [Implementation of OpenAppend](#)
-

OpenWrite

```
OpenWrite()
```

creates an empty file in the default temporary-file directory and returns an `OutputStream`.

```
OpenWrite("file-name")
```

opens a file and returns an `OutputStream`.

Examples

```
>> f = FileNameJoin({$TemporaryDirectory, \"test_open.txt\"})  
/tmp/test_open.txt  
  
>> str = OpenWrite(f);  
  
>> Write(str, x^2 - y^2)  
  
>> Close(str);
```

`FilePrint` prints the file content:

```
>> FilePrint(f)
```

as:

```
x^2 - y^2
```

```
>> str = OpenAppend(f);  
  
>> Write(str, x^2 + y^2)  
  
>> Close(str);
```

`FilePrint` prints the file content:

```
>> FilePrint(f)
```

as:

$x^2 - y^2$ $x^2 + y^2$

Implementation status

- - partially implemented

Github

- [Implementation of OpenWrite](#)
-

Operate

```
Operate(p, expr)
```

applies `p` to the head of `expr`.

```
Operate(p, expr, n)
```

applies `p` to the `n`th head of `expr`.

Examples

```
>> Operate(p, f(a, b))  
p(f)[a,b]
```

The default value of `n` is `1`:

```
>> Operate(p, f(a, b), 1)  
p(f)[a,b]
```

With `n = 0`, `Operate` acts like `Apply`:

```
>> Operate(p, f(a)[b][c], 0)  
p(f(a)[b][c])
```

```
>> Operate(p, f(a)[b][c])  
p(f(a)[b])[c]
```

```
>> Operate(p, f(a)[b][c], 1)  
p(f(a)[b])[c]
```

```
>> Operate(p, f(a)[b][c], 2)  
p(f(a))[b][c]
```

```
>> Operate(p, f(a)[b][c], 3)  
p(f)[a][b][c]
```

```
>> Operate(p, f(a)[b][c], 4)  
f(a)[b][c]
```

```
>> Operate(p, f)  
f
```

```
>> Operate(p, f, 0)  
p(f)
```

Non-negative integer expected at position 3 in `operate(p, f, -1)`.

```
>> Operate(p, f, -1)
Operate(p, f, -1)
```

Implementation status

- - full supported

Github

- [Implementation of Operate](#)
-

OptimizeExpression

```
OptimizeExpression(function)
```

common subexpressions elimination for a complicated `function` by generating "dummy" variables for these subexpressions.

See

- [Wikipedia - Common subexpression elimination](#)

Examples

```
>> OptimizeExpression( Sin(x) + Cos(Sin(x)) )
{v1+Cos(v1), {v1->Sin(x)}}

>> OptimizeExpression((3 + 3*a^2 + Sqrt(5 + 6*a + 5*a^2) + a*(4 + Sqrt(5 + 6*a + 5*a^2)
{1/6*(3+3*v1+v2+a*(4+v2)), {v1->a^2, v2->Sqrt(5+6*a+5*v1)}})
```

Create the original expression:

```
>> ReplaceRepeated(1/6*(3+3*v1+v2+a*(4+v2)), {v1->a^2, v2->Sqrt(5+6*a+5*v1)})
1/6*(3+3*a^2+Sqrt(5+6*a+5*a^2)+a*(4+Sqrt(5+6*a+5*a^2)))
```

Related terms

[Compile](#), [CompilePrint](#), [Share](#)

Implementation status

- - partially implemented

Github

- [Implementation of OptimizeExpression](#)
-

Optional

```
Optional(patt, default)
```

or

```
patt : default
```

is a pattern which matches `patt`, which if omitted should be replaced by `default`.

Examples

```
>> f(x_, y_:1) := {x, y}

>> f(1, 2)
{1, 2}

>> f(a)
{a, 1}
```

Implementation status

- - full supported

Github

- [Implementation of Optional](#)
-

Options

```
Options(symbol)
```

gives a list of optional arguments to `symbol` and their default values.

Examples

You can assign values to 'Options' to specify options.

```
>> Options(f) = {n -> 2}
{n->2}

>> Options(f)
{n->2}

>> f(x_, OptionsPattern(f)) := x ^ OptionValue(n)

>> f(x)
x^2

>> f(x, n -> 3)
x^3
```

Delayed option rules are evaluated just when the corresponding 'OptionValue' is called:

```
>> f[a :> Print["value"]] /. f[OptionsPattern[{}]] :> (OptionValue[a]; Print["between"])
value
between
value
```

In contrast to that, normal option rules are evaluated immediately:

```
>> f[a -> Print["value"]] /. f[OptionsPattern[{}]] :> (OptionValue[a]; Print["between"])
value
between
```

Options must be rules or delayed rules:

```
>> Options[f] = {a}
{a} is not a valid list of option rules.
{a}
```

A single rule need not be given inside a list:


```
>> Options[f] = a -> b
a -> b

>> Options[f]
{a :> b}
```

Options can only be assigned to symbols:

```
>> Options[a + b] = {a -> b}
Argument a + b at position 1 is expected to be a symbol.
{a -> b}
```

Related terms

[OptionValue](#), [FilterRules](#)

Implementation status

- - full supported

Github

- [Implementation of Options](#)
-

OptionsPattern

```
OptionsPattern(x)
```

is a pattern that stands for a sequence of options given to a function, with default values taken from `options(x)`. The options can be of the form `opt->value` or `opt:>value`, and might be in arbitrarily nested lists.

```
OptionsPattern({opt1->value1, ...})
```

takes explicit default values from the given list. The list may also contain symbols `symbol`, for which `options(symbol)` is taken into account; it may be arbitrarily nested. `OptionsPattern({})` does not use any default values.

Examples

The option values can be accessed using `OptionValue`.

```
>> f(x_, OptionsPattern({n->2})) := x ^ OptionValue(n)

>> f(x)
x ^ 2

>> f(x, n->3)
x ^ 3
```

Delayed rules as options:

```
>> e = f(x, n:>a)
x ^ a

>> a = 5
5

>> e
x ^ 5
```

Options might be given in nested lists:

```
>> f(x, {{{n->4}}})
x ^ 4
```

Related terms

[Options](#), [OptionValue](#), [FilterRules](#)

Implementation status

- - full supported

Github

- [Implementation of OptionsPattern](#)
-

OptionValue

```
OptionValue(name)
```

gives the value of the option `name` as specified in a call to a function with `OptionsPattern`.

```
OptionValue(f, name)
```

recover the value of the option `name` associated to the symbol `f`.

```
OptionValue(f, optvals, name)
```

recover the value of the option `name` associated to the symbol `f`, extracting the values from `optvals` if available.

```
OptionValue(..., list)
```

recover the value of the options in `list`.

Examples

You can assign values to 'Options' to specify options.

```
>> f(a->3) /. f(OptionsPattern({})) -> {OptionValue(a)}  
{3}  
  
>> f(a->3) /. f(OptionsPattern({})) -> {OptionValue(b)}  
{b}  
  
>> f(a->3) /. f(OptionsPattern({})) -> {OptionValue(a+b)}  
{a + b}
```

However, it can be evaluated dynamically:

```
>> f(a->5) /. f(OptionsPattern({})) -> {OptionValue(Symbol("a"))}  
{5}
```

Related terms

[Options](#)

Implementation status

- - full supported

Github

- [Implementation of OptionValue](#)
-

Or

```
Or(expr1, expr2, ...)
```

`expr1 || expr2 || ...` evaluates each expression in turn, returning `True` as soon as an expression evaluates to `True`. If all expressions evaluate to `False`, `or` returns `False`.

See

- [Wikipedia - Logical disjunction](#)

Examples

```
>> False || True
True
```

If an expression does not evaluate to `True` or `False`, `or` returns a result in symbolic form:

```
>> a || False || b
a || b
```

Implementation status

- - full supported

Github

- [Implementation of Or](#)
-

Orange

Orange

RGB color value for the color orange

Examples

```
>> Orange  
RGBColor(1.0,0.5,0.0)
```

Order

```
Order(a, b)
```

is 0 if `a` equals `b`. Is -1 or 1 according to canonical order of `a` and `b`.

See

- [Wikipedia - Order theory](#)

Examples

```
>> Order(3,4)
1

>> Order(4,3)
-1
```

Related terms

[NumericalOrder](#), [NumericalSort](#), [Sort](#)

Implementation status

- - full supported

Github

- [Implementation of Order](#)
-

OrderedQ

```
OrderedQ({a, b, ...})
```

is `True` if `a` sorts before `b` according to canonical ordering for all adjacent elements.

```
OrderedQ({a, b, ...}, comparator)
```

is `True` if `a` sorts before `b` according to the `comparator` functions value for all adjacent elements. If the `comparator` function returns `False` or `-1` the `OrderedQ` function returns `False`. If the `comparator` function doesn't return `False` or `-1` for all adjacent elements the `OrderedQ` function returns `True`.

See

- [Wikipedia - Order theory](#)

Examples

```
>> OrderedQ({a, b})
True

>> OrderedQ({b, a})
False
```

Implementation status

- - full supported

Github

- [Implementation of OrderedQ](#)
-

Ordering

```
Ordering(list)
```

calculate the permutation list of the elements in the sorted `list`.

```
Ordering(list, n)
```

calculate the first `n` indexes of the permutation list of the elements in the sorted `list`.

```
Ordering(list, -n)
```

calculate the last `n` indexes of the permutation list of the elements in the sorted `list`.

```
Ordering(list, n, head)
```

calculate the first `n` indexes of the permutation list of the elements in the sorted `list` using comparator operation `head`.

See

- [Wikipedia - Order theory](#)

Examples

```
>> Ordering({1,3,4,2,5,9,6})
{1,4,2,3,5,7,6}

>> Ordering({1,3,4,2,5,9,6}, All, Greater)
{6,7,5,3,2,4,1}
```

Implementation status

- - full supported

Github

- [Implementation of Ordering](#)
-

Orderless

Orderless

is an attribute indicating that the leaves in an expression `f(a, b, c)` can be placed in any order.

See

- [Wikipedia - Commutative property](#)
- [Wikipedia - Order theory](#)

Examples

The leaves of an `orderless` function are automatically sorted:

```
>> SetAttributes(f, Orderless)
>> f(c, a, b, a + b, 3, 1.0)
f(1.0, 3, a, b, a+b, c)
```

A symbol with the `orderless` attribute represents a commutative mathematical operation.

```
>> f(a, b) == f(b, a)
True
```

`orderless` affects pattern matching:

```
>> SetAttributes(f, Flat)
>> f(a, b, c) /. f(a, c) -> d
f(b, d)
```

Related terms

[Constant](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#)

Orthogonalize

```
Orthogonalize(matrix)
```

returns a basis for the orthogonalized set of vectors defined by `matrix`.

See

- [Wikipedia - Gram–Schmidt process \(linear algebra\)](#)
- [Wikipedia - Orthogonal_matrix](#)

Examples

```
>> Orthogonalize({{3,1},{2,2}})
{{3/Sqrt(10),1/Sqrt(10)},{-Sqrt(5/2)/5,3/5*Sqrt(5/2)}}
```

Implementation status

- - full supported

Github

- [Implementation of Orthogonalize](#)
-

OrthogonalMatrixQ

```
OrthogonalMatrixQ(matrix)
```

returns `True`, if `matrix` is an orthogonal matrix. `False` otherwise.

See

- [Wikipedia - Orthogonal_matrix](#)

Examples

Permutation of coordinate axes.

```
>> OrthogonalMatrixQ({{0, 0, 0, 1}, {0, 0, 1, 0}, {1, 0, 0, 0}, {0, 1, 0, 0}})
True
```

Implementation status

- - full supported

Github

- [Implementation of OrthogonalMatrixQ](#)
-

Out

`Out(k)`

gives the result of the `k`th input line.

Examples

```
>> 42
```

```
42
```

```
>> %
```

```
42
```

Implementation status

- - partially implemented

Github

- [Implementation of Out](#)
-

Outer

```
Outer(f, x, y)
```

computes a generalised outer product of `x` and `y`, using the function `f` in place of multiplication.

See

- Wikipedia - Outer product

Examples

```
>> Outer(f, {a, b}, {1, 2, 3})
{{f(a, 1), f(a, 2), f(a, 3)}, {f(b, 1), f(b, 2), f(b, 3)}}
```

Outer product of two matrices:

```
>> Outer(Times, {{a, b}, {c, d}}, {{1, 2}, {3, 4}})
{{{a, 2 a}, {3 a, 4 a}}, {{b, 2 b}, {3 b, 4 b}}}, {{{c, 2 c}, {3 c, 4 c}}, {{d, 2 d}, {3 d, 4 d}}}
```

Outer of multiple lists:

```
>> Outer(f, {a, b}, {x, y, z}, {1, 2})
{{{f(a, x, 1), f(a, x, 2)}, {f(a, y, 1), f(a, y, 2)}, {f(a, z, 1), f(a, z, 2)}}, {{f(b,
```

Arrays can be ragged:

```
>> Outer(Times, {{1, 2}}, {{a, b}, {c, d, e}})
{{{a, b}, {c, d, e}}, {{2 a, 2 b}, {2 c, 2 d, 2 e}}}
```

Word combinations:

```
>> Outer(StringJoin, {"", "re", "un"}, {"cover", "draw", "wind"}, {"", "ing", "s"})
{{{ "cover", "covering", "covers"}, {"draw", "drawing", "draws"}, {"wind", "winding", "w"
```

Compositions of trigonometric functions:

```
>> trigs = Outer(Composition, {Sin, Cos, Tan}, {ArcSin, ArcCos, ArcTan})
{{Composition(Sin, ArcSin), Composition(Sin, ArcCos), Composition(Sin, ArcTan)}, {Compo
```

Evaluate at 0:

```
>> Map(#(0) &, trigs, {2})  
{{0, 1, 0}, {1, 0, 1}, {0, ComplexInfinity, 0}}
```

Implementation status

- - partially implemented

Github

- [Implementation of Outer](#)
-

OutputStream

```
OutputStream("file-name")
```

opens a file and returns an OutputStream.

Examples

```
>> f = FileNameJoin({$TemporaryDirectory, \"test_open.txt\"})  
/tmp/test_open.txt  
  
>> str = OpenWrite(f);  
  
>> Write(str, x^2 - y^2)  
  
>> Close(str);
```

`FilePrint` prints the file content:

```
>> FilePrint(f)
```

as:

```
x^2 - y^2
```

```
>> str = OpenAppend(f);  
  
>> Write(str, x^2 + y^2)  
  
>> Close(str);
```

`FilePrint` prints the file content:

```
>> FilePrint(f)
```

as:

```
x^2 - y^2  
x^2 + y^2
```

Implementation status

- - partially implemented

Github

- [Implementation of OutputStream](#)
-

Overflow

```
Overflow( )
```

represents a number too large to be represented by Symja.

Examples

Implementation status

- - full supported

Github

- [Implementation of Overflow](#)
-

OwnValues

```
OwnValues(symbol)
```

prints the own-value rule associated with `symbol`.

Examples

```
>> a=42
42

>> OwnValues(a)
{HoldPattern(a):>42}
```

Implementation status

- - full supported

Github

- [Implementation of OwnValues](#)
-

PadLeft

```
PadLeft(list, n)
```

pads `list` to length `n` by adding `0` on the left.

```
PadLeft(list, n, x)
```

pads `list` to length `n` by adding `x` on the left.

```
PadLeft(list)
```

turns the ragged list `list` into a regular list by adding '0' on the left.

Examples

```
>> PadLeft({1, 2, 3}, 5)
{0, 0, 1, 2, 3}

>> PadLeft(x(a, b, c), 5)
x(0, 0, a, b, c)

>> PadLeft({1, 2, 3}, 2)
{2, 3}

>> PadLeft({}, {1, 2}, {1, 2, 3})
{{0, 0, 0}, {0, 1, 2}, {1, 2, 3}}
```

[OEIS - A196023](#):

```
>> Select(Table(FromDigits@Join(Flatten@IntegerDigits@PadLeft({666}, 2, n), Reverse@Int
{16661, 76667, 3166613, 3466643, 7466647, 7666667, 145666541, 148666841, 152666251,
155666551, 169666961, 176666671, 181666181, 304666403, 305666503, 307666703, 308666803,
329666923, 347666743, 349666943, 373666373, 374666473, 383666383, 391666193, 397666793})
```

Implementation status

- - full supported

Github

- [Implementation of PadLeft](#)
-

PadRight

```
PadRight(list, n)
```

pads `list` to length `n` by adding `0` on the right.

```
PadRight(list, n, x)
```

pads `list` to length `n` by adding `x` on the right.

```
PadRight(list)
```

turns the ragged list `list` into a regular list by adding '0' on the right.

Examples

```
>> PadRight({1, 2, 3}, 5)
{1,2,3,0,0}

>> PadRight(x(a, b, c), 5)
x(a,b,c,0,0)

>> PadRight({1, 2, 3}, 2)
{1,2}

>> PadRight({{ }, {1, 2}, {1, 2, 3}})
{{0,0,0},{1,2,0},{1,2,3}}
```

[OEIS - A134860](#):

```
>> With({r = Map(Fibonacci, Range(2, 14))}, Position(#, {1, 0, 1})[[All, 1]] &@ Table(I
{4,12,17,25,33,38,46,51,59,67,72,80,88,93,101,106,114,122,127,135,140,148,156,
161,169,177,182,190,195,203,211,216,224,232,237,245,250,258,266,271,279,284,292,
300,305,313,321,326,334,339,347,355,360,368,373}
```

Implementation status

- - full supported

Github

- [Implementation of PadRight](#)

ParametricPlot

```
ParametricPlot({function1, function2}, {t, tMin, tMax})
```

generate a JavaScript control for the parametric expressions `function1`, `function2` in the `t` range `{t, tMin, tMax}`.

Note: This feature is available in the console app and in the web interface.

Examples

In the console apps, this command shows an HTML page with a JavaScript parametric plot control.

```
>> ParametricPlot({Sin(t), Cos(t^2)}, {t, 0, 2*Pi})
```

With `JSForm` you can display the generated JavaScript form of the `ParametricPlot` function

```
>> ParametricPlot({Sin(t), Cos(t^2)}, {t, 0, 2*Pi}) // JSForm
```

Related terms

[JSForm](#), [Manipulate](#), [Plot](#), [PolarPlot](#), [Plot3D](#)

Implementation status

- - experimental

Github

- [Implementation of ParametricPlot](#)
-

Parenthesis

```
Parenthesis(expr)
```

print `expr` with parenthesis surrounded in output forms.

Examples

```
>> x+Parenthesis(a+b+c)
x+(a+b+c)

>> TeXForm(x+Parenthesis(a+b+c))
x+(a+b+c)

>> MathMLForm(x+Parenthesis(a+b+c))
```

ParetoDistribution

```
ParetoDistribution(k,a)
```

```
ParetoDistribution(k,a,m)
```

```
ParetoDistribution(k,a,g,m)
```

returns a Pareto distribution.

See

- [Wikipedia - Pareto distribution](#)

Examples

The variance of the Pareto distribution is

```
>> Variance(ParetoDistribution(k,a))  
Piecewise({{(a*k^2)/((1-a)^2*(-2+a)),a>2}},Indeterminate)
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of ParetoDistribution](#)
-

Part

```
Part(expr, i)
```

or

```
expr[[i]]
```

returns part `i` of `expr`.

Examples

Extract an element from a list:

```
>> A = {a, b, c, d}

>> A[[3]]
c
```

Negative indices count from the end:

```
>> {a, b, c}][[-2]]
b
```

`Part` can be applied on any expression, not necessarily lists.

```
>> (a + b + c)[[2]]
b
```

`expr[[0]]` gives the head of `expr`:

```
>> (a + b + c)[[0]]
Plus
```

Parts of nested lists:

```
>> M = {{a, b}, {c, d}}

>> M[[1, 2]]
b
```

You can use `span` to specify a range of parts:

```
>> {1, 2, 3, 4}[[2;;4]]
{2,3,4}

>> {1, 2, 3, 4}[[2;;-1]]
{2,3,4}
```

A list of parts extracts elements at certain indices:

```
>> {a, b, c, d}[[{1, 3, 3}]]
{a,c,c}
```

Get a certain column of a matrix:

```
>> B = {{a, b, c}, {d, e, f}, {g, h, i}}
>> B[[;;, 2]]
{b, e, h}
```

Extract a submatrix of 1st and 3rd row and the two last columns:

```
>> B = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}
>> B[[{1, 3}, -2;;-1]]
{{2,3},{8,9}}
```

Further examples:

```
>> (a+b+c+d)[[-1;;-2]]
0
```

Part specification is longer than depth of object.

```
>> x[[2]]
x[[2]]
```

Assignments to parts are possible:

```
>> B[[;;, 2]] = {10, 11, 12}
{10, 11, 12}

>> B
{{1, 10, 3}, {4, 11, 6}, {7, 12, 9}}

>> B[[;;, 3]] = 13
```

13

```
>> B
{{1, 10, 13}, {4, 11, 13}, {7, 12, 13}}

>> B[[1;;-2]] = t
>> B
{t,t,{7,12,13}}

>> F = Table(i*j*k, {i, 1, 3}, {j, 1, 3}, {k, 1, 3})
>> F[[;; All, 2 ;; 3, 2]] = t
>> F
{{{1,2,3},{2,t,6},{3,t,9}},{{2,4,6},{4,t,12},{6,t,18}},{{3,6,9},{6,t,18},{9,t,27}}}}

>> F[[;; All, 1 ;; 2, 3 ;; 3]] = k
>> F
{{{1,2,k},{2,t,k},{3,t,9}},{{2,4,k},{4,t,k},{6,t,18}},{{3,6,k},{6,t,k},{9,t,27}}}}
```

Of course, part specifications have precedence over most arithmetic operations:

```
>> A[[1]] + B[[2]] + C[[3]] // Hold // FullForm
"Hold(Plus(Plus(Part(A, 1), Part(B, 2)), Part(C, 3)))"

>> a = {2,3,4}; i = 1; a[[i]] = 0; a
{0, 3, 4}
```

Negative step

```
>> {1,2,3,4,5}[[3;;1;;-1]]
{3,2,1}

>> {1, 2, 3, 4, 5}[[;; ;; -1]]
{5, 4, 3, 2, 1}

>> Range(11)[[-3 ;; 2 ;; -2]]
{9,7,5,3}

>> Range(11)[[-3 ;; -7 ;; -3]]
{9,6}

>> Range(11)[[7 ;; -7;; -2]]
{7,5}
```

Cannot take positions 1 through 3 in {1, 2, 3, 4}.

```
>> {1, 2, 3, 4}[[1;;3;;-1]]
{1,2,3,4}[[1;;3;;-1]]
```

Cannot take positions 3 through 1 in {1, 2, 3, 4}.

```
>> {1, 2, 3, 4}[[3;;1]]  
{1,2,3,4}[[3;;1]]
```

Implementation status

- - full supported

Github

- [Implementation of Part](#)
-

Partition

```
Partition(list, n)
```

partitions `list` into sublists of length `n`.

```
Partition(list, n, d)
```

partitions `list` into sublists of length `n` which overlap `d` indices.

See

- [Wikipedia - Partition of a set](#)

Examples

```
>> Partition({a, b, c, d, e, f}, 2)
{{a,b},{c,d},{e,f}}

>> Partition({a, b, c, d, e, f}, 3, 1)
{{a,b,c},{b,c,d},{c,d,e},{d,e,f}}

>> Partition({a, b, c, d, e}, 2)
{{a,b},{c,d}}
```

Implementation status

- - full supported

Github

- [Implementation of Partition](#)
-

PartitionsP

PartitionsP(*n*)

gives the number of unrestricted partitions of the integer *n*.

See

- [Wikipedia - Partition \(number theory\)](#)
- [Fungrim - Partition function](#)

Examples

```
>> PartitionsP(50)
204226

>> PartitionsP(6)
11

>> IntegerPartitions(6)
{{6}, {5, 1}, {4, 2}, {4, 1, 1}, {3, 3}, {3, 2, 1}, {3, 1, 1, 1}, {2, 2, 2}, {2, 2, 1, 1}, {2, 1, 1, 1, 1}, {1, 1, 1, 1, 1, 1}}
```

Implementation status

- - full supported

Github

- [Implementation of PartitionsP](#)
-

PartitionsQ

`PartitionsQ(n)`

gives the number of partitions of the integer `n` into distinct parts

See

- [Wikipedia - Partition \(number theory\)](#)

Examples

```
>> PartitionsQ(50)
3658

>> PartitionsQ(6)
4
```

Implementation status

- - full supported

Github

- [Implementation of PartitionsQ](#)
-

PathGraph

```
PathGraph({vertex1, vertex2, ...})
```

create a new path graph with the given vertices `vertex1, vertex2, ....`

See

- [Wikipedia - Path graph](#)

Examples

```
>> PathGraph({1,2,3,4}) // AdjacencyMatrix // Normal
{{0,1,0,0},
 {1,0,1,0},
 {0,1,0,1},
 {0,0,1,0}}
```

Implementation status

- - partially implemented

Github

- [Implementation of PathGraph](#)
-

PatternTest

```
PatternTest(pattern, test)
```

or

```
pattern ? test
```

constrains `pattern` to match `expr` only if the evaluation of `test(expr)` yields `True`.

Examples

```
>> MatchQ(3, _Integer?(#>0&))
True

>> MatchQ(-3, _Integer?(#>0&))
False
```

Implementation status

- - full supported

Github

- [Implementation of PatternTest](#)
-

PauliMatrix

```
PauliMatrix(n)
```

returns the n th Pauli spin 2×2 matrix for n between 0 and 4.

See

- [Wikipedia - Pauli matrices](#)

Examples

```
>> PauliMatrix(2)
{{0, -I}, {I, 0}}
```

`PauliMatrix` has attribute `Listable`.

```
>> PauliMatrix({0,1,2,3,4})
{{{1, 0}, {0, 1}}, {{0, 1}, {1, 0}}, {{0, -I}, {I, 0}}, {{1, 0}, {0, -1}}, {{1, 0}, {0, 1}}}
```

Implementation status

- - full supported

Github

- [Implementation of PauliMatrix](#)
-

Pause

```
Pause(seconds)
```

pause the thread for the number of `seconds`.

Examples

Pause 5 seconds.

```
>> Pause(5)
```

Related terms

[TimeConstrained](#), [Timing](#)

Implementation status

- - full supported

Github

- [Implementation of Pause](#)
-

PDF

```
PDF(distribution, value)
```

returns the probability density function of `value`.

```
PDF(distribution, {list} )
```

returns the probability density function of the values of `list`.

See:

- [Wikipedia - probability density function](#)

PDF can be applied to the following distributions:

[BernoulliDistribution](#), [BinomialDistribution](#), [DiscreteUniformDistribution](#),
[ErlangDistribution](#), [ExponentialDistribution](#), [FrechetDistribution](#), [GammaDistribution](#),
[GeometricDistribution](#), [GumbelDistribution](#), [HypergeometricDistribution](#),
[LogNormalDistribution](#), [NakagamiDistribution](#), [NormalDistribution](#),
[PoissonDistribution](#), [StudentTDistribution](#), [WeibullDistribution](#)

Examples

```
>> PDF(NormalDistribution(n, m))  
1/(Sqrt(2)*E^((-n+#1)^2/(2*m^2))*m*Sqrt(Pi))&  
  
>> PDF(GumbelDistribution(n, m),k)  
E^(-E^((k-n)/m)+(k-n)/m)/m  
  
>> Table(PDF(NormalDistribution( ), x), {m, {-1, 1, 2}},{x, {-1, 1, 2}})/N  
{0.24197,0.24197,0.05399},{0.24197,0.24197,0.05399},{0.24197,0.24197,0.05399}}
```

Related terms

[CDF](#), [InverseCDF](#)

Implementation status

- - full supported

Github

- [Implementation of PDF](#)

PearsonCorrelationTest

```
PearsonCorrelationTest(real-vector1, real-vector2)
```

```
PearsonCorrelationTest(real-vector1, real-vector2, "value")
```

"value" can be "TestStatistic", "TestData" or "PValue". In statistics, the Pearson correlation coefficient (PCC) is a correlation coefficient that measures linear correlation between two sets of data.

See

- [Wikipedia - Pearson correlation coefficient](#)

Examples

```
>> PearsonCorrelationTest({1, 2, 3, 5, 8}, {0.11, 0.12, 0.13, 0.15, 0.18}, \"TestData\"  
{1.0,0.0})
```

Implementation status

- - partially implemented

Github

- [Implementation of PearsonCorrelationTest](#)
-

PerfectNumber

PerfectNumber(n)

returns the n th perfect number. In number theory, a perfect number is a positive integer that is equal to the sum of its proper positive divisors, that is, the sum of its positive divisors excluding the number itself.

See

- [Wikipedia - Perfect number](#)
- [Wikipedia - List of perfect numbers](#)

Examples

```
>> Table(PerfectNumber(i), {i, 5})  
{6, 28, 496, 8128, 33550336}
```

Implementation status

- - partially implemented

Github

- [Implementation of PerfectNumber](#)
-

PerfectNumberQ

PerfectNumberQ(n)

returns `True` if `n` is a perfect number. In number theory, a perfect number is a positive integer that is equal to the sum of its proper positive divisors, that is, the sum of its positive divisors excluding the number itself.

See

- [Wikipedia - Perfect number](#)
- [Wikipedia - List of perfect numbers](#)

Examples

```
>> Select(Range(1000), PerfectNumberQ)
{6, 28, 496}
```

Implementation status

- - partially implemented

Github

- [Implementation of PerfectNumberQ](#)
-

Perimeter

```
Perimeter(geometric-form)
```

returns the perimeter of the `geometric-form`.

See:

- [Wikipedia - Perimeter](#)

Examples

```
>> Perimeter(Disk({1,2}))  
2*Pi
```

Implementation status

- - partially implemented

Github

- [Implementation of Perimeter](#)
-

PermutationCycles

```
PermutationCycles(permutation-list)
```

generate a `Cycles({{...},{...}, ...})` expression from the `permutation-list`.

See

- [Wikipedia - Permutation](#)

Examples

```
>> PermutationCycles({3, 1, 2, 5, 4})  
Cycles({{1,3,2},{4,5}})
```

Related terms

[Cycles](#), [FindPermutations](#), [PermutationCyclesQ](#), [PermutationList](#), [PermutationListQ](#), [PermutationReplace](#), [Permutations](#), [Permute](#)

Implementation status

- - full supported

Github

- [Implementation of PermutationCycles](#)
-

PermutationCyclesQ

```
PermutationCyclesQ(cyclesExpression)
```

if `cyclesExpression` is a valid `Cycles({{...},{...}, ...})` expression return `True`.

See

- [Wikipedia - Permutation](#)

Examples

```
>> PermutationCyclesQ(Cycles({{1, 6, 2}, {4, 11, 12, 3}}))
True
```

Related terms

[Cycles](#), [FindPermutation](#), [PermutationCycles](#), [PermutationList](#), [PermutationListQ](#), [PermutationReplace](#), [Permutations](#), [Permute](#)

Implementation status

- - full supported

Github

- [Implementation of PermutationCyclesQ](#)
-

PermutationList

```
PermutationList(Cycles({{...},{...}, ...}))
```

get the permutation list representation from the `cycles({{...},{...}, ...})` expression.

See

- [Wikipedia - Permutation](#)

Examples

```
>> PermutationList(Cycles({{3, 2}, { 6, 7},{11,17}}))  
{1,3,2,4,5,7,6,8,9,10,17,12,13,14,15,16,11}
```

Related terms

[Cycles](#), [FindPermutation](#), [PermutationCycles](#), [PermutationCyclesQ](#), [PermutationListQ](#), [PermutationReplace](#), [Permutations](#), [Permute](#)

Implementation status

- - full supported

Github

- [Implementation of PermutationList](#)
-

PermutationListQ

```
PermutationListQ(permutation-list)
```

if `permutation-list` is a valid permutation list return `True`.

See

- [Wikipedia - Permutation](#)

Examples

```
>> PermutationListQ({5, 7, 6, 1, 3, 4, 2, 8})  
True
```

Related terms

[Cycles](#), [FindPermutation](#), [PermutationCycles](#), [PermutationCyclesQ](#), [PermutationList](#), [PermutationReplace](#), [Permutations](#), [Permute](#)

Implementation status

- - full supported

Github

- [Implementation of PermutationListQ](#)
-

PermutationReplace

```
PermutationReplace(list-or-integer, Cycles({{...},{...}, ...}))
```

replace the arguments of the first expression with the corresponding element from the `Cycles({{...},{...}, ...})` expression.

See

- [Wikipedia - Permutation](#)

Examples

```
>> PermutationReplace({1, b, 3, 4, 5}, Cycles({{1, 5,8}, {2, 7}}))  
{5,b,3,4,8}
```

Related terms

[Cycles](#), [FindPermutation](#), [PermutationCycles](#), [PermutationCyclesQ](#), [PermutationList](#), [PermutationListQ](#), [Permutations](#), [Permute](#)

Implementation status

- - full supported

Github

- [Implementation of PermutationReplace](#)
-

Permutations

```
Permutations(list)
```

gives all possible orderings of the items in `list`.

```
Permutations(list, n)
```

gives permutations up to length `n`.

```
Permutations(list, {n})
```

finds a list of all possible permutations containing exactly `n` elements.

See

- [Wikipedia - Permutation](#)

Examples

```
>> Permutations({a, b, c})
{{a,b,c},{a,c,b},{b,a,c},{b,c,a},{c,a,b},{c,b,a}}

>> Permutations({1, 2, 3}, 2)
{{},{1},{2},{3},{1,2},{1,3},{2,1},{2,3},{3,1},{3,2}}

>> Permutations({a, b, c}, {2})
{{a,b},{a,c},{b,a},{b,c},{c,a},{c,b}}
```

Related terms

[Cycles](#), [FindPermutation](#), [PermutationCycles](#), [PermutationCyclesQ](#), [PermutationList](#), [PermutationListQ](#), [PermutationReplace](#)

Implementation status

- - full supported

Github

- [Implementation of Permutations](#)
-

Permute

```
Permute(list, Cycles({permutationCycles}))
```

permutes the `list` from the cycles in `permutationCycles`.

```
Permute(list, permutation-list)
```

permutes the `list` from the permutations defined in `permutation-list`.

See

- [Wikipedia - Permutation](#)

Examples

```
>> Permute(CharacterRange("v", "z"), Cycles({{1, 5, 3}}))  
{x, w, z, y, v}
```

Related terms

[Cycles](#), [FindPermutation](#), [PermutationCycles](#), [PermutationCyclesQ](#), [PermutationList](#), [PermutationListQ](#), [PermutationReplace](#), [Permutations](#)

Implementation status

- - full supported

Github

- [Implementation of Permute](#)
-

PetersenGraph

```
PetersenGraph()
```

create a `PetersenGraph(5, 2)` graph.

```
PetersenGraph(order, k)
```

create a new Petersen graph with `2*order` number of total vertices.

See

- [Wikipedia - Petersen graph](#)

Examples

```
>> PetersenGraph()  
Graph({1, 2, 3, 4, 5, 6, 7, 8, 9, 10}, {1<->3, 1<->2, 2<->6, 3<->5, 3<->4, 4<->8, 5<->7, 5<->6, 6<->10, 7<
```

Implementation status

- - partially implemented

Github

- [Implementation of PetersenGraph](#)
-

Pi

```
Pi
```

is the constant `Pi`.

See

- [Wikipedia - Pi](#)
- [Fungrim - Pi](#)

Examples

```
>> N(Pi)
3.141592653589793
```

Related terms

[E](#), [EulerGamma](#)

Implementation status

- - full supported

Github

- [Implementation of Pi](#)
-

Pick

```
Pick(nestedList, nestedSelection)
```

returns the elements of `nestedList` that have value `True` in the corresponding position in `nestedSelection`.

```
Pick(nestedList, nestedSelection, pattern)
```

returns the elements of `nestedList` those values in the corresponding position in `nestedSelection` match the `pattern`.

Examples

```
>> Pick({{1, 2}, {2, 3}, {5, 6}}, {{1}, {2, 3}, {{3, 4}, {4, 5}}}, {1} | 2 | {4, 5})
{{1,2},{2},{6}}

>> Pick({a, b, c}, {False, True, False})
{b}

>> Pick(f(g(1, 2), h(3, 4)), {{True, False}, {False, True}})
f(g(1),h(4))

>> Pick({a, b, c, d, e}, {1, 2, 3.5, 4, 5.5}, _Integer)
{a,b,d}
```

Related terms

[Cases](#), [Select](#)

Implementation status

- - full supported

Github

- [Implementation of Pick](#)
-

Piecewise

```
Piecewise({{expr1, cond1}, ...})
```

represents a piecewise function.

```
Piecewise({{expr1, cond1}, ...}, expr)
```

represents a piecewise function with default `expr`.

See:

- [Wikipedia - Piecewise](#)
- [Wikipedia - Step function](#)
- [Wikipedia - Heaviside step function](#)

Examples

```
>> Piecewise({{-x, x<0}, {x, x>=0}})/.{x->-3, {x->-1/3}, {x->0}, {x->1/2}, {x->5}}
{3,1/3,0,1/2,5}
```

Heaviside function

```
>> Piecewise({{0, x <= 0}}, 1)
Piecewise({{0, x <= 0}}, 1)
```

Piecewise defaults to `0`, if no other case is matching.

```
>> Piecewise({{1, False}})
0

>> Piecewise({{0 ^ 0, False}}, -1)
-1
```

Related terms

[UnitStep](#)

Implementation status

- - full supported

Github

- [Implementation of Piecewise](#)
-

PiecewiseExpand

```
PiecewiseExpand(function)
```

expands piecewise expressions into a `Piecewise` function. Currently only `Abs`, `Clip`, `If`, `Ramp`, `UnitStep` are converted to `Piecewise` expressions.

```
PiecewiseExpand(function, Reals)
```

expands `function` piecewise under the assumption, that all variables are real numbers.

See:

- [Wikipedia - Piecewise](#)
- [Wikipedia - Step function](#)
- [Wikipedia - Heaviside step function](#)

Examples

```
>> PiecewiseExpand(Abs(x), Reals)
Piecewise({{-x, x<0}}, x)
```

Related terms

[Abs](#), [Clip](#), [If](#), [Piecewise](#), [Ramp](#), [UnitStep](#)

Implementation status

- - full supported

Github

- [Implementation of PiecewiseExpand](#)
-

PieChart

```
PieChart(list-of-values)
```

plot a pie chart from a `list-of-values`.

Examples

```
>> PieChart({25, 33, 33, 10})
```

Implementation status

- - experimental

Github

- [Implementation of PieChart](#)
-

Pink

Pink

RGB color value for the color pink

Examples

```
>> Pink  
RGBColor(1.0,0.5,0.5)
```

PlanarGraphQ

```
PlanarGraphQ(g)
```

Returns `True` if `g` is a planar graph and `False` otherwise.

See

- [Wikipedia - Planar graph](#)

Examples

```
>> PlanarGraphQ(CycleGraph(4))
True

>> PlanarGraphQ(CompleteGraph(5))
False

>> PlanarGraphQ(CompleteGraph(4))
True

>> PlanarGraphQ("abc")
False
```

Implementation status

- - partially implemented

Github

- [Implementation of PlanarGraphQ](#)
-

Plot

```
Plot(function, {x, xMin, xMax}, PlotRange->{yMin,yMax})
```

generate a JavaScript control for the expression `function` in the `x` range `{x, xMin, xMax}` and `{yMin, yMax}` in the `y` range.

Note: This feature is available in the console app and in the web interface.

Examples

In the console apps, this command shows an HTML page with a JavaScript plot control.

```
>> Plot(Sin(x)*Cos(1 + x), {x, 0, 2*Pi})
```

With `JSForm` you can display the generated JavaScript form of the `Manipulate` function

```
>> Plot(Sin(x)*Cos(1 + x), {x, 0, 2*Pi}) // JSForm
```

Related terms

[JSForm](#) [Manipulate](#) [ParametricPlot](#) [Plot3D](#)

Implementation status

- - partially implemented

Github

- [Implementation of Plot](#)
-

Plot3D

```
Plot3D(function, {x, xMin, xMax}, {y,yMin,yMax})
```

generate a JavaScript control for the expression `function` in the `x` range `{x, xMin, xMax}` and `{yMin, yMax}` in the `y` range.

```
Plot3D(function, {x, xMin, xMax}, {y,yMin,yMax}, ColorFunction->"color-map")
```

set the color map as: `CherryTones`, `Rainbow`, `RustTones`, `SunsetColors`, `TemperatureMap`, `ThermometerColors`, `WatermelonColors`

Note: This function is available in the console app and in the web interface.

Examples

In the console apps, this command shows an HTML page with a JavaScript of a 3D surface plot control:

```
>> Plot3D(Sin(x*y), {x, -1.5, 1.5}, {y, -1.5, 1.5})
```

With `JSForm` you can display the generated JavaScript form of the `Manipulate` function

```
>> Plot3D(Sin(x*y), {x, -1.5, 1.5}, {y, -1.5, 1.5}) // JSForm
```

Related terms

[JSForm](#) [Manipulate](#) [ParametricPlot](#) [Plot](#)

Implementation status

- - experimental

Github

- [Implementation of Plot3D](#)
-

Plus

```
Plus(a, b, ...)
```

```
a + b + ...
```

represents the sum of the terms `a, b, ...`.

Note: the `Plus` operator has the attribute `Flat` ([associative property](#)), `Orderless` ([commutative property](#)), `OneIdentity` and `Listable`.

Examples

```
>> 1 + 2  
3
```

`Plus` performs basic simplification of terms:

```
>> a + b + a  
2*a+b  
  
>> a + a + 3 * a  
5*a  
  
>> a + b + 4.5 + a + b + a + 2 + 1.5 * b  
6.5+3.0*a+3.5*b
```

Apply `Plus` on a list to sum up its elements:

```
>> Plus @@ {2, 4, 6}  
12
```

The sum of the first `1000` integers:

```
>> Plus @@ Range(1000)  
500500
```

`Plus` has default value `0`:

```
>> a /. n_. + x_ :> {n, x}  
{0,a}  
  
>> -2*a - 2*b
```

$-2a-2b$

```
>> -4+2*x+2*Sqrt(3)
-4+2*x+2*Sqrt(3)
```

```
>> 1 - I * Sqrt(3)
1-I*Sqrt(3)
```

```
>> Head(3 + 2 I)
Complex
```

```
>> N(Pi, 30) + N(E, 30)
5.85987448204883847382293085463
```

```
>> N(Pi, 30) + N(E, 30) // Precision
30
```

Related terms

[Flat](#), [Listable](#), [OneIdentity](#), [Orderless](#)

Implementation status

- - full supported

Github

- [Implementation of Plus](#)
-

PlusMinus

```
PlusMinus(a, b, ...)
```

```
a ± b ± ...
```

or

```
PlusMinus(a)
```

```
± a
```

has no built-in evaluating function, but represents the structure of the $\pm$ operator.

See

- [Wikipedia - Plus-minus sign](#)

Examples

```
>> \[PlusMinus] x
```

```
±x
```

```
>> x \[PlusMinus] y
```

```
x±y
```

Pochhammer

```
Pochhammer(a, n)
```

returns the pochhammer symbol for a rational number `a` and an integer number `n`.

See

- [Wikipedia - Falling and rising factorials](#)

Examples

```
>> Pochhammer(4, 8)
6652800
```

Implementation status

- - full supported

Github

- [Implementation of Pochhammer](#)
-

Point

```
Point({point_1, point_2 ...})
```

represents the point primitive.

```
Point({{p_11, p_12, ...}, {p_21, p_22, ...}, ...})
```

represents a number of point primitives.

Examples

```
>> Graphics3D({Orange, PointSize(0.05), Point(Table({Sin(t), Cos(t), 0}, {t, 0, 2*Pi, P  
-Graphics3D-
```

Implementation status

- - full supported

Github

- [Implementation of Point](#)
-

PoissonDistribution

```
PoissonDistribution(m)
```

returns a Poisson distribution.

See

- [Wikipedia - Poisson distribution](#)

Examples

The variance of the Poisson distribution is

```
>> Variance(PoissonDistribution(n))  
n
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of PoissonDistribution](#)
-

PolarPlot

```
PolarPlot(function, {t, tMin, tMax})
```

generate a JavaScript control for the polar plot expressions `function` in the `t` range `{t, tMin, tMax}`.

See

- [Wikipedia - Polar coordinate system](#)

Examples

A polar coordinate system is a two-dimensional coordinate system in which each point on a plane is determined by a distance from a reference point and an angle from a reference direction.

This command shows an HTML page with a JavaScript parametric plot control.

```
>> PolarPlot(1-Cos(t), {t, 0, 2*Pi})
```

With `JSForm` you can display the generated JavaScript form of the `PolarPlot` function

```
>> PolarPlot(1-Cos(t), {t, 0, 2*Pi}) // JSForm
```

Here is a 5-blade propeller, or maybe a flower, using `PolarPlot`:

```
>> PolarPlot(Cos(5*t), {t, 0, Pi})
```

The number of blades and be change by adjusting the `t` multiplier. A slight change adding `Abs` turns this a clump of grass:

```
>> PolarPlot(Abs(Cos(5*t)), {t, 0, Pi})
```

Coils around a ring:

```
>> PolarPlot({1, 1 + Sin(20*t) / 5}, {t, 0, 2*Pi})
```

A spring having 16 turns:

```
>> PolarPlot(Sqrt(t), {t, 0, 16*Pi})
```

Related terms

[JSForm](#), [Manipulate](#), [ParametricPlot](#) [Plot](#), [Plot3D](#)

Implementation status

- - experimental

Github

- [Implementation of PolarPlot](#)
-

PolyGamma

```
PolyGamma(value)
```

return the digamma function of the `value`. The digamma function is defined as the logarithmic derivative of the gamma function.

```
PolyGamma(m, value)
```

return the polygamma function of order `m` of the `value`. The polygamma function of order `m` is a meromorphic function on the complex numbers defined as the $(m + 1)$ -th derivative of the logarithm of the gamma function.

See

- [Wikipedia: Polygamma function](#)
- [Wikipedia: Digamma function](#)
- [Fungrim - Digamma function](#)

Examples

```
>> PolyGamma(-1, 1)
0
```

Implementation status

- - partially implemented

Github

- [Implementation of PolyGamma](#)
 - [Rule definitions of PolyGamma](#)
-

Polygon

```
Polygon({point_1, point_2 ...})
```

represents the filled polygon primitive.

```
Polygon({{p_11, p_12, ...}, {p_21, p_22, ...}, ...})
```

represents a number of filled polygon primitives.

Examples

```
>> Graphics3D(Polygon({{0,0,0},{0,1,1},{1,0,0}}))  
-Graphics3D-
```

Implementation status

- - full supported

Github

- [Implementation of Polygon](#)
-

PolygonalNumber

```
PolygonalNumber(nPoints)
```

returns the triangular number for `nPoints`.

```
PolygonalNumber(rSides, nPoints)
```

returns the polygonal number for `nPoints` and `rSides`.

See:

- [Wikipedia - Polygonal number](#)

Examples

```
>> Table(PolygonalNumber(r, 3), {r, 1, 5})  
{0,3,6,9,12}  
  
>> PolygonalNumber(4, -2)  
4  
  
>> PolygonalNumber(-3, -2)  
-17
```

Implementation status

- - full supported

Github

- [Implementation of PolygonalNumber](#)
-

PolyLog

```
PolyLog(s, z)
```

returns the polylogarithm function.

See:

- [Wikipedia - Polylogarithm](#)

Examples

```
>> PolyLog(-3, f(x))  
(f(x)+4*f(x)^2+f(x)^3)/(1-f(x))^4
```

Implementation status

- - partially implemented

Github

- [Implementation of PolyLog](#)
 - [Rule definitions of PolyLog](#)
-

PolynomialExtendedGCD

```
PolynomialExtendedGCD(p, q, x)
```

returns the extended GCD ('greatest common divisor') of the univariate polynomials p and q .

```
PolynomialExtendedGCD(p, q, x, Modulus -> prime)
```

returns the extended GCD ('greatest common divisor') of the univariate polynomials p and q modulus the $prime$ integer.

See

- [Wikipedia: Polynomial extended Euclidean algorithm](#)

Examples

```
>> PolynomialExtendedGCD(x^8 + x^4 + x^3 + x + 1, x^6 + x^4 + x + 1, x, Modulus->2)
{1, {1+x^2+x^3+x^4+x^5, x+x^3+x^6+x^7}}
```

Implementation status

- - full supported

Github

- [Implementation of PolynomialExtendedGCD](#)
-

PolynomialGCD

```
PolynomialGCD(p, q)
```

returns the GCD ('greatest common divisor') of the polynomials `p` and `q`.

```
PolynomialGCD(p, q, Modulus -> prime)
```

returns the GCD ('greatest common divisor') of the polynomials `p` and `q` modulus the `prime` integer.

See

- [Wikipedia: Polynomial greatest common divisor](#)

Examples

```
>> PolynomialGCD(x^2 + 7*x + 6, x^2-5*x-6)  
1+x
```

Implementation status

- - full supported

Github

- [Implementation of PolynomialGCD](#)
-

PolynomialLCM

```
PolynomialLCM(p, q)
```

returns the LCM ('least common multiple') of the polynomials `p` and `q`.

```
PolynomialLCM(p, q, Modulus -> prime)
```

returns the LCM ('least common multiple') of the polynomials `p` and `q` modulus the `prime` integer.

See

- [Wikipedia: Polynomial greatest common divisor](#)

Examples

```
>> PolynomialLCM(x^2 + 7*x + 6, x^2-5*x-6)
-36-36*x+x^2+x^3
```

Implementation status

- - full supported

Github

- [Implementation of PolynomialLCM](#)
-

PolynomialQ

```
PolynomialQ(p, x)
```

return `True` if `p` is a polynomial for the variable `x`. Return `False` in all other cases.

```
PolynomialQ(p, {x, y, ...})
```

return `True` if `p` is a polynomial for the variables `x, y, ...` defined in the list.
Return `False` in all other cases.

See

- [Wikipedia: Polynomial](#)

Examples

```
>> PolynomialQ(x^2 + 7*x + 6)
True

>> PolynomialQ(Cos(x*y), Cos(x*y))
True

>> PolynomialQ(2*x^3, x^2)
False
```

Implementation status

- - full supported

Github

- [Implementation of PolynomialQ](#)
-

PolynomialQuotient

```
PolynomialQuotient(p, q, x)
```

returns the polynomial quotient of the polynomials `p` and `q` for the variable `x`.

```
PolynomialQuotient(p, q, x, Modulus -> prime)
```

returns the polynomial quotient of the polynomials `p` and `q` for the variable `x` modulus the `prime` integer.

See

- [Wikipedia: Polynomial long division](#)

Examples

```
>> PolynomialQuotient(x^2 + 7*x + 6, x^2-5*x-6, x)
1
```

Implementation status

- - full supported

Github

- [Implementation of PolynomialQuotient](#)
-

PolynomialQuotientRemainder

```
PolynomialQuotientRemainder(p, q, x)
```

returns a list with the polynomial quotient and remainder of the polynomials `p` and `q` for the variable `x`.

```
PolynomialQuotientRemainder(p, q, x, Modulus -> prime)
```

returns list with the polynomial quotient and remainder of the polynomials `p` and `q` for the variable `x` modulus the `prime` integer.

See

- [Wikipedia: Polynomial long division](#)

Examples

```
>> PolynomialQuotientRemainder(x^2 + 7*x + 6, x^2-5*x-6, x)
{1, 12+12*x}
```

Implementation status

- - full supported

Github

- [Implementation of PolynomialQuotientRemainder](#)
-

PolynomialRemainder

```
PolynomialRemainder(p, q, x)
```

returns the polynomial remainder of the polynomials `p` and `q` for the variable `x`.

```
PolynomialRemainder(p, q, x, Modulus -> prime)
```

returns the polynomial remainder of the polynomials `p` and `q` for the variable `x` modulus the `prime` integer.

See

- [Wikipedia: Polynomial long division](#)

Examples

```
>> PolynomialRemainder(x^2 + 7*x + 6, x^2-5*x-6, x)
12+12*x
```

Implementation status

- - full supported

Github

- [Implementation of PolynomialRemainder](#)
-

Position

```
Position(expr, patt)
```

returns the list of positions for which `expr` matches `patt`.

```
Position(expr, patt, ls)
```

returns the positions on levels specified by level specification `ls`.

Examples

```
>> Position({1, 2, 2, 1, 2, 3, 2}, 2)
{{2},{3},{5},{7}}
```

Find positions upto 3 levels deep

```
>> Position({1 + Sin(x), x, (Tan(x) - y)^2}, x, 3)
{{1,2,1},{2}}
```

Find all powers of x

```
>> Position({1 + x^2, x y ^ 2, 4 y, x ^ z}, x^_)
{{1,2},{4}}
```

Use Position as an operator

```
>> Position(_Integer)({1.5, 2, 2.5})
{{2}}
```

Implementation status

- - full supported

Github

- [Implementation of Position](#)
-

Positive

```
Positive(x)
```

returns `True` if `x` is a positive real number.

Examples

```
>> Positive(1)
True
```

`Positive` returns `False` if `x` is zero or a complex number:

```
>> Positive(0)
False

>> Positive(1 + 2 I)
False

>> Positive(Pi)
True

>> Positive(x)
Positive(x)

>> Positive(Sin({11, 14}))
{False, True}
```

Implementation status

- - full supported

Github

- [Implementation of Positive](#)
-

PossibleZeroQ

PossibleZeroQ(expr)

returns `True` if basic symbolic and numerical methods suggests that `expr` has value zero, and `False` otherwise.

See

- [Wikipedia - Constant problem](#)

Examples

Test whether a numeric expression is zero:

```
>> PossibleZeroQ(E^(I*Pi/4)-(-1)^(1/4))
True
```

The determination is approximate.

Test whether a symbolic expression is likely to be identically zero:

```
>> PossibleZeroQ((x + 1) (x - 1) - x^2 + 1)
True

>> PossibleZeroQ(1/x + 1/y - (x + y)/(x y))
True
```

Decide that a numeric expression is zero, based on approximate computations:

```
>> PossibleZeroQ(2^(2*I) - 2^(-2*I) - 2*I*Sin(Log(4)))
True

>> PossibleZeroQ((E + Pi)^2 - E^2 - Pi^2 - 2*E*Pi)
True

>> PossibleZeroQ(Sqrt(x^2) - x)
False
```

Implementation status

- - full supported

Github

- [Implementation of PossibleZeroQ](#)
-

Power

```
Power(a, b)
```

```
a ^ b
```

represents `a` raised to the power of `b`.

Note: `Power` has the `Listable` attribute and `Power(x,y,z)` is grouped as `x^(y^z)`

You can't do matrix calculations with the `^` operator:

```
>> {{2,1},{1,1}} ^ 2
```

is computed as:

```
{{2^2,1^2},{1^2,1^2}}
```

Use `Inverse({{2,1},{1,1}})` or `MatrixPower({{2,1},{1,1}},2)` to calculate matrix inverses and powers.

Don't confuse the `^` operator with the `^^` operator, which can be used for integer number bases other than `10`. Here's an example for a hexadecimal number:

```
>> 16^^abcdefff  
2882400255
```

See

- [Wikipedia - Exponentiation](#)
- [Fungrim - Powers](#)

Examples

```
>> 4 ^ (1/2)  
2
```

```
>> 4 ^ (1/3)  
4^(1/3)
```

```
>> 3^123  
48519278097689642681155855396759336072749841943521979872827
```

```
>> (y ^ 2) ^ (1/2)
```

```
Sqrt(y^2)

>> (y ^ 2) ^ 3
y^6
```

Use a decimal point to force numeric evaluation:

```
>> 4.0 ^ (1/3)
1.5874010519681994
```

`Power` has default value `1` for its second argument:

```
>> a /. x_ ^ n_. :> {x, n}
{a, 1}
```

`Power` can be used with complex numbers:

```
>> (1.5 + 1.0*I) ^ 3.5
-3.682940057821917+I*6.951392664028508

>> (1.5 + 1.0*I) ^ (3.5 + 1.5*I)
-3.1918162904562815+I*0.6456585094161581
```

Infinite expression $0^{(\text{negative number})}$

```
>> 1/0
ComplexInfinity

>> 0 ^ -2
ComplexInfinity

>> 0 ^ (-1/2)
ComplexInfinity

>> 0 ^ -Pi
ComplexInfinity
```

Indeterminate expression $0^{(\text{complex number})}$ encountered.

```
>> 0 ^ (2*I*E)
Indeterminate

>> 0 ^ - (Pi + 2*E*I)
ComplexInfinity
```

Indeterminate expression 0^0 encountered.

```
>> 0 ^ 0
Indeterminate

>> Sqrt(-3+2.*I)
0.5502505227003375+I*1.8173540210239707

>> Sqrt(-3+2*I)
Sqrt(-3+I*2)

>> (3/2+1/2I)^2
2+I*3/2

>> I ^ I
I^I

>> 2 ^ 2.0
4.0

>> Pi ^ 4.
97.40909103400242

>> a ^ b
a^b

>> Power(x,y,z)
Power(x,Power(y,z))
```

Related terms

[BaseForm](#), [MatrixPower](#), [Times](#)

Implementation status

- - full supported

Github

- [Implementation of Power](#)
 - [Rule definitions of Power](#)
-

PowerExpand

`PowerExpand(expr)`

expands out powers of the form $(x^y)^z$ and $(x*y)^z$ in `expr`.

Examples

```
>> PowerExpand((a ^ b) ^ c)
a^(b*c)

>> PowerExpand((a * b) ^ c)
a^c*b^c

>> PowerExpand(Log(x/y))
Log(x) - Log(y)

>> PowerExpand(Log(-13/350))
I*Pi - Log(2) - 2*Log(5) - Log(7) + Log(13)
```

`PowerExpand` is not correct without certain assumptions:

```
>> PowerExpand((x ^ 2) ^ (1/2))
x
```

Implementation status

- - full supported

Github

- [Implementation of PowerExpand](#)
-

PowerMod

```
PowerMod(x, y, m)
```

computes x^y modulo m . x and m must be Gaussian integers and the y must be an integer or rational number.

See

- [Wikipedia - Exponentiation](#)

Examples

```
>> PowerMod(2, 10000000, 3)
1
>> PowerMod(3, -2, 10)
9
```

0 is not invertible modulo 2.

```
>> PowerMod(0, -1, 2)
PowerMod(0, -1, 2)
```

The third argument of `PowerMod` should be nonzero.

```
>> PowerMod(5, 2, 0)
PowerMod(5, 2, 0)
```

`PowerMod` works on square roots.

```
>> PowerMod(3, 1/2, 2)
1
```

`PowerMod` works on Gaussian integers.

```
>> PowerMod(2+I, 2, 19)
3+I*4
```

Implementation status

- - full supported

Github

- [Implementation of PowerMod](#)
-

PowerRange

```
PowerRange(base)
```

Generates a list of powers from exponent `1` to `max`. Max is the largest power of '10 less equal `base`'.

```
PowerRange(min, base)
```

Generates a list of powers from `min` to `max`, with a factor of `10`.

```
PowerRange(min, base, factor)
```

Generates a list of powers from `min` to `max`, with a factor of `factor`.

Examples

```
>> PowerRange(1, x^10, Sqrt(x))  
{1, Sqrt(x), x, x^(3/2), x^2, x^(5/2), x^3, x^(7/2), x^4, x^(9/2), x^5, x^(11/2), x^6, x^(13/2), x^7,
```

Implementation status

- - full supported

Github

- [Implementation of PowerRange](#)
-

PowersRepresentations

PowersRepresentations(intNumber, k, exponent)

computes the representations of the `intNumber` as sum of x^{exponent} terms which occur `k` times.

See

- Wikipedia - Sum of squares function

Examples

```
>> PowersRepresentations(8174, 6, 3)
{{0, 0, 4, 10, 13, 17}, {0, 3, 6, 7, 9, 19}, {0, 7, 10, 12, 12, 15}, {1, 1, 1, 11, 14, 16}, {1, 3, 5, 12, 13, 16}, {1, 3, 5, 12, 13, 16}}
>> 2^3+3^3+4^3+6^3+10^3+19^3
8174
```

Implementation status

- - partially implemented

Github

- Implementation of PowersRepresentations

PreDecrement

```
PreDecrement(x)
```

```
--x
```

decrements `x` by `1`, returning the new value of `x`.

Examples

`--a` is equivalent to `a = a - 1`:

```
>> a = 2
```

```
>> --a  
1
```

```
>> a  
1
```

Implementation status

- - full supported

Github

- [Implementation of PreDecrement](#)
-

PreIncrement

```
PreIncrement(x)
```

```
++x
```

increments `x` by `1`, returning the new value of `x`.

Examples

`++a` is equivalent to `a = a + 1`:

```
>> a = 2
>> ++a
3

>> a
3
```

Implementation status

- - full supported

Github

- [Implementation of PreIncrement](#)
-

Prepend

```
Prepend(expr, item)
```

returns `expr` with `item` prepended to its leaves.

Examples

`Prepend` is similar to `Append`, but adds `item` to the beginning of `expr`:

```
>> Prepend({2, 3, 4}, 1)
{1, 2, 3, 4}
```

`Prepend` works on expressions with heads other than 'List':

```
>> Prepend(f(b, c), a)
f(a, b, c)
```

Unlike `Join`, `Prepend` does not flatten lists in `item`:

```
>> Prepend({c, d}, {a, b})
{{a, b}, c, d}
```

Nonatomic expression expected.

```
>> Prepend(a, b)
Prepend(a, b)
```

Implementation status

- - full supported

Github

- [Implementation of Prepend](#)
-

PrependTo

```
PrependTo(s, item)
```

prepend `item` to value of `s` and sets `s` to the result.

Examples

Assign `s` to a list

```
>> s = {1, 2, 4, 9}
{1, 2, 4, 9}
```

Add a new value at the beginning of the list:

```
>> PrependTo(s, 0)
{0, 1, 2, 4, 9}
```

The value assigned to `s` has changed:

```
>> s
{0, 1, 2, 4, 9}
```

`PrependTo` works with a head other than `List`:

```
>> y = f(a, b, c)
>> PrependTo(y, x)
f(x, a, b, c)

>> y
f(x, a, b, c)
```

`{a, b}` is not a variable with a value, so its value cannot be changed.

```
>> PrependTo({a, b}, 1)
PrependTo({a, b}, 1)
```

`a` is not a variable with a value, so its value cannot be changed.

```
>> PrependTo(a, b)
PrependTo(a, b)
```



```
>> x = 1 + 2  
3
```

Nonatomic expression expected at position 1 in `PrependTo`

```
>> PrependTo(x, {3, 4})  
PrependTo(x, {3, 4})
```

Implementation status

- - full supported

Github

- [Implementation of PrependTo](#)
-

Prime

Prime(*n*)

returns the *n*th prime number.

See

- [Wikipedia - Prime number](#)
- [Wikipedia - Prime number theorem](#)
- [Wikipedia - Bertrand's postulate](#)

Examples

Note that the first prime is 2, not 1:

```
>> Prime(1)
2

>> Prime(167)
991
```

When given a list of integers, a list is returned:

```
>> Prime({5, 10, 15})
{11, 29, 47}
```

Implementation status

- - partially implemented

Github

- [Implementation of Prime](#)
-

PrimeOmega

PrimeOmega(n)

returns the sum of the exponents of the prime factorization of `n`.

See

- [Wikipedia - Prime number](#)
- [Wikipedia - Integer factorization](#)

Examples

```
>> PrimeOmega(990)
{{2, 1}, {3, 2}, {5, 1}, {11, 1}}

>> PrimeOmega(341550071728321)
{{10670053, 1}, {32010157, 1}}

>> PrimeOmega(2010)
{{2, 1}, {3, 1}, {5, 1}, {67, 1}}
```

Implementation status

- - partially implemented

Github

- [Implementation of PrimeOmega](#)
-

PrimePi

PrimePi(x)

gives the number of primes less than or equal to x .

See

- [Wikipedia - Prime-counting function](#)
- [Wikipedia - On the Number of Primes Less Than a Given Magnitude](#)
- [Fungrim - Prime numbers](#)

Examples

```
>> PrimePi(100)
25

>> PrimePi(-1)
0

>> PrimePi(3.5)
2

>> PrimePi(E)
1
```

Implementation status

- - partially implemented

Github

- [Implementation of PrimePi](#)
-

PrimePowerQ

PrimePowerQ(n)

returns `True` if `n` is a power of a prime number.

Examples

```
>> PrimePowerQ(9)
True

>> PrimePowerQ(52142)
False

>> PrimePowerQ(-8)
True

>> PrimePowerQ(371293)
True

>> PrimePowerQ(1)
False
```

Implementation status

- - partially implemented

Github

- [Implementation of PrimePowerQ](#)
-

PrimeQ

```
PrimeQ(n)
```

returns `True` if `n` is a integer prime number.

```
PrimeQ(n, GaussianIntegers->True)
```

returns `True` if `n` is a Gaussian prime number.

For very large numbers, `PrimeQ` uses [probabilistic prime testing](#), so it might be wrong sometimes (i.e. a number might be composite even though `PrimeQ` says it is prime).

See

- [Wikipedia - Prime number](#)
- [Wikipedia - Gaussian primes](#)

Examples

```
>> PrimeQ(2)
True
>> PrimeQ(-3)
True
>> PrimeQ(137)
True
>> PrimeQ(2 ^ 127 - 1)
True
>> PrimeQ(1)
False
>> PrimeQ(2 ^ 255 - 1)
False
```

All prime numbers between `1` and `100`:

```
>> Select(Range(100), PrimeQ)
{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83}
```

`PrimeQ` has attribute `Listable`:

```
>> PrimeQ(Range(20))
{False, True, True, False, True, False, True, False, False, False, True, False, True, F
```

The Gaussian integer $2 = (1 + i)(1 - i)$ isn't a Gaussian prime number:

```
>> PrimeQ(2, GaussianIntegers->True)
False

>> PrimeQ(5+2*I, GaussianIntegers->True)
True
```

Implementation status

- - full supported

Github

- [Implementation of PrimeQ](#)
-

PrimeZetaP

PrimeZetaP(z)

returns the prime zeta function of z .

See

- [Wikipedia - Prime zeta function](#)

Examples

```
>> Table(PrimeZetaP(s), {s, {2, 3, 4, 5, 6}}) // N  
{0.452247, 0.174763, 0.0769931, 0.035755, 0.0170701}
```

PrimitiveRootList

```
PrimitiveRootList(n)
```

returns the list of the primitive roots of n .

See

- [Wikipedia - Primitive root modulo \$n\$](#)

Examples

```
>> PrimitiveRootList(37)
{2, 5, 13, 15, 17, 18, 19, 20, 22, 24, 32, 35}

>> PrimitiveRootList(127)
{3, 6, 7, 12, 14, 23, 29, 39, 43, 45, 46, 48, 53, 55, 56, 57, 58, 65, 67, 78, 83, 85, 86, 91, 92, 93, 96, 97, 101, 1
```

Implementation status

- - experimental

Github

- [Implementation of PrimitiveRootList](#)
-

Print

```
Print(expr)
```

print the `expr` to the default output stream and return `Null`.

Examples

```
>> Do(Print(i0);If(i0>4,Return(toobig)), {i0,1,10})
```

prints these numbers

```
1  
2  
3  
4  
5
```

and returns

```
toobig
```

Implementation status

- - full supported

Github

- [Implementation of Print](#)
-

PrintableASCIIQ

```
PrintableASCIIQ(str)
```

returns `True` if all characters in `str` are ASCII characters.

See:

- [Wikipedia - ASCII](#)
- [Wikipedia - UTF-8](#)

Examples

```
>> PrintableASCIIQ("Symja")  
True
```

Implementation status

- - full supported

Github

- [Implementation of PrintableASCIIQ](#)
-

Probability

```
Probability(pure-function, data-set)
```

returns the probability of the `pure-function` for the given `data-set`.

See

- [Wikipedia - Probability](#)

Examples

```
>> Probability(#^2 + 3*# < 11 &, {-0.21848,1.67503,0.78687,4.9887,7.06587,-1.27856,0.797/10

>> Probability(x^2 + 3*x < 11,Distributed(x,{-0.21848,1.67503,0.78687,0.9887,2.06587,-19/10
```

Implementation status

- - experimental

Github

- [Implementation of Probability](#)
-

Product

```
Product(expr, {i, imin, imax})
```

evaluates the discrete product of `expr` with `i` ranging from `imin` to `imax`.

```
Product(expr, {i, imin, imax, di})
```

`i` ranges from `imin` to `imax` in steps of `di`.

```
Product(expr, {i, imin, imax}, {j, jmin, jmax}, ...)
```

evaluates `expr` as a multiple sum, with `{i, ...}, {j, ...}, ...` being in outermost-to-innermost order.

See

- [Wikipedia - Multiplication - Capital_pi_notation](#)

Examples

```
>> Product(k, {k, 1, 10})
3628800

>> 10!
3628800

>> Product(x^k, {k, 2, 20, 2})
x^110

>> Product(2 ^ i, {i, 1, n})
2^(1/2*n*(1+n))
```

Symbolic products involving the factorial are evaluated:

```
>> Product(k, {k, 3, n})
n! / 2
```

Evaluate the `n`th primorial:

```
>> primorial(0) = 1;
>> primorial(n_Integer) := Product(Prime(k), {k, 1, n});
```

```
>> primorial(12)
7420738134810
```

Implementation status

- - experimental

Github

- [Implementation of Product](#)
 - [Rule definitions of Product](#)
-

ProductLog

`ProductLog(z)`

returns the value of the Lambert W function at `z`.

See

- [Wikipedia - Lambert W function](#)
- [Wikipedia - Omega constant](#)
- [Fungrim - Lambert W function](#)

Examples

The defining equation:

```
>> z == ProductLog(z) * E ^ ProductLog(z)
True
```

Some special values:

```
>> ProductLog(0)
0

>> ProductLog(E)
1
```

Implementation status

- - partially implemented

Github

- [Implementation of ProductLog](#)
 - [Rule definitions of ProductLog](#)
-

Projection

```
Projection(vector1, vector2)
```

Find the orthogonal projection of `vector1` onto another `vector2`.

```
Projection(vector1, vector2, ipf)
```

Find the orthogonal projection of `vector1` onto another `vector2` using the inner product function `ipf`.

See

- [Wikipedia - Vector projection](#)

Examples

```
>> Projection({5, I, 7}, {1, 1, 1})  
{4+I*1/3, 4+I*1/3, 4+I*1/3}  
  
>> {u, v} = RandomReal(1, {2, 6})  
  
>> Abs(VectorAngle(Projection(u, v), v)) < (0.0 + 10^-7)  
True
```

Implementation status

- - full supported

Github

- [Implementation of Projection](#)
-

PseudoInverse

```
PseudoInverse(matrix)
```

computes the Moore-Penrose pseudoinverse of the `matrix`. If `matrix` is invertible, the pseudoinverse equals the inverse.

See

- [Wikipedia - Moore-Penrose pseudoinverse](#)

Examples

```
>> PseudoInverse({{1, 2}, {2, 3}, {3, 4}})
{{1.00000000000000002, 2.0000000000000001},
 {1.9999999999999976, 2.999999999999996},
 {3.0000000000000001, 4.0}}
>> PseudoInverse({{1, 2, 0}, {2, 3, 0}, {3, 4, 1}})
{{-2.999999999999998, 1.9999999999999967, 4.440892098500626E-16},
 {1.999999999999999, -0.9999999999999982, -2.7755575615628914E-16},
 {0.999999999999999, -1.999999999999991, 1.0}}
>> PseudoInverse({{1.0, 2.5}, {2.5, 1.0}})
{{-0.19047619047619038, 0.47619047619047616},
 {0.47619047619047616, -0.1904761904761904}}
```

Argument `{1, {2}}` at position 1 is not a non-empty rectangular matrix.

```
>> PseudoInverse({1, {2}})
PseudoInverse({1, {2}})
```

Implementation status

- - full supported

Github

- [Implementation of PseudoInverse](#)
-

Purple

Purple

RGB color value for the color purple

Examples

```
>> Purple  
RGBColor(0.5,0.0,0.5)
```

QRDecomposition

QRDecomposition(A)

computes the QR decomposition of the matrix `A`. The QR decomposition is a decomposition of a matrix `A` into a product $A = Q \cdot R$ of an unitary matrix `Q` and an upper triangular matrix `R`.

See

- [Wikipedia - QR decomposition](#)

Examples

```
>> QRDecomposition({{1, 2}, {3, 4}, {5, 6}})
{
  {-0.16903085094570325, 0.8970852271450604, 0.4082482904638636},
  {-0.50709255283711, 0.2760262237369421, -0.8164965809277258},
  {-0.8451542547285165, -0.3450327796711773, 0.40824829046386274}},
{-5.916079783099616, -7.437357441610944},
{0.0, 0.828078671210824},
{0.0, 0.0}}
```

Implementation status

- - full supported

Github

- [Implementation of QRDecomposition](#)
-

QuadraticIrrationalQ

```
QuadraticIrrationalQ(expr)
```

returns `True`, if the `expr` is of the form $(p + s \cdot \text{Sqrt}(d)) / q$ for integers p, q, d, s .

See

- [Wikipedia - Quadratic irrational number](#)
- [Wikipedia - Periodic continued fraction](#)

Examples

```
>> QuadraticIrrationalQ(5*Sqrt(11))
True

>> QuadraticIrrationalQ((7*Sqrt(2) + 1)/11)
True

>> QuadraticIrrationalQ(42)
False
```

Related terms

[ContinuedFraction](#)

Implementation status

- - full supported

Github

- [Implementation of QuadraticIrrationalQ](#)
-

Quantile

```
Quantile(list, q)
```

returns the q -Quantile of `list`.

```
Quantile(list, {q1, q2, ...})
```

returns a list of the q -Quantiles of `list`.

```
Quantile(list, q, {{a,b},{c,d}})
```

returns the q -Quantile of `list` with the quantile definition $\{\{a,b\},\{c,d\}\}$. The default parameters for the quantile definition are $\{\{0,0\},\{1,0\}\}$.

Common choices of parameters $\{\{a,b\},\{c,d\}\}$ include:

- $\{\{0, 0\}, \{1, 0\}\}$ - inverse empirical CDF (default)
- $\{\{0, 0\}, \{0, 1\}\}$ - linear interpolation (California method)

In statistics and probability, quantiles are cut points dividing the range of a probability distribution into continuous intervals with equal probabilities, or dividing the observations in a sample in the same way. Quantile is also known as value at risk (VaR) or fractile.

See

- [Wikipedia - Quantile](#)

`quantile` can be applied to the following distributions:

[BernoulliDistribution](#), [ErlangDistribution](#), [ExponentialDistribution](#), [FrechetDistribution](#),
[GammaDistribution](#), [GumbelDistribution](#), [LogNormalDistribution](#),
[NakagamiDistribution](#), [NormalDistribution](#), [StudentTDistribution](#), [WeibullDistribution](#)

Examples

```
>> Quantile({1,2}, 0.5)
1
```

```
>> Quantile(NormalDistribution(m, s), q)
ConditionalExpression(m-Sqrt(2)*s*InverseErfc(2*q), 0<=q<=1)
```

```
>> Quantile(Range(11), 1/3)
4
```

```
>> Quantile(Range(17), 1/4)
5

>> Quantile({1, 2, 3, 4, 5, 6, 7}, {1/4, 3/4})
{2,6}
```

Related terms

[FiveNum](#), [Quartiles](#)

Implementation status

- - full supported

Github

- [Implementation of Quantile](#)
 - [Rule definitions of Quantile](#)
-

Quantity

```
Quantity(value, unit)
```

returns the quantity for `value` and `unit`

See

- [Wikipedia - International System of Units](#)

Examples

```
>> Quantity(50, "s") + Quantity(1, "min")  
110[s]
```

Implementation status

- - partially implemented

Github

- [Implementation of Quantity](#)
-

QuantityMagnitude

```
QuantityMagnitude(quantity)
```

returns the value of the `quantity`

```
QuantityMagnitude(quantity, unit)
```

returns the value of the `quantity` for the given `unit`

See

- [Wikipedia - International System of Units](#)

Examples

Convert from degrees to radians

```
>> QuantityMagnitude(Quantity(360, "deg"), "rad")  
2*Pi
```

Implementation status

- - partially implemented

Github

- [Implementation of QuantityMagnitude](#)
-

QuantityUnit

```
QuantityUnit(quantity)
```

return the unit of the `quantity`

See

- [Wikipedia - International System of Units](#)

Examples

```
>> QuantityUnit(Quantity(42, "Kilograms"))  
Kilograms
```

Implementation status

- - experimental

Github

- [Implementation of QuantityUnit](#)
-

Quartiles

```
Quartiles(arg)
```

returns a list of the $1/4$, $1/2$ and $3/4$ quantile of `arg`.

```
Quartiles(arg, {{a,b},{c,d}})
```

use the quantile parameters `{{a,b},{c,d}}`. The default parameters for the quantile definition are `{{0,0},{1,0}}`.

See

- [Wikipedia - Quartile](#)
- [Wikipedia - Quantile](#)

Examples

Method 1 from [Wikipedia - Quartile](#)

```
>> Quartiles({6, 7, 15, 36, 39, 40, 41, 42, 43, 47, 49}, {{0, 0}, {1, 0}}) // N
{15.0,40.0,43.0}
```

Method 3 from [Wikipedia - Quartile](#)

```
>> Quartiles({6, 7, 15, 36, 39, 40, 41, 42, 43, 47, 49}) // N
{20.25,40.0,42.75}
```

Related terms

[FiveNum](#), [Median](#), [Quantile](#)

Implementation status

- - full supported

Github

- [Implementation of Quartiles](#)
-

Quiet

```
Quiet(expr)
```

evaluates `expr` in "quiet" mode (i.e. no warning messages are shown during evaluation).

Examples

No error is printed for the division by 0

```
>> Quiet(1/0)
ComplexInfinity
```

Implementation status

- - full supported

Github

- [Implementation of Quiet](#)
-

Quotient

```
Quotient(m, n)
```

computes the integer quotient of `m` and `n`.

See

- [Wikipedia - Quotient](#)
- [Wikipedia - Remainder](#)

Examples

```
>> Quotient(23, 7)
3
```

Infinite expression `Quotient(13, 0)` encountered.

```
>> Quotient(13, 0)
ComplexInfinity

>> Quotient(-17, 7)
-3

>> Quotient(-17, -4)
4

>> Quotient(19, -4)
-5
```

Implementation status

- - full supported

Github

- [Implementation of Quotient](#)
-

QuotientRemainder

```
QuotientRemainder(m, n)
```

computes a list of the quotient and remainder from division of `m` and `n`.

See

- [Wikipedia - Quotient](#)
- [Wikipedia - Remainder](#)

Examples

```
>> QuotientRemainder(23, 7)
{3, 2}
```

Infinite expression `QuotientRemainder(13, 0)` encountered.

```
>> QuotientRemainder(13, 0)
QuotientRemainder(13, 0)

>> QuotientRemainder(-17, 7)
{-3, 4}

>> QuotientRemainder(-17, -4)
{4, -1}

>> QuotientRemainder(19, -4)
{-5, -1}
```

Implementation status

- - full supported

Github

- [Implementation of QuotientRemainder](#)
-

Ramp

`Ramp(z)`

The `Ramp` function is a unary real function, whose graph is shaped like a ramp.

See

- [Wikipedia - Ramp function](#)

Examples

```
>> PiecewiseExpand(Ramp(x))  
Piecewise({{x, x>=0}}, 0)
```

Related terms

[Piecewise](#), [PiecewiseExpand](#)

Implementation status

- - full supported

Github

- [Implementation of Ramp](#)
-

RamseyNumber

`RamseyNumber(r, s)`

returns the Ramsey number $R(r, s)$. Currently not all values are known for $1 \leq r \leq 4$. The function returns unevaluated if the value is unknown.

See

- [Wikipedia - Ramsey's theorem](#)
- [OEIS - A212954](#)

Examples

```
>> Table(RamseyNumber(1, j), {j, 1, 20})
{1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1}

>> Table(RamseyNumber(2, j), {j, 1, 20})
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}

>> Table(RamseyNumber(i, j), {i, 1, 5}, {j, 1, 10})
{{1, 1, 1, 1, 1, 1, 1, 1, 1, 1}, {2, 3, 4, 5, 6, 7, 8, 9, 10}, {6, 9, 14, 18, 23, 28, 36, RamseyNumber(3, 10)}, {18, 21, 25, 30, 35, 40, 45, 50, 55, 60}}
```

Implementation status

- - partially implemented

Github

- [Implementation of RamseyNumber](#)
-

RandomReal

```
Random()
```

gives a pseudorandom real number in the range 0.0 to 1.0.

```
Random(number-type, range)
```

gives a pseudorandom number of the type `number-type`, in the specified interval `range`. Possible `number-types` are `Integer`, `Real` or `Complex`.

Deprecated function superseded by: [RandomComplex](#), [RandomInteger](#), [RandomReal](#)

Examples

```
>> Random( )  
0.53275
```

Eight random integers in the range 1..100:

```
>> Table(Random(Integer, {1, 100}), {8})  
...
```

RandomChoice

```
RandomChoice({item1, item2, item3,...})
```

randomly picks one `item` from `items`.

```
RandomChoice({item1, item2, item3,...}, n)
```

randomly picks `n` items from `items`. Each pick in the `n` picks happens from the given set of items, so each item can be picked any number of times.

```
RandomChoice({item1, item2, item3,...}, {n1, n2, ...})
```

randomly picks items from `items` and arranges the picked items in the nested list structure described by `{n1, n2, ...}`.

```
RandomChoice(weights -> items, n)
```

randomly picks `n` items from `items` and uses the corresponding numeric values in `weights` to determine how probable it is for each item in `items` to get picked (in the long run, items with higher weights will get picked more often than ones with lower weight).

```
RandomChoice[weights -> items]
```

randomly picks one item from `items` using weights `weights`.

```
RandomChoice(weights -> items, {n1, n2,...})
```

randomly picks a structured list of items from `items` using weights `weights`.

Examples

```
>> RandomChoice({1,2,3,4,5,6,7})
```

```
5
```

```
>> RandomChoice({a,b,c},20)
```

```
{b,a,b,b,b,c,b,c,c,b,a,a,a,c,b,a,b,a,b,c}
```

```
>> RandomChoice({a, b, c}, {5, 2})
```

```
{{c,a},{c,c},{c,a},{c,c},{a,a}}
```

```
>> RandomChoice({1, 10, 5} -> {a, b, c}, 20)
{c, a, a, b, c, c, b, b, a, c, b, b, c, b, b, c, b, b, b, b}
```

Implementation status

- - full supported

Github

- [Implementation of RandomChoice](#)
-

RandomComplex

```
RandomComplex[{z_min, z_max}]
```

yields a pseudo-random complex number in the rectangle with complex corners `z_min` and `z_max`.

```
RandomComplex[z_max]
```

yields a pseudo-random complex number in the rectangle with corners at the origin and at `z_max`.

```
RandomComplex()
```

yields a pseudo-random complex number with real and imaginary parts from 0 to 1.

```
RandomComplex(range, n)
```

gives a list of `n` pseudo-random complex numbers.

```
RandomComplex(range, {n1, n2, ...})
```

gives a nested list

Examples

```
>> RandomComplex( )  
0.313565+I*0.954076
```

```
>> RandomComplex({1+I, 2+2I}, {2, 2, 3})  
{{{1.61894+I*1.62895, 1.42982+I*1.64042, 1.88055+I*1.24075}, {1.27681+I*1.09553, 1.29139+I*
```

Implementation status

- - full supported

Github

- [Implementation of RandomComplex](#)
-

RandomGraph

```
RandomGraph({number-of-vertices,number-of-edges})
```

create a random graph with `number-of-vertices` vertices and `number-of-edges` edges.

See

- [Wikipedia - Random graph](#)

Examples

```
>> RandomGraph({5,10})  
Graph({1,2,3,4,5},{5<->3,4<->2,5<->1,2<->3,5<->4,2<->5,2<->1,3<->4,3<->1,1<->4})
```

Implementation status

- - partially implemented

Github

- [Implementation of RandomGraph](#)
-

RandomInteger

```
RandomInteger(n)
```

create a random integer number between 0 and n.

Examples

```
>> RandomInteger(100)
88
```

Implementation status

- - full supported

Github

- [Implementation of RandomInteger](#)
-

RandomPermutation

```
RandomPermutation(s)
```

create a pseudo random permutation between `1` and `s`.

```
RandomPermutation(s, n)
```

create `n` pseudo random permutations.

See

- [Wikipedia - Permutation](#)

Examples

```
>> RandomPermutation(10)
Cycles({{1,2,7,3,8,10,5,9,4,6}})

>> RandomPermutation(10,3)
{Cycles({{1,6,4,5,7,10,9,2,3,8}}),Cycles({{1,10,9,4,2,6,3,8,7,5}}),Cycles({{1,4,2,6,8,9
```

Implementation status

- - full supported

Github

- [Implementation of RandomPermutation](#)
-

RandomPrime

```
RandomPrime({imin, imax})
```

create a random prime integer number between `imin` and `imax` inclusive.

```
RandomPrime(imax)
```

create a random prime integer number between 2 and `imax` inclusive.

```
RandomPrime(range, n)
```

gives a list of `n` random primes in `range`.

Examples

```
>> RandomPrime(1000000000000000000000000000000)
338802421239407

>> RandomPrime({14, 17})
17

>> RandomPrime({14, 16}, 1)
:: There are no primes in the specified interval.
RandomPrime({14, 16}, 1)

>> RandomPrime({8,12}, 3)
{11, 11, 11}

>> RandomPrime({10,30}, {2,5})
{{13,13,23,29,23},{13,11,23,13,11}}

>> RandomPrime({10,12}, {2,2})
{{11, 11}, {11, 11}}

>> RandomPrime(2, {3,2})
{{2, 2}, {2, 2}, {2, 2}}
```

Implementation status

- - full supported

Github

- Implementation of RandomPrime

RandomReal

```
RandomReal()
```

create a random number between `0.0` and `1.0`.

Examples

```
>> RandomReal( )  
0.53275
```

Implementation status

- - full supported

Github

- [Implementation of RandomReal](#)
-

RandomSample

```
RandomSample(items)
```

create a random sample for the arguments of the `items`.

```
RandomSample(items, n)
```

randomly picks `n` items from `items`. Each pick in the `n` picks happens after the previous items picked have been removed from `items`, so each item can be picked at most once.

Examples

```
>> RandomSample(f(1, 2, 3, 4, 5))  
f(3, 4, 5, 1, 2)  
  
>> RandomSample(f(1, 2, 3, 4, 5), 3)  
f(3, 4, 1)
```

Implementation status

- - full supported

Github

- [Implementation of RandomSample](#)
-

RandomVariate

```
RandomVariate(distribution)
```

create a pseudo random variate from the `distribution`.

```
RandomVariate(distribution, numberOfVariates)
```

create `numberOfVariates` pseudo random variates from the `distribution`.

```
RandomVariate(distribution, {n1, n2, ...})
```

create a `{n1, n2, ...}` tensor of pseudo random variates.

Examples

The random variates of a binomial distribution can be generated with function `RandomVariate`.

```
>> RandomVariate(BinomialDistribution(10,0.25), 10^1)
{1,2,1,1,4,1,1,3,2,5}
```

```
>> RandomVariate(NormalDistribution(2,3),{2,3,4})
{{{4.62132,1.44626,-2.01995,4.19106},{1.84104,-2.36264,5.36121,-1.24045},{1.99449,2.598
```

Implementation status

- - full supported

Github

- [Implementation of RandomVariate](#)
-

Range

`Range(n)`

returns a list of integers from `1` to `n`.

`Range(a, b)`

returns a list of integers from `a` to `b`.

`Range[a, b, di]`

returns a list of values from `a` to `b` using step `$di$`. More specifically, 'Range' starts from `a` and successively adds increments of `di` until the result is greater (if `di > 0`) or less (if `di < 0`) than `b`.

Examples

```
>> Range(5)
{1, 2, 3, 4, 5}

>> Range(-3, 2)
{-3, -2, -1, 0, 1, 2}

>> Range(0, 2, 1/3)
{0, 1/3, 2/3, 1, 4/3, 5/3, 2}

>> x^Range(n, n + 10, 2)
{x^n, x^(2+n), x^(4+n), x^(6+n), x^(8+n), x^(10+n)}

>> Range(a, a + 12*n, 2*n)
{a, a+2*n, a+4*n, a+6*n, a+8*n, a+10*n, a+12*n}

>> a + Range(0, 3, Pi/8)
{a, a+Pi/8, a+Pi/4, a+3/8*Pi, a+Pi/2, a+5/8*Pi, a+3/4*Pi, a+7/8*Pi}

>> x * Range(-1, 1, 1/5)
{-x, -4/5*x, -3/5*x, -2/5*x, -x/5, 0, x/5, 2/5*x, 3/5*x, 4/5*x, x}
```

Implementation status

- - full supported

Github

- [Implementation of Range](#)
-

RankedMax

```
RankedMax({e_1, e_2, ..., e_i}, n)
```

returns the n-th largest real value in the list `{e_1, e_2, ..., e_i}`.

```
RankedMax({e_1, e_2, ..., e_i}, -n)
```

returns the n-th smallest real value in the list `{e_1, e_2, ..., e_i}`.

Examples

```
>> RankedMax({Pi, Sqrt(3), 2.95, 3}, 3)  
2.95
```

Implementation status

- - full supported

Github

- [Implementation of RankedMax](#)
-

RankedMin

```
RankedMin({e_1, e_2, ..., e_i}, n)
```

returns the n-th smallest real value in the list `{e_1, e_2, ..., e_i}`.

```
RankedMin({e_1, e_2, ..., e_i}, -n)
```

returns the n-th largest real value in the list `{e_1, e_2, ..., e_i}`.

Examples

```
>> RankedMin({Pi, Sqrt(3), 2.95, 3}, 3)  
3
```

Implementation status

- - full supported

Github

- [Implementation of RankedMin](#)
-

Rational

```
Rational
```

is the head of rational numbers.

```
Rational(a, b)
```

constructs the rational number `a / b`.

See

- [Wikipedia - Rational number](#)

Examples

```
>> Head(1/2)
Rational

>> Rational(1, 2)
1/2

>> -2/3
-2/3
```

Implementation status

- - full supported

Github

- [Implementation of Rational](#)
-

Rationalize

```
Rationalize(expression)
```

convert numerical real or imaginary parts in (sub-)expressions into rational numbers.

Examples

```
>> Rationalize(6.75)
27/4

>> Rationalize(0.25+I*0.33333)
1/4+I*33333/100000
```

Implementation status

- - full supported

Github

- [Implementation of Rationalize](#)
-

Ratios

```
Ratios({x1, x2, ...})
```

computes the ratios $\{x_2/x_1, x_3/x_2, x_4/x_2, x_5/x_4\}$.

```
Ratios({x1, x2, ...}, n)
```

computes the n -th ratio iteration of $\{x_1, x_2, \dots\}$.

```
Ratios({x1, x2, ...}, {n1, n2, ..., nk})
```

computes the n_k -th ratio iteration of $\{x_1, x_2, \dots\}$. at level k .

Examples

```
>> Ratios({a,b,c,d,e,f,g,h},5)
{(b^5*d^10*f)/(a*c^10*e^5), (c^5*e^10*g)/(b*d^10*f^5), (d^5*f^10*h)/(c*e^10*g^5)}
```

Implementation status

- - full supported

Github

- [Implementation of Ratios](#)
-

Re

`Re(z)`

returns the real component of the complex number `z`.

See

- [Wikipedia - Complex number](#)

Examples

```
>> Re(3+4*I)
3

>> Im(0.5 + 2.3*I)
2.3
```

Related terms

[Complex](#), [Im](#), [ReIm](#)

Implementation status

- - full supported

Github

- [Implementation of Re](#)
-

Read

```
Read(input-stream)
```

reads the `input-stream` and return one expression.

```
Read(input-stream, object-type)
```

reads the `input-stream` and return one expression of type `object-type` (`Character`, `Expression`, `Word`, `Number`).

```
Read(input-stream, {object-type1, object-type2, ...})
```

reads the `input-stream` and return a list of expressions.

Examples

```
>> str = StringToStream("4711 dummy 0815")
InputStream[Name: String Unique-ID: 1]

>> Read(str, Number)
4711

>> Read(str, {Word, Number})
{dummy, 0815}

>> Close(str)
String
```

Implementation status

- - partially implemented

Github

- [Implementation of Read](#)
-

ReadList

```
ReadList(input-stream)
```

reads all the expressions until the end of file and return a list of these expressions.

```
ReadList(input-stream, object-type)
```

reads the the expressions of type `object-type` and return a list of these expressions.

Examples

```
>> ReadList(StringToStream("(**)\\n\\n\\n{0, x, 1, 0}\\n{1, x, 1, x}"))
{Null, {0, x, 1, 0}, {1, x, 1, x}}

>> ReadList(StringToStream("(**)\\n\\n\\n{0, x, 1, 0}\\n{1, x, 1, x}"), Expression)
{Null, {0, x, 1, 0}, {1, x, 1, x}}

>> ReadList(StringToStream("(**)\\n\\n\\n{0, x, 1, 0}\\n{1, x, 1, x}"), Expression, 2)
{Null, {0, x, 1, 0}}

>> ReadList(StringToStream("(**)\\n\\n\\n{0, x, 1, 0}\\n{1, x, 1, x}"), String) // InputFor
{"(**)", "{0, x, 1, 0}", "{1, x, 1, x}"}
```

Implementation status

- - partially implemented

Github

- [Implementation of ReadList](#)
-

Real

```
Real
```

is the head of real (floating point) numbers.

Examples

```
>> Head(1.5)  
Real
```

RealAbs

`RealAbs(x)`

returns the absolute value of the real number `x`. For complex number arguments the function will be left unevaluated.

See:

- [Wikipedia - Absolute value](#)

Examples

```
>> RealAbs(-3)
3

>> RealAbs(-Infinity)
Infinity
```

Implementation status

- - full supported

Github

- [Implementation of RealAbs](#)
-

Reals

Reals

is the set of real numbers.

Examples

```
>> Refine(D(Abs(x),x), Element(x, Reals))  
x/Abs(x)
```

```
>> Element(Infinity, Reals)  
False
```

RealSign

`RealSign(x)`

gives `-1`, `0` or `1` depending on whether `x` is negative, zero or positive. For complex number arguments the function will be left unevaluated.

See

- [Wikipedia - Sign function](#)

Examples

```
>> RealSign(-2.5)
-1

>> RealSign(-Infinity)
-1
```

Implementation status

- - full supported

Github

- [Implementation of RealSign](#)
-

RealValuedNumberQ

```
RealValuedNumberQ(expr)
```

returns `True` if `expr` is an explicit real number with no imaginary component.

See

- [Wikipedia - Complex number](#)

Examples

```
>> RealValuedNumberQ[10]
= True

>> RealValuedNumberQ[4.0]
= True

>> RealValuedNumberQ[1+I]
= False

>> RealValuedNumberQ[0 * I]
= True

>> RealValuedNumberQ[0.0 * I]
= False
```

Implementation status

- - full supported

Github

- [Implementation of RealValuedNumberQ](#)
-

Reap

```
Reap(expr)
```

gives the result of evaluating `expr`, together with all values sown during this evaluation. Values sown with different tags are given in different lists.

Examples

```
>> Reap(Sow(3); Sow(1))  
{1, {{3, 1}}}
```

[OEIS - A020652](#):

```
>> Reap(Do(If(GCD(num, den) == 1, Sow(num)), {den, 1, 20}, {num, 1, den-1}))[[2, 1]]  
{1, 1, 2, 1, 3, 1, 2, 3, 4, 1, 5, 1, 2, 3, 4, 5, 6, 1, 3, 5, 7, 1, 2, 4, 5, 7, 8, 1, 3, 7, 9, 1, 2, 3, 4, 5, 6, 7, 8, 9,  
10, 1, 5, 7, 11, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 3, 5, 9, 11, 13, 1, 2, 4, 7, 8, 11, 13, 14, 1, 3, 5, 7,  
9, 11, 13, 15, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1, 5, 7, 11, 13, 17, 1, 2, 3, 4, 5, 6, 7, 8,  
9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 1, 3, 7, 9, 11, 13, 17, 19}
```

Related terms

[Sow](#)

Implementation status

- - full supported

Github

- [Implementation of Reap](#)
-

Red

Red

RGB color value for the color red

Examples

```
>> Red  
RGBColor(1.0,0.0,0.0)
```

Reduce

```
Reduce(logic-expression, var)
```

returns the reduced `logic-expression` for the variable `var`. Reduce works only for the `Reals` domain.

Examples

```
>> Reduce(x^6-1==0 && x>0, x)
x==1
```

Related terms

[Solve](#), [NSolve](#), [Roots](#), [NRoots](#)

Implementation status

- - experimental

Github

- [Implementation of Reduce](#)
-

Refine

```
Refine(expression, assumptions)
```

evaluate the `expression` for the given `assumptions`.

Examples

```
>> Refine(Abs(n+Abs(m)), n>=0)
Abs(m)+n

>> Refine(-Infinity<x, x>0)
True

>> Refine(Max(Infinity,x,y), x>0)
Max(Infinity,y)

>> Refine(Sin(k*Pi), Element(k, Integers))
0

>> Sin(k*Pi)
Sin(k*Pi)

>> Refine(D(Abs(x),x), Element(x, Reals))
x/Abs(x)

>> Refine(2/3*Round(x), Element(x,Integers))
2/3*x
```

Implementation status

- - full supported

Github

- [Implementation of Refine](#)
-

RegularExpression

```
RegularExpression("regex")
```

represents the regular expression specified by the string `"regex"`.

Examples

```
>> StringSplit("1.23, 4.56 7.89", RegularExpression("(\\s|,)+"))  
{1.23,4.56,7.89}
```

ReIm

`ReIm(z)`

returns a list of the real and imaginary component of the complex number `z`.

See

- [Wikipedia - Complex number](#)

Examples

```
>> ReIm(3+4*I)
{3,4}

>> ReIm(0.5 + 2.3*I)
{0.5,2.3}
```

Related terms

[Complex](#), [Im](#), [Re](#)

Implementation status

- - full supported

Github

- [Implementation of ReIm](#)
-

ReleaseHold

ReleaseHold(expr)

removes any `Hold`, `HoldForm`, `HoldPattern` or `HoldComplete` head from `expr`.

Examples

```
>> x = 3;

>> Hold(x)
Hold(x)

>> ReleaseHold(Hold(x))
3

>> ReleaseHold(y)
y
```

Related terms

[Hold](#), [HoldComplete](#), [HoldForm](#), [HoldPattern](#)

Implementation status

- - full supported

Github

- [Implementation of ReleaseHold](#)
-

RemoveDiacritics

```
RemoveDiacritics("string")
```

returns a version of `string` with all diacritics removed.

See:

- [Wikipedia - Diacritic](#)

Examples

```
>> RemoveDiacritics("éeàââ")  
eeaaa
```

Github

- [Implementation of RemoveDiacritics](#)
-

RepeatedTiming

```
RepeatedTiming(x)
```

returns a list with the first entry containing the average evaluation time of `x` and the second entry containing the evaluation result of `x`.

Examples

```
>> RepeatedTiming(FactorInteger(2^32-1))  
{0.0003, {{3, 1}, {5, 1}, {17, 1}, {257, 1}, {65537, 1}}}
```

Implementation status

- - full supported

Github

- [Implementation of RepeatedTiming](#)
-

Replace

```
Replace(expr, lhs -> rhs)
```

replaces the left-hand-side pattern expression `lhs` in `expr` with the right-hand-side `rhs`.

```
Replace(expr, {lhs1 -> rhs1, lhs2 -> rhs2, ... } )
```

replaces the left-hand-side patterns expression `lhsX` in `expr` with the right-hand-side `rhsX`.

Examples

```
>> Replace(x, {{e->q, x -> a}, {x -> b}})
{a,b}
```

Related terms

[ReplaceAll](#), [ReplaceList](#), [ReplacePart](#), [ReplaceRepeated](#)

Implementation status

- - full supported

Github

- [Implementation of Replace](#)
-

ReplaceAll

```
ReplaceAll(expr, i -> new)
```

or

```
expr /. i -> new
```

replaces all `i` in `expr` with `new`.

```
ReplaceAll(expr, {i1 -> new1, i2 -> new2, ... } )
```

or

```
expr /. {i1 -> new1, i2 -> new2, ... }
```

replaces all `i`s in `expr` with `new`s.

Examples

```
>> f(a) + f(b) /. f(x_) -> x^2  
a^2+b^2
```

Related terms

[Dispatch](#), [Replace](#), [ReplaceList](#), [ReplacePart](#), [ReplaceRepeated](#)

Implementation status

- - full supported

Github

- [Implementation of ReplaceAll](#)
-

ReplaceAt

```
ReplaceAt(expr, lhs -> rhs, position)
```

replaces the given `position` in `expr` which matches `lhs` with `rhs`.

```
ReplaceAt(expr, lhs -> rhs, list-of-positions)
```

replaces the given `list-of-positions` in `expr` which matches `lhs` with `rhs`.

Examples

```
>> ReplaceAt({a, a, a, a, a}, a -> xx, 2 ;; 5)
{a,xx,xx,xx,xx}
```

Related terms

[ReplaceAll](#), [ReplaceList](#), [ReplacePart](#), [ReplaceRepeated](#)

Implementation status

- - full supported

Github

- [Implementation of ReplaceAt](#)
-

ReplaceList

```
ReplaceList(expr, lhs -> rhs)
```

or

```
ReplaceList(expr, lhs :> rhs)
```

replaces the left-hand-side pattern expression `lhs` in `expr` with the right-hand-side `rhs`.

Examples

```
>> ReplaceList(a+b+c, (x_+y_) :> {{x},{y}})
{{{a},{b+c}}, {{b},{a+c}}, {{c},{a+b}}, {{a+b},{c}}, {{a+c},{b}}, {{b+c},{a}}}
```

Related terms

[Replace](#), [ReplaceAll](#), [ReplacePart](#), [ReplaceRepeated](#)

Implementation status

- - full supported

Github

- [Implementation of ReplaceList](#)
-

ReplacePart

```
ReplacePart(expr, i -> new)
```

replaces part `i` in `expr` with `new`.

```
ReplacePart(expr, {{i, j} -> e1, {k, l} -> e2})'
```

replaces parts `i` and `j` with `e1`, and parts `k` and `l` with `e2`.

Examples

```
>> ReplacePart({a, b, c}, 1 -> t)
{t,b,c}

>> ReplacePart({{a, b}, {c, d}}, {2, 1} -> t)
{{a,b},{t,d}}

>> ReplacePart({{a, b}, {c, d}}, {{2, 1} -> t, {1, 1} -> t})
{{t,b},{t,d}}

>> ReplacePart({a, b, c}, {{1}, {2}} -> t)
{t,t,c}
```

Delayed rules are evaluated once for each replacement:

```
>> n = 1
>> ReplacePart({a, b, c, d}, {{1}, {3}} :> n++)
{1,b,2,d}
```

Non-existing parts are simply ignored:

```
>> ReplacePart({a, b, c}, 4 -> t)
{a,b,c}
```

You can replace heads by replacing part `0`:

```
>> ReplacePart({a, b, c}, 0 -> Times)
a*b*c
```

Negative part numbers count from the end:


```
>> ReplacePart({a, b, c}, -1 -> t)
{a,b,t}
```

Related terms

[Replace](#), [ReplaceAll](#), [ReplaceList](#), [ReplaceRepeated](#)

Implementation status

- - full supported

Github

- [Implementation of ReplacePart](#)
-

ReplaceRepeated

```
ReplaceRepeated(expr, lhs -> rhs)
```

```
expr //. lhs -> rhs
```

or

```
ReplaceRepeated(expr, lhs :> rhs)
```

```
expr //. lhs :> rhs
```

repeatedly applies the rule `lhs -> rhs` to `expr` until the result no longer changes.

Examples

```
>> a+b+c //. c->d  
a+b+d
```

Simplification of logarithms:

```
>> logrules = {Log(x_ * y_) :> Log(x) + Log(y), Log(x_^y_) :> y * Log(x)};  
  
>> Log(a * (b * c) ^ d ^ e * f) //. logrules  
Log(a)+d^e*(Log(b)+Log(c))+Log(f)
```

`ReplaceAll` just performs a single replacement:

```
>> Log(a * (b * c) ^ d ^ e * f) /. logrules  
Log(a)+Log((b*c)^d^e*f)
```

Related terms

[Dispatch](#), [Replace](#), [ReplaceAll](#), [ReplaceList](#), [ReplacePart](#)

Implementation status

- - full supported

Github

- [Implementation of ReplaceRepeated](#)
-

Rescale

```
Rescale(list)
```

```
returns Rescale(list, {Min(list), Max(list)}).
```

```
Rescale(x, {xmin, xmax})
```

```
returns  $x/(x_{\max}-x_{\min})-x_{\min}/(x_{\max}-x_{\min})$ .
```

```
Rescale(x, {xmin, xmax}, {ymin, ymax})
```

```
returns  $(x*(y_{\max}-y_{\min}))/((x_{\max}-x_{\min})-(x_{\min}*y_{\max}-x_{\max}*y_{\min})/(x_{\max}-x_{\min}))$ .
```

Examples

```
>> Rescale({a,b})
{a/(Max(a,b)-Min(a,b))-Min(a,b)/(Max(a,b)-Min(a,b)), b/(Max(a,b)-Min(a,b))-Min(a,b)/(Max(a,b)-Min(a,b))}

>> Rescale({1, 2, 3, 4, 5}, {-100, 100})
{101/200, 51/100, 103/200, 13/25, 21/40}

>> Rescale(x, {xmin, xmax}, {ymin, ymax})
(x*(ymax-ymin))/((xmax-xmin)-(xmin*ymax-xmax*ymin)/(xmax-xmin))
```

Implementation status

- - full supported

Github

- [Implementation of Rescale](#)
-

Rest

```
Rest(expr)
```

returns `expr` with the first element removed.

`Rest(expr)` is equivalent to `expr[[2; ;]]`.

Examples

```
>> Rest({a, b, c})
{b, c}

>> Rest(a + b + c)
b+c
```

Nonatomic expression expected.

```
>> Rest(x)
Rest(x)
```

Implementation status

- - full supported

Github

- [Implementation of Rest](#)
-

Resultant

```
Resultant(polynomial1, polynomial2, var)
```

computes the resultant of the polynomials `polynomial1` and `polynomial2` with respect to the variable `var`.

See

- [Wikipedia - Resultant](#)

Examples

```
>> Resultant((x-y)^2-2 , y^3-5, y)
17-60*x+12*x^2-10*x^3-6*x^4+x^6
```

Implementation status

- - partially implemented

Github

- [Implementation of Resultant](#)
-

Return

```
Return(expr)
```

aborts a function call and returns `expr`.

Examples

```
>> f(x_) := (If(x < 0, Return(0)); x)

>> f(-1)
0

>> Do(If(i > 3, Return()); Print(i), {i, 10})
| 1
| 2
| 3
```

`Return` only exits from the innermost control flow construct.

```
>> g(x_) := (Do(If(x < 0, Return(0)), {i, {2, 1, 0, -1}}); x)

>> g(-1)
-1

>> h(x_) := (If(x < 0, Return()); x)

>> h(1)
1

>> h(-1) // FullForm
Null

>> f(x_) := Return(x)

>> g(y_) := Module({}, z = f(y); 2)

>> g(1)
2
```

Implementation status

- - full supported

Github

- [Implementation of Return](#)
-

Reverse

```
Reverse(list)
```

reverse the elements of the `list`.

Examples

```
>> Reverse({1, 2, 3})  
{3, 2, 1}
```

```
>> Reverse(x(a, b, c))  
x(c, b, a)
```

Implementation status

- - full supported

Github

- [Implementation of Reverse](#)
-

ReverseSort

```
ReverseSort(list)
```

sorts `list` (or the leaves of any other expression) according to reversed canonical ordering.

```
ReverseSort(list, p)
```

sorts in reversed order using `p` to determine the order of two elements.

Examples

```
>> ReverseSort({4, 1.0, a, 3+I})  
{a, 4, 3+I, 1.0}
```

Related terms

[NumericalOrder](#), [NumericalSort](#), [Order](#), [Sort](#)

Implementation status

- - full supported

Github

- [Implementation of ReverseSort](#)
-

RiccatiSolve

```
RiccatiSolve({A,B},{Q,R})
```

An algebraic Riccati equation is a type of nonlinear equation that arises in the context of infinite-horizon optimal control problems in continuous time or discrete time. The continuous time algebraic Riccati equation (CARE): $A^T \cdot X + X \cdot A - X \cdot B \cdot R^{-1} \cdot B^T \cdot X + Q = 0$. And the respective linear controller is: $K = R^{-1} \cdot B^T \cdot X \cdot P$. The solver receives A , B , Q and R and computes P .

Note: the `Eigensystem` method is used to extract the complex eigen vectors.

See:

- [Wikipedia - Algebraic Riccati equation](#)

Examples

```
>> RiccatiSolve({ {{3, -2}, {4, -1}}, {{0}, {1}} }, { {{1.0,0.0},{0.0,1.0}}, {{1.0}} })  
{{19.75982, -7.64298},  
{-7.64298, 4.70718}}
```

Implementation status

- - partially implemented

Github

- [Implementation of RiccatiSolve](#)
-

Riffle

```
Riffle(list1, list2)
```

insert elements of `list2` between the elements of `list1`.

Examples

```
>> Riffle({a, b, c}, x)
{a,x,b,x,c}

>> Riffle({a, b, c}, {x, y, z})
{a,x,b,y,c,z}
```

Implementation status

- - full supported

Github

- [Implementation of Riffle](#)
-

RightComposition

```
RightComposition(sym1, sym2,...)[arg1, arg2,...]
```

creates a composition of the symbols applied in reversed order at the arguments.

Examples

```
>> RightComposition(u, v, w)[x, y]  
w(v(u(x,y)))
```

Implementation status

- - full supported

Github

- [Implementation of RightComposition](#)
-

RogersTanimotoDissimilarity

```
RogersTanimotoDissimilarity(u, v)
```

returns the Rogers-Tanimoto dissimilarity between the two boolean 1-D lists `u` and `v`, which is defined as $R / (c_{tt} + c_{ff} + R)$ where n is `len(u)`, c_{ij} is the number of occurrences of `u(k)=i` and `v(k)=j` for $k < n$, and $R = 2 * (c_{tf} + c_{ft})$.

Examples

```
>> RogersTanimotoDissimilarity({1, 0, 1, 1, 0, 1, 1}, {0, 1, 1, 0, 0, 0, 1})  
8/11
```

Implementation status

- - full supported

Github

- [Implementation of RogersTanimotoDissimilarity](#)
-

RomanNumeral

```
RomanNumeral(positive-int-value)
```

converts the given `positive-int-value` to a roman numeral string.

See

- [Wikipedia - Roman numerals](#)
- [Unicode Number Forms - Archaic Roman Numerals](#)

Examples

```
>> RomanNumeral({4548, 3267, 3603, 1929, 2575, 746, 666, 4108, 1457, 3828})  
{MDXLVIII, MMCCCLXVII, MMMDCIII, MCMXXIX, MMDLXXV, DCCXLVI, DCLXVI, MCVIII, MCDLVII, MMMDCCCXXVI}
```

Zeros are represented by `N`

```
>> RomanNumeral(0)  
N
```

Related terms

[FromRomanNumeral](#)

Implementation status

- - partially implemented

Github

- [Implementation of RomanNumeral](#)
-

RootMeanSquare

```
RootMeanSquare(list)
```

calculate the root mean square

See

- [Wikipedia - Root mean square](#)

Examples

```
>> RootMeanSquare({x,y,z})  
Sqrt(x^2+y^2+z^2)/Sqrt(3)  
  
>> RootMeanSquare({42,43})  
Sqrt(3613/2)
```

Implementation status

- - experimental

Github

- [Implementation of RootMeanSquare](#)
-

Roots

```
Roots(polynomial-equation, var)
```

determine the roots of a univariate polynomial equation with respect to the variable `var`.

Examples

```
>> Roots(3*x^3-5*x^2+5*x-2==0,x)  
x==2/3 || x==1/2-I*1/2*Sqrt(3) || x==1/2+I*1/2*Sqrt(3)
```

Implementation status

- - full supported

Github

- [Implementation of Roots](#)
-

RotateLeft

```
RotateLeft(list)
```

rotates the items of `list` by one item to the left.

```
RotateLeft(list, n)
```

rotates the items of `list` by `n` items to the left.

Examples

```
>> RotateLeft({1, 2, 3})
{2, 3, 1}

>> RotateLeft(Range(10), 3)
{4, 5, 6, 7, 8, 9, 10, 1, 2, 3}

>> RotateLeft(x(a, b, c), 2)
x(c, a, b)
```

Implementation status

- - full supported

Github

- [Implementation of RotateLeft](#)
-

RotateRight

```
RotateRight(list)
```

rotates the items of `list` by one item to the right.

```
RotateRight(list, n)
```

rotates the items of `list` by `n` items to the right.

Examples

```
>> RotateRight({1, 2, 3})
{3,1,2}

>> RotateRight(Range(10), 3)
{8,9,10,1,2,3,4,5,6,7}

>> RotateRight(x(a, b, c), 2)
x(b,c,a)
```

Implementation status

- - full supported

Github

- [Implementation of RotateRight](#)
-

RotationMatrix

```
RotationMatrix(theta)
```

yields a rotation matrix for the angle `theta`.

See

- [Wikipedia - Rotation matrix](#)

Examples

```
>> RotationMatrix(90*Degree)
{{0, -1}, {1, 0}}

>> RotationMatrix(t, {1, 0, 0})
{{1, 0, 0}, {0, Cos(t), -Sin(t)}, {0, Sin(t), Cos(t)}}
```

Implementation status

- - full supported

Github

- [Implementation of RotationMatrix](#)
-

RotationTransform

```
RotationTransform(phi)
```

gives a rotation by `phi`

```
RotationTransform(phi, p)
```

gives a rotation by `phi` around the point `p`.

See

- [Wikipedia - Rotation \(mathematics\)](#)

Examples

```
>> RotationTransform(Pi).TranslationTransform({1, -1})
TransformationFunction(
  {{-1,0,-1},
   {0,-1,1},
   {0,0,1}})

>> TranslationTransform({1, -1}).RotationTransform(Pi)
TransformationFunction(
  {{-1,0,1},
   {0,-1,-1},
   {0,0,1}})
```

Related terms

[TransformationFunction](#), [TranslationTransform](#)

Implementation status

- - full supported

Github

- [Implementation of RotationTransform](#)
-

Round

```
Round(expr)
```

round a given `expr` to nearest integer.

```
Round(expr, factor)
```

round a given `expr` up to the next nearest multiple of a `factor`.

See

- [Wikipedia - Floor and ceiling functions](#)

Examples

```
>> Round(-3.6)
-4

>> Round(1.234512, 0.01)
1.23

>> Round(1.235512, 0.01)
1.24
```

Round to nearest multiple of 5

```
>> Round({12.5, 62.1, 68.3, 74.5, 80.7}, 5)
{10, 60, 70, 75, 80}
```

Related terms

[IntegerPart](#), [Ceiling](#), [Floor](#), [FractionalPart](#)

Implementation status

- - full supported

Github

- [Implementation of Round](#)
-

RowReduce

```
RowReduce(matrix)
```

returns the reduced row-echelon form of `matrix`.

See:

- [Wikipedia - Row echelon form](#)

Examples

```
>> RowReduce({{1, 1, 0, 1, 5}, {1, 0, 0, 2, 2}, {0, 0, 1, 4, -1}, {0, 0, 0, 0, 0}})
{{1, 0, 0, 2, 2},
 {0, 1, 0, -1, 3},
 {0, 0, 1, 4, -1},
 {0, 0, 0, 0, 0}}

>> RowReduce({{1, 0, a}, {1, 1, b}})
{{1, 0, a},
 {0, 1, -a+b}}

>> RowReduce({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}})
{{1, 0, -1},
 {0, 1, 2},
 {0, 0, 0}}
```

Argument `{{1, 0}, {0}}` at position 1 is not a non-empty rectangular matrix.

```
>> RowReduce({{1, 0}, {0}})
RowReduce({{1, 0}, {0}})
```

Implementation status

- - full supported

Github

- [Implementation of RowReduce](#)
-

RSolve

```
RSolve(equation, y(var), var)
```

attempts to solve a recurrence `equation` for the function `y(var)` and variable `var`.

See:

- [Wikipedia - Recurrence relation](#)
- [Wikipedia - Z-transform](#)

Examples

RSolveValue

```
RSolveValue(equation, f(var), var)
```

attempts to solve a recurrence `equation` for the function `y(var)` and variable `var`.

See:

- [Wikipedia - Recurrence relation](#)
- [Wikipedia - Z-transform](#)

Examples

Rule

```
Rule(x, y)
```

```
x -> y
```

represents a rule replacing `x` with `y`.

Examples

```
>> a+b+c /. c->d  
a+b+d
```

```
>> {x,x^2,y} /. x->3  
{3,9,y}
```

Rule called with 3 arguments; 2 arguments are expected.

```
>> a /. Rule(1, 2, 3) -> t  
a
```

Implementation status

- - full supported

Github

- [Implementation of Rule](#)
-

RuleDelayed

```
RuleDelayed(x, y)
```

```
x :> y
```

represents a rule replacing `x` with `y`, with `y` held unevaluated.

Examples

```
>> Cases({1, f(2), f(3, 3, 3), 4, f(5, 5)}, f(x__) :> Plus(x))  
{2, 9, 10}
```

```
>> Cases({1, f(2), f(3, 3, 3), 4, f(5, 5)}, f(x__) -> Plus(x))  
{2, 3, 3, 3, 5, 5}
```

Implementation status

- - full supported

Github

- [Implementation of RuleDelayed](#)
-

RussellRaoDissimilarity

```
RussellRaoDissimilarity(u, v)
```

returns the Russell-Rao dissimilarity between the two boolean 1-D lists `u` and `v`, which is defined as $(n - c_{tt}) / c_{tt}$ where `n` is `len(u)` and `cij` is the number of occurrences of `u(k)=i` and `v(k)=j` for `k < n`.

Examples

```
>> RussellRaoDissimilarity({1, 0, 1, 1, 0, 1, 1}, {0, 1, 1, 0, 0, 0, 1})  
5/7
```

Implementation status

- - full supported

Github

- [Implementation of RussellRaoDissimilarity](#)
-

SameObjectQ

```
SameObjectQ[java-object1, java-object2]
```

gives `True` if the Java `==` operator for the Java objects gives true. `False` otherwise.

Note: the Java specific functions which call Java native classes are only available in the MMA mode in a local installation. All symbol and method names have to be case sensitive.

Examples

```
>> loc1= JavaNew["java.util.Locale","US"]
JavaObject[class java.util.Locale]

>> loc2= JavaNew["java.util.Locale","US"]
JavaObject[class java.util.Locale]

>> SameObjectQ[loc1, loc2]
False

>> SameObjectQ[loc1, loc1]
True
```

Related terms

[InstanceOf](#), [JavaClass](#), [JavaNew](#), [JavaObject](#), [JavaObjectQ](#)

Implementation status

- - supported on Java virtual machine

Github

- [Implementation of SameObjectQ](#)
-

SameQ

```
SameQ(x, y)
```

```
x===y
```

returns `True` if `x` and `y` are structurally identical.

See

- [Wikipedia - Computer algebra - Equality](#)

Examples

Any object is the same as itself:

```
>> a===a
True
```

Unlike `Equal`, `SameQ` only yields `True` if `x` and `y` have the same type:

```
>> {1==1., 1===1.}
{True,False}
```

Related terms

[Equal](#), [Unequal](#), [UnsameQ](#)

Implementation status

- - full supported

Github

- [Implementation of SameQ](#)
-

SASTriangle

```
SASTriangle(a, gamma, b)
```

returns a triangle from 2 sides `a`, `b` and angle `gamma` (which is between the sides).

See:

- [Wikipedia - Solution of triangles](#)

Examples

```
>> SASTriangle(1, Pi/3, 1)
Triangle({{0,0},{1,0},{1/2,Sqrt(3)/2}})
```

Related terms

[AASTriangle](#), [ASATriangle](#), [SSSTriangle](#)

Implementation status

- - partially implemented

Github

- [Implementation of SASTriangle](#)
-

SatisfiabilityCount

```
SatisfiabilityCount(boolean-expr)
```

test whether the `boolean-expr` is satisfiable by a combination of boolean `False` and `True` values for the variables of the boolean expression and return the number of possible combinations.

```
SatisfiabilityCount(boolean-expr, list-of-variables)
```

test whether the `boolean-expr` is satisfiable by a combination of boolean `False` and `True` values for the `list-of-variables` and return the number of possible combinations.

See

- [Wikipedia - Boolean satisfiability problem](#)

Examples

```
>> SatisfiabilityCount((a || b) && (! a || ! b), {a, b})  
2
```

Implementation status

- - full supported

Github

- [Implementation of SatisfiabilityCount](#)
-

SatisfiabilityInstances

```
SatisfiabilityInstances(boolean-expr, list-of-variables)
```

test whether the `boolean-expr` is satisfiable by a combination of boolean `False` and `True` values for the `list-of-variables` and return exactly one instance of `True`, `False` combinations if possible.

```
SatisfiabilityInstances(boolean-expr, list-of-variables, combinations)
```

test whether the `boolean-expr` is satisfiable by a combination of boolean `False` and `True` values for the `list-of-variables` and return up to `combinations` instances of `True`, `False` combinations if possible.

See

- [Wikipedia - Boolean satisfiability problem](#)

Examples

```
>> SatisfiabilityInstances((a || b) && (! a || ! b), {a, b}, All)
{{False,True},{True,False}}
```

Implementation status

- - full supported

Github

- [Implementation of SatisfiabilityInstances](#)
-

SatisfiableQ

```
SatisfiableQ(boolean-expr, list-of-variables)
```

test whether the `boolean-expr` is satisfiable by a combination of boolean `False` and `True` values for the `list-of-variables`.

See

- [Wikipedia - Boolean satisfiability problem](#)

Examples

```
>> SatisfiableQ((a || b) && (! a || ! b), {a, b})  
True
```

Implementation status

- - full supported

Github

- [Implementation of SatisfiableQ](#)
-

Save

```
Save("path-to-filename", expression)
```

if the file system is enabled, export the `FullDefinition` of the `expression` to the "path-to-filename" file. The saved file can be imported with `Get`.

```
Save("path-to-filename", "Global`*")
```

if the file system is enabled, export the `FullDefinition` of all symbols in the `"Global`*"` context to the "path-to-filename" file.

Examples

Save a definition with dependent symbol definitions into temporary file

```
>> g(x_) := x^3;

>> g(x_,y_) := f(x,y);

>> SetAttributes(f, Listable);

>> f(x_) := g(x^2);

>> temp = FileNameJoin({$TemporaryDirectory, \"savedlist.txt\"});Print(temp);

>> Save(temp, {f,g})

>> ClearAll(f,g)

>> "Attributes(f)

>> {f(2),g(7)}

>> Get(temp)

>> {f(2),g(7)}
{64,343}

>> Attributes(f)
{Listable}
```

Related terms

[BinaryDeserialize](#), [BinarySerialize](#), [ByteArray](#), [ByteArrayQ](#), [Export](#), [Import](#)

Implementation status

- - partially implemented

Github

- [Implementation of Save](#)
-

SawtoothWave

```
SawtoothWave(expr)
```

returns the sawtooth wave value of `expr`.

```
SawtoothWave({minimum, maximum}, expr)
```

returns the sawtooth wave value of `expr` in the range `minimum`, `maximum`

See

- [Wikipedia - Sawtooth wave](#)

Examples

```
>> SawtoothWave(41/42)
41/42

>> SawtoothWave(42/41)
1/41
```

Implementation status

- - full supported

Github

- [Implementation of SawtoothWave](#)
-

ScalingTransform

```
ScalingTransform(v)
```

gives a scaling transform of v . v may be a scalar or a vector.

```
ScalingTransform(v, p)
```

gives a scaling transform of v that is centered at the point p .

See

- [Wikipedia - Transformation matrix](#)

Examples

Implementation status

- - full supported

Github

- [Implementation of ScalingTransform](#)
-

Scan

```
Scan(f, expr)
```

applies `f` to each element of `expr` and returns `Null`.

```
Scan(f, expr, levelspec)
```

applies `f` to each level specified by `levelspec` of `expr`.

Examples

```
>> Scan(Print, {1, 2, 3})
1
2
3

>> Scan(Print, f(g(h(x))), 2)
h(x)
g(h(x))

>> Scan(Print)({1, 2})
1
2

>> Scan(Return, {1, 2})
1
```

Implementation status

- - full supported

Github

- [Implementation of Scan](#)
-

SchurDecomposition

```
SchurDecomposition(matrix)
```

calculate the Schur-decomposition as a list $\{q, t\}$ of a square `matrix`.

See:

- [Wikipedia - Schur decomposition](#)

Examples

Implementation status

- - full supported

Github

- [Implementation of SchurDecomposition](#)
-

Sec

`Sec(z)`

returns the secant of `z`.

See

- [Wikipedia - Trigonometric functions](#)

Examples

```
>> Sec(0)
1

>> Sec(1)
Sec(1)

>> Sec(1.)
1.8508157176809255
```

Implementation status

- - full supported

Github

- [Implementation of Sec](#)
 - [Rule definitions of Sec](#)
-

Sech

Sech(z)

returns the hyperbolic secant of z .

See

- [Wikipedia - Hyperbolic function](#)

Examples

```
>> Sech(0)  
1
```

Implementation status

- - full supported

Github

- [Implementation of Sech](#)
 - [Rule definitions of Sech](#)
-

Select

```
Select({e1, e2, ...}, head)
```

returns a list of the elements `ei` for which `head(ei)` returns `True`.

```
Select({e1, e2, ...}, head, n)
```

returns a list of the first `n` elements `ei` for which `head(ei)` returns `True`.

Examples

Find numbers greater than zero:

```
>> Select({-3, 0, 1, 3, a}, #>0&)  
{1,3}
```

Find the first number that is greater than zero in a list:

```
>> Select({-3, 0, 10, 3, a}, #>0&, 1)  
{10}
```

`select` works on an expression with any head:

```
>> Select(f(a, 2, 3), NumberQ)  
f(2,3)
```

Nonatomic expression expected.

```
>> Select(a, True)  
Select(a,True)
```

Select all `Listable` system function names.

```
>> Select(Names("System`*"), MemberQ(Attributes(#), Listable) &)
```

Related terms

[Cases](#), [Pick](#)

Implementation status

- - full supported

Github

- [Implementation of Select](#)
-

SelectFirst

```
SelectFirst({e1, e2, ...}, f)
```

returns the first of the elements `ei` for which `f(ei)` returns `True`.

```
SelectFirst({e1, e2, ...}, f, default)
```

if `f(ei)` returns `False`, the `default` value will be returned

Examples

Find first number greater than zero:

```
>> SelectFirst({-3, 0, 1, 3, a}, #>0 &)  
1  
  
>> SelectFirst({-3, 0, 1, 3, a}, #>3 &)  
Missing(NotFound)
```

[OEIS - A134860](#):

```
>> With({r = Map(Fibonacci, Range(2, 14))}, Position(#, {1, 0, 1})[[All, 1]] &@ Table(I  
{4, 12, 17, 25, 33, 38, 46, 51, 59, 67, 72, 80, 88, 93, 101, 106, 114, 122, 127, 135, 140, 148, 156, 161, 169, 1
```

Implementation status

- - full supported

Github

- [Implementation of SelectFirst](#)
-

SemanticImport

```
SemanticImport("path-to-filename")
```

if the file system is enabled, import the data from CSV files and do a semantic interpretation of the columns.

Dataset uses:

- [Github - JTablesaw - Java dataframe and visualization library](#)

Examples

```
>> SemanticImport("./data/test.csv")
```

Related terms

[Dataset](#), [SemanticImportString](#)

SemanticImportString

```
SemanticImportString("string-content")
```

import the data from a content string in CSV format and do a semantic interpretation of the columns.

Dataset uses:

- [Github - JTablesaw - Java dataframe and visualization library](#)

Examples

```
>> SemanticImportString("Products,Sales,Market_Share  
a,5500,3  
b,12200,4  
c,60000,33")
```

Related terms

[Dataset](#), [SemanticImport](#)

Sequence

```
Sequence[x1, x2, ...]
```

represents a sequence of arguments to a function.

Examples

`Sequence` is automatically spliced in, except when a function has attribute `SequenceHold` (like assignment functions).

```
>> f(x, Sequence(a, b), y)
f(x,a,b,y)

>> Attributes(Set)
{HoldFirst, Protected, SequenceHold}

>> a = Sequence(b, c);

>> a
Sequence(b,c)
```

Apply `Sequence` to a list to splice in arguments:

```
>> lst = {1, 2, 3};

>> f(Sequence @@ lst)
f(1,2,3)
```

Inside `Hold` or a function with a held argument, `Sequence` is spliced in at the first level of the argument:

```
>> Hold(a, Sequence(b, c), d)
Hold(a,b,c,d)
```

If `Sequence` appears at a deeper level, it is left unevaluated:

```
>> Hold({a, Sequence(b, c), d})
Hold({a, Sequence(b,c), d})
```

Implementation status

- - full supported

Github

- [Implementation of Sequence](#)
-

SequenceHold

SequenceHold

is an attribute specifying that in all arguments of a function the `Sequence` expressions shouldn't be flattened out.

Related terms

[Attributes](#), [ClearAttributes](#), [Constant](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#), [Sequence](#), [SetAttributes](#)

Series

```
Series(expr, {x, x0, n})
```

create a power series of `expr` up to order $(x - x_0)^n$ at the point $x = x_0$

See:

- [Wikipedia - Taylor series](#)
- [Wikipedia - Big O notation](#)
- [Wikipedia - Formal power series](#)

Examples

```
>> Series(f(x), {x, a, 3})  
f(a)+f'(a)*(-a+x)+1/2*f''(a)*(-a+x)^2+1/6*Derivative(3)[f][a]*(-a+x)^3+O(-a+x)^4
```

The [A053614 Numbers that are not the sum of distinct triangular numbers](#). integer sequence

```
>> nn=10; t=Rest(CoefficientList(Series(Product((1+x^(k*(k+1)/2))), {k, nn}), {x, 0, nn*  
{2, 5, 8, 12, 23, 33}
```

Related terms

[ComposeSeries](#), [InverseSeries](#), [SeriesCoefficient](#), [SeriesData](#)

Implementation status

- - partially implemented

Github

- [Implementation of Series](#)
-

SeriesCoefficient

```
SeriesCoefficient(expr, {x, x0, n})
```

get the coefficient of $(x - x_0)^n$ at the point $x = x_0$

See:

- [Wikipedia - Taylor series](#)
- [Wikipedia - Big O notation](#)
- [Wikipedia - Formal power series](#)

Examples

```
>> SeriesCoefficient(Sin(x), {x, f+g, n})  
Piecewise({{Sin(f+g+1/2*n*Pi)/n!, n>=0}}, 0)
```

Related terms

[ComposeSeries](#), [InverseSeries](#), [Series](#), [SeriesData](#)

Implementation status

- - partially implemented

Github

- [Implementation of SeriesCoefficient](#)
 - [Rule definitions of SeriesCoefficient](#)
-

SeriesData

```
SeriesData(x, x0, {coeff0, coeff1, coeff2,...}, nMin, nMax, denominator)
```

internal structure of a power series at the point $x = x_0$ the `coeff_i` are coefficients of the power series.

See:

- [Wikipedia - Taylor series](#)
- [Wikipedia - Big O notation](#)
- [Wikipedia - Formal power series](#)

Examples

```
>> SeriesData(x, 0, {1, 0, -1/6, 0, 1/120, 0, -1/5040, 0, 1/362880}, 1, 11, 2)
Sqrt(x)-x^(3/2)/6+x^(5/2)/120-x^(7/2)/5040+x^(9/2)/362880+O(x)^(11/2)
```

Related terms

[ComposeSeries](#), [InverseSeries](#), [SeriesCoefficient](#), [Series](#)

Implementation status

- - partially implemented

Github

- [Implementation of SeriesData](#)
-

Set

```
Set(expr, value)
```

```
expr = value
```

evaluates `value` and assigns it to `expr`.

```
{s1, s2, s3} = {v1, v2, v3}
```

sets multiple symbols (`s1, s2, ...`) to the corresponding values (`v1, v2, ...`).

Examples

`set` can be used to give a symbol a value:

```
>> a = 3
3

>> a
3
```

You can set multiple values at once using lists:

```
>> {a, b, c} = {10, 2, 3}
{10,2,3}

>> {a, b, {c, {d}}} = {1, 2, {{c1, c2}, {a}}}
{1,2,{{c1,c2},{10}}}

>> d
10
```

`set` evaluates its right-hand side immediately and assigns it to the left-hand side:

```
>> a
1

>> x = a
1

>> a = 2
2
```

```
>> x
1
```

'Set' always returns the right-hand side, which you can again use in an assignment:

```
>> a = b = c = 2
>> a == b == c == 2
True
```

'Set' supports assignments to parts:

```
>> A = {{1, 2}, {3, 4}}
>> A[[1, 2]] = 5
5

>> A
{{1, 5}, {3, 4}}

>> A[;;, 2] = {6, 7}
{6, 7}

>> A
{{1, 6}, {3, 7}}
```

Set a submatrix:

```
>> B = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}
>> B[[1;;2, 2;;-1]] = {{t, u}, {y, z}}
>> B
{{1, t, u}, {4, y, z}, {7, 8, 9}}
```

Related terms

[SetDelayed](#), [TagSet](#), [TagSetDelayed](#)

Implementation status

- - full supported

Github

- [Implementation of Set](#)
-

SetAttributes

```
SetAttributes(symbol, attrib)
```

adds `attrib` to `symbol`'s attributes.

Examples

```
>> SetAttributes(f, Flat)

>> Attributes(f)
{Flat}
```

Multiple attributes can be set at the same time using lists:

```
>> SetAttributes({f, g}, {Flat, Orderless})
>> Attributes(g)
{Flat, Orderless}
```

Related terms

[Attributes](#), [ClearAttributes](#), [Constant](#), [Flat](#), [HoldAll](#), [HoldFirst](#), [HoldRest](#), [Listable](#), [NHoldAll](#), [NHoldFirst](#), [NHoldRest](#), [Orderless](#)

Implementation status

- - full supported

Github

- [Implementation of SetAttributes](#)
-

SetDelayed

```
SetDelayed(expr, value)
```

```
expr := value
```

assigns `value` to `expr`, without evaluating `value`.

Examples

`SetDelayed` is like `Set`, except it has attribute `HoldAll`, thus it does not evaluate the right-hand side immediately, but evaluates it when needed.

```
>> Attributes(SetDelayed)
{HoldAll}

>> a = 1
1

>> x := a
>> x
1
```

Changing the value of `a` affects `x`:

```
>> a = 2
2

>> x
2
```

`Condition (/;)` can be used with `SetDelayed` to make an assignment that only holds if a condition is satisfied:

```
>> f(x_) := p(x) /; x>0
>> f(3)
p(3)

>> f(-3)
f(-3)
```

Related terms

[Set](#), [TagSet](#), [TagSetDelayed](#)

Implementation status

- - full supported

Github

- [Implementation of SetDelayed](#)
-

Share

```
Share(function)
```

replace internally equal common subexpressions in `function` by the same reference to reduce memory consumption and return the number of times where `Share(function)` could replace a common subexpression.

See

- [Wikipedia - Common subexpression elimination](#)

Examples

```
>> people = <|236234 -> <|"name" -> "bob", "age" -> 18, "sex" -> "M"|>, 253456 -> <|"name" -> "sue", "age" -> 25, "sex" -> "F"|>, 323442 -> <|"name" -> "bob", "age" -> 20, "sex" -> "M"|>
<|236234-><|name->bob, age->20, sex->M|>, 253456-><|name->sue, age->25, sex->F|>, 323442-><|name->bob, age->20, sex->M|>>
```

`"age" -> 18` and `"sex" -> "F"` could be replaced once each times by the same reference internally:

```
>> Share(people)
2
```

```
>> people
<|236234-><|name->bob, age->20, sex->M|>, 253456-><|name->sue, age->25, sex->F|>, 323442-><|name->bob, age->20, sex->M|>>
```

Related terms

[Compile](#), [CompilePrint](#), [OptimizeExpression](#)

Implementation status

- - experimental

Github

- [Implementation of Share](#)
-

ShearingTransform

```
ShearingTransform(phi, {1, 0}, {0, 1})
```

gives a horizontal shear by the angle `phi`.

```
ShearingTransform(phi, {0, 1}, {1, 0})
```

gives a vertical shear by the angle `phi`.

```
ShearingTransform(phi, u, v, p)
```

gives a shear centered at the point `p`.

See

- [Wikipedia - Transformation matrix](#)

Examples

Implementation status

- - full supported

Github

- [Implementation of ShearingTransform](#)
-

Sign

`Sign(x)`

gives `-1`, `0` or `1` depending on whether `x` is negative, zero or positive. For complex numbers `Sign` is defined as `x/Abs(x)`, if `x` is nonzero.

See

- [Wikipedia - Sign function](#)

Examples

```
>> Sign(-2.5)
-1
```

Implementation status

- - full supported

Github

- [Implementation of Sign](#)
-

Signature

```
Signature(permutation-list)
```

determine if the `permutation-list` has odd (`-1`) or even (`1`) parity. Returns `0` if two elements in the `permutation-list` are equal.

See

- [Wikipedia - Permutation](#)

Examples

```
>> Signature({1,2,3,4})
1

>> Signature({1,4,3,2})
-1

>> Signature({1,2,3,2})
0
```

Implementation status

- - full supported

Github

- [Implementation of Signature](#)
-

Simplify

```
Simplify(expr)
```

simplifies `expr`

```
Simplify(expr, option1, option2, ...)
```

simplify `expr` with some additional options set

- Assumptions - use assumptions to simplify the expression
- ComplexFunction - use this function to determine the "weight" of an expression.

Examples

```
>> Simplify(1/2*(2*x+2))
x+1

>> Simplify(2*Sin(x)^2 + 2*Cos(x)^2)
2

>> Simplify(x)
x

>> Simplify(f(x))
f(x)

>> Simplify(a*x^2+b*x^2)
(a+b)*x^2
```

Simplify with an assumption:

```
>> Simplify(Sqrt(x^2), Assumptions -> x>0)
x
```

For `Assumptions` you can define the assumption directly as second argument:

```
>> Simplify(Sqrt(x^2), x>0)
x
```

```
>> Simplify(Abs(x), x<0)
Abs(x)
```

With this "complexity function" the `Abs` expression gets a "heavier weight".

```
>> complexity(x_) := 2*Count(x, _Abs, {0, 10}) + LeafCount(x)

>> Simplify(Abs(x), x<0, ComplexityFunction->complexity)
-x
```

Related terms

[FullSimplify](#)

Implementation status

- - partially implemented

Github

- [Implementation of Simplify](#)
-

Sin

```
Sin(expr)
```

returns the sine of `expr` (measured in radians).

`Sin(expr)` will evaluate automatically in the case `expr` is a multiple of `Pi`, `Pi/2`, `Pi/3`, `Pi/4` and `Pi/6`.

See

- [Wikipedia - Sine](#)
- [Fungrim - Sine](#)
- [Wikipedia - Radian](#)
- [Wikipedia - Degree](#)

Examples

```
>> Sin(0)
0

>> Sin(0.5)
0.479425538604203

>> Sin(3*Pi)
0

>> Sin(1.0 + I)
1.2984575814159773+I*0.6349639147847361
```

Implementation status

- - full supported

Github

- [Implementation of Sin](#)
 - [Rule definitions of Sin](#)
-

Sinc

```
Sinc(expr)
```

the sinc function $\sin(\text{expr})/\text{expr}$ for $\text{expr} \neq 0$. $\text{Sinc}(0)$ returns 1.

See

- [Wikipedia - Sinc function](#)
- [Fungrim - Sinc function](#)

Examples

```
>> Sinc(0)  
1
```

Implementation status

- - full supported

Github

- [Implementation of Sinc](#)
 - [Rule definitions of Sinc](#)
-

SingularValueDecomposition

```
SingularValueDecomposition(matrix)
```

calculates the singular value decomposition for the `matrix`.

`SingularValueDecomposition` returns `u`, `s`, `w` such that `matrix = u s v`, `u' u=1`, `v' v=1`, and `s` is diagonal.

See:

- [Wikipedia: Singular value decomposition](#)

Examples

```
>> SingularValueDecomposition({{1.5, 2.0}, {2.5, 3.0}})
{
{{0.5389535334972082, 0.8423354965397538},
 {0.8423354965397537, -0.5389535334972083}},
{{4.635554529660638, 0.0},
 {0.0, 0.10786196059193007}},
{{0.6286775450376476, -0.7776660879615599},
 {0.7776660879615599, 0.6286775450376476}}}
```

Symbolic SVD is not implemented, performing numerically.

```
>> SingularValueDecomposition({{3/2, 2}, {5/2, 3}})
{
{{0.5389535334972082, 0.8423354965397538},
 {0.8423354965397537, -0.5389535334972083}},
{{4.635554529660638, 0.0},
 {0.0, 0.10786196059193007}},
{{0.6286775450376476, -0.7776660879615599},
 {0.7776660879615599, 0.6286775450376476}}}
```

Argument `{1, {2}}` at position 1 is not a non-empty rectangular matrix.

```
>> SingularValueDecomposition({1, {2}})
SingularValueDecomposition({1, {2}})
```

Implementation status

- - full supported

Github

- [Implementation of SingularValueDecomposition](#)
-

Sinh

```
Sinh(z)
```

returns the hyperbolic sine of `z`.

See

- [Wikipedia - Hyperbolic function](#)

Examples

```
>> Sinh(0)  
0
```

Implementation status

- - full supported

Github

- [Implementation of Sinh](#)
 - [Rule definitions of Sinh](#)
-

SinhIntegral

```
SinhIntegral(expr)
```

returns the hyperbolic sine integral of `expr`.

See

- [Wikipedia - Trigonometric integral](#)

Examples

```
>> SinhIntegral(0)  
0
```

Implementation status

- - full supported

Github

- [Implementation of SinhIntegral](#)
-

SinIntegral

```
SinIntegral(expr)
```

returns the sine integral of `expr`.

See

- [Wikipedia - Trigonometric integral](#)

Examples

```
>> SinIntegral(0)  
0
```

Implementation status

- - full supported

Github

- [Implementation of SinIntegral](#)
-

SixJSymbol

```
SixJSymbol({j1,j2,j3},{j4,j5,j6})
```

get the 6-j symbol coefficients.

See:

- [Wikipedia - 6-j symbol](#)

Examples

```
>> SixJSymbol({1, 2, 3}, {1, 2, 3})  
1/105
```

Implementation status

- - partially implemented

Github

- [Implementation of SixJSymbol](#)
-

Skewness

```
Skewness(list)
```

gives Pearson's moment coefficient of skewness for `list` (a measure for estimating the symmetry of a distribution).

See

- [Wikipedia - Skewness](#)

Examples

```
>> Skewness({1.1, 1.2, 1.4, 2.1, 2.4})  
0.40704
```

Implementation status

- - full supported

Github

- [Implementation of Skewness](#)
-

Slot

```
#
```

is a short-hand for `#1`.

```
#n
```

represents the `n`-th argument of a pure function.

```
#0
```

represents the pure function itself.

See:

- [Wikipedia - Pure function](#)

Examples

Unused arguments are simply ignored:

```
>> {#1, #2, #3}&[1, 2, 3, 4, 5]  
{1,2,3}
```

Recursive pure functions can be written using `#0`:

```
>> If(#1<=1, 1, #1 #0(#1-1))& [10]  
3628800  
  
>> # // InputForm  
#1  
  
>> #0 // InputForm  
#0  
  
>> f := # ^ 2 &  
>> f(3)  
9  
  
>> #^3& /@ {1, 2, 3}  
{1,8,27}
```

```
>> #1+#2&[4, 5]  
9
```

Related terms

[Function](#), [SlotSequence](#)

SlotSequence

```
##
```

is the sequence of arguments supplied to a pure function.

```
##n
```

starts with the `n`-th argument.

Examples

```
>> Plus(##)& [1, 2, 3]
5

>> Plus(##2)& [1, 2, 3]
5

>> ## // InputForm
##1
```

Related terms

[Function](#), [Slot](#)

SokalSneathDissimilarity

```
SokalSneathDissimilarity(u, v)
```

returns the Sokal-Sneath dissimilarity between the two boolean 1-D lists `u` and `v`, which is defined as $R / (c_{tt} + R)$ where n is `len(u)`, c_{ij} is the number of occurrences of `u(k)=i` and `v(k)=j` for $k < n$, and $R = 2 * (c_{tf} + c_{ft})$.

Examples

```
>> SokalSneathDissimilarity({1, 0, 1, 1, 0, 1, 1}, {0, 1, 1, 0, 0, 0, 1})  
4/5
```

Implementation status

- - full supported

Github

- [Implementation of SokalSneathDissimilarity](#)
-

Solve

```
Solve(equations, vars)
```

attempts to solve `equations` for the variables `vars`.

```
Solve(equations, vars, domain)
```

attempts to solve `equations` for the variables `vars` in the given `domain`.

Options

- `GenerateConditions` - if `True` (default value) the solutions for multi-valued inverse functions are generated with the `ConditionalExpression` function; if `False` some solutions for multi-valued inverse functions get lost.
- `MaxRoots` the maximum number of roots, which should be returned

Examples

It's important to use the `==` operator to define the equations. If you have unintentionally assigned a value to the variables `x`, `y` with the `=` operator you have to call `clear(x,y)` to clear the definitions for these variables.

```
>> Solve({x^2==4,x+y^2==6}, {x,y})
{{x->2,y->2},{x->2,y->-2},{x->-2,y->2*2^(1/2)},{x->-2,y->(-2)*2^(1/2)}}

>> Solve({2*x + 3*y == 4, 3*x - 4*y <= 5,x - 2*y > -21}, {x, y}, Integers)
{{x->-7,y->6},{x->-4,y->4},{x->-1,y->2}}

>> Solve(Xor(a, b, c, d) && (a || b) && ! (c || d), {a, b, c, d}, Booleans)
{{a->False,b->True,c->False,d->False},{a->True,b->False,c->False,d->False}}

>> Solve(30*p+7*q==1,{p,q}, Integers, MaxRoots->10)
{{p->-31,q->133},{p->-24,q->103},{p->-17,q->73},{p->-10,q->43},{p->-3,q->13},{p->4,q->-6}

>> Solve(a+b+c==100,{a,b,c},Primes, MaxRoots->100)
{{a->2,b->19,c->79},{a->2,b->31,c->67},{a->2,b->37,c->61},{a->2,b->61,c->37},{a->2,b->6
```

The solutions for multi-valued inverse functions are generated with the `ConditionalExpression` function.

```
>> Solve(Sin(x^2)==0,x)
{{x->ConditionalExpression(Sqrt(2*Pi)*Sqrt(C(1)),C(1)Integers)},{x->ConditionalExpressio
```

```
>> Solve(Sin(x^2)==0,x,GenerateConditions->False)
{{x->0}}
```

Related terms

[DSolve](#), [Eliminate](#), [GroebnerBasis](#), [FindInstance](#), [FindRoot](#), [NRoots](#), [NSolve](#), [Reduce](#), [Roots](#)

Implementation status

- - partially implemented

Github

- [Implementation of Solve](#)
-

Sort

```
Sort(list)
```

sorts `list` (or the leaves of any other expression) according to canonical ordering.

```
Sort(list, p)
```

sorts using `p` to determine the order of two elements.

Examples

```
>> Sort({4, 1.0, a, 3+I})  
{1.0, 4, 3+I, a}
```

Related terms

[NumericalOrder](#), [NumericalSort](#), [Order](#), [ReversedSort](#)

Implementation status

- - full supported

Github

- [Implementation of Sort](#)
-

SortBy

```
SortBy(list, f)
```

sorts `list` (or the elements of any other expression) according to canonical ordering of the keys that are extracted from the `list`'s elements using `f`. Chunks of leaves that appear the same under `f` are sorted according to their natural order (without applying `f`).

```
Sort(f)
```

creates an operator function that, when applied, sorts by `f`.

Examples

```
>> SortBy({{5, 1}, {10, -1}}, Last)
{{10, -1}, {5, 1}}

>> SortBy(Total)[{{5, 1}, {10, -9}}]
{{10, -9}, {5, 1}}
```

Implementation status

- - full supported

Github

- [Implementation of SortBy](#)
-

Sow

```
Sow(expr)
```

sends the value `expr` to the innermost `Reap`.

Examples

```
>> Reap(Sow(3); Sow(1))  
{1, {{3, 1}}}
```

[OEIS - A020652](#):

```
>> Reap(Do(If(GCD(num, den) == 1, Sow(num)), {den, 1, 20}, {num, 1, den-1}))[[2, 1]]  
{1, 1, 2, 1, 3, 1, 2, 3, 4, 1, 5, 1, 2, 3, 4, 5, 6, 1, 3, 5, 7, 1, 2, 4, 5, 7, 8, 1, 3, 7, 9, 1, 2, 3, 4, 5, 6, 7, 8, 9,  
10, 1, 5, 7, 11, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 3, 5, 9, 11, 13, 1, 2, 4, 7, 8, 11, 13, 14, 1, 3, 5, 7,  
9, 11, 13, 15, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1, 5, 7, 11, 13, 17, 1, 2, 3, 4, 5, 6, 7, 8,  
9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 1, 3, 7, 9, 11, 13, 17, 19}
```

Related terms

[Reap](#)

Implementation status

- - full supported

Github

- [Implementation of Sow](#)
-

Span

Span

is the head of span ranges like `1;;3`.

Examples

```
>> ;; // FullForm
Span(1, All)

>> 1;;4;;2 // FullForm
Span(1, 4, 2)

>> 2;;-2 // FullForm
Span(2, -2)

>> ;;3 // FullForm
Span(1, 3)
```

SparseArray

```
SparseArray(nestedList)
```

create a sparse array from a `nestedList` structure.

```
SparseArray(arrayRules, listOfIntegers, defaultValue)
```

create a sparse array from `arrayRules` with dimension `listOfIntegers` and undefined elements are having `defaultValue`.

```
SparseArray(Automatic, listOfIntegers, defaultValue, crsList)
```

create a sparse array from the compressed-row-storage `crsList` with dimension `listOfIntegers` and undefined elements are having `defaultValue`.

See

- [Netlib - Compressed Row Storage \(CRS\)](#)

Examples

```
>> SparseArray({{1, 1} -> 1, {2, 2} -> 2, {3, 3} -> 3, {1, 3} -> 4})
SparseArray(Number of elements: 4 Dimensions: {3,3} Default value: 0)

>> SparseArray({{1, 1} -> 1, {2, 2} -> 2, {3, 3} -> 3, {1, 3} -> 4}, Automatic, 0)
SparseArray(Number of elements: 4 Dimensions: {3,3} Default value: 0)
```

Related terms

[SparseArrayQ](#), [MatrixForm](#), [Normal](#)

Implementation status

- - partially implemented

Github

- [Implementation of SparseArray](#)
-

SparseArrayQ

```
SparseArrayQ(expr)
```

return `True` if `expr` is a sparse array.

Examples

```
>> s = SparseArray({{1, 1} -> 1, {2, 3} -> 4, {3, 1} -> -1});  
  
>> SparseArrayQ(s)  
True
```

Related terms

[SparseArray](#), [MatrixForm.md](#), [Normal](#)

Implementation status

- - full supported

Github

- [Implementation of SparseArrayQ](#)
-

SpecialsFreeQ

```
SpecialsFreeQ(expr)
```

returns `True` if `expr` does not contain the symbols `DirectedInfinity` or `Indeterminate`.

Examples

```
>> SpecialsFreeQ(-Infinity)
False

>> SpecialsFreeQ(1+x^2)
True
```

Implementation status

- - full supported

Github

- [Implementation of SpecialsFreeQ](#)
-

Sphere

```
Sphere({x, y, z})
```

is a sphere of radius `1` centered at the point `{x, y, z}`.

```
Sphere({x, y, z}, r)
```

is a sphere of radius `r` centered at the point `{x, y, z}`.

```
Sphere({{x1, y1, z1}, {x2, y2, z2}, ... }, r)
```

is a collection of spheres of radius `r` centered at the points `{x1, y2, z2}, {x2, y2, z2}, ... }`

Examples

```
>> Graphics3D(Sphere({0, 0, 0}, 1))  
-Graphics3D-  
  
>> Graphics3D({Yellow, Sphere({{-1, 0, 0}, {1, 0, 0}, {0, 0, Sqrt(3.)}}, 1)})  
-Graphics3D-
```

Implementation status

- - full supported

Github

- [Implementation of Sphere](#)
-

SphericalBesselJ

```
SphericalBesselJ(n, z)
```

spherical Bessel function $J(n, x)$.

See

- [Wikipedia - Bessel function - Spherical Bessel function](#)

Examples

```
>> SphericalBesselJ(2.5, -5)  
I*0.204488
```

Implementation status

- - experimental

Github

- [Implementation of SphericalBesselJ](#)
 - [Rule definitions of SphericalBesselJ](#)
-

SphericalBessely

SphericalBessely(n, z)

spherical Bessel function $y(n, x)$.

See

- [Wikipedia - Bessel function - Spherical Bessel function](#)

Examples

Implementation status

- - experimental

Github

- [Implementation of SphericalBessely](#)
 - [Rule definitions of SphericalBessely](#)
-

SphericalHarmonicY

```
SphericalHarmonicY(l, m, theta, phi)
```

returns the spherical harmonic function $Y_l^m(\theta, \phi)$.

See

- [Wikipedia - Spherical harmonics](#)

Examples

```
>> SphericalHarmonicY(3/4, 0.5, Pi/5, Pi/3)
0.254247+I*0.14679
```

Implementation status

- - full supported

Github

- [Implementation of SphericalHarmonicY](#)
 - [Rule definitions of SphericalHarmonicY](#)
-

Splice

```
Splice(list-of-elements)
```

the `list-of-elements` will automatically be converted into a `Sequence` of elements.

```
Splice(list-of-elements, head-pattern)
```

the `list-of-elements` will automatically be converted into a `Sequence` of elements, if the calling expression matches the `head-pattern`.

Examples

```
>> h(a, b, c, Splice({{1,1},{2,2}}, h), d, e)
h(a,b,c,{1,1},{2,2},d,e)
```

Implementation status

- - partially implemented

Github

- [Implementation of Splice](#)
-

Split

```
Split(list)
```

splits `list` into collections of consecutive identical elements.

```
Split(list, test)
```

splits `list` based on whether the function `test` yields 'True' on consecutive elements.

Examples

```
>> Split({x, x, x, y, x, y, y, z})
{{x,x,x},{y},{x},{y,y},{z}}

>> Split({x, x, x, y, x, y, y, z}, x)
{{x},{x},{x},{y},{x},{y},{y},{z}}
```

Split into increasing or decreasing runs of elements

```
>> Split({1, 5, 6, 3, 6, 1, 6, 3, 4, 5, 4}, Less)
{{1,5,6},{3,6},{1,6},{3,4,5},{4}}

>> Split({1, 5, 6, 3, 6, 1, 6, 3, 4, 5, 4}, Greater)
{{1},{5},{6,3},{6,1},{6,3},{4},{5,4}}
```

Split based on first element

```
>> Split({x -> a, x -> y, 2 -> a, z -> c, z -> a}, First(#1) === First(#2) &)
{{x->a,x->y},{2->a},{z->c,z->a}}

>> Split({})
{}
```

Implementation status

- - full supported

Github

- [Implementation of Split](#)
-

SplitBy

```
SplitBy(list, f)
```

splits `list` into collections of consecutive elements that give the same result when `f` is applied.

Examples

```
>> SplitBy(Range(1, 3, 1/3), Round)
{{1, 4/3}, {5/3, 2, 7/3}, {8/3, 3}}
{{1, 4 / 3}, {5 / 3, 2, 7 / 3}, {8 / 3, 3}}
```

```
>> SplitBy({1, 2, 1, 1.2}, {Round, Identity})
{{{1}}, {{2}}, {{1}, {1.2}}}
```

```
>> SplitBy(Tuples({1, 2}, 3), First)
{{{1, 1, 1}, {1, 1, 2}, {1, 2, 1}, {1, 2, 2}}, {{2, 1, 1}, {2, 1, 2}, {2, 2, 1}, {2, 2, 2}}}
```

Implementation status

- - full supported

Github

- [Implementation of SplitBy](#)
-

Sqrt

```
Sqrt(expr)
```

returns the square root of `expr`.

See

- [Wikipedia - Square root](#)
- [Fungrim - Square roots](#)

Examples

```
>> Sqrt(4)
2

>> Sqrt(5)
Sqrt(5)

>> Sqrt(5) // N
2.23606797749979

>> Sqrt(a)^2
a
```

Complex numbers:

```
>> Sqrt(-4)
I*2

>> I == Sqrt(-1)
True

>> N(Sqrt(2), 50)
1.41421356237309504880168872420969807856967187537694
```

Implementation status

- - full supported

Github

- [Implementation of Sqrt](#)
-

SquaredEuclideanDistance

```
SquaredEuclideanDistance(u, v)
```

returns squared the euclidean distance between `u` and `v`.

See

- [Wikipedia - Euclidean distance - Squared Euclidean distance](#)

Examples

```
>> SquaredEuclideanDistance({-1, -1}, {1, 1})  
8
```

Related terms

[FindClusters](#), [BinaryDistance](#), [BrayCurtisDistance](#), [ChessboardDistance](#), [CanberraDistance](#), [CosineDistance](#), [EuclideanDistance](#), [ManhattanDistance](#)

Implementation status

- - full supported

Github

- [Implementation of SquaredEuclideanDistance](#)
-

SquareFreeQ

SquareFreeQ(n)

returns `True` if `n` is a square free integer number or a square free univariate polynomial.

See

- [Wikipedia - Square-free polynomial](#)

Examples

```
>> SquareFreeQ(9)
False

>> SquareFreeQ(105)
True
```

Implementation status

- - full supported

Github

- [Implementation of SquareFreeQ](#)
-

SquareMatrixQ

```
SquareMatrixQ(m)
```

returns `True` if `m` is a square matrix.

See

- [Wikipedia - Square matrix](#)
- [Wikipedia - Matrix \(mathematics\)](#)

Examples

```
>> SquareMatrixQ({{1, 3 + 4*I}, {3 - 4*I, 2}})
True

>> SquareMatrixQ({{}})
False
```

Implementation status

- - full supported

Github

- [Implementation of SquareMatrixQ](#)
-

SquaresR

```
SquaresR(k, intNumber)
```

counts the numbers of the representation of `intNumber` as sum of x^2 terms which occur `k` times.

See

- [Wikipedia - Sum of squares function](#)
- [Wikipedia - Sum of two squares function](#)

Examples

```
>> SquaresR(8, 30)
395136
```

Implementation status

- - partially implemented

Github

- [Implementation of SquaresR](#)
-

SSSTriangle

```
SSSTriangle(a, b, c)
```

returns a triangle from 3 sides `a`, `b` and `c`.

See:

- [Wikipedia - Solution of triangles](#)

Examples

```
>> SSSTriangle(10,10,10)
Triangle({{0,0},{10,0},{5,5*Sqrt(3)}})
```

Related terms

[AASTriangle](#), [ASATriangle](#), [SASTriangle](#)

Implementation status

- - partially implemented

Github

- [Implementation of SSSTriangle](#)
-

Stack

```
Stack( )
```

return a list of the heads of the current stack wrapped by `HoldForm`.

```
Stack(_)
```

return a list of the expressions of the current stack wrapped by `HoldForm`.

Examples

Related terms

[StackBegin](#), [Trace](#)

Implementation status

- - full supported

Github

- [Implementation of Stack](#)
-

StackBegin

```
Stack(expr)
```

begin a new stack and evaluate `expr`. Use `Stack(_)` as a subexpression in `expr` to return the stack elements.

Examples

Related terms

[Stack](#), [Trace](#)

Implementation status

- - full supported

Github

- [Implementation of StackBegin](#)
-

StandardDeviation

```
StandardDeviation(list)
```

computes the standard deviation of `list`. `list` may consist of numerical values or symbols. Numerical values may be real or complex.

`StandardDeviation({{a1, a2, ...}, {b1, b2, ...}, ...})` will yield `{StandardDeviation({a1, b1, ...}, StandardDeviation({a2, b2, ...}), ...}`.

`StandardDeviation` can be applied to the following distributions:

[BernoulliDistribution](#), [BinomialDistribution](#), [DiscreteUniformDistribution](#),
[ErlangDistribution](#), [ExponentialDistribution](#), [FrechetDistribution](#), [GammaDistribution](#),
[GeometricDistribution](#), [GumbelDistribution](#), [HypergeometricDistribution](#),
[LogNormalDistribution](#), [NakagamiDistribution](#), [NormalDistribution](#),
[PoissonDistribution](#), [StudentTDistribution](#), [WeibullDistribution](#)

See

- [Wikipedia - Standard deviation](#)

Examples

```
>> StandardDeviation({1, 2, 3})
1

>> StandardDeviation({7, -5, 101, 100})
Sqrt(13297)/2

>> StandardDeviation({a, a})
0

>> StandardDeviation({{1, 10}, {-1, 20}})
{Sqrt(2), 5*Sqrt(2)}

>> StandardDeviation(LogNormalDistribution(0, 1))
Sqrt((-1+E)*E)
```

Implementation status

- - full supported

Github

- [Implementation of StandardDeviation](#)
 - [Rule definitions of StandardDeviation](#)
-

Standardize

```
Standardize(list-of-values)
```

shifts the `list-of-values` by `Mean(list-of-values)` and scales by `StandardDeviation(list-of-values)`

```
Standardize(list-of-values, function1, function2)
```

shifts the `list-of-values` by `function1(list-of-values)` and scales by `function2(list-of-values)`

See

- [Wikipedia - Standard score](#)

Examples

```
>> Standardize(N(Range(5)), Mean, 1 &)  
{-2.0, -1.0, 0.0, 1.0, 2.0}  
  
>> Standardize(N(Range(5)), Median, InterquartileRange)  
{-0.8, -0.4, 0.0, 0.4, 0.8}
```

Related terms

[PrincipleComponents](#)

Implementation status

- - full supported

Github

- [Implementation of Standardize](#)
-

StarGraph

```
StarGraph(order)
```

create a new star graph with `order` number of total vertices including the center vertex.

See

- [Wikipedia - Star \(graph theory\)](#)

Examples

```
>> StarGraph(4) // AdjacencyMatrix // Normal
{{0,1,1,1},
 {1,0,0,0},
 {1,0,0,0},
 {1,0,0,0}}

>> StarGraph(6)
Graph({1,2,3,4,5,6},{2<->1,3<->1,4<->1,5<->1,6<->1})
```

Implementation status

- - partially implemented

Github

- [Implementation of StarGraph](#)
-

StartOfLine

StartOfLine

begin a new stack and evaluate `expr`. Use `stack(_)` as a subexpression in `expr` to return the stack elements.

Examples

```
>> StringSplit("abc\ndef\nhij",StartOfLine)
{abc,def,hij}
```

StartOfString

StartOfString

represents the start of a string.

Examples

Test whether strings start with "a":

```
>> StringMatchQ(#[, StartOfString ~~"a" ~~__)&/@ {"apple", "banana", "artichoke"}  
{True,False,True}
```

StieltjesGamma

```
StieltjesGamma(a)
```

returns Stieltjes constant.

See

- [Wikipedia - Stieltjes constants](#)
- [Fungrim - Stieltjes constants](#)

Examples

```
>> StieltjesGamma(0)
EulerGamma

>> StieltjesGamma(0, a)
-PolyGamma(0, a)
```

Implementation status

- - experimental

Github

- [Implementation of StieltjesGamma](#)
 - [Rule definitions of StieltjesGamma](#)
-

StirlingS1

```
StirlingS1(n, k)
```

returns the Stirling numbers of the first kind.

See

- [Wikipedia - Stirling numbers of the first kind](#)
- [Fungrim - Stirling numbers](#)

Examples

```
>> StirlingS1(9, 6)  
-4536
```

Implementation status

- - partially implemented

Github

- [Implementation of StirlingS1](#)
-

StirlingS2

```
StirlingS2(n, k)
```

returns the Stirling numbers of the second kind. `StirlingS2(n, k)` is the number of ways of partitioning an `n`-element set into `k` non-empty subsets.

See

- [Wikipedia - Stirling numbers of the second kind](#)
- [Fungrim - Stirling numbers](#)

Examples

```
>> StirlingS2(10, 6)
22827
```

Implementation status

- - partially implemented

Github

- [Implementation of StirlingS2](#)
-

String

```
String
```

is the head of strings..

Examples

```
>> Head("abc")  
String  
  
>> "abc"  
"abc"
```

Use `InputForm` to display quotes around strings:

```
>> InputForm("abc")  
"abc"
```

`FullForm` also displays quotes:

```
>> FullForm("abc" + 2)  
Plus(2, "abc")
```

StringCases

```
StringCases(string, pattern)
```

gives all occurrences of `pattern` in `string`.

Examples

```
>> StringCases("12341235678", "123" | "78")
{123,123,78}

>> StringCases("a#ä_123", WordCharacter)
{a,ä,1,2,3}

>> StringCases("a#ä_123", LetterCharacter)
{a,ä}
```

Related terms

[StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringCases](#)
-

StringContainsQ

```
StringContainsQ(str1, str2)
```

return a list of matches for "p1", "p2", ... list of strings in the string `str`.

Examples

```
>> StringContainsQ({"the quick brown fox", "jumps", "over the lazy dog"}, "the")  
{True, False, True}
```

Related terms

[StringCases](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringContainsQ](#)
-

StringCount

```
StringCount(string, pattern)
```

counts all occurrences of `pattern` in `string`.

Examples

```
>> StringCount("a#ä_123", WordCharacter)
5

>> StringCount("a#ä_123", LetterCharacter)
2
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringCount](#)
-

StringExpression

```
StringExpression(s_1, s_2, ...)
```

or

```
s_1 ~~ s_2 ~~ ...
```

represents a sequence of strings and symbolic string objects `s_i`.

Examples

```
>> "a" ~~ "b" ~~ "c" // FullForm  
"abc"
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringExpression](#)
-

StringFreeQ

```
StringFreeQ("string", patt)
```

returns `True` if no substring in `string` matches the string expression `patt`, and returns `False` otherwise.

```
StringFreeQ({"s1", "s2", ...}, patt)
```

returns the list of results for each element of the string list.

```
StringFreeQ("string", {p1, p2, ...})
```

returns `True` if no substring matches any of the `pi`.

```
StringFreeQ(patt)
```

represents an operator form of `StringFreeQ` that can be applied to an expression.

See

- [Wikipedia - Regular expression](#)

Examples

```
>> StringFreeQ("symja", "s" ~~__ ~~"a")
False

>> StringFreeQ("symja", "y" ~~__~~"s")
True

>> StringFreeQ("Symja", "SY" ,IgnoreCase -> True)", //
False

>> StringFreeQ({"g", "a", "laxy", "universe", "sun"}, "u")
{True,True,True,False,False}

>> StringFreeQ("e" ~~__ ~~"u") /@ {"The Sun", "Mercury", "Venus", "Earth", "Mars", "Ju
{False,False,False,True,True,True,True,True,False}

>> StringFreeQ({"A", "Galaxy", "Far", "Far", "Away"}, {"F" ~~__ ~~"r", "aw" ~~__}, Ign
{True,True,False,False,False}
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringInsert](#), [StringJoin](#),
[StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#),
[StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringFreeQ](#)
-

StringInsert

```
StringInsert(string, new-string, position)
```

returns a string with `new-string` inserted starting at `position` in `string`.

```
StringInsert(string, new-string, -position)
```

returns a string with `new-string` inserted starting at `position` from the end of `string`.

```
StringInsert(string, new-string, {pos1, pos2,...})
```

returns a string with `new-string` inserted at each position `posN` in `string`.

```
StringInsert({str1, str2,...}, new-string, position)
```

gives the list of results for each of the strings `strN`

Examples

```
>> StringInsert({"", "Symja"}, "X", {1, 1, -1})  
{XXX,XXSymjaX}
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringInsert](#)
-

StringJoin

```
StringJoin(str1, str2, ... strN)
```

or

```
str1 <> str2 <> ... <> strN
```

returns the concatenation of the strings `str1, str2, ... strN`.

Examples

```
>> "Java" <> ToString(8)
Java8

>> StringJoin("Java", ToString(8))
Java8

>> StringJoin({"a", "b"})// InputForm
"ab"
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringJoin](#)
-

StringLength

```
StringLength(string)
```

gives the length of `string`.

Examples

```
>> StringLength("symja")
5

>> StringLength[{"a", "bc"}]
{1, 2}
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringLength](#)
-

StringMatchQ

```
StringMatchQ(string, regex-pattern)
```

check if the regular expression `regex-pattern` matches the `string`.

See

- [Wikipedia - Regular expression](#)

Examples

```
>> StringMatchQ({"ExpandAll", "listable", "test"}, RegularExpression("li(.+?)le"))
{False,True,False}

>> StringMatchQ("15a94xcZ6", (DigitCharacter | LetterCharacter)..)
True
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringMatchQ](#)
-

StringPart

```
StringPart(str, pos)
```

return the character at position `pos` from the `str` string expression.

```
StringPart(str, {pos1, pos2, pos3, ...})
```

return the characters at the position in the list `{pos1, pos2, pos3, ...}` from the `str` string expression.

Examples

```
>> StringPart("1234567890", {1, 3, 10})  
{1, 3, 0}
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringPart](#)
-

StringPosition

```
StringPosition("string", patt)
```

gives a list of starting and ending positions where `patt` matches `"string"`.

```
StringPosition("string", patt, n)
```

returns the first `n` matches only.

```
StringPosition("string", {patt1, patt2,...}, n)
```

matches multiple patterns.

```
StringPosition({s1, s2, ...}, patt)
```

returns a list of matches for multiple strings.

See

- [Wikipedia - Regular expression](#)

Examples

```
>> StringPosition("123ABCxyABCzzzABCABC", "ABC")
{{4,6},{9,11},{15,17},{18,20}}

>> StringPosition("123ABCxyABCzzzABCABC", "ABC", 2)
{{4,6},{9,11}}
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringPosition](#)
-

StringQ

```
StringQ(x)
```

is `True` if `x` is a string object, or `False` otherwise.

Examples

```
>> StringQ(a)
False

>> StringQ("a")
True
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Github

- [Implementation of StringQ](#)
-

StringReplace

```
StringReplace(string, fromStr -> toStr)
```

replaces each occurrence of `fromStr` with `toStr` in `string`.

Examples

```
>> StringReplace("the quick brown fox jumps over the lazy dog", "the" -> "a")
a quick brown fox jumps over a lazy dog

>> StringReplace("01101100010", "01" .. -> "x")
x1x100x0
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringReplace](#)
-

StringReverse

```
StringReplace(string)
```

reverse the `string`.

See

- [Wikipedia - Palindrome](#)

Examples

A famous palindrome:

```
>> StringReverse("Never odd or even")
neve ro ddo reveN
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringReverse](#)
-

StringRiffle

```
StringRiffle({s1, s2, s3, ...})
```

returns a new string by concatenating all the `si`, with spaces inserted between them.

```
StringRiffle(list, "sep")
```

inserts the separator `sep` between all elements in list.

```
StringRiffle(list, {"left", "sep", "right"})
```

use `left` and `right` as delimiters after concatenation.

Examples

```
>> StringRiffle({"a", "b", "c", "d", "e"})
a b c d e

>> StringRiffle({"a", "b", "c", "d", "e"}, ", ")
a, b, c, d, e

>> StringRiffle({"a", "b", "c", "d", "e"}, {"(", " ", ")"})
(a b c d e)
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringRiffle](#)
-

StringSplit

```
StringSplit(str)
```

split the string `str` by whitespaces into a list of strings.

```
StringSplit(str1, str2)
```

split the string `str1` by `str2` into a list of strings.

```
StringSplit(str1, RegularExpression(str2))
```

split the string `str1` by the regular expression `str2` into a list of strings.

See

- [Wikipedia - Regular expression](#)

Examples

```
>> StringSplit("128.0.0.1", ".")
{128,0,0,1}

>> StringSplit("128.0.0.1", RegularExpression("\\\\W+"))
{128,0,0,1}

>> StringSplit("a1b2.2c0.333d4444.0efghijklm", NumberString)
{a,b,c,d,efghijklm}
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringTake](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringSplit](#)
-

StringTake

```
StringTake("string", n)
```

gives the first `n` characters in `string`.

```
StringTake("string", -n)
```

gives the last `n` characters in `string`.

```
StringTake("string", {n})
```

gives the `n`th character in `string`.

```
StringTake("string", {m, n})
```

gives characters `m` through `n` in `string`.

```
StringTake("string", {m, n, s})
```

gives characters `m` through `n` in steps of `s`.

Examples

```
>> StringTake("abcde", 2)
ab

>> StringTake("abcde", 0)

>> StringTake("abcde", -2)
de

>> StringTake("abcde", {2})
b

>> StringTake("abcd", {2,3})
bc

>> StringTake("abcdefgh", {1, 5, 2})
ace

>> StringTake("abcdef", All)
abcdef
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringToByteArray](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringTake](#)
-

StringTemplate

```
StringTemplate(string)
```

gives a `StringTemplate` expression with name `string`.

Examples

```
>> StringTemplate("The quick brown `a` jumps over the lazy `b`.") [<|"a" -> "fox", "b" -  
The quick brown fox jumps over the lazy dog.  
  
>> TemplateApply(StringTemplate("The quick brown `a` jumps over the lazy `b`."), <|"a" -  
The quick brown fox jumps over the lazy dog.
```

Related terms

[TemplateApply](#), [TemplateIf](#), [TemplateSlot](#)

Implementation status

- - full supported

Github

- [Implementation of StringTemplate](#)
-

StringToByteArray

```
StringToByteArray(string)
```

encodes the `string` into a sequence of bytes using the default character set `UTF-8`, storing the result into a `ByteArray`.

See:

- [Wikipedia - Base64](#)

Examples

The example from Wikipedia:

```
>> ba1 = BaseEncode(StringToByteArray("Man is distinguished, not only by his reason, but by this singular passion from other a
TWFuIGlzIGRpc3Rpbmd1aXNoZWQsIG5vdCBvbmx5IGJ5IGhpcyByZWZb24sIGJ1dCBieSB0aGlzIHNpbmd1bGF
>> ba2 = BaseDecode(ba1)
ByteArray[269 Bytes]
>> ByteArrayToString(ba2)
Man is distinguished, not only by his reason, but by this singular passion from other a
```

Related terms

[BaseDecode](#), [BaseEncode](#), [ByteArrayToString](#), [StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringTrim](#)

Implementation status

- - full supported

Github

- [Implementation of StringToByteArray](#)
-

StringToStream

```
StringToStream("string")
```

converts a `string` to an open input stream.

Examples

```
>> str = StringToStream("4711 dummy 0815")
InputStream[Name: String Unique-ID: 1]

>> Read(str, Number)
4711

>> Read(str, {Word, Number})
{dummy, 0815}

>> Close(str)
String
```

Implementation status

- - partially implemented

Github

- [Implementation of StringToStream](#)
-

StringTrim

```
StringTrim(s)
```

returns a version of `s` with whitespace removed from start and end.

```
StringTrim(s, patt)
```

returns a version of `s` with strings matching `patt` removed from start and end.

Examples

```
>> StringJoin("a", StringTrim(" \\tb\\n "), "c")
abc

>> StringTrim("ababaxababyaabab", RegularExpression("(ab)+"))
axababya
```

Related terms

[StringCases](#), [StringContainsQ](#), [StringCount](#), [StringExpression](#), [StringFreeQ](#), [StringInsert](#), [StringJoin](#), [StringLength](#), [StringMatchQ](#), [StringPart](#), [StringPosition](#), [StringQ](#), [StringReplace](#), [StringRiffle](#), [StringSplit](#), [StringTake](#), [StringToByteArray](#)

Implementation status

- - full supported

Github

- [Implementation of StringTrim](#)
-

StruveH

```
StruveH(n, z)
```

returns the Struve function $H_n(z)$.

See

- [Wikipedia - Struve function](#)

Examples

```
>> StruveH(1.5, 3.5)  
1.1319901169676305
```

Implementation status

- - partially implemented

Github

- [Implementation of StruveH](#)
 - [Rule definitions of StruveH](#)
-

StruveL

```
StruveL(n, z)
```

returns the modified Struve function $L_n(z)$.

See

- [Wikipedia - Struve function](#)

Examples

```
>> StruveL(1.5, 3.5)
4.414171898670692
```

Implementation status

- - partially implemented

Github

- [Implementation of StruveL](#)
 - [Rule definitions of StruveL](#)
-

StudentTDistribution

```
StudentTDistribution(v)
```

returns a Student's t-distribution.

See

- [Wikipedia - Student's t-distribution](#)

Examples

The variance of Student's t-distribution is

```
>> Variance(StudentTDistribution(n))  
Piecewise({{n/(-2+n), n>2}}, Indeterminate)  
  
>> Variance(StudentTDistribution(4))  
2
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of StudentTDistribution](#)
-

Subdivide

`Subdivide(n)`

returns a list with $n+1$ entries obtained by subdividing the range 0 to 1.

`Subdivide(to, n)`

returns a list with $n+1$ entries obtained by subdividing the range 0 to to.

`Subdivide(from, to, n)`

returns a list with $n+1$ entries obtained by subdividing the range from to to.

Examples

```
>> Subdivide(5)
{0, 1/5, 2/5, 3/5, 4/5, 1}

>> Subdivide(10, 4)
{0, 5/2, 5, 15/2, 10}

>> Subdivide(-1, -4, 3)
{-1, -2, -3, -4}

>> Subdivide({10,5}, {5,15}, 4)
{{10,5}, {35/4, 15/2}, {15/2, 10}, {25/4, 25/2}, {5,15}}

>> N@Subdivide(-5, 5, 6)
{-5.0, -3.33333, -1.66667, 0.0, 1.66667, 3.33333, 5.0}
```

Implementation status

- - full supported

Github

- [Implementation of Subdivide](#)
-

Subfactorial

```
Subfactorial(n)
```

returns the subfactorial number of the integer `n`

See

- [Wikipedia - Derangement](#)

Examples

```
>> Subfactorial(12)
176214841
```

Implementation status

- - partially implemented

Github

- [Implementation of Subfactorial](#)
-

Subresultants

```
Subresultants(polynomial1, polynomial2, var)
```

computes the subresultants of the polynomials `polynomial1` and `polynomial2` with respect to the variable `var`.

```
Subresultants(polynomial1, polynomial2, var, Modulus->prime)
```

computes the subresultants modulo a `prime` number.

See

- [Wikipedia - Polynomial greatest common divisor - Subresultants](#)
- [Wikipedia - Resultant](#)

Examples

```
>> Subresultants((x-y)^2-2, y^3-5, y)[[1]]  
17-60*x+12*x^2-10*x^3-6*x^4+x^6
```

SubsetCases

```
SubsetCases(list, sublist -> rhs)
```

or

```
SubsetCases(list, sublist :> rhs)
```

returns a list of the right-hand-side `rhs` for the matching sublist pattern expression `sublist` in `list`.

```
SubsetCases(list, sublist)
```

returns a list of the matching sublist pattern expressions `sublist` in `list`.

Note: this function doesn't support pattern sequences at the moment.

Examples

```
>> SubsetCases({a,b,c,d},{x_,y_} :> f(x,y))
{f(a,b),f(c,d)}

>> SubsetCases({a,b,c,d},{x_,y_})
{{a,b},{c,d}}
```

Related terms

[SubsetReplace](#), [SubsetQ](#)

Implementation status

- - experimental

Github

- [Implementation of SubsetCases](#)
-

SubsetQ

```
SubsetQ(set1, set2)
```

returns `True` if `set2` is a subset of `set1`.

See:

- [Wikipedia - Subset](#)

Examples

```
>> SubsetQ({a, b, c}, {a, b})  
True
```

Related terms

[SubsetCases.md](#), [SubsetReplace](#)

Implementation status

- - full supported

Github

- [Implementation of SubsetQ](#)
-

SubsetReplace

```
SubsetReplace(list, sublist -> rhs)
```

or

```
SubsetReplace(list, sublist := rhs)
```

replaces the sublist pattern expression `sublist` in `list` with the right-hand-side `rhs`.

Note: this function doesn't support pattern sequences at the moment.

Examples

```
>> SubsetReplace({a,b,c,d},{x_,y_} := f(x,y))  
{f(a,b),f(c,d)}
```

Related terms

[SubsetCases](#), [SubsetQ](#)

Implementation status

- - experimental

Github

- [Implementation of SubsetReplace](#)
-

Subsets

```
Subsets(list)
```

finds a list of all possible subsets of `list`.

```
Subsets(list, n)
```

finds a list of all possible subsets containing at most `n` elements.

```
Subsets(list, {n})
```

finds a list of all possible subsets containing exactly `n` elements.

See:

- [Wikipedia - Combination](#)

Examples

```
>> Subsets({a, b, c})
{{}, {a}, {b}, {c}, {a, b}, {a, c}, {b, c}, {a, b, c}}

>> Subsets({a, b, c}, 2)
{{}, {a}, {b}, {c}, {a, b}, {a, c}, {b, c}}

>> Subsets({a, b, c}, {2})
{{a, b}, {a, c}, {b, c}}

>> Subsets({})
{{}}

>> Subsets()
Subsets()
```

The [A018900 Sum of two distinct powers of 2](#) integer sequence

```
>> Union(Total/@Subsets(2^Range(0, 10), {2}))
{3, 5, 6, 9, 10, 12, 17, 18, 20, 24, 33, 34, 36, 40, 48, 65, 66, 68, 72, 80, 96, 129, 130, 132, 136, 144, 160, 192}
```

Implementation status

- - full supported

Github

- [Implementation of Subsets](#)
-

Subtract

```
Subtract(a, b)
```

```
a - b
```

represents the subtraction of `b` from `a`.

Examples

```
>> 5 - 3
```

```
2
```

```
>> a - b // FullForm
```

```
"Plus(a, Times(-1, b))"
```

```
>> a - b - c
```

```
a-b-c
```

```
>> a - (b - c)
```

```
a-b+c
```

Implementation status

- - full supported

Github

- [Implementation of Subtract](#)
-

SubtractFrom

```
SubtractFrom(x, dx)
```

```
x -= dx
```

is equivalent to `x = x - dx`.

Examples

```
>> a = 10
```

```
>> a -= 2  
8
```

```
>> a  
8
```

Implementation status

- - full supported

Github

- [Implementation of SubtractFrom](#)
-

SubtractSides

```
SubtractSides(compare-expr, value)
```

subtracts `value` from all elements of the `compare-expr`. `compare-expr` can be `True`, `False` or a comparison expression with head `Equal`, `Unequal`, `Less`, `LessEqual`, `Greater`, `GreaterEqual`.

Examples

```
>> SubtractSides(a==a, x)
True

>> SubtractSides(a==b, x)
a-x==b-x
```

Implementation status

- - full supported

Github

- [Implementation of SubtractSides](#)
-

SudokuSolve

```
SudokuSolve(matrix)
```

In Sudoku, the objective is to fill a 9×9 `matrix` with digits so that each column, each row, and each of the nine 3×3 subgrids that compose the grid (also called "boxes", "blocks", or "regions") contains all of the digits from 1 to 9. Every input which is not a number between 1 and 9 will be replaced with the correct number to fully solve the sudoku.

See

- [Wikipedia - Sudoku](#)

Examples

```
>> SudokuSolve({{0,0,3,0,0,0,0,0,0},{4,0,0,0,8,0,0,3,6},{0,0,8,0,0,0,1,0,0},{0,4,0,0,6,0,0,0,0},
{{1,2,3,4,5,6,7,8,9},
{4,5,7,1,8,9,2,3,6},
{9,6,8,3,2,7,1,5,4},
{2,4,9,5,6,1,8,7,3},
{5,7,6,9,3,8,4,1,2},
{8,3,1,7,4,2,6,9,5},
{3,1,4,2,7,5,9,6,8},
{6,9,5,8,1,4,3,2,7},
{7,8,2,6,9,3,5,4,1}}}
```

Implementation status

- - full supported

Github

- [Implementation of SudokuSolve](#)
-

Sum

```
Sum(expr, {i, imin, imax})
```

evaluates the discrete sum of `expr` with `i` ranging from `imin` to `imax`.

```
Sum(expr, {i, imax})
```

same as `Sum(expr, {i, 1, imax})`.

```
Sum(expr, {i, imin, imax, di})
```

`i` ranges from `imin` to `imax` in steps of `di`.

```
Sum(expr, {i, imin, imax}, {j, jmin, jmax}, ...)
```

evaluates `expr` as a multiple sum, with `{i, ...}`, `{j, ...}`, ... being in outermost-to-innermost order.

See

- [Wikipedia - Summation](#)
- [Wikipedia - Faulhaber's formula](#)

Examples

```
>> Sum(k, {k, 1, 10})  
55
```

Double sum:

```
>> Sum(i * j, {i, 1, 10}, {j, 1, 10})  
3025  
  
>> Table(Sum(i * j, {i, 0, n}, {j, 0, n}), {n, 0, 4})  
{0,1,9,36,100}
```

Symbolic sums are evaluated:

```
>> Sum(k, {k, 1, n})  
1/2*n*(1+n)
```

```
>> Sum(k, {k, n, 2*n})
3/2*n*(1+n)

>> Sum(k, {k, I, I + 1})
1+I*2

>> Sum(1 / k ^ 2, {k, 1, n})
HarmonicNumber(n, 2)
```

Verify algebraic identities:

```
>> Simplify(Sum(x ^ 2, {x, 1, y}) - y * (y + 1) * (2 * y + 1) / 6)
0
```

Infinite sums:

```
>> Sum(1 / 2 ^ i, {i, 1, Infinity})
1

>> Sum(1 / k ^ 2, {k, 1, Infinity})
Pi^2/6

>> Sum(x^k*Sum(y^l,{l,0,4}},{k,0,4})
1+y+y^2+y^3+y^4+x*(1+y+y^2+y^3+y^4)+(1+y+y^2+y^3+y^4)*x^2+(1+y+y^2+y^3+y^4)*x^3+(1+y+y^2+y^3+y^4)*x^4

>> Sum(2^(-i), {i, 1, Infinity})
1

>> Sum(i / Log(i), {i, 1, Infinity})
Sum(i/Log(i),{i,1,Infinity})

>> Sum(Cos(Pi i), {i, 1, Infinity})
Sum(Cos(i*Pi),{i,1,Infinity})
```

Non-integer bounds:

```
>> Sum(i, {i, 1, 2.5})
3.0

>> Sum(i, {i, 1.1, 2.5})
3.2

>> Sum(k, {k, I, I+1.5})
1.0+2.0*I
```

Implementation status

- - partially implemented

Github

- [Implementation of Sum](#)
 - [Rule definitions of Sum](#)
-

Surd

```
Surd(expr, n)
```

returns the n -th root of `expr`. If the result is defined, it's a real value.

Examples

```
>> Surd(16.0, 3)
2.51984
```

Related terms

[CubeRoot](#)

Implementation status

- - full supported

Github

- [Implementation of Surd](#)
-

SurvivalFunction

```
SurvivalFunction(dist, x)
```

returns the survival function for the distribution `dist` evaluated at `x`.

```
SurvivalFunction(dist, {x1, x2, ...})
```

returns the survival function at `{x1, x2, ...}`.

```
SurvivalFunction(dist)
```

returns the survival function as a pure function.

See

- [Wikipedia - Survival function](#)

Examples

```
>> SurvivalFunction(NormalDistribution(0, 1), x)  
1-Erfc(-x/Sqrt(2))/2
```

Implementation status

- - full supported

Github

- [Implementation of SurvivalFunction](#)
-

Switch

```
Switch(expr, pattern1, value1, pattern2, value2, ...)
```

yields the first `value` for which `expr` matches the corresponding pattern.

Examples

```
>> Switch(2, 1, x, 2, y, 3, z)
y

>> Switch(5, 1, x, 2, y)
Switch(5, 1, x, 2, y)

>> Switch(5, 1, x, 2, y, _, z)
z
```

Switch called with 2 arguments. Switch must be called with an odd number of arguments.

```
>> Switch(2, 1)
Switch(2, 1)
```

Switch called with 2 arguments. Switch must be called with an odd number of arguments.

```
>> a; Switch(b, b)
Switch(b, b)
```

Switch called with 2 arguments. Switch must be called with an odd number of arguments.

```
>> z = Switch(b, b);

>> z

Switch(b, b)
```

Implementation status

- - full supported

Github

- [Implementation of Switch](#)
-

Symbol

```
Symbol
```

is the head of symbols.

Examples

```
>> Head(x)
Symbol
```

You can use `Symbol` to create symbols from strings:

```
>> Symbol("x") + Symbol("x")
2*x

>> {\[Eta], \[CapitalGamma]\[Beta], Z\[Infinity], \[Angle]XYZ, \[FilledSquare]r, i\[Ellipsis]
{\u03b7, \u0393\u03b2, Z\u221e, \u2220XYZ, \u25a0r, i\u2026}}
```

Implementation status

- - full supported

Github

- [Implementation of Symbol](#)
-

SymbolName

`SymbolName(s)`

returns the name of the symbol `s` (without any leading context name).

Examples

```
>> SymbolName(x) // InputForm
x

>> SymbolName(a`b`x) // InputForm
x
```

Implementation status

- - full supported

Github

- [Implementation of SymbolName](#)
-

SymbolQ

```
SymbolQ(x)
```

is `True` if `x` is a symbol, or `False` otherwise.

Examples

```
>> SymbolQ(a)
True

>> SymbolQ(1)
False

>> SymbolQ(a + b)
False
```

Github

- [Implementation of SymbolQ](#)
-

SymmetricMatrixQ

```
SymmetricMatrixQ(m)
```

returns `True` if `m` is a symmetric matrix.

See

- [Wikipedia - Symmetric matrix](#)

Examples

```
>> SymmetricMatrixQ({{1,7,3}, {7,4,-5}, {3,-5,6}})
True
```

Implementation status

- - full supported

Github

- [Implementation of SymmetricMatrixQ](#)
-

SyntaxQ

```
SyntaxQ(str)
```

is `True` if the given `str` is a string which has the correct syntax.

Examples

```
>> SyntaxQ("Integrate(f(x), {x, 0, 10})")  
True
```

Implementation status

- - full supported

Github

- [Implementation of SyntaxQ](#)
-

SystemDialogInput

```
SystemDialogInput("FileOpen")
```

if the file system is enabled, open a file chooser dialog box.

```
SystemDialogInput("FileSave")
```

if the file system is enabled, open a file chooser dialog box.

```
SystemDialogInput("Directory")
```

if the file system is enabled, open a directory chooser dialog box.

Examples

Table

```
Table(expr, {i, n})
```

evaluates `expr` with `i` ranging from `1` to `n`, returning a list of the results.

```
Table(expr, {i, start, stop, step})
```

evaluates `expr` with `i` ranging from `start` to `stop`, incrementing by `step`.

```
Table(expr, {i, {e1, e2, ..., ei}})
```

evaluates `expr` with `i` taking on the values `e1, e2, ..., ei`.

Examples

```
>> Table(x!, {x, 8})
{1, 2, 6, 24, 120, 720, 5040, 40320}

>> Table(x, {4})
{x, x, x, x}

>> n=0
>> Table(n= n + 1, {5})
{1, 2, 3, 4, 5}

>> Table(i, {i, 4})
{1, 2, 3, 4}

>> Table(i, {i, 2, 5})
{2, 3, 4, 5}

>> Table(i, {i, 2, 6, 2})
{2, 4, 6}

>> Table(i, {i, Pi, 2*Pi, Pi / 2})
{Pi, 3/2*Pi, 2*Pi}

>> Table(x^2, {x, {a, b, c}})
{a^2, b^2, c^2}

>> Table(a + dx, {dx, 0, 3, Pi/8})
{a, a+Pi/8, a+Pi/4, a+3/8*Pi, a+Pi/2, a+5/8*Pi, a+3/4*Pi, a+7/8*Pi}
```

`Table` supports multi-dimensional tables:

```
>> Table({i, j}, {i, {a, b}}, {j, 1, 2})  
{{{a, 1}, {a, 2}}, {{b, 1}, {b, 2}}}  
  
>> Table(x, {x, 0, 1/3})  
{0}  
  
>> Table(x, {x, -0.2, 3.9})  
{-0.2, 0.8, 1.8, 2.8, 3.8}
```

Implementation status

- - full supported

Github

- [Implementation of Table](#)
-

TagSet

```
TagSet(f, expr, value)
```

or

```
f /: expr = value
```

assigns the evaluated `value` to `expr` and associates the corresponding rule with the symbol `f`.

Examples

Create an upvalue without using `UpSet`:

```
>> x /: f(x) = 2
2

>> f(x)
2

>> DownValues(f)
{}

>> UpValues(x)
{HoldPattern(f(x)):>2}
```

The symbol `f` must appear as the ultimate head of `lhs` or as the head of a leaf in `lhs`:

```
>> x /: f(g(x)) = 3
: Tag x not found or too deep for an assigned rule.

>> g /: f(g(x)) = 3;

>> f(g(x))
3
```

Related terms

[Set](#), [SetDelayed](#), [TagSetDelayed](#)

Implementation status

- - full supported

Github

- [Implementation of TagSet](#)
-

TagSetDelayed

```
TagSetDelayed(f, expr, value)
```

or

```
f /: expr := value
```

assigns `value` to `expr`, without evaluating `value` and associates the corresponding rule with the symbol `f`.

Examples

Related terms

[Set](#), [SetDelayed](#), [TagSet](#)

Implementation status

- - full supported

Github

- [Implementation of TagSetDelayed](#)
-

Take

```
Take(expr, n)
```

returns `expr` with all but the first `n` leaves removed.

Examples

```
>> Take({a, b, c, d}, 3)
{a,b,c}

>> Take({a, b, c, d}, -2)
{c,d}

>> Take({a, b, c, d, e}, {2, -2})
{b,c,d}
```

Take a submatrix:

```
>> A = {{a, b, c}, {d, e, f}}
>> Take(A, 2, 2)
{{a,b},{d,e}}
```

Take a single column:

```
>> Take(A, All, {2})
{{b},{e}}

>> Take(Range(10), {8, 2, -1})
{8,7,6,5,4,3,2}

>> Take(Range(10), {-3, -7, -2})
{8,6,4}
```

Cannot take positions `-5` through `-2` in `{1, 2, 3, 4, 5, 6}`.

```
>> Take(Range(6), {-5, -2, -2})
Take({1, 2, 3, 4, 5, 6}, {-5, -2, -2})
```

Nonatomic expression expected at position `1` in `Take(l, {-1})`.

```
>> Take(l, {-1})
Take(l,{-1})
```

Empty case

```
>> Take({1, 2, 3, 4, 5}, {-1, -2})
{}

>> Take({1, 2, 3, 4, 5}, {0, -1})
{}

>> Take({1, 2, 3, 4, 5}, {1, 0})
{}

>> Take({1, 2, 3, 4, 5}, {2, 1})
{}

>> Take({1, 2, 3, 4, 5}, {1, 0, 2})
{}

```

Cannot take positions 1 through 0 in {1, 2, 3, 4, 5}.

```
>> Take({1, 2, 3, 4, 5}, {1, 0, -1})
Take({1, 2, 3, 4, 5}, {1, 0, -1})

```

Implementation status

- - full supported

Github

- [Implementation of Take](#)
-

TakeLargest

```
TakeLargest({e_1, e_2, ..., e_i}, n)
```

returns the n largest real values from the list $\{e_1, e_2, \dots, e_i\}$.

Examples

```
>> TakeLargest(Prime(Range(10)), 3)
{29, 23, 19}
```

Implementation status

- - full supported

Github

- [Implementation of TakeLargest](#)
-

TakeLargestBy

```
TakeLargestBy({e_1, e_2, ..., e_i}, function, n)
```

returns the `n` values from the list `{e_1, e_2, ..., e_i}`, where `function(e_i)` is largest.

Examples

```
>> TakeLargestBy(Prime(Range(10)),Mod( #,7)&, 3)  
{13,5,19}
```

Implementation status

- - full supported

Github

- [Implementation of TakeLargestBy](#)
-

TakeSmallest

```
TakeSmallest({e_1, e_2, ..., e_i}, n)
```

returns the n smallest real values from the list $\{e_1, e_2, \dots, e_i\}$.

Examples

```
>> TakeSmallest(Prime(Range(10)), 3)
{2,3,5}
```

Implementation status

- - full supported

Github

- [Implementation of TakeSmallest](#)
-

TakeSmallestBy

```
TakeSmallestBy({e_1, e_2, ..., e_i}, function, n)
```

returns the `n` values from the list `{e_1, e_2, ..., e_i}`, where `function(e_i)` is smallest.

Examples

```
>> TakeSmallestBy(Prime(Range(10)), Mod(7), 3)
{7, 29, 2}
```

Implementation status

- - full supported

Github

- [Implementation of TakeSmallestBy](#)
-

TakeWhile

```
TakeWhile({e1, e2, ...}, head)
```

returns the list of elements `ei` at the start of list for which `head(ei)` returns `True`.

Examples

```
>> TakeWhile({1, 2, 3, 10, 5, 8, 42, 11}, # < 10 &)  
{1,2,3}
```

Implementation status

- - full supported

Github

- [Implementation of TakeWhile](#)
-

Tally

```
Tally(list)
```

or

```
Tally(list, binaryPredicate)
```

return the elements and their number of occurrences in `list` in a new result list.
The `binaryPredicate` tests if two elements are equivalent. `Sameq` is used as the default `binaryPredicate`.

Examples

```
>> str="The quick brown fox jumps over the lazy dog";  
  
>> Tally(Characters(str)) // InputForm  
{{"T",1},{ "h",2},{ "e",3},{ " ",8},{ "q",1},{ "u",2},{ "i",1},{ "c",1},{ "k",1},{ "b",1},{ "r",2}}
```

Related terms

[Commonest](#), [Counts](#)

Implementation status

- - full supported

Github

- [Implementation of Tally](#)
-

Tan

`Tan(expr)`

returns the tangent of `expr` (measured in radians).

`Tan(expr)` will evaluate automatically in the case `expr` is a multiple of `Pi`.

See:

- [Wikipedia - Trigonometric functions](#)
- [Wikipedia - Radian](#)
- [Wikipedia - Degree](#)

Examples

```
>> Tan(1/4*Pi)
1

>> Tan(0)
0

>> Tan(Pi / 2)
ComplexInfinity

>> Tan(0.5 Pi)
1.633123935319537E16
```

Implementation status

- - full supported

Github

- [Implementation of Tan](#)
 - [Rule definitions of Tan](#)
-

Tanh

```
Tanh(z)
```

returns the hyperbolic tangent of `z`.

See

- [Wikipedia - Hyperbolic function](#)

Examples

```
>> Tanh(0)  
0
```

Implementation status

- - full supported

Github

- [Implementation of Tanh](#)
 - [Rule definitions of Tanh](#)
-

TautologyQ

```
TautologyQ(boolean-expr, list-of-variables)
```

test whether the `boolean-expr` is satisfiable by all combinations of boolean `False` and `True` values for the `list-of-variables`.

See

- [Wikipedia - Tautology \(logic\)](#)

Examples

```
>> TautologyQ(a || !a)
True
```

Implementation status

- - full supported

Github

- [Implementation of TautologyQ](#)
-

TemplateApply

```
TemplateApply(string, values)
```

renders a `StringTemplate` expression by replacing `TemplateSlot`s with mapped values.

Examples

```
>> TemplateApply("The quick brown `a` jumps over the lazy `b`.", <|"a" -> "fox", "b" ->
The quick brown fox jumps over the lazy dog.

>> TemplateApply(StringTemplate("The quick brown `a` jumps over the lazy `b`."), <|"a" -
The quick brown fox jumps over the lazy dog.
```

Related terms

[StringTemplate](#), [TemplateIf](#), [TemplateSlot](#)

Implementation status

- - full supported

Github

- [Implementation of TemplateApply](#)
-

TemplateIf

```
TemplateIf(condition-expression, true-expression, false-expression)
```

in `TemplateApply` evaluation insert `true-expression` if `condition-expression` evaluates to `true`, otherwise insert `false-expression`.

Examples

```
>> t=TemplateIf( TemplateSlot("summer"), "in summer ice cream is delicious", "ice cream  
in summer ice cream is delicious
```

Related terms

[StringTemplate](#), [TemplateApply](#), [TemplateSlot](#)

Implementation status

- - full supported

Github

- [Implementation of TemplateIf](#)
-

TemplateSlot

```
TemplateSlot(string)
```

gives a `TemplateSlot` expression with name `string`.

Examples

Related terms

[StringTemplate](#), [TemplateApply](#), [Templatelf](#)

Implementation status

- - full supported

Github

- [Implementation of TemplateSlot](#)
-

TensorDimensions

```
TensorDimensions(t)
```

return the dimensions of the tensor `t`.

Examples

```
>> TensorDimensions({{1,2},{3,4},{a,b},{c,d}})
{4,2}
```

Implementation status

- - full supported

Github

- [Implementation of TensorDimensions](#)
-

TensorProduct

```
TensorProduct(t1, t2, ...)
```

product of the tensors `t1`, `t2`,

See

- [Wikipedia - Tensor product](#)

Examples

```
>> TensorProduct({{{1,2,3},{4,5,6},{7,8,9}},{2,0,0},{0,3,0},{0,0,1}},{2,1,4},{0,3,0}
{{{2,1,4},{0,3,0},{0,0,1}},{4,2,8},{0,6,0},{0,0,2}},{6,3,12},{0,9,0},{0,0,3}}},{{{
8,4,16},{0,12,0},{0,0,4}},{10,5,20},{0,15,0},{0,0,5}},{12,6,24},{0,18,0},{0,0,
6}}},{{{14,7,28},{0,21,0},{0,0,7}},{16,8,32},{0,24,0},{0,0,8}},{18,9,36},{0,27,
0},{0,0,9}}},{{{4,2,8},{0,6,0},{0,0,2}},{0,0,0},{0,0,0},{0,0,0}},{0,0,0},{0,
0,0},{0,0,0}}},{{{0,0,0},{0,0,0},{0,0,0}},{6,3,12},{0,9,0},{0,0,3}},{0,0,0},{0,
0,0},{0,0,0}}},{{{0,0,0},{0,0,0},{0,0,0}},{0,0,0},{0,0,0},{0,0,0}},{2,1,4},{0,
3,0},{0,0,1}}}}
```

Implementation status

- - partially implemented

Github

- [Implementation of TensorProduct](#)
-

TensorRank

```
TensorRank(t)
```

return the rank of the tensor `t`.

Examples

```
>> TensorRank({{1,2},{3,4},{a,b},{c,d}})
2
```

Implementation status

- - partially implemented

Github

- [Implementation of TensorRank](#)
-

TestReport

```
TestReport("file-name-string")
```

load the unit tests from a `file-name-string` and print a summary of the `VerificationTest` included in the file.

```
TestReport("url-string")
```

load the unit tests from a URL `url-string` (starting with `http://` or `https://`) and print a summary of the `VerificationTest` included in the file.

See

- [Wikipedia - Unit_testing](#)

Examples

In the [MMA console](#) execute a test located in a Github repository

```
>> TestReport["https://raw.githubusercontent.com/antononcube/MathematicaForPrediction/m
```

Related terms

[TestResultObject](#), [VerificationTest](#)

Implementation status

- - partially implemented

Github

- [Implementation of TestReport](#)
-

TestResultObject

```
TestResultObject( ... )
```

is an association wrapped in a `TestResultObject` returned from `VerificationTest` which stores the results from executing a single unit test.

Examples

```
>> VerificationTest(3! < 3^3)
TestResultObject(Outcome->Success, TestID->None)
```

Related terms

[TestReport](#), [VerificationTest](#)

TeXForm

```
TeXForm(expr)
```

returns the TeX form of the evaluated `expr`.

See

- [Wikipedia - LaTeX](#)

Examples

```
>> TeXForm(D(sin(x)*cos(x), x))  
"{\cos(x)}^{\!2}-{\sin(x)}^{\!2}"
```

Implementation status

- - partially implemented

Github

- [Implementation of TeXForm](#)
-

Thread

```
Thread(f(args))
```

threads `f` over any lists that appear in `args`.

```
Thread(f(args), h)
```

threads over any parts with head `h`.

Examples

```
>> Thread(f({a, b, c}))
{f(a), f(b), f(c)}

>> Thread(f({a, b, c}, t))
{f(a, t), f(b, t), f(c, t)}

>> Thread(f(a + b + c), Plus)
f(a)+f(b)+f(c)

>> Thread(Tuples({0, 1}, 2) -> {a, b, c, d})
{{0, 0}->a, {0, 1}->b, {1, 0}->c, {1, 1}->d}
```

Functions with attribute `Listable` are automatically threaded over lists:

```
>> {a, b, c} + {d, e, f} + g
{a+d+g, b+e+g, c+f+g}
```

Implementation status

- - full supported

Github

- [Implementation of Thread](#)
-

ThreeJSymbol

```
ThreeJSymbol({j1,m1},{j2,m2},{j3,m3})
```

get the 3-j symbol coefficients.

See:

- [Wikipedia - 3-j symbol](#)

Examples

```
>> ThreeJSymbol({3/2, -3/2}, {3/2, 3/2}, {1, 0})  
Sqrt(3/5)/2
```

Implementation status

- - partially implemented

Github

- [Implementation of ThreeJSymbol](#)
-

Through

```
Through(p(f)[x])
```

gives `p(f(x))`.

Examples

```
>> Through(f(g)[x])
f(g(x))

>> Through(p(f, g)[x])
p(f(x), g(x))

>> Through(p(f, g)[x, y])
p(f(x, y), g(x, y))

>> Through(p(f, g)[])
p(f(), g())

>> Through(p(f, g))
Through(p(f, g))

>> Through(f())[x]
f()
```

Implementation status

- - full supported

Github

- [Implementation of Through](#)
-

Throw

```
Throw(value)
```

stops evaluation and returns `value` as the value of the nearest enclosing `catch`.

`Catch(value, tag)` is caught only by `Catch(expr, form)`, where `tag` matches `form`.

Examples

Using `Throw` can affect the structure of what is returned by a function:

```
>> NestList(#^2 + 1 &, 1, 7)
{1, 2, 5, 26, 677, 458330, 210066388901, 44127887745906175987802}

>> Catch(NestList(If(# > 1000, Throw(#), #^2 + 1) &, 1, 7))
458330
```

A `Throw` without an enclosing `catch` prints the message: `Uncaught Throw(1) returned to top level.`

```
>> Throw[1]
Hold(Throw(1))
```

Implementation status

- - full supported

Github

- [Implementation of Throw](#)
-

TimeConstrained

```
TimeConstrained(expression, seconds)
```

stop evaluation of `expression` if time measurement of the evaluation exceeds `seconds` and return `$Aborted`.

```
TimeConstrained(expression, seconds, default)
```

return `default` instead of `$Aborted` if the evaluation exceeds `seconds`.

Examples

```
>> TimeConstrained(Pause(5), 2)
$Aborted

>> TimeConstrained(Pause(1); "Hello World", 10)
Hello World

>> TimeConstrained(Pause(5), 2, "test")
test
```

Related terms

[Pause](#), [Timing](#)

Implementation status

- - full supported

Github

- [Implementation of TimeConstrained](#)
-

TimeObject

```
TimeObject()
```

returns the current time

```
TimeObject({H, M, S})
```

return the time object for `H` hours, `M` minutes and `S` seconds

Examples

```
>> TimeObject({10, 45})  
10:45:00
```

Implementation status

- - experimental

Github

- [Implementation of TimeObject](#)
-

Times

```
Times(a, b, ...)
```

```
a * b * ...
```

represents the product of the terms `a, b, ...`.

Note: the `Times` operator has the attribute `Flat` ([associative property](#)), `Orderless` ([commutative property](#)), `OneIdentity` and `Listable`.

Examples

```
>> 10*2
20

>> a * a
a^2

>> x ^ 10 * x ^ -2
x^8

>> {1, 2, 3} * 4
{4,8,12}

>> Times @@ {1, 2, 3, 4}
24

>> IntegerLength(Times@@Range(100))
158
```

`Times` has default value `1`:

```
>> a /. n_. * x_ :> {n, x}
{1,a}

>> -a*b // FullForm
"Times(-1, a, b)"

>> -(x - 2/3)
2/3-x

>> -x^2
-2 x

>> -(h/2) // FullForm
"Times(Rational(-1,2), h)"
```

```

>> x / x
1

>> 2*x^2 / x^2
2

>> 3.*Pi
9.42477796076938

>> Head(3 * I)
Complex

>> Head(Times(I, 1/2))
Complex

>> Head(Pi * I)
Times

>> -2.123456789 * x
-2.123456789*x

>> -2.123456789 * I
I*(-2.123456789)

>> N(Pi, 30) * I
I*3.14159265358979323846264338327

>> N(I*Pi, 30)
I*3.14159265358979323846264338327

>> N(Pi * E, 30)
8.53973422267356706546355086954

>> N(Pi, 30) * N(E, 30)
8.53973422267356706546355086954

>> N(Pi, 30) * E
8.53973422267356649108017774746

>> N(Pi, 30) * E // Precision
30

```

Related terms

[Flat](#), [Listable](#), [MatrixPower](#), [OnIdentity](#), [Orderless](#), [Power](#)

Implementation status

- - full supported

Github

- [Implementation of Times](#)
-

TimesBy

```
TimesBy(x, dx)
```

```
x *= dx
```

is equivalent to `x = x * dx`.

Examples

```
>> a = 10
```

```
>> a *= 2  
20
```

```
>> a  
20
```

Implementation status

- - full supported

Github

- [Implementation of TimesBy](#)
-

TimeValue

```
TimeValue(p, i, n)
```

returns a time value calculation.

See

- [Wikipedia - Time value of money](#)

Examples

```
>> TimeValue(Annuity(100, 12), .01, 0)
1125.508

>> TimeValue(Annuity(100, 12), EffectiveInterest(.01, 0.25), 12)
1268.515

>> TimeValue(AnnuityDue(100, 12), 0.1, 0)
749.5061
```

Related terms

[Annuity](#), [AnnuityDue](#), [EffectiveInterest](#)

Implementation status

- - full supported

Github

- [Implementation of TimeValue](#)
-

Timing

Timing(x)

returns a list with the first entry containing the evaluation CPU time of `x` and the second entry is the evaluation result of `x`.

Note: if the Java Management Extensions (JMX) library is not available, this function switch's to `AbsoluteTiming` function.

Examples

```
>> Timing(FactorInteger(3225275494496681))  
{0.547, {{56791489, 1}, {56791529, 1}}}
```

Suppress the output of the result:

```
>> Timing(FactorInteger(3225275494496681);)  
{0.393, Null}
```

Related terms

[AbsoluteTiming](#), [Pause](#), [TimeConstrained](#)

Implementation status

- - full supported

Github

- [Implementation of Timing](#)
-

ToCharacterCode

```
ToCharacterCode(string)
```

converts `string` into a list of corresponding integer character codes.

Examples

```
>> ToCharacterCode("ABCD abcd")  
{65,66,67,68,32,97,98,99,100}  
  
>> "a-3" // ToCharacterCode  
{97,45,51}
```

Related terms

[FromCharCode](#), [ToUnicode](#)

Implementation status

- - full supported

Github

- [Implementation of ToCharacterCode](#)
-

ToeplitzMatrix

```
ToeplitzMatrix(n)
```

gives a toeplitz matrix with the dimension `n`.

```
ToeplitzMatrix(vector)
```

gives a toeplitz matrix of the elements of `vector`.

See

- [Wikipedia - Toeplitz matrix](#)

Examples

```
>> ToeplitzMatrix(3)
{{1,2,3},
 {2,1,2},
 {3,2,1}}

>> ToeplitzMatrix({1, 2, 3, 4, 5, 6}, {1, a, b, c})
{{1,a,b,c},
 {2,1,a,b},
 {3,2,1,a},
 {4,3,2,1},
 {5,4,3,2},
 {6,5,4,3}}

>> ToeplitzMatrix({1, a, b, c}, {1, 2, 3, 4, 5, 6})
{{1,2,3,4,5,6},
 {a,1,2,3,4,5},
 {b,a,1,2,3,4},
 {c,b,a,1,2,3}}
```

Implementation status

- - full supported

Github

- [Implementation of ToeplitzMatrix](#)
-

ToExpression

```
ToExpression("string")
```

interprets a given string as Symja input.

```
ToExpression("string", form)
```

converts the `string` given in `form` into an expression.

```
ToExpression("string", form, head)
```

applies the `head` to the expression before evaluating it.

Examples

```
>> ToExpression("1+2")
3

>> ToExpression("{2, 3, 1}", InputForm, Max)
3

>> ToExpression("2 3", InputForm)
6

>> ToExpression("1 + 2 - x \\times 4 \\div 5", TeXForm)
3-4/5*x
```

Implementation status

- - full supported

Github

- [Implementation of ToExpression](#)
-

Together

```
Together(expr)
```

writes sums of fractions in `expr` together.

Examples

```
>> Together(a/b+x/y)
(a*y+b*x)*b^(-1)*y^(-1)

>> Together(a / c + b / c)
(a+b)/c
```

`Together` operates on lists:

```
>> Together({x / (y+1) + x / (y+1)^2})
{x (2 + y) / (1 + y) ^ 2}
```

But it does not touch other functions:

```
>> Together(f(a / c + b / c))
f(a/c+b/c)

>> f(x)/x+f(x)/x^2//Together
f(x)/x^2+f(x)/x
```

Implementation status

- - full supported

Github

- [Implementation of Together](#)
-

ToLowerCase

```
ToLowerCase(string)
```

converts `string` into a string of corresponding lowercase character codes.

Examples

```
>> ToLowerCase("This is a Test")  
this is a test
```

Implementation status

- - full supported

Github

- [Implementation of ToLowerCase](#)
-

ToPolarCoordinates

```
ToPolarCoordinates({x, y})
```

return the polar coordinates for the cartesian coordinates $\{x, y\}$.

```
ToPolarCoordinates({x, y, z})
```

return the polar coordinates for the cartesian coordinates $\{x, y, z\}$.

See

- [Wikipedia - Polar coordinate system](#)
- [Wikipedia - Cartesian coordinate system](#)

Examples

```
>> ToPolarCoordinates({x, y})  
{Sqrt(x^2+y^2), ArcTan(x, y)}  
  
>> ToPolarCoordinates({x, y, z})  
{Sqrt(x^2+y^2+z^2), ArcCos(x/Sqrt(x^2+y^2+z^2)), ArcTan(y, z)}
```

Related terms

[FromPolarCoordinates](#)

Implementation status

- - partially implemented

Github

- [Implementation of ToPolarCoordinates](#)
-

ToSphericalCoordinates

```
ToSphericalCoordinates({x, y, z})
```

returns the spherical coordinates for the cartesian coordinates `{x, y, z}`.

See

- [Wikipedia - Spherical coordinate system](#)
- [Wikipedia - Polar coordinate system](#)
- [Wikipedia - Cartesian coordinate system](#)

Examples

```
>> ToSphericalCoordinates({x, y, z})  
{Sqrt(x^2+y^2+z^2), ArcTan(z, Sqrt(x^2+y^2)), ArcTan(x, y)}
```

Implementation status

- - partially implemented

Github

- [Implementation of ToSphericalCoordinates](#)
-

ToString

```
ToString(expr)
```

converts `expr` into a string.

Examples

```
>> "Java" <> ToString(8)  
Java8
```

Implementation status

- - full supported

Github

- [Implementation of ToString](#)
-

Total

```
Total(list)
```

adds all values in `list`.

```
Total(list, n)
```

adds all values up to level `n`.

```
Total(list, {n})
```

totals only the values at level `{n}`.

Examples

```
>> Total({1, 2, 3})
6

>> Total({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}})
{12,15,18}

>> Total({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}, 2)
45

>> Total({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}, {2})
{6,15,24}
```

Implementation status

- - full supported

Github

- [Implementation of Total](#)
-

ToUnicode

```
ToUnicode(string)
```

converts `string` into a string of corresponding unicode character codes.

Examples

```
>> ToUnicode("123abcABC")  
"\u0031\u0032\u0033\u0061\u0062\u0063\u0041\u0042\u0043"
```

Related terms

[FromCharCode](#), [ToCharCode](#)

Implementation status

- - full supported

Github

- [Implementation of ToUnicode](#)
-

ToUpperCase

```
ToUpperCase(string)
```

converts `string` into a string of corresponding uppercase character codes.

Examples

```
>> ToUpperCase("This is a Test")  
THIS IS A TEST
```

Implementation status

- - full supported

Github

- [Implementation of ToUpperCase](#)
-

Tr

```
Tr(matrix)
```

computes the trace of the `matrix`.

See

- [Wikipedia - Trace \(linear algebra\)](#)

Examples

```
>> Tr({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}})
15
```

Symbolic trace:

```
>> Tr({{a, b, c}, {d, e, f}, {g, h, i}})
a+e+i
```

Implementation status

- - full supported

Github

- [Implementation of Tr](#)
-

Trace

Trace(expr)

return the evaluation steps which are used to get the result.

Examples

```
>> Trace(D(Sin(x),x))  
{{Cos(#1)&[x], Cos(x)}, D(x,x)*Cos(x), {D(x,x), 1}, 1*Cos(x), Cos(x)}
```

Related terms

[Stack](#), [StackBegin](#)

Implementation status

- - full supported

Github

- [Implementation of Trace](#)
-

TransformationFunction

TransformationFunction(m)

represents a transformation.

See

- [Wikipedia - Transformation matrix](#)

Examples

```
>> RotationTransform(Pi).TranslationTransform({1, -1})
TransformationFunction(
  {{-1, 0, -1},
   {0, -1, 1},
   {0, 0, 1}})

>> TranslationTransform({1, -1}).RotationTransform(Pi)
TransformationFunction(
  {{-1, 0, 1},
   {0, -1, -1},
   {0, 0, 1}})
```

Related terms

[RotationTransform](#), [TranslationTransform](#)

Implementation status

- - full supported

Github

- [Implementation of TransformationFunction](#)
-

TranslationTransform

```
TranslationTransform(v)
```

gives a `TransformationFunction` that translates points by vector `v`.

See

- [Wikipedia - Transformation matrix](#)

Examples

```
>> TranslationTransform({1, 2})  
TransformationFunction(  
  {{1, 0, 1},  
   {0, 1, 2},  
   {0, 0, 1}})
```

Related terms

[RotationTransform](#), [TransformationFunction](#)

Implementation status

- - full supported

Github

- [Implementation of TranslationTransform](#)
-

Transliterate

```
Transliterate("string")
```

try converting the given string to a similar ASCII string

See:

- [Wikipedia - Transliteration](#)
- [unicode-org.github.io - General Transforms](#)

Examples

ASCII transliteration are for examples:

- Russian language
- Hiragana

```
>> Transliterate(", ")  
Gorbacev, Mihail Sergeevic
```

Github

- [Implementation of Transliterate](#)
-

Transpose

`Transpose(m)`

transposes rows and columns in the matrix `m`.

`Transpose(tensor, permutation-list)`

transposes rows and columns in the `tensor` according to `permutation-list`. If `tensor` is a `SparseArray` the result will also be a `SparseArray`.

See:

- [Wikipedia - Transpose](#)

Examples

```
>> Transpose({{1, 2, 3}, {4, 5, 6}})
{{1, 4}, {2, 5}, {3, 6}}

>> MatrixForm(%)
1    4
2    5
3    6

>> Transpose(x)
Transpose(x)

>> Transpose({{1, 2, 3}, {4, 5, 6}}, {2,1})
{{1,4},{2,5},{3,6}}

>> Transpose({{1, 2, 3}, {4, 5, 6}}, {1,2})
{{1,2,3},{4,5,6}}
```

Implementation status

- - partially implemented

Github

- [Implementation of Transpose](#)
-

TreeForm

```
TreeForm(expr)
```

create a tree visualization from the given expression `expr`.

```
TreeForm(expr, n)
```

create a tree visualization from the given expression `expr` from level `0` to `n`.

See:

- [Wikipedia - Binary expression tree](#)

Examples

Generate a [Graphics](#) output to visualize the structure of the input expression:

```
>> TreeForm(a+(b*q*s)^(2*y)+Sin(c)^(3-z))
```

Related terms

[Manipulate Graph](#)

Implementation status

- - full supported

Github

- [Implementation of TreeForm](#)
-

TreePlot

```
TreePlot(graph-expr)
```

create a tree plot from the given graph expression `graph-expr`.

See:

- [Wikipedia - Tree structure](#)

Examples

Generate a [Graphics](#) output to visualize the structure of the input expression:

```
>> TreePlot(KaryTree(7, 3))  
-Graphics-
```

Implementation status

- - experimental

Github

- [Implementation of TreePlot](#)
-

TrigExpand

```
TrigExpand(expr)
```

expands out trigonometric expressions in `expr`.

Examples

```
>> TrigExpand(Sin(x+y))  
Cos(x)*Sin(y)+Cos(y)*Sin(x)
```

Implementation status

- - partially implemented

Github

- [Implementation of TrigExpand](#)
-

TrigReduce

`TrigReduce(expr)`

rewrites products and powers of trigonometric functions in `expr` in terms of trigonometric functions with combined arguments.

Examples

```
>> TrigReduce(2*Sin(x)*Cos(y))  
Sin(-y+x)+Sin(y+x)
```

Implementation status

- - partially implemented

Github

- [Implementation of TrigReduce](#)
-

TrigToExp

```
TrigToExp(expr)
```

converts trigonometric functions in `expr` to exponentials.

See

- [Wikipedia - Euler's formula](#)

Examples

```
>> TrigToExp(Cos(x))  
1/2*E^(I*x)+1/2*E^(-I*x)
```

Implementation status

- - partially implemented

Github

- [Implementation of TrigToExp](#)
-

True

```
True
```

the constant `True` represents the boolean value **true**

See:

- [Wikipedia - Truth value](#)

Examples

```
>> Pi > E
True
```

Github

- [Implementation of True](#)
-

TrueQ

```
TrueQ(expr)
```

returns `True` if and only if `expr` is `True`.

See:

- [Wikipedia - Truth value](#)

Examples

```
>> TrueQ(True)
True

>> TrueQ(False)
False

>> TrueQ(a)
False
```

Implementation status

- - full supported

Github

- [Implementation of TrueQ](#)
-

TTest

```
TTest(real-vector)
```

Returns the *observed significance level*, or *p-value*, associated with a one-sample, two-tailed t-test comparing the mean of the input vector with the constant `0.0`.

```
TTest({real-vector1, real-vector2})
```

Returns the *observed significance level*, or *p-value*, associated with a two-sample, two-tailed t-test comparing the means of the input vectors, under the assumption that the two samples are drawn from subpopulations with equal variances.

Examples

Implementation status

- - experimental

Github

- [Implementation of TTest](#)
-

Tuples

```
Tuples(list, n)
```

creates a list of all `n`-tuples of elements in `list`.

```
Tuples({list1, list2, ...})
```

returns a list of tuples with elements from the given lists.

See

- [Wikipedia - Tuple](#)

Examples

```
>> Tuples({a, b, c}, 2)
{{a, a}, {a, b}, {a, c}, {b, a}, {b, b}, {b, c}, {c, a}, {c, b}, {c, c}}

>> Tuples[{{a, b}, {1, 2, 3}}]
{{a, 1}, {a, 2}, {a, 3}, {b, 1}, {b, 2}, {b, 3}}

>> Thread(Tuples({0, 1}, 2) -> {a, b, c, d})
{{0, 0}->a, {0, 1}->b, {1, 0}->c, {1, 1}->d}
```

The head of list need not be `List`:

```
>> Tuples(f(a, b, c), 2)
{f(a, a), f(a, b), f(a, c), f(b, a), f(b, b), f(b, c), f(c, a), f(c, b), f(c, c)}
```

However, when specifying multiple expressions, `List` is always used:

```
>> Tuples({f(a, b), g(x, y)})
{{a, x}, {a, y}, {b, x}, {b, y}}
```

Implementation status

- - full supported

Github

- [Implementation of Tuples](#)
-

Uncompress

```
Uncompress(string)
```

an expression compressed by the `Compress` function can be fully reconstructed using the `Uncompress` function.

Examples

```
>> str = Compress(Expand((a+b)^3))  
H4sIAAAAAAAAAA/0uMM1bQVjDWSowz0kqCsLSS4oyArKQ4YwDZmlsNHQAAAA==  
  
>> Uncompress(str)  
a^3+3*a^2*b+3*a*b^2+b^3
```

Related terms

[Compress](#)

Implementation status

- - full supported

Github

- [Implementation of Uncompress](#)
-

Undefined

```
Undefined
```

represents an undefined result for example in the `ConditionalExpression` function.

Examples

If the condition in `ConditionalExpression` is `False`, `Undefined` will be returned:

```
>> ConditionalExpression(Sin(x)^2, 42==1)
Undefined
```

Related terms

[ConditionalExpression](#),

Underflow

```
Underflow( )
```

represents a number too small to be represented by Symja.

Examples

Implementation status

- - full supported

Github

- [Implementation of Underflow](#)
-

UndirectedEdge

```
UndirectedEdge(a, b)
```

is an undirected edge between the vertices `a` and `b` in a `graph` object.

See:

- [Wikipedia - Directed graph](#)
- [Wikipedia - Graph theory](#)

Examples

```
>> Graph({1 <-> 2, 2 <-> 3, 1 <-> 3, 4 <-> 2})
```

Related terms

[DirectedEdge](#), [GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Unequal

```
Unequal(x, y)
```

```
x != y
```

yields `False` if `x` and `y` are known to be equal, or `True` if `x` and `y` are known to be unequal.

```
lhs != rhs
```

represents the inequality `lhs <> rhs`.

See

- [Wikipedia - Computer algebra - Equality](#)

Examples

```
>> 1 != 1
False
```

Lists are compared based on their elements:

```
>> {1} != {2}
True

>> {1, 2} != {1, 2}
False

>> {a} != {a}
False

>> "a" != "b"
True

>> "a" != "a"
False

>> Pi != N(Pi)
False

>> a_ != b_
a_ != b_

>> a != a != a
```

```
False
```

```
>> "abc" != "def" != "abc"
```

```
False
```

```
>> a != a != b
```

```
False
```

```
>> a != b != a
```

```
a != b != a
```

```
>> {Unequal(), Unequal(x), Unequal(1)}
```

```
{True, True, True}
```

Related terms

[Equal](#), [SameQ](#) , [UnsameQ](#)

Implementation status

- - full supported

Github

- [Implementation of Unequal](#)
-

Unevaluated

```
Unevaluated(expr)
```

temporarily leaves `expr` in an unevaluated form when it appears as a function argument.

Examples

`Unevaluated` is automatically removed when function arguments are evaluated:

```
>> Sqrt(Unevaluated(x))  
Sqrt(x)
```

The `Length` value is 4 because we do not evaluate the `' Plus '` operator:

```
>> Length(Unevaluated(1+2+3+4))  
4
```

`Unevaluated` has attribute `HoldAllComplete`:

```
>> Attributes(Unevaluated)  
{HoldAllComplete, Protected}
```

The `Unevaluated` function call is kept in arguments of non-executed functions:

```
>> f(Unevaluated(x))  
f(Unevaluated(x))
```

Functions that have the `Flat` property, `Unevaluated` propagates into function's arguments:

```
>> Attributes(f) = {Flat};  
  
>> f(a, Unevaluated(f(b, c)))  
f(a, Unevaluated(b), Unevaluated(c))
```

In `Sequences` containing `Unevaluated` functions:

```
>> g(a, Sequence(Unevaluated(b), Unevaluated(c)))
```

However, when surrounding a `Sequence` by `Unevaluated`, no proliferation of `Unevaluated` function calls occurs:

```
>> g[Unevaluated[Sequence[a, b, c]]]  
g(a, Unevaluated(b), Unevaluated(c))
```

Related terms

[Hold](#), [HoldComplete](#), [HoldForm](#), [HoldPattern](#), [ReleaseHold](#)

Implementation status

- - full supported

Github

- [Implementation of Unevaluated](#)
-

UniformDistribution

```
UniformDistribution({min, max})
```

returns a uniform distribution.

```
UniformDistribution( )
```

returns a uniform distribution with `min = 0` and `max = 1`.

See

- [Wikipedia - Uniform distribution \(continuous\)](#)

Examples

The variance of the uniform distribution is

```
>> Variance(DiscreteUniformDistribution({l, r}))  
1/12*(-1+(1-l+r)^2)
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of UniformDistribution](#)
-

Union

```
Union(set1, set2)
```

get the union set from `set1` and `set2`.

See

- [Wikipedia - Union \(set theory\)](#)

Examples

```
>> Union({1, 2, 3}, {2, 3, 4})  
{1, 2, 3, 4}
```

Related terms

[Complement](#), [Intersection](#)

Implementation status

- - full supported

Github

- [Implementation of Union](#)
-

Unique

```
Unique(expr)
```

create a unique symbol of the form `expr$. . .`.

```
Unique("expr")
```

create a unique symbol of the form `expr. . .`.

Examples

```
>> Unique(xy)
xy$1

>> Unique("a")
a1
```

Implementation status

- - full supported

Github

- [Implementation of Unique](#)
-

UnitaryMatrixQ

```
UnitaryMatrixQ(u)
```

returns `True` if a complex square matrix `u` is unitary, that is, if its conjugate transpose `u^(*)` is also its inverse, that is, if $u^*(*) \cdot u = u \cdot u^*(*) = u \cdot u^*(-1) = 1 = \mathbf{I}$ where $\mathbf{I}$ is the identity matrix.

See:

- [Wikipedia - Unitary matrix](#)

Examples

```
>> u = {{0, I}, {I, 0}};  
  
>> UnitaryMatrixQ(u)  
True
```

Implementation status

- - full supported

Github

- [Implementation of UnitaryMatrixQ](#)
-

UnitConvert

```
UnitConvert(quantity)
```

convert the `quantity` to the base unit

```
UnitConvert(quantity, unit)
```

convert the `quantity` to the given `unit`.

See

- [Wikipedia - International System of Units](#)

Examples

Convert radian to degree.

```
>> UnitConvert(Quantity(Pi, "rad"), "deg")
180[deg]

>> UnitConvert(Quantity(Pi, "deg"), "rad")
Pi^2/180[rad]
```

Implementation status

- - partially implemented

Github

- [Implementation of UnitConvert](#)
-

Unitize

```
Unitize(expr)
```

maps a non-zero `expr` to `1`, and a zero `expr` to `0`.

Examples

```
>> Unitize((E + Pi)^2 - E^2 - Pi^2 - 2*E*Pi)
0
```

Implementation status

- - full supported

Github

- [Implementation of Unitize](#)
-

UnitStep

```
UnitStep(expr)
```

returns `0`, if `expr` is less than `0` and returns `1`, if `expr` is greater equal than `0`.

```
UnitStep(x1, x2, ...)
```

return the multidimensional unit step function which is `1` only if none of the `xi` are negative.

See

- [Wikipedia - Heaviside step function](#)
- [Wikipedia - Step function](#)

Examples

```
>> UnitStep(-42)
0
```

Related terms

[Piecewise](#)

Implementation status

- - full supported

Github

- [Implementation of UnitStep](#)
-

UnitVector

```
UnitVector(position)
```

returns a unit vector with element `1` at the given `position`.

```
UnitVector(dimension, position)
```

returns a unit vector with dimension `dimension` and an element `1` at the given `position`.

See:

- [Wikipedia - Unit vector](#)

Examples

```
>> UnitVector(4,3)
{0, 0, 1, 0}
```

Implementation status

- - full supported

Github

- [Implementation of UnitVector](#)
-

UnsameQ

```
UnsameQ(x, y)
```

```
x!=y
```

returns `True` if `x` and `y` are not structurally identical.

See

- [Wikipedia - Computer algebra - Equality](#)

Examples

Any object is the same as itself:

```
>> a!=a
False

>> 1!=1.0
True
```

Related terms

[Equal](#), [SameQ](#) , [Unequal](#)

Implementation status

- - full supported

Github

- [Implementation of UnsameQ](#)
-

Unset

```
Unset(expr)
```

or

```
expr =.
```

removes any definitions belonging to the left-hand-side `expr`.

Examples

```
>> a = 2
2

>> a =.

>> a
a
```

Unsetting an already unset or never defined variable will not change anything:

```
>> a =.

>> b =.
```

`unset` can unset particular function values. It will print a message if no corresponding rule is found.

```
>> f[x_] =.
Assignment not found for: f(x_)

>> f(x_) := x ^ 2

>> f(3)
9

>> f(x_) =.

>> f(3)
f(3)
```

Implementation status

- - full supported

Github

- [Implementation of Unset](#)
-

UpperCaseQ

```
UpperCaseQ(str)
```

is `True` if the given `str` is a string which only contains upper case characters.

Examples

```
>> UpperCaseQ("ABCDEFGHIJKLMNOPQRSTUVWXYZ")
True

>> UpperCaseQ("ABCDEFGHIJKLMNopqRSTUVWXYZ")
False
```

Implementation status

- - full supported

Github

- [Implementation of UpperCaseQ](#)
-

UpperTriangularize

```
UpperTriangularize(matrix)
```

create a upper triangular matrix from the given `matrix`.

```
UpperTriangularize(matrix, n)
```

create a upper triangular matrix from the given `matrix` above the `n`-th subdiagonal.

See

- [Wikipedia - Triangular matrix](#)

Examples

```
>> UpperTriangularize({{a,b,c,d}, {d,e,f,g}, {h,i,j,k}, {l,m,n,o}}, 1)
{{0,b,c,d}
 {0,0,f,g}
 {0,0,0,k}
 {0,0,0,0}}
```

Implementation status

- - partially implemented

Github

- [Implementation of UpperTriangularize](#)
-

UpperTriangularMatrixQ

```
UpperTriangularMatrixQ(matrix)
```

```
UpperTriangularMatrixQ(matrix, diagonal)
```

returns `True` if `matrix` is upper triangular.

Examples

Implementation status

- - full supported

Github

- [Implementation of UpperTriangularMatrixQ](#)
-

UpValues

```
UpValues(symbol)
```

prints the up-value rules associated with `symbol`.

Examples

```
>> u /: v(x_u) := {x}

>> UpValues(u)
{HoldPattern(v(x_u)):>{x}}
```

Implementation status

- - full supported

Github

- [Implementation of UpValues](#)
-

URLDecode

```
URLDecode(string)
```

the `URLDecode` function decodes a URL-encoded string, converting it back to its original human-readable format. This is the inverse operation of `URLEncode`.

Examples

Related terms

[URLEncode](#)

Implementation status

- - full supported

Github

- [Implementation of URLDecode](#)
-

URLEncode

```
URLEncode(string)
```

the `URLEncode` function converts a string into a URL-encoded format, making it safe for inclusion in URL query strings. This is the inverse operation of `URLDecode`.

Examples

Related terms

[URLDecode](#)

Implementation status

- - full supported

Github

- [Implementation of URLEncode](#)
-

ValueQ

ValueQ(expr)

returns `True` if and only if `expr` is defined.

Examples

```
>> ValueQ(x)
False

>> x=1;

>> ValueQ(x)
True

>> ValueQ(True)
False
```

Implementation status

- - full supported

Github

- [Implementation of ValueQ](#)
-

Values

```
Values(association)
```

return a list of values of the `association`.

Examples

```
>> Values(<|ahey->avalue, bkey->bvalue, ckey->cvalue|>)  
{avalue,bvalue,cvalue}
```

Related terms

[Association](#), [AssociationQ](#), [Counts](#), [Keys](#), [KeySort](#), [Lookup](#)

Implementation status

- - full supported

Github

- [Implementation of Values](#)
-

VandermondeMatrix

```
VandermondeMatrix(n)
```

gives the Vandermonde matrix with `n` rows and columns.

See

- [Wikipedia - Vandermonde matrix](#)

Examples

```
>> VandermondeMatrix({a,b,c})  
{{1,a,a^2},  
 {1,b,b^2},  
 {1,c,c^2}}
```

Implementation status

- - full supported

Github

- [Implementation of VandermondeMatrix](#)
-

Variables

`Variables(expr)`

gives a list of the variables that appear in the polynomial `expr`.

Examples

```
>> Variables(a x^2 + b x + c)
{a,b,c,x}

>> Variables({a + b x, c y^2 + x/2})
{a,b,c,x,y}

>> Variables(x + Sin(y))
{x,Sin(y)}
```

Related terms

[BooleanVariables](#)

Implementation status

- - full supported

Github

- [Implementation of Variables](#)
-

Variance

```
Variance(list)
```

computes the variance of `list`. `list` may consist of numerical values or symbols. Numerical values may be real or complex.

`Variance({{a1, a2, ...}, {b1, b2, ...}, ...})` will yield `{Variance({a1, b1, ...}, Variance({a2, b2, ...}), ...}`.

`Variance` can be applied to the following distributions:

[BernoulliDistribution](#), [BinomialDistribution](#), [DiscreteUniformDistribution](#),
[ErlangDistribution](#), [ExponentialDistribution](#), [FrechetDistribution](#), [GammaDistribution](#),
[GeometricDistribution](#), [GumbelDistribution](#), [HypergeometricDistribution](#),
[LogNormalDistribution](#), [NakagamiDistribution](#), [NormalDistribution](#),
[PoissonDistribution](#), [StudentTDistribution](#), [WeibullDistribution](#)

See

- [Wikipedia - Variance](#)

Examples

```
>> Variance({1, 2, 3})
1

>> Variance({7, -5, 101, 3})
7475/3

>> Variance({1 + 2*I, 3 - 10*I})
74

>> Variance({a, a})
0

>> Variance({{1, 3, 5}, {4, 10, 100}})
{9/2, 49/2, 9025/2}
```

Implementation status

- - full supported

Github

- [Implementation of Variance](#)
-

VectorAngle

```
VectorAngle(u, v)
```

gives the angles between vectors `u` and `v`

Examples

```
>> VectorAngle({1, 0}, {0, 1})  
Pi/2  
  
>> VectorAngle({1, 2}, {3, 1})  
Pi/4  
  
>> VectorAngle({1, 1, 0}, {1, 0, 1})  
Pi/3  
  
>> VectorAngle({0, 1}, {0, 1})  
0
```

Implementation status

- - partially implemented

Github

- [Implementation of VectorAngle](#)
-

VectorGreater

```
VectorGreater({vector1, vector2})
```

the `VectorGreater` function is used to compare two vectors, `vector1` and `vector2` recursively. It returns `True` if each corresponding element of `vector1` is greater than the corresponding element of `vector2`, and `False` otherwise.

Examples

```
>> VectorGreater({{1, 2, 7}}, {3, 4}, {{0, -1}, {2, -1}})
True

>> VectorGreater({{1, 2, 7}}, {3, 4}, {{0, 2}, {2, -1}})
False
```

Implementation status

- - partially implemented

Github

- [Implementation of VectorGreater](#)
-

VectorGreaterEqual

```
VectorGreaterEqual({vector1, vector2})
```

the `VectorGreaterEqual` function is used to compare two vectors, `vector1` and `vector2` recursively. It returns `True` if each corresponding element of `vector1` is greater or equal than the corresponding element of `vector2`, and `False` otherwise.

Examples

```
>> VectorGreaterEqual({{1, {2,7}}, {3,4}}, {{0, -1}, {2, -1}})
True

>> VectorGreaterEqual({{1, {2,7}}, {3,4}}, {{0, 2}, {2, -1}})
True
```

Implementation status

- - partially implemented

Github

- [Implementation of VectorGreaterEqual](#)
-

VectorLess

```
VectorLess({vector1, vector2})
```

the `VectorLess` function is used to compare two vectors, `vector1` and `vector2` recursively. It returns `True` if each corresponding element of `vector1` is less than the corresponding element of `vector2`, and `False` otherwise.

Examples

```
>> VectorLess({{{0, -1}, {2, -1}}, {{1, {2, 7}}, {3, 4}}})
True

>> VectorLess({{{0, 2}, {2, -1}}, {{1, {2, 7}}, {3, 4}}})
False
```

Implementation status

- - partially implemented

Github

- [Implementation of VectorLess](#)
-

VectorLessEqual

```
VectorLessEqual({vector1, vector2})
```

the `VectorLessEqual` function is used to compare two vectors, `vector1` and `vector2` recursively. It returns `True` if each corresponding element of `vector1` is less or equal than the corresponding element of `vector2`, and `False` otherwise.

Examples

```
>> VectorLessEqual({{0, -1}, {2, -1}}, {{1, {2, 7}}, {3, 4}})
True

VectorLessEqual({{0, 2}, {2, -1}}, {{1, {2, 7}}, {3, 4}})
True
```

Implementation status

- - partially implemented

Github

- [Implementation of VectorLessEqual](#)
-

VectorQ

```
VectorQ(v)
```

returns `True` if `v` is a list of elements which are not themselves lists.

```
VectorQ(v, f)
```

returns `True` if `v` is a vector and `f(x)` returns `True` for each element `x` of `v`.

See

- [Wikipedia - Vector \(mathematics and physics\)](#)
- [Youtube - Vectors, what even are they? | Essence of linear algebra, chapter 1](#)
- [Youtube - Linear combinations, span, and basis vectors | Essence of linear algebra, chapter 2](#)

Examples

```
>> VectorQ({a, b, c})  
True
```

Implementation status

- - full supported

Github

- [Implementation of VectorQ](#)
-

Verbatim

```
Verbatim(expr)
```

prevents pattern constructs in `expr` from taking effect, allowing them to match themselves.

Examples

```
>> _ /. Verbatim(_)->t  
t  
  
>> x /. Verbatim[_]->t  
x
```

VerificationTest

```
VerificationTest(test-expr)
```

create a `TestResultObject` by testing if `test-expr` evaluates to `True`.

```
VerificationTest(test-expr, expected-expr)
```

create a `TestResultObject` by testing if `test-expr` evaluates to `expected-expr`.

`SameTest` can be used as an option. The default option used is `SameTest->SameQ`.

See

- [Wikipedia - Unit_testing](#)

Examples

```
>> VerificationTest(3^3, 27)
TestResultObject(Outcome->Success, TestID->None)
```

Related terms

[TestReport](#), [TestResultObject](#)

Implementation status

- - partially implemented

Github

- [Implementation of VerificationTest](#)
-

EdgeCount

```
VertexCount(graph)
```

return the number of vertices of the `graph`

See:

- [Wikipedia - Graph theory](#)

Examples

```
>> VertexCount(CompleteGraph(4))  
4
```

Related terms

[EdgeCount](#), [GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of VertexCount](#)
-

VertexEccentricity

```
VertexEccentricity(graph, vertex)
```

compute the eccentricity of `vertex` in the `graph`. It's the length of the longest shortest path from the `vertex` to every other vertex in the `graph`.

See

- [Wikipedia - Distance \(graph_theory\)](#)

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindShortestTour](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of VertexEccentricity](#)
-

VertexList

```
VertexList(graph)
```

convert the `graph` into a list of vertices.

See

- [Wikipedia - Graph theory](#)

Examples

```
>> VertexList(Graph({1 -> 2, 2 -> 3, 1 -> 3, 4 -> 2}))  
{1,2,3,4}
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindShortestTour](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of VertexList](#)
-

VertexQ

```
VertexQ(graph, vertex)
```

test if `vertex` is a vertex in the `graph` object.

See:

- [Wikipedia - Graph theory](#)

Examples

```
>> VertexQ(Graph({1 -> 2, 2 -> 3, 1 -> 3, 4 -> 2}),3)
True

>> VertexQ(Graph({1 -> 2, 2 -> 3, 1 -> 3, 4 -> 2}),5)
False
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindShortestTour](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#)

Implementation status

- - partially implemented

Github

- [Implementation of VertexQ](#)
-

WeibullDistribution

```
WeibullDistribution(a, b)
```

returns a Weibull distribution.

See

- [Wikipedia - Weibull distribution](#)

Examples

The variance of the Weibull distribution is

```
>> Variance(WeibullDistribution(n, m))  
m^2* (-Gamma(1+1/n)^2+Gamma(1+2/n))  
  
>> Plot(Table(PDF(WeibullDistribution(a, 2), x), {a, {0.5, 1.0, 3.0}}) // Evaluate, {x,
```

Related terms

[CDF](#), [Mean](#), [Median](#), [PDF](#), [Quantile](#), [StandardDeviation](#), [Variance](#)

Implementation status

- - full supported

Github

- [Implementation of WeibullDistribution](#)
-

WeierstrassP

```
WeierstrassP(expr, {n1, n2})
```

Weierstrass elliptic function.

See

- [Wikipedia - Weierstrass's elliptic functions](#)
- [Fungrim - Weierstrass elliptic functions](#)

Examples

```
>> WeierstrassP(z, {3, 1})  
1+3/2*Cot(Sqrt(3/2)*z)^2
```

Implementation status

- - full supported

Github

- [Implementation of WeierstrassP](#)
-

WeightedAdjacencyMatrix

```
WeightedAdjacencyMatrix(graph)
```

convert the `graph` into a weighted adjacency matrix in sparse array format.

See:

- [Wikipedia - Adjacency matrix](#)

Examples

The sparse array can be converted to an adjacency matrix in `List` format with the `Normal` function:

```
>> WeightedAdjacencyMatrix(Graph({1 -> 2, 2 -> 3, 1 -> 3, 4 -> 2}, EdgeWeight->{5.061, 2.282, 0.061, 1.707}, Normal)
{{0, 5.061, 5.086, 0}, \
 {0, 0, 2.282, 0},
 {0, 0, 0, 0},
 {0, 1.707, 0, 0}}
```

Related terms

[AdjacencyMatrix](#), [GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [Normal](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of WeightedAdjacencyMatrix](#)
-

WeightedGraphQ

```
WeightedGraphQ(expr)
```

test if `expr` is an explicit weighted graph object.

See:

- [Wikipedia - Graph](#)
- [Wikipedia - Graph theory](#)
- [Wikipedia - Calculus on finite weighted graphs](#)

Examples

```
>> WeightedGraphQ(Graph({1 -> 2, 2 -> 3, 1 -> 3, 4 -> 2}) )
True

>> WeightedGraphQ( Sin(x) )
False

>> WeightedGraphQ( Graph({1->2, 2->3, 3->1}, EdgeWeight->{5.061,2.282,5.086}) )
True
```

Related terms

[GraphCenter](#), [GraphDiameter](#), [GraphPeriphery](#), [GraphRadius](#), [AdjacencyMatrix](#), [EdgeList](#), [EdgeQ](#), [EulerianGraphQ](#), [FindEulerianCycle](#), [FindHamiltonianCycle](#), [FindVertexCover](#), [FindShortestPath](#), [FindSpanningTree](#), [Graph](#), [GraphQ](#), [HamiltonianGraphQ](#), [VertexEccentricity](#), [VertexList](#), [VertexQ](#)

Implementation status

- - partially implemented

Github

- [Implementation of WeightedGraphQ](#)
-

WheelGraph

```
WheelGraph(order)
```

in graph theory, a wheel graph is a graph formed by connecting a single universal vertex to all vertices of a cycle.

See

- [Wikipedia - Wheel graph](#)

Examples

```
>> WheelGraph(4) // AdjacencyMatrix // Normal
{{0,1,1,1},
 {1,0,1,1},
 {1,1,0,1},
 {1,1,1,0}}
```

Implementation status

- - partially implemented

Github

- [Implementation of WheelGraph](#)
-

Which

```
Which(cond1, expr1, cond2, expr2, ...)
```

yields `expr1` if `cond1` evaluates to `True`, `expr2` if `cond2` evaluates to `True`, etc.

Examples

```
>> n=5;
>> Which(n == 3, x, n == 5, y)
y

>> f(x_) := Which(x < 0, -x, x == 0, 0, x > 0, x)
>> f(-3)
3
```

If no test yields `True`, `Which` returns `Null`:

```
>> Which(False, a)
```

If a test does not evaluate to `True` or `False`, evaluation stops and a `which` expression containing the remaining cases is returned:

```
>> Which(False, a, x, b, True, c)
Which(x, b, True, c)
```

`which` must be called with an even number of arguments:

```
>> Which(a, b, c)
Which(a, b, c)
```

Implementation status

- - full supported

Github

- [Implementation of Which](#)
-

While

```
While(test, body)
```

evaluates `body` as long as test evaluates to `True`.

```
While(test)
```

runs the loop without any body.

Examples

Compute the GCD of two numbers

```
>> {a, b} = {27, 6};

>> While(b != 0, {a, b} = {b, Mod(a, b)});

>> a
3

>> i = 1; While(True, If(i^2 > 100, Return(i + 1), i++))
12
```

Related terms

[Break](#), [Continue](#), [Do](#), [For](#)

Implementation status

- - full supported

Github

- [Implementation of While](#)
-

White

White

RGB color value for the color white

Examples

```
>> White  
RGBColor(1.0,1.0,1.0)
```

Whitespace

Whitespace

represents a sequence of whitespace characters.

Examples

```
>> StringMatchQ("\r\n", Whitespace)
True

>> StringSplit("a\nb\r\ncd", Whitespace)
{a,b,c,d}
```

WhitespaceCharacter

WhitespaceCharacter

represents a single whitespace character.

Examples

```
>> StringMatchQ("\n", WhitespaceCharacter)
True

>> StringSplit("a\nb\nc d", WhitespaceCharacter)
{a,b,c,d}
```

WhittakerM

```
WhittakerM(a, b, z)
```

returns the Whittaker function $M_{a,b}(z)$.

See

- [Wikipedia - Whittaker function](#)

Examples

```
>> N(WhittakerM(2, 1/2, 2*2*I))  
0.3080150149255876+I*(-8.938966760794024)
```

Implementation status

- - experimental

Github

- [Implementation of WhittakerM](#)
 - [Rule definitions of WhittakerM](#)
-

WhittakerW

```
WhittakerW(a, b, z)
```

returns the Whittaker function $w_{a,b}(z)$.

See

- [Wikipedia - Whittaker function](#)

Examples

```
>> N(WhittakerW(2, 1/2, 2*2*I))  
-0.6160300298511752+I*17.877933521588048
```

Implementation status

- - experimental

Github

- [Implementation of WhittakerW](#)
 - [Rule definitions of WhittakerW](#)
-

WignerD

```
WignerD({j,m1,m2},a,b,c)
```

the Wigner D-function returns the matrix element of a rotation operator.

See:

- [Wikipedia - Wigner D-matrix](#)

Examples

```
>> WignerD({3/2,-3/2,-3/2}, ps, th, ph)  
Cos(th/2)^3/E^(I*3/2*ph+I*3/2*ps)
```

Implementation status

- - experimental

Github

- [Implementation of WignerD](#)
-

With

```
With({list_of_local_variables}, expr )
```

evaluates `expr` for the `list_of_local_variables` by replacing the local variables in `expr`.

Examples

```
>> With({x = a}, (1 + x^2) &)  
1+a^2&
```

Related terms

[Block](#), [Module](#)

Implementation status

- - full supported

Github

- [Implementation of With](#)
-

WordBoundary

WordBoundary

represents the boundary between words.

Examples

```
>> StringReplace("apple banana orange artichoke", "e" ~~ WordBoundary -> "E")
applE banana orangE artichokE
```

Xor

```
Xor(arg1, arg2, ...)
```

Logical XOR (exclusive OR) function. Returns `True` if an odd number of the arguments are `True` and the rest are `False`. Returns `False` if an even number of the arguments are `True` and the rest are `False`.

See: [Wikipedia: Exclusive or](#)

Examples

```
>> Xor(False, True)
True

>> Xor(True, True)
False
```

If an expression does not evaluate to `True` or `False`, `Xor` returns a result in symbolic form:

```
>> Xor(a, False, b)
Xor(a,b)

>> Xor()
False

>> Xor(a)
a

>> Xor(False)
False

>> Xor(True)
True

>> Xor(a, b)
Xor(a,b)
```

Implementation status

- - full supported

Github

- [Implementation of Xor](#)

Yellow

Yellow

RGB color value for the color yellow

Examples

```
>> Yellow  
RGBColor(1.0,1.0,0.0)
```

YuleDissimilarity

```
YuleDissimilarity(u, v)
```

returns the Yule dissimilarity between the two boolean 1-D lists `u` and `v`, which is defined as $R / (c_{tt} * c_{ff} + R / 2)$ where `n` is `len(u)`, `cij` is the number of occurrences of `u(k)=i` and `v(k)=j` for `k < n`, and $R = 2 * c_{tf} * c_{ft}$.

Examples

```
>> YuleDissimilarity({1, 0, 1, 1, 0, 1, 1}, {0, 1, 1, 0, 0, 0, 1})  
6/5
```

Implementation status

- - full supported

Github

- [Implementation of YuleDissimilarity](#)
-

ZernikeR

`ZernikeR(n,m,p)`

returns the radial Zernike polynomial

See:

- [Wikipedia - Zernike polynomials](#)

Examples

```
>> ZernikeR(0,0,p)
1

>> ZernikeR(1,1,p)
p

>> ZernikeR(2,0,p)
-1+2*p^2

>> ZernikeR(2,2,p)
p^2

>> ZernikeR(3,1,p)
-2*p+3*p^3

>> ZernikeR(3,3,p)
p^3

>> ZernikeR(6,4,p)
-5*p^4+6*p^6
```

Implementation status

- - experimental

Github

- [Implementation of ZernikeR](#)
-

Zeta

```
Zeta(z)
```

returns the Riemann zeta function of `z`.

See:

- [Wikipedia - Riemann zeta function](#)
- [Wikipedia - Particular values of the Riemann zeta function](#)
- [Fungrim - Riemann zeta function](#)

Examples

```
>> Zeta(2)  
Pi^2/6
```

Implementation status

- - partially implemented

Github

- [Implementation of Zeta](#)
-

ZTransform

```
ZTransform(x,n,z)
```

returns the Z-Transform of x .

See:

- [Wikipedia - Z-transform](#)

Examples

```
>> ZTransform(a*f(n)+ b*g(n), n, z)
a*ZTransform(f(n),n,z)+b*ZTransform(g(n),n,z)
```

Implementation status

- - experimental

Github

- [Implementation of ZTransform](#)
 - [Rule definitions of ZTransform](#)
-